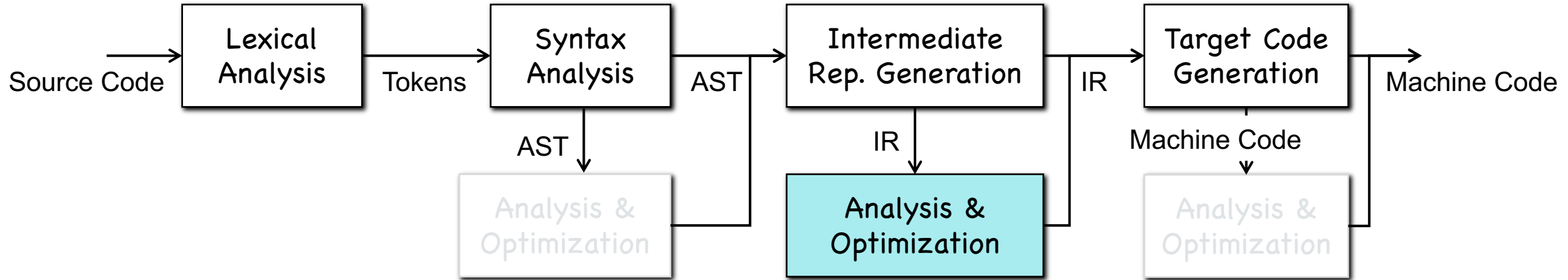


Chapter 9-1

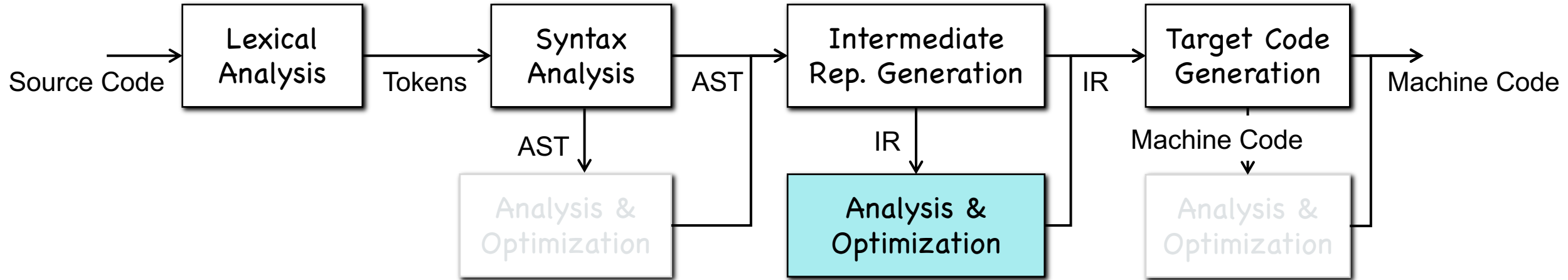
Data Flow Analysis (DFA)

Data Flow Analysis



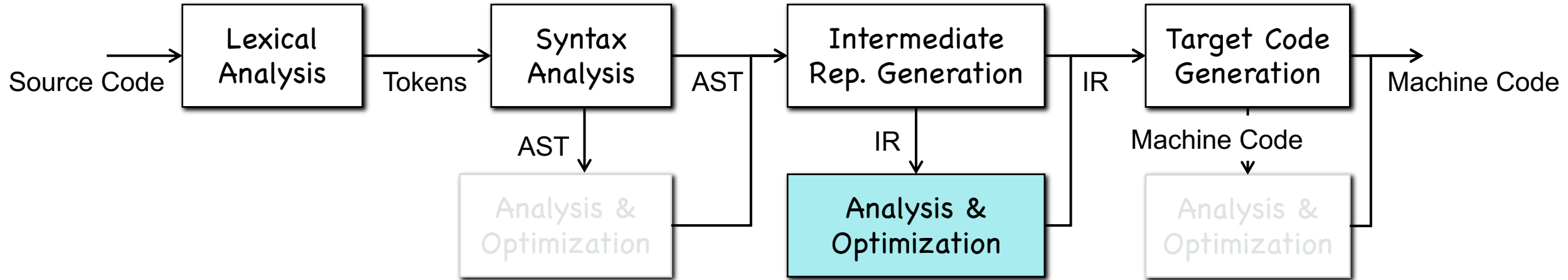
- IR is friendly to analysis and optimization in compilers
 - Rich source code information like types (compared to machine code)
 - Standard format, control flow graphs, etc. (compared to source code)
 - ...

Data Flow Analysis



- Data flow analysis is a form of static program analysis
 - Machine-independent compiler optimization
 - Detection of software errors and vulnerabilities
 - ...

Data Flow Analysis



- Data flow analysis is a form of static program analysis
 - **Machine-independent compiler optimization**
 - Detection of software errors and vulnerabilities
 - ...

Sources of Optimization

- Redundancy in source code

Sources of Optimization

- Redundancy in source code

```
std::vector<int> Vec;  
...  
for (int K = 0; K < Vec.size(); ++K) {  
    ...  
}
```

Sources of Optimization

- Redundancy in source code

```
std::vector<int> Vec;  
...  
for (int K = 0; K < Vec.size(); ++K) {  
    ...  
}
```

Sources of Optimization

- Redundancy in source code

```
std::vector<int> Vec;  
...  
for (int K = 0; K < Vec.size(); ++K) {  
    ...  
}
```

```
struct A { int x; int y; } ;  
...  
A *Obj = new A;  
printf("%d\n", Obj->y);  
printf("%d\n", Obj->y);
```


Sources of Optimization

- Redundancy in source code

```
std::vector<int> Vec;
...
for (int K = 0; K < Vec.size();
    ...
}
```

```
struct A { int x; int y; } ;
...
A *Obj = new A;
printf("%d\n", Obj->y);
printf("%d\n", Obj->y);
```

```
; Function Attrs: noline norecurse optnone ssp uwtable mustprogress
define dso_local i32 @main() #0 {
    %1 = alloca i32, align 4
    %2 = alloca %struct.A*, align 8
    store i32 0, i32* %1, align 4
    %3 = call noalias nonnull i8* @_Znwm(i64 8) #3
    %4 = bitcast i8* %3 to %struct.A*
    store %struct.A* %4, %struct.A** %2, align 8
    %5 = load %struct.A*, %struct.A** %2, align 8
    %6 = getelementptr inbounds %struct.A, %struct.A* %5, i32 0, i32 1
    store i32 1, i32* %6, align 4
    %7 = load %struct.A*, %struct.A** %2, align 8
    %8 = getelementptr inbounds %struct.A, %struct.A* %7, i32 0, i32 1
    %9 = load i32, i32* %8, align 4
    %10 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0,
    %11 = load %struct.A*, %struct.A** %2, align 8
    %12 = getelementptr inbounds %struct.A, %struct.A* %11, i32 0, i32 1
    %13 = load i32, i32* %12, align 4
    %14 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0,
    ret i32 0
}
```

Sources of Optimization

- Redundancy in source code

```
std::vector<int> Vec;
...
for (int K = 0; K < Vec.size();
    ...
}
```

```
struct A { int x; int y; };
...
A *Obj = new A;
printf("%d\n", Obj->y);
printf("%d\n", Obj->y);
```

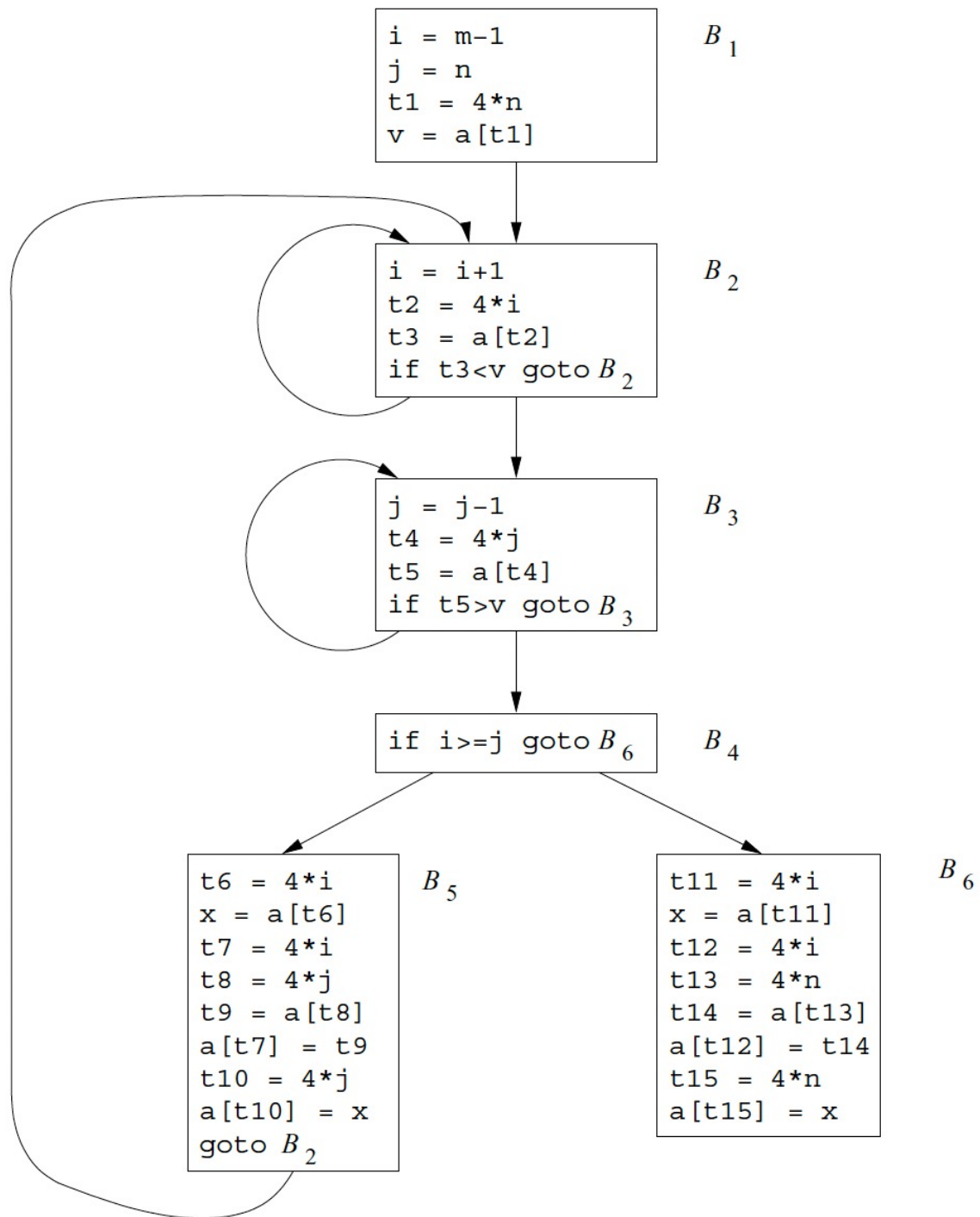
```
; Function Attrs: noline norecurse optnone ssp uwtable mustprogress
define dso_local i32 @main() #0 {
    %1 = alloca i32, align 4
    %2 = alloca %struct.A*, align 8
    store i32 0, i32* %1, align 4
    %3 = call noalias nonnull i8* @_Znwm(i64 8) #3
    %4 = bitcast i8* %3 to %struct.A*
    store %struct.A* %4, %struct.A** %2, align 8
    %5 = load %struct.A*, %struct.A** %2, align 8
    %6 = getelementptr inbounds %struct.A, %struct.A* %5, i32 0, i32 1
    store i32 1, i32* %6, align 4
    %7 = load %struct.A*, %struct.A** %2, align 8
    %8 = getelementptr inbounds %struct.A, %struct.A* %7, i32 0, i32 1
    %9 = load i32, i32* %8, align 4
    %10 = call i32 @__printf(i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0,
    %11 = load %struct.A*, %struct.A** %2, align 8
    %12 = getelementptr inbounds %struct.A, %struct.A* %11, i32 0, i32 1
    %13 = load i32, i32* %12, align 4
    %14 = call i32 @__printf(i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @.str, i64 0,
    ret i32 0
}
```

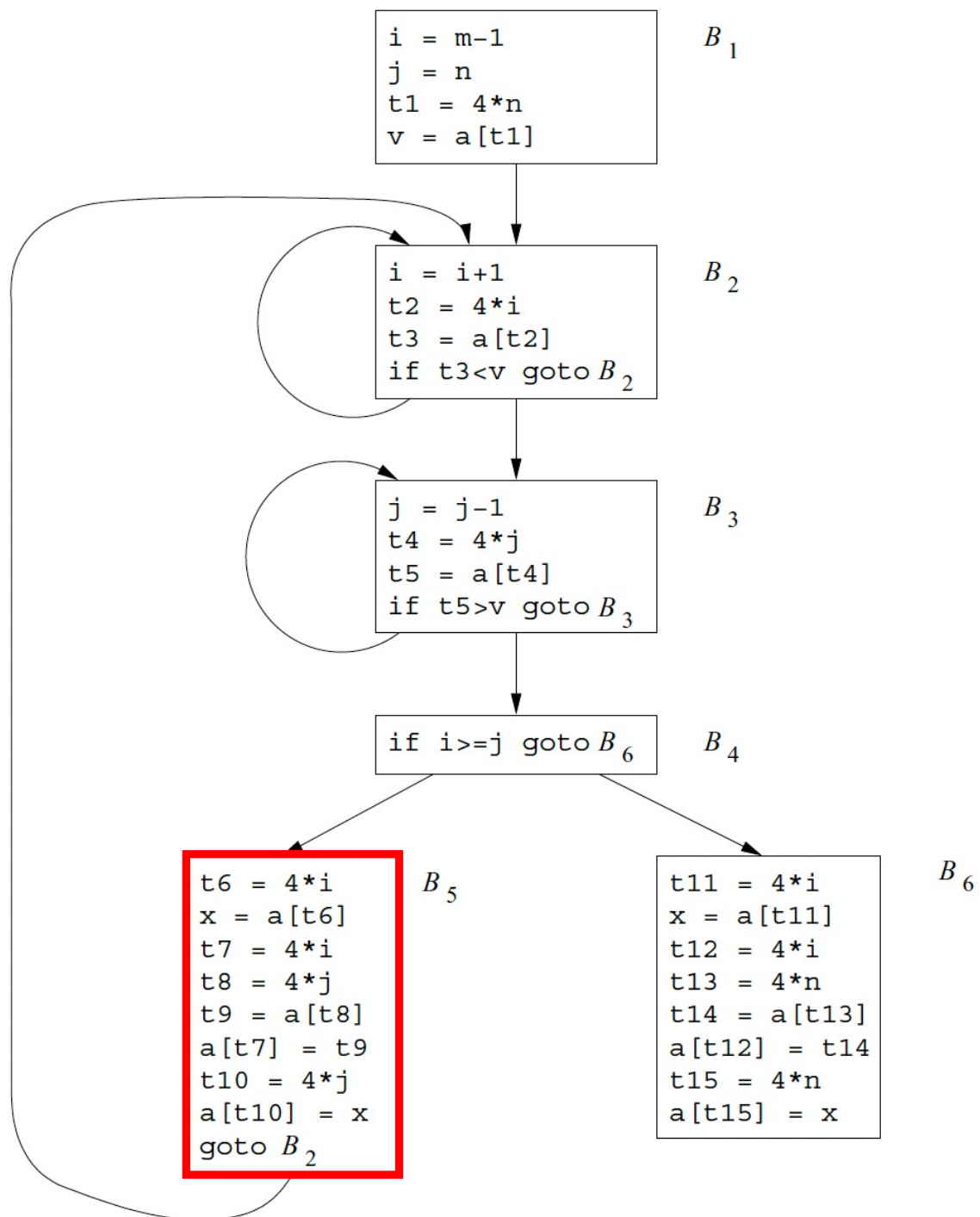
Quick Sort: A Running Example

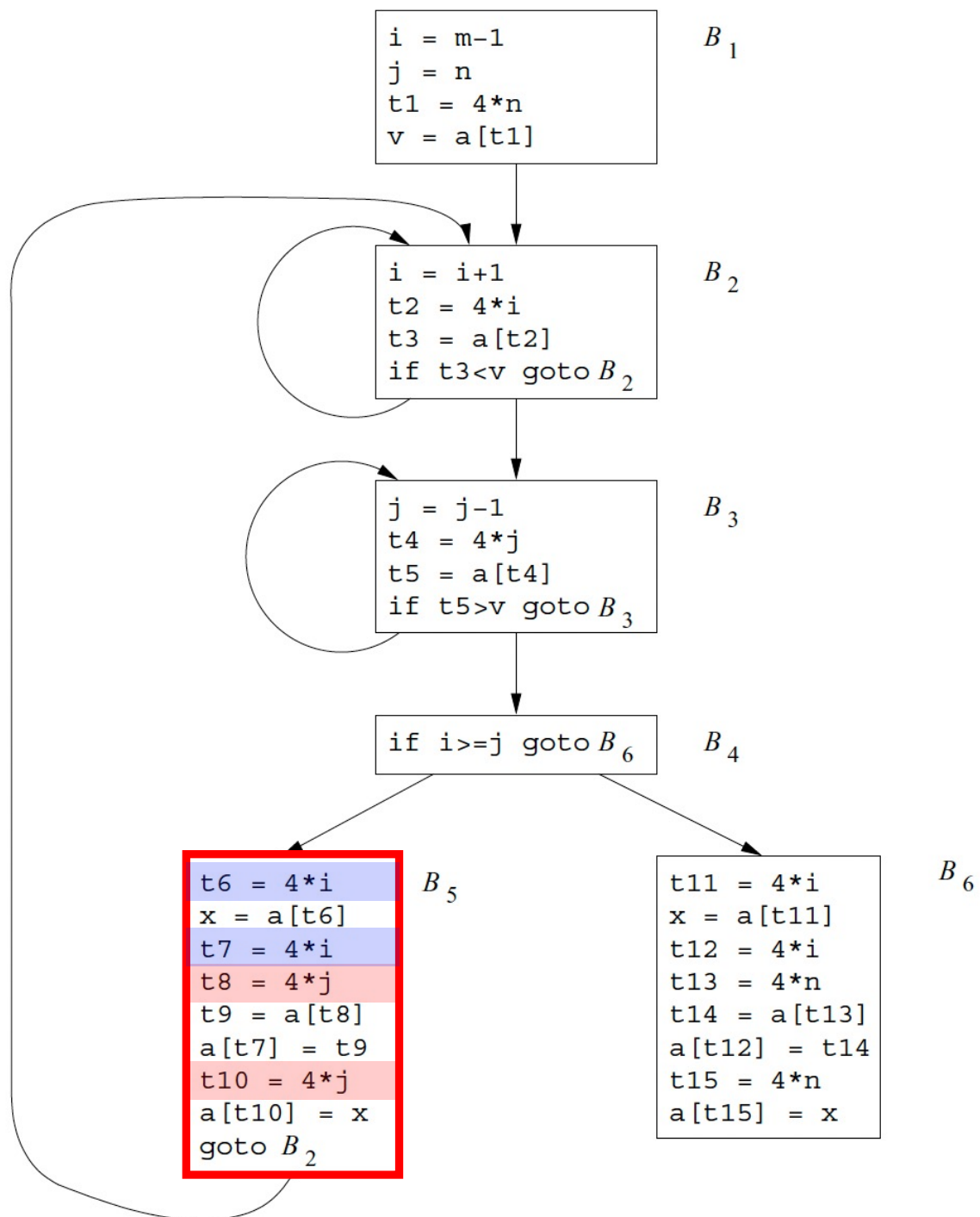
```
void quicksort(int m, int n)
    /* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x; /* swap a[i], a[j] */
    }
    x = a[i]; a[i] = a[n]; a[n] = x; /* swap a[i], a[n] */
    /* fragment ends here */
    quicksort(m, j); quicksort(i+1, n);
}
```

Quick Sort: A Running Example

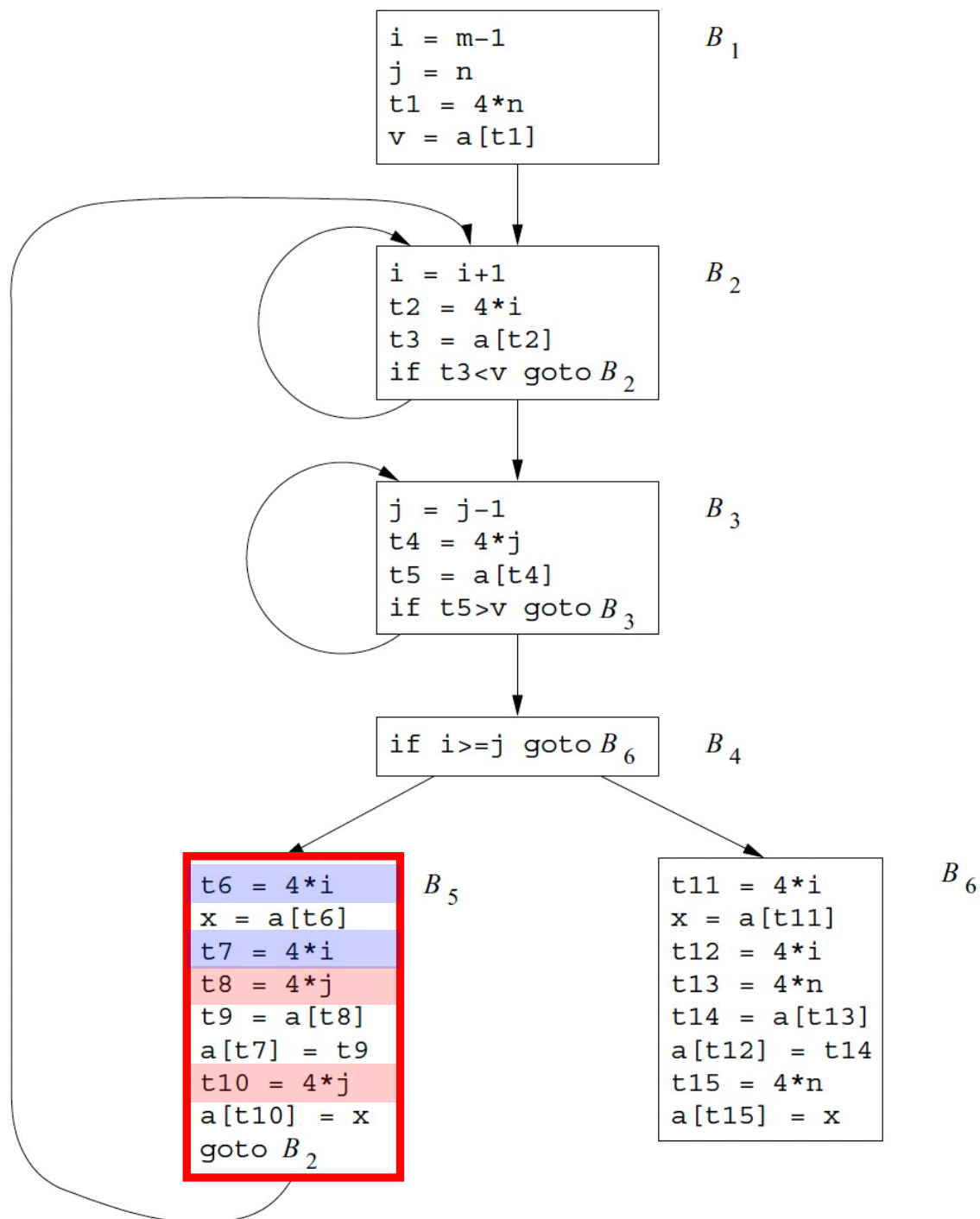
(1)	i = m-1	(16)	t7 = 4*i
(2)	j = n	(17)	t8 = 4*j
(3)	t1 = 4*n	(18)	t9 = a[t8]
(4)	v = a[t1]	(19)	a[t7] = t9
(5)	i = i+1	(20)	t10 = 4*j
(6)	t2 = 4*i	(21)	a[t10] = x
(7)	t3 = a[t2]	(22)	goto (5)
(8)	if t3<v goto (5)	(23)	t11 = 4*i
(9)	j = j-1	(24)	x = a[t11]
(10)	t4 = 4*j	(25)	t12 = 4*i
(11)	t5 = a[t4]	(26)	t13 = 4*n
(12)	if t5>v goto (9)	(27)	t14 = a[t13]
(13)	if i>=j goto (23)	(28)	a[t12] = t14
(14)	t6 = 4*i	(29)	t15 = 4*n
(15)	x = a[t6]	(30)	a[t15] = x



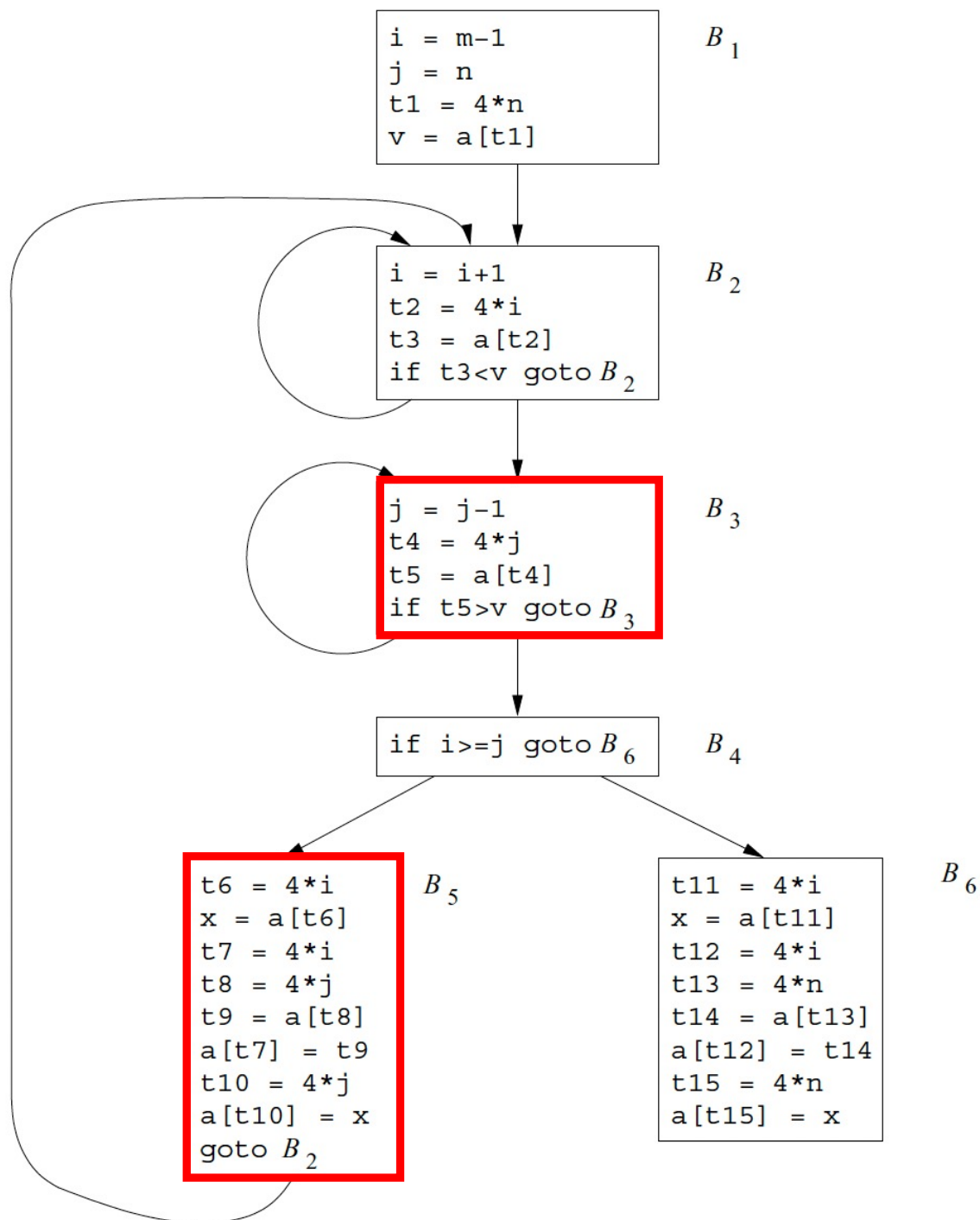




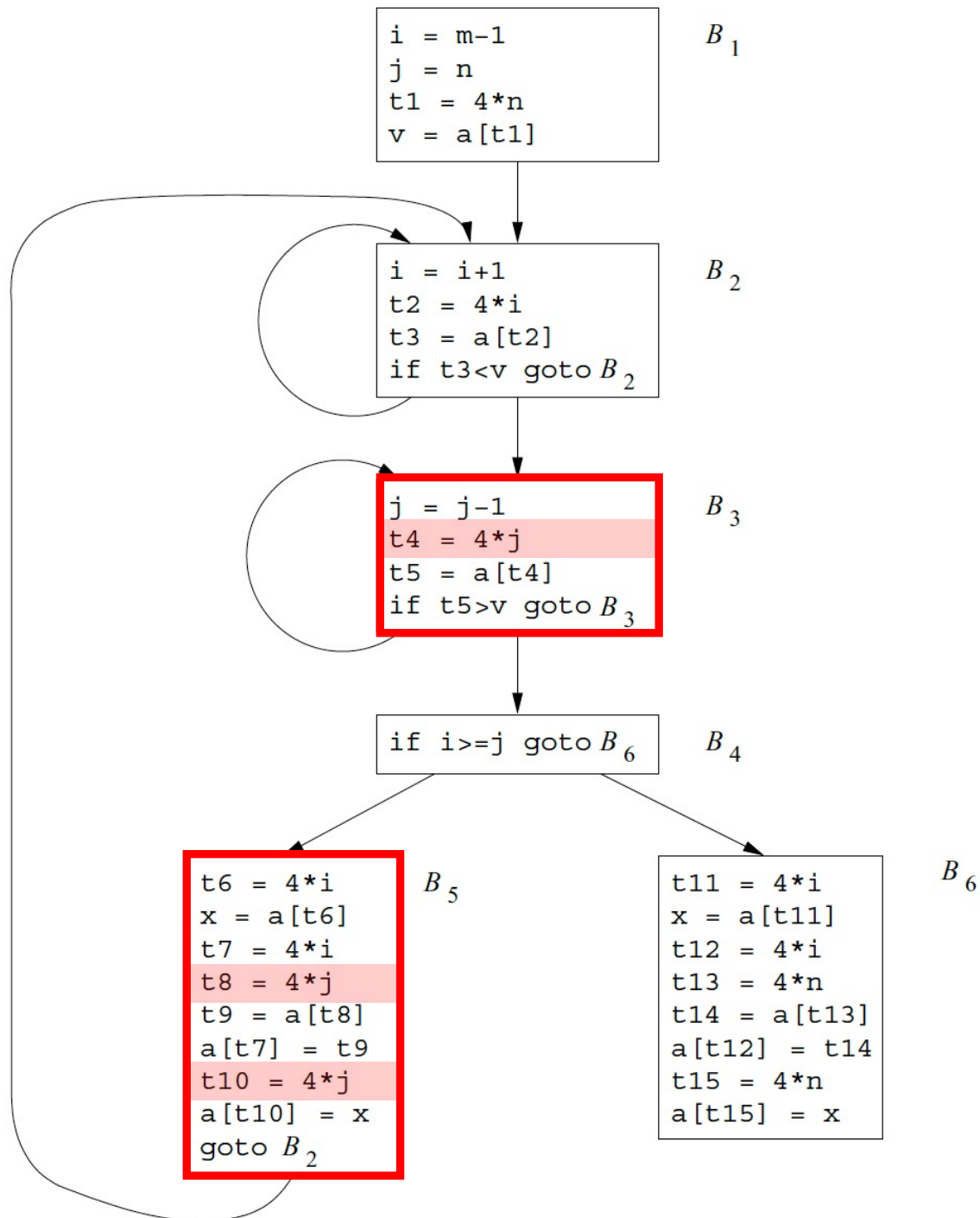
1. Local common subexpression

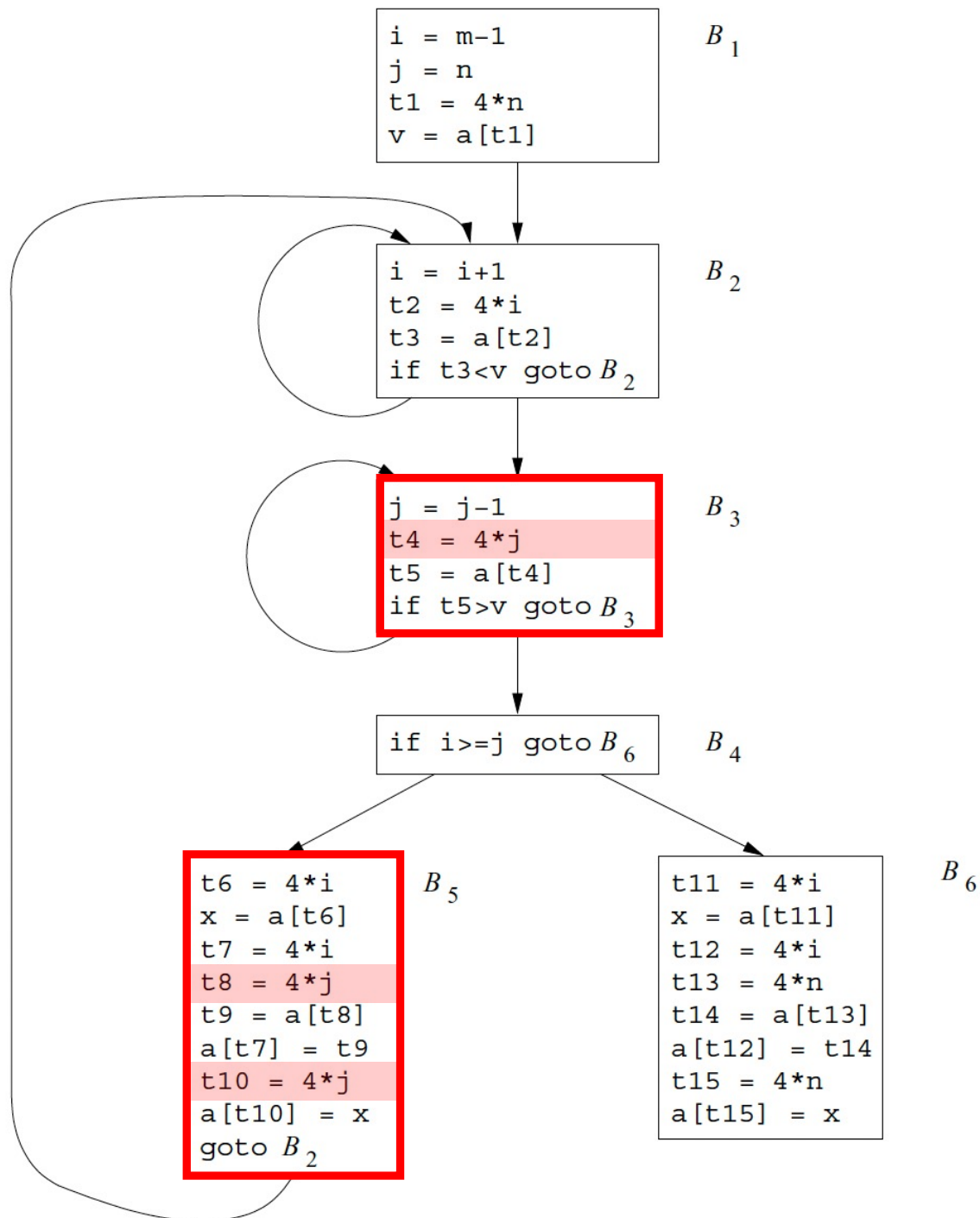


1. Local common subexpression

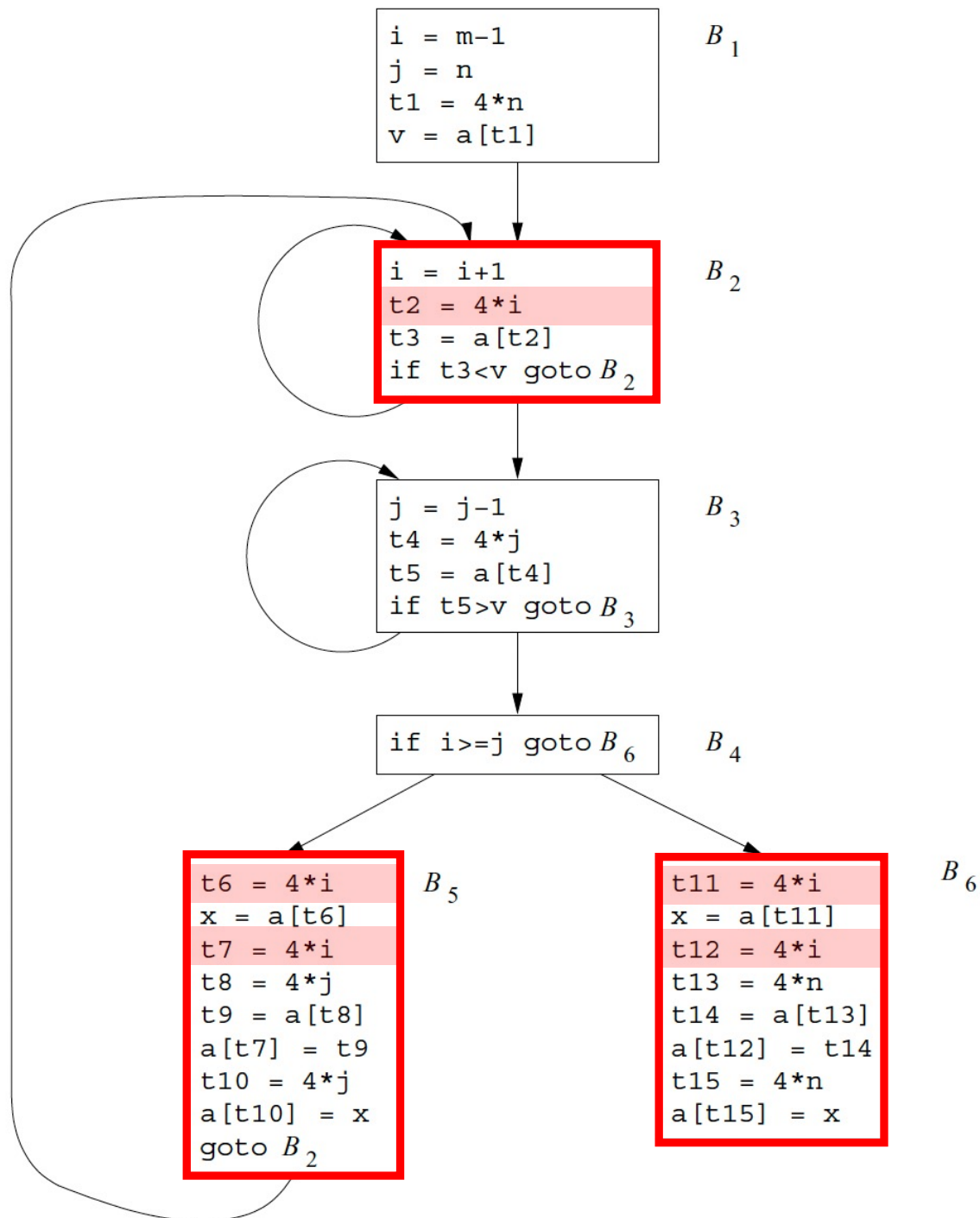


1. Local common subexpression

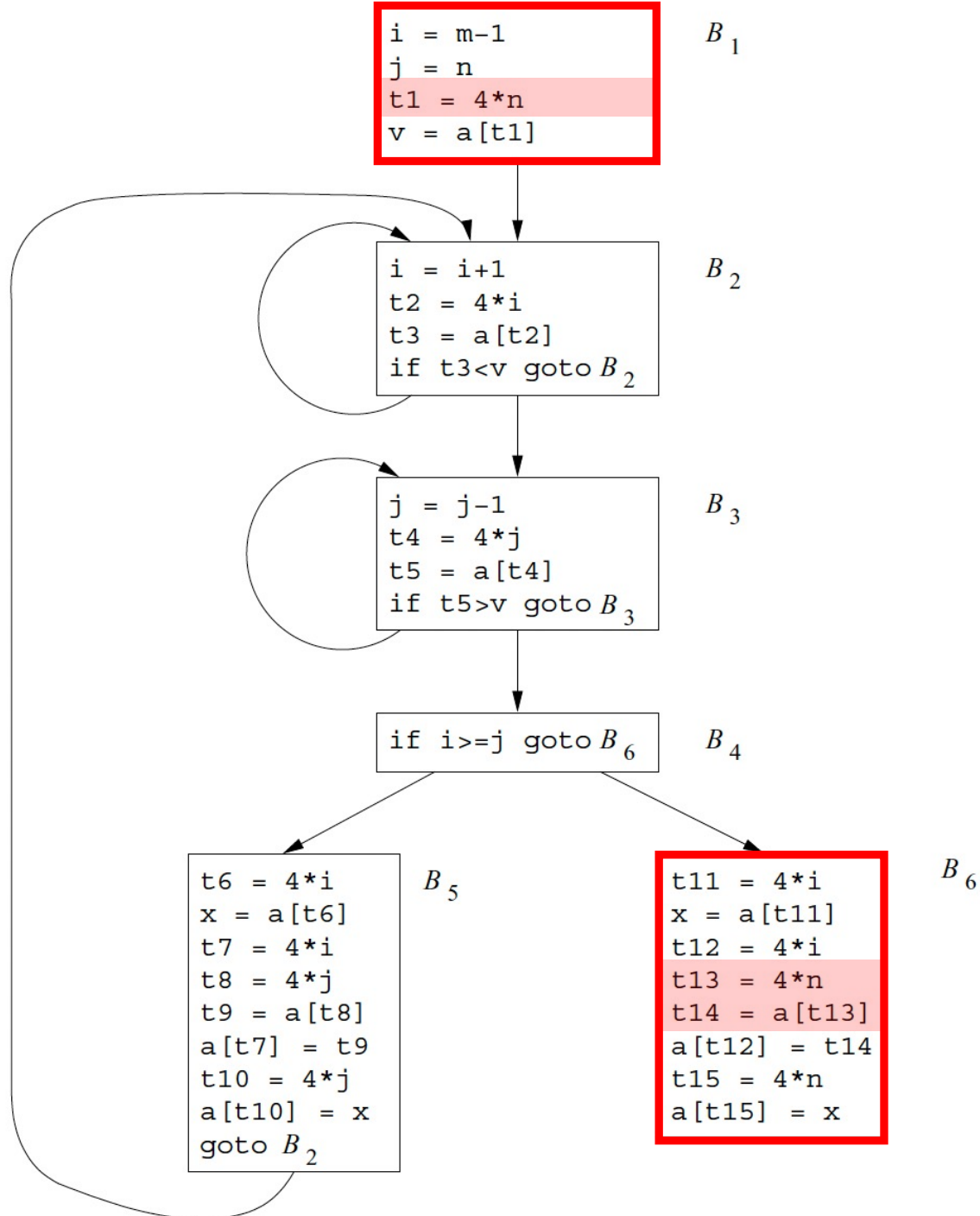




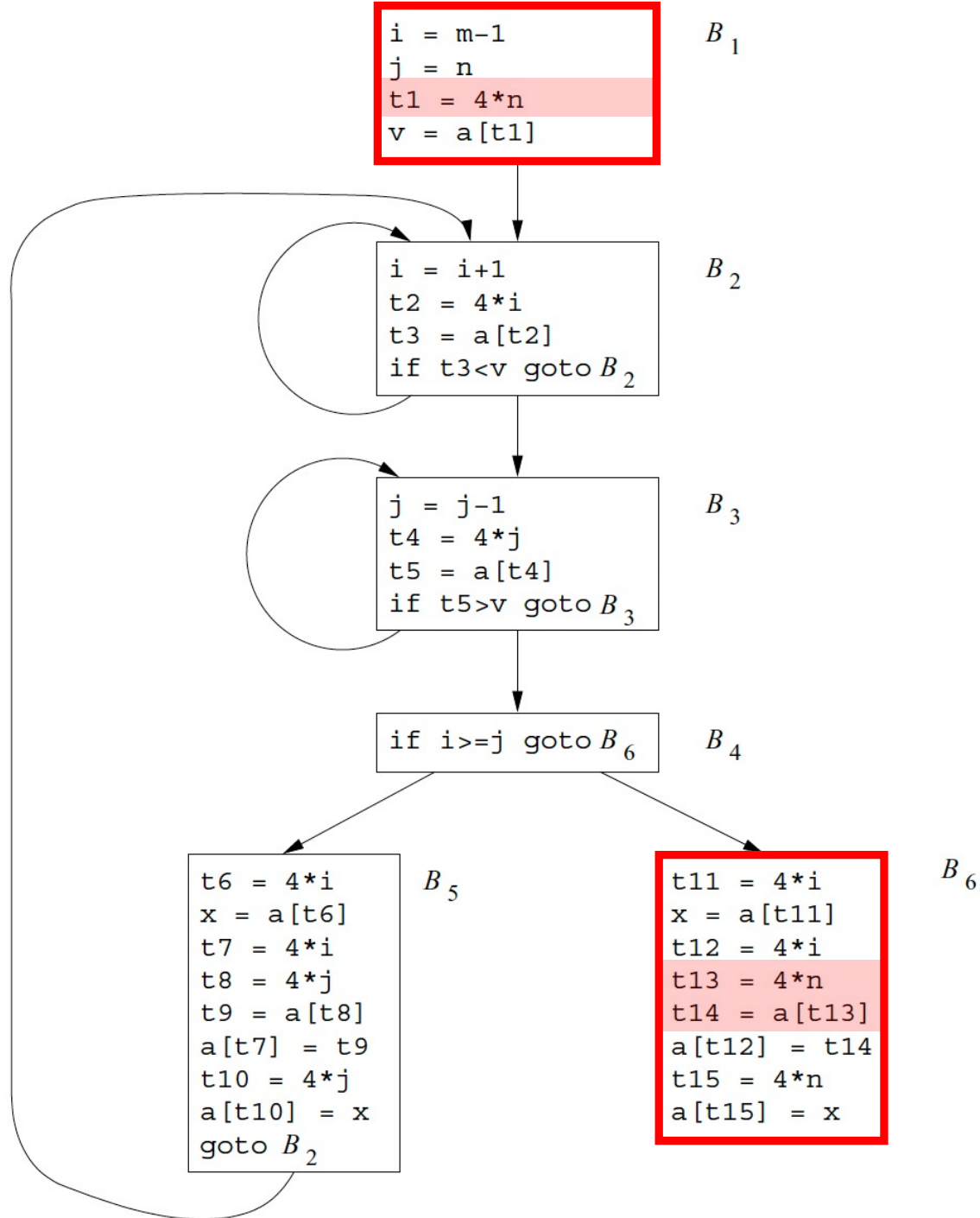
1. Local common subexpression
2. **Global common subexpression**



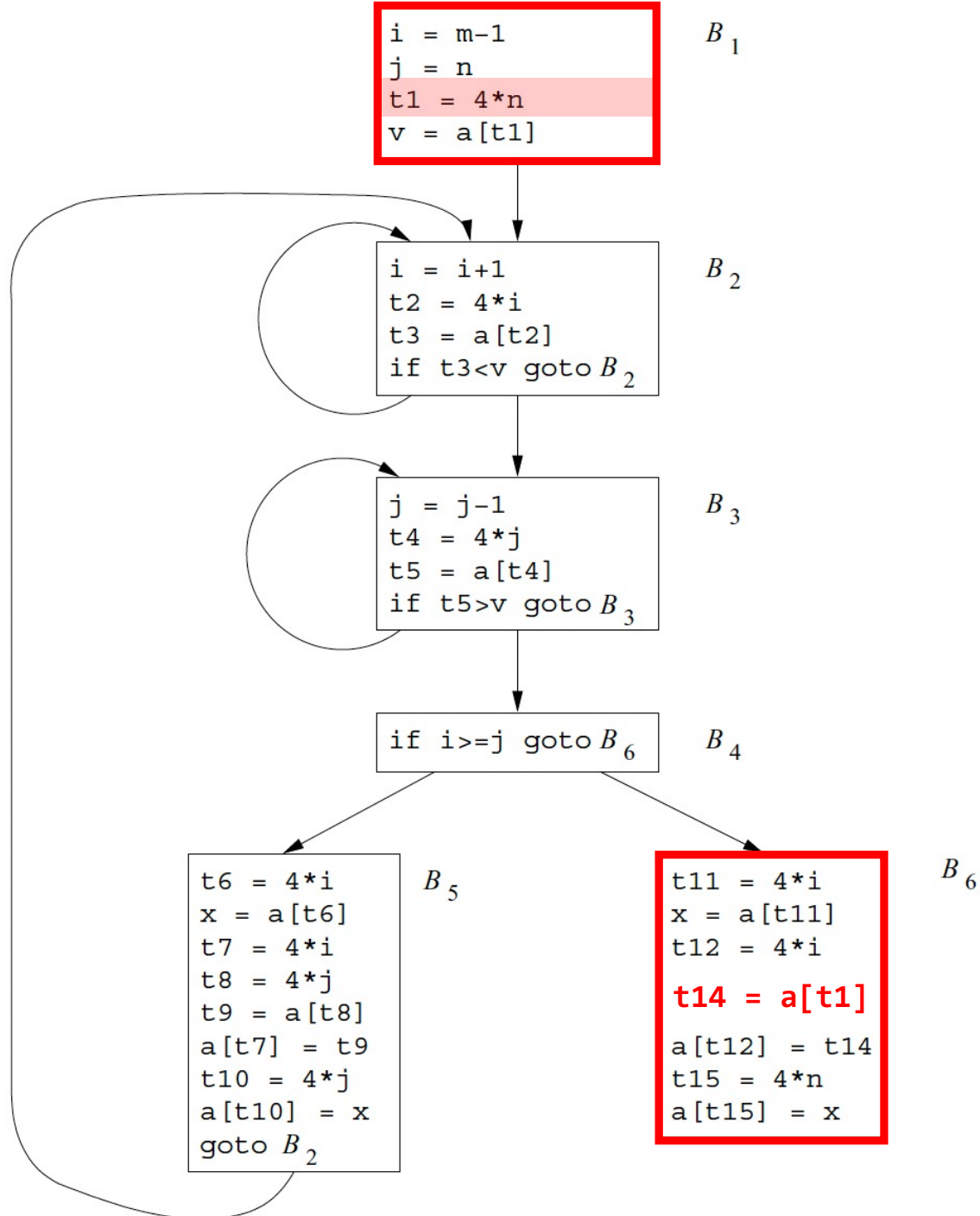
1. Local common subexpression
2. **Global common subexpression**



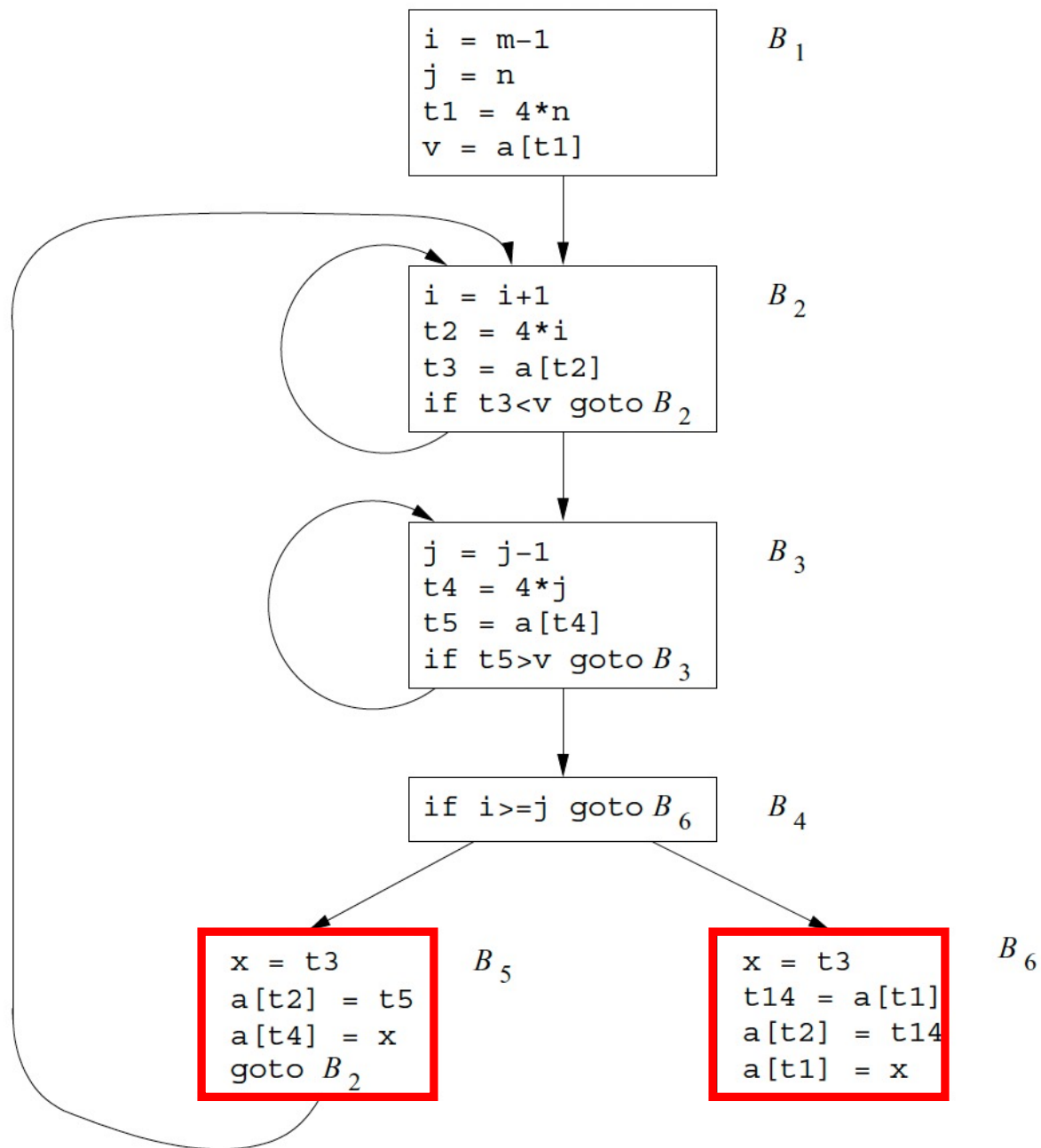
1. Local common subexpression
2. **Global common subexpression**



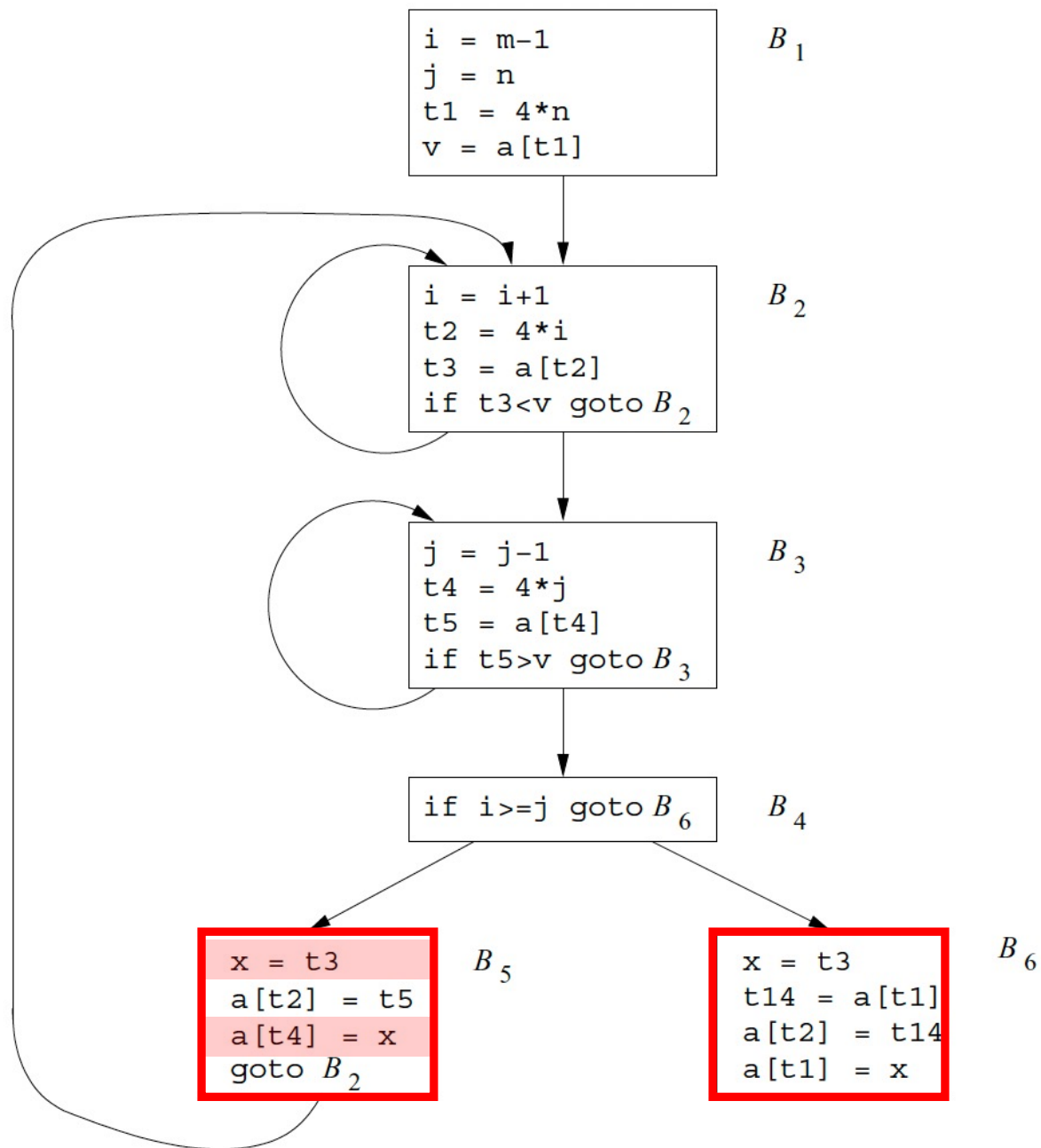
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation



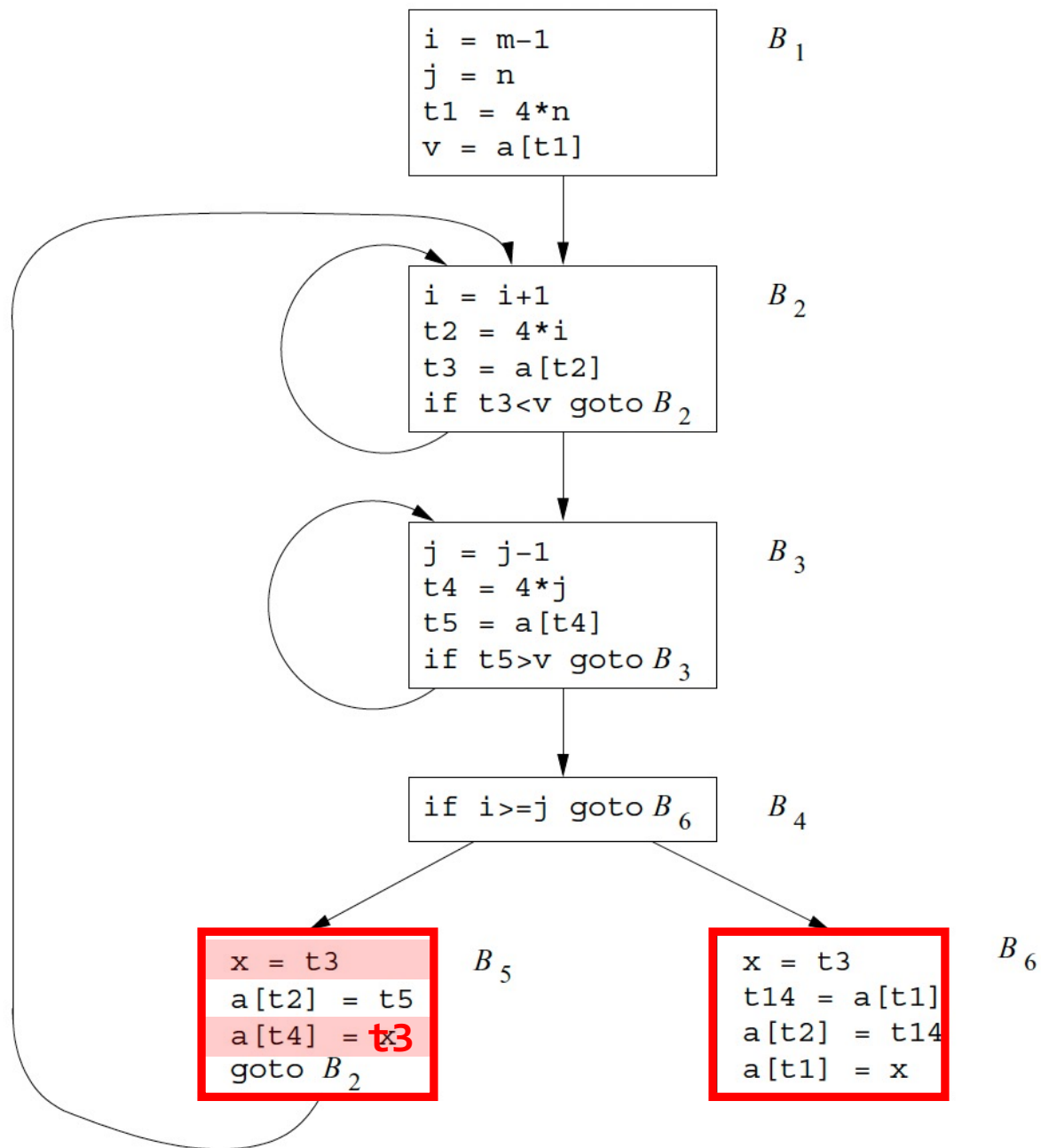
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation



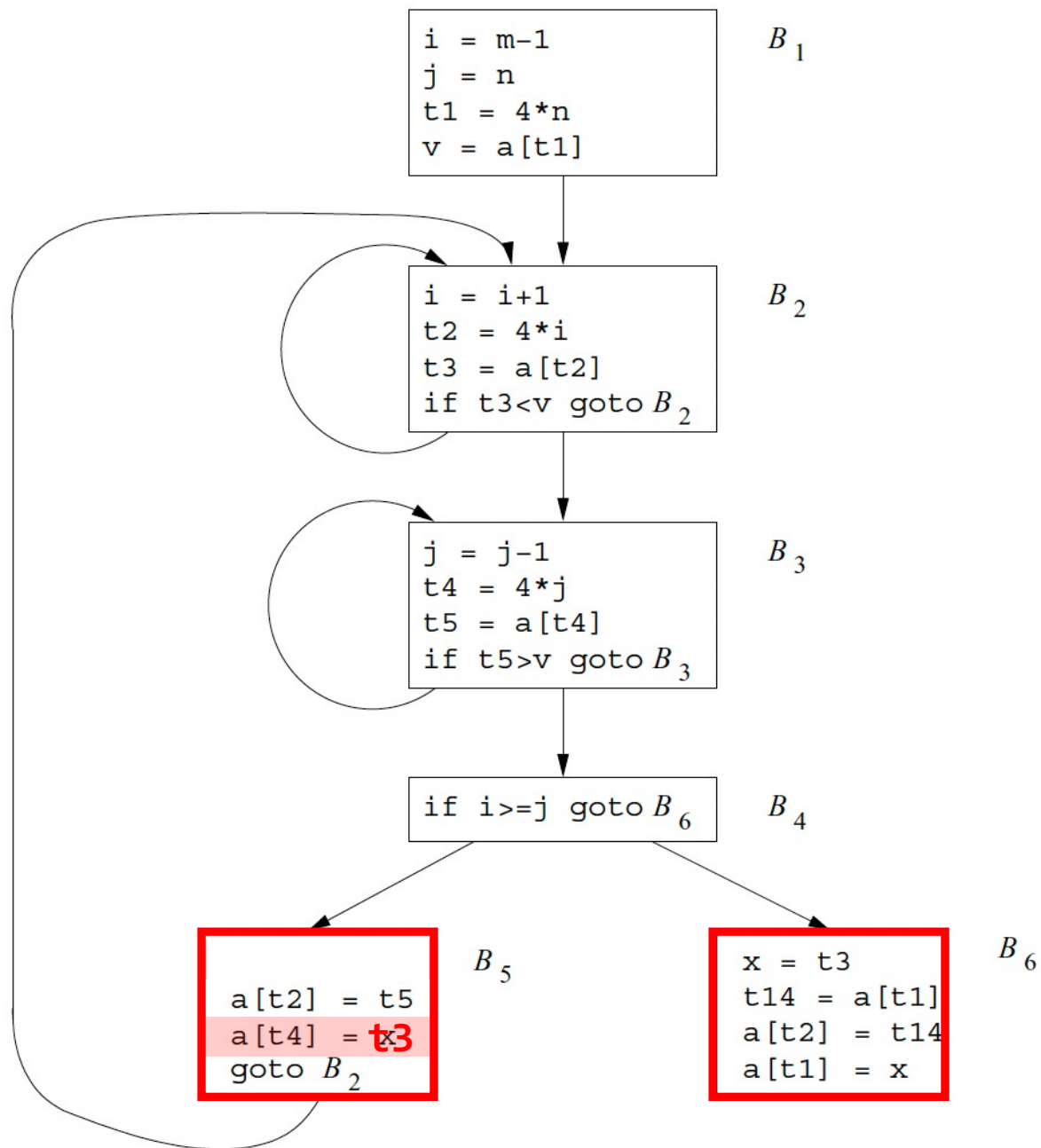
1. Local common subexpression
2. **Global common subexpression**
3. **Copy propagation**



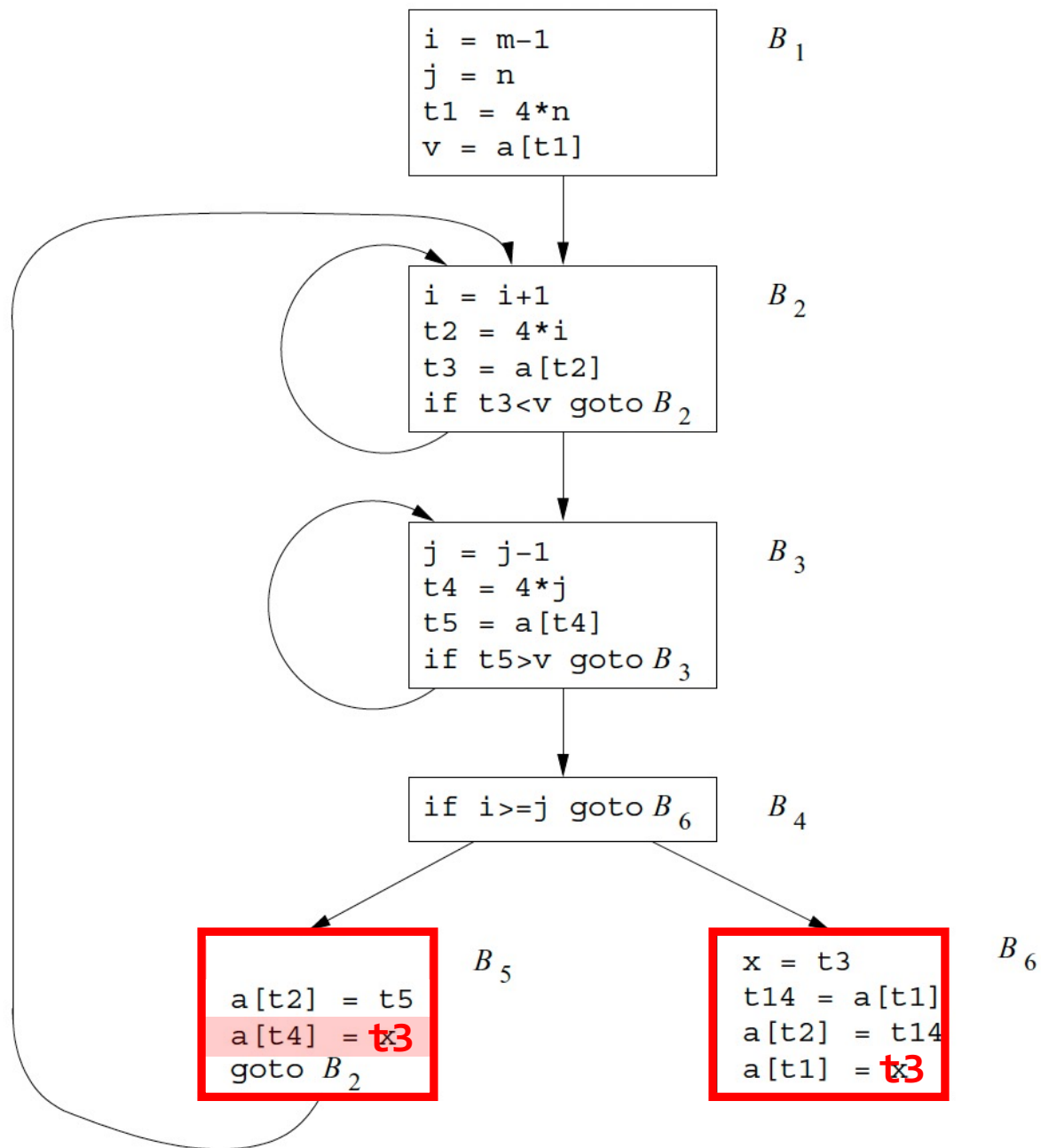
1. Local common subexpression
2. **Global common subexpression**
3. **Copy propagation**



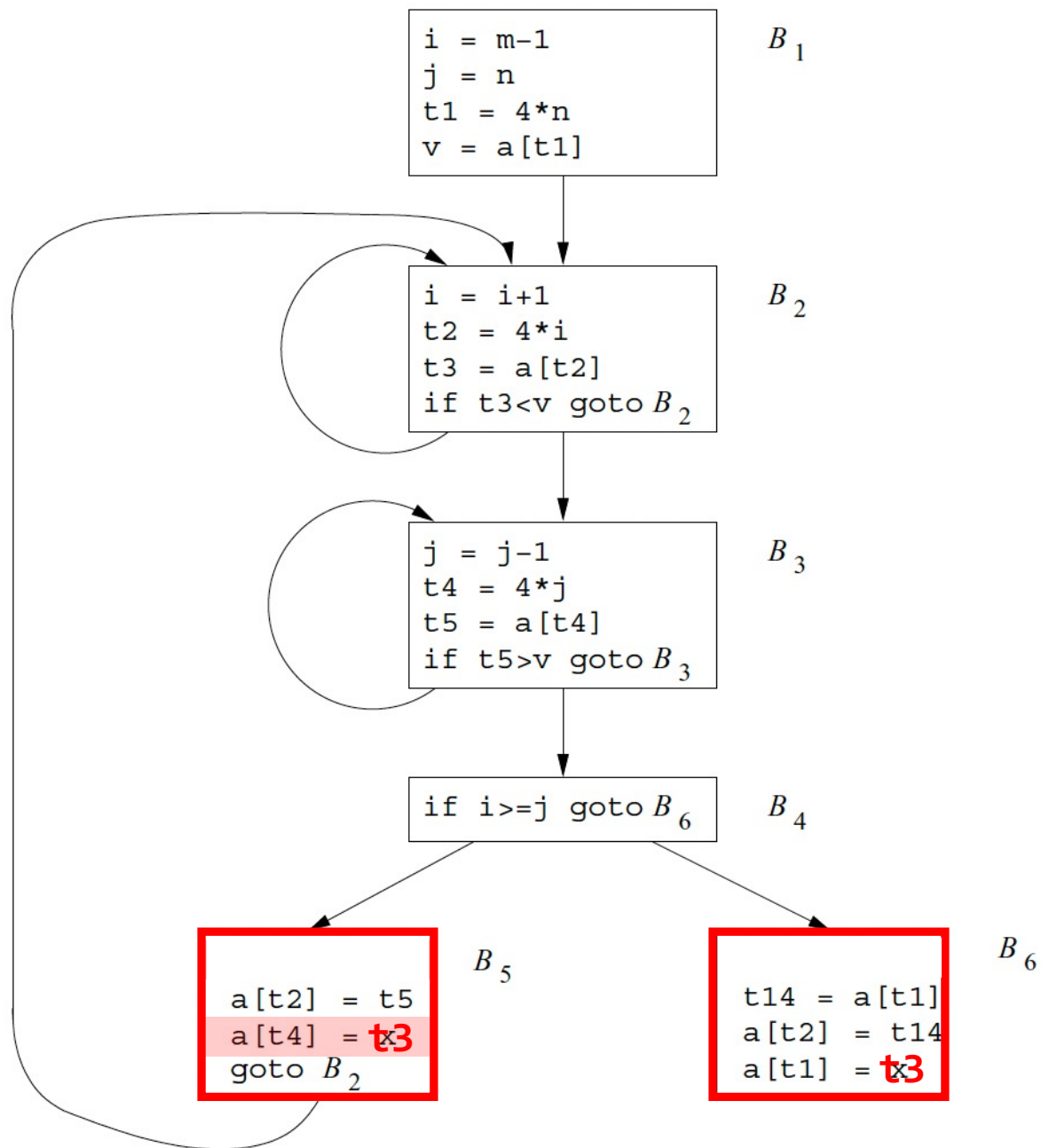
1. Local common subexpression
2. **Global common subexpression**
3. **Copy propagation**



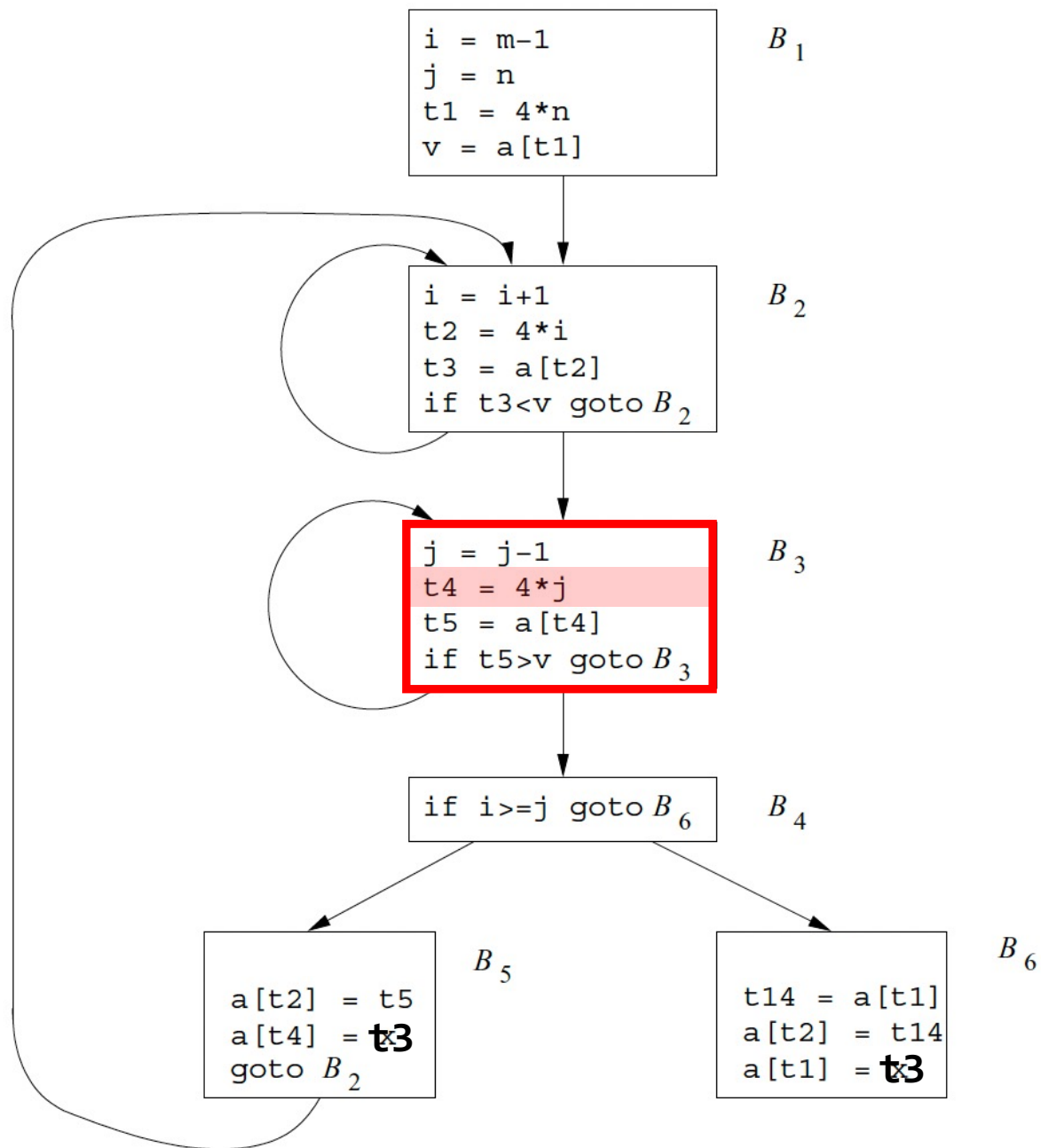
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation



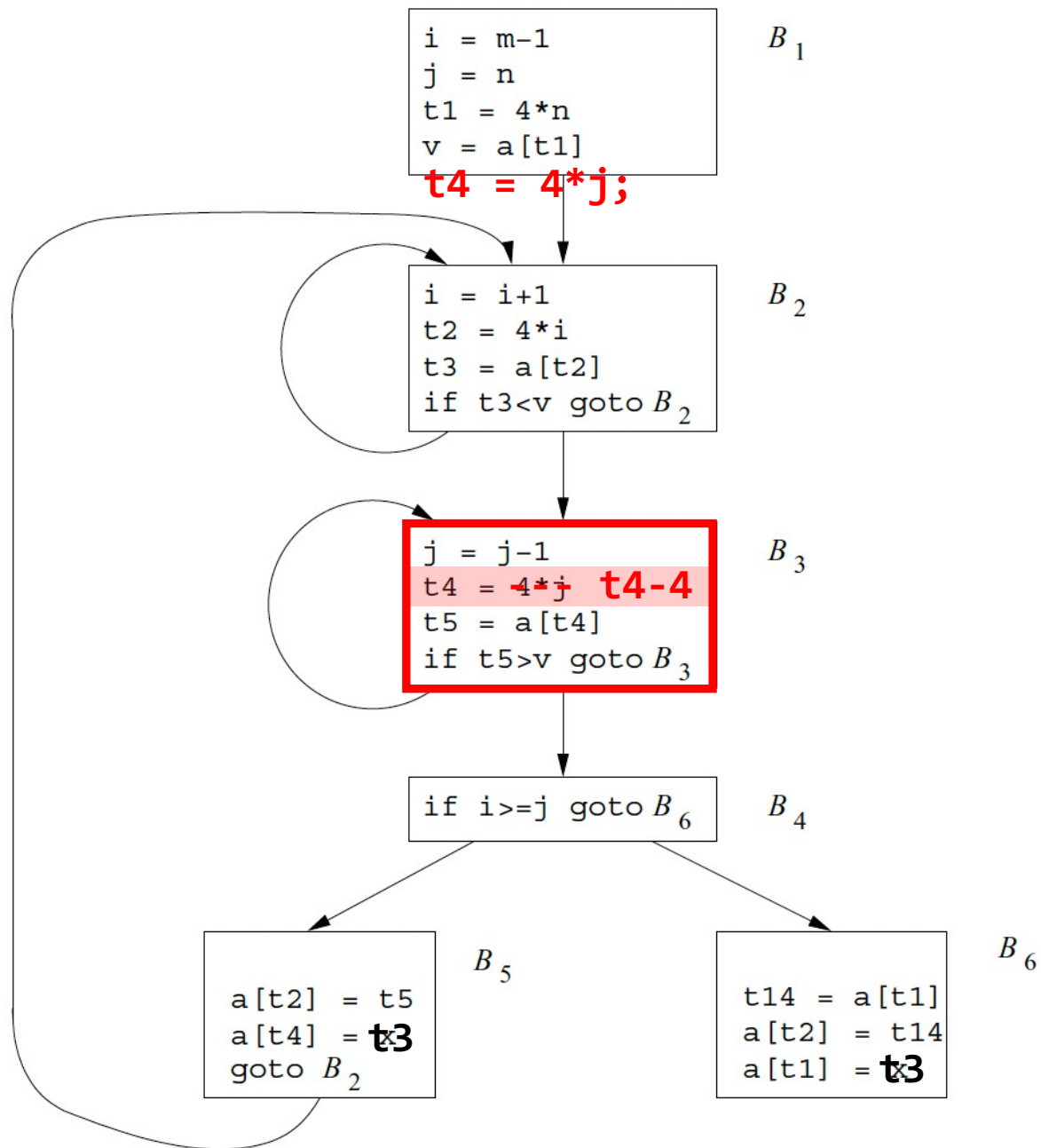
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation



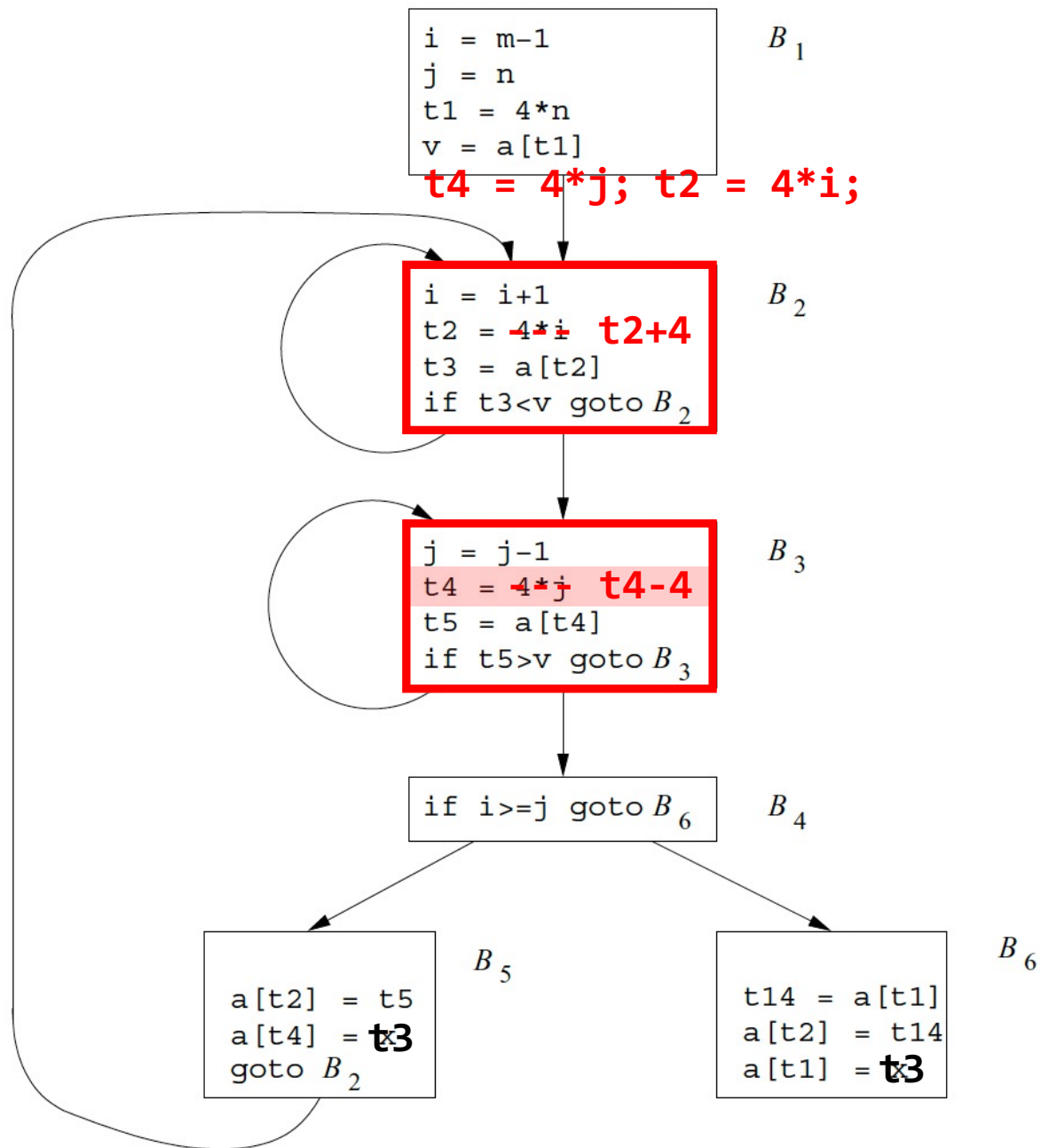
1. Local common subexpression
2. **Global common subexpression**
3. **Copy propagation**



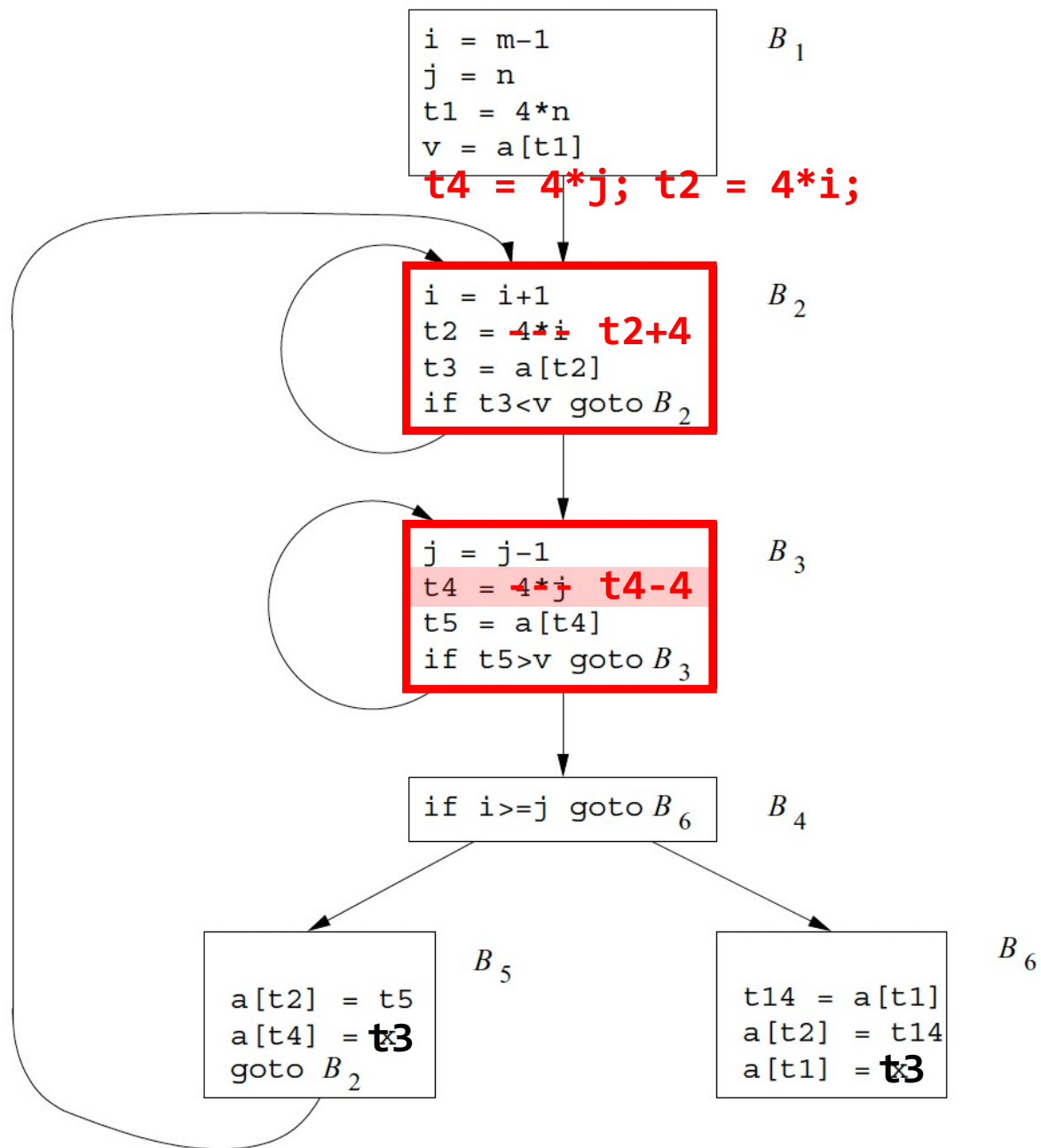
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. **Strength reduction**



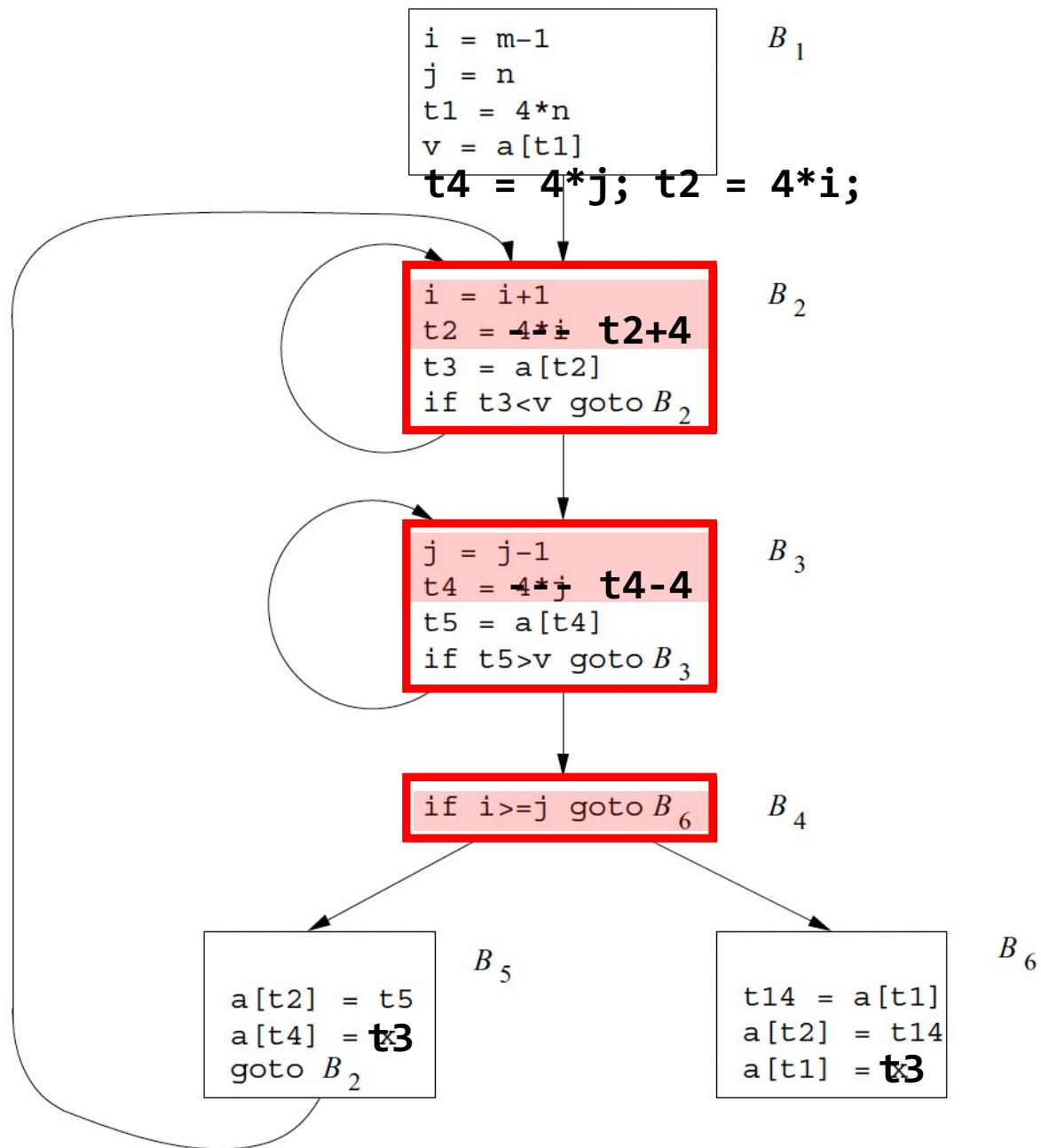
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. Strength reduction



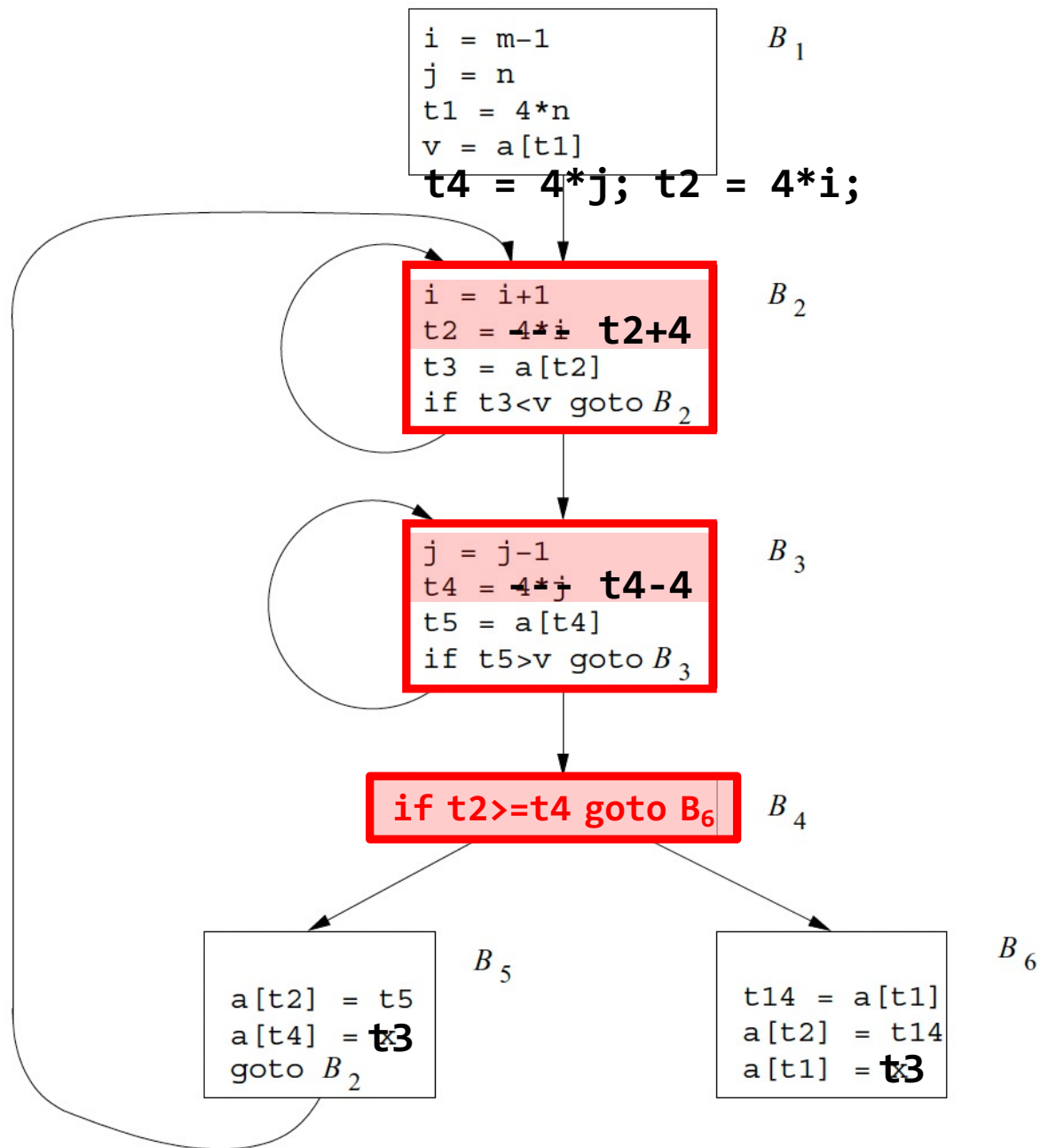
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. Strength reduction



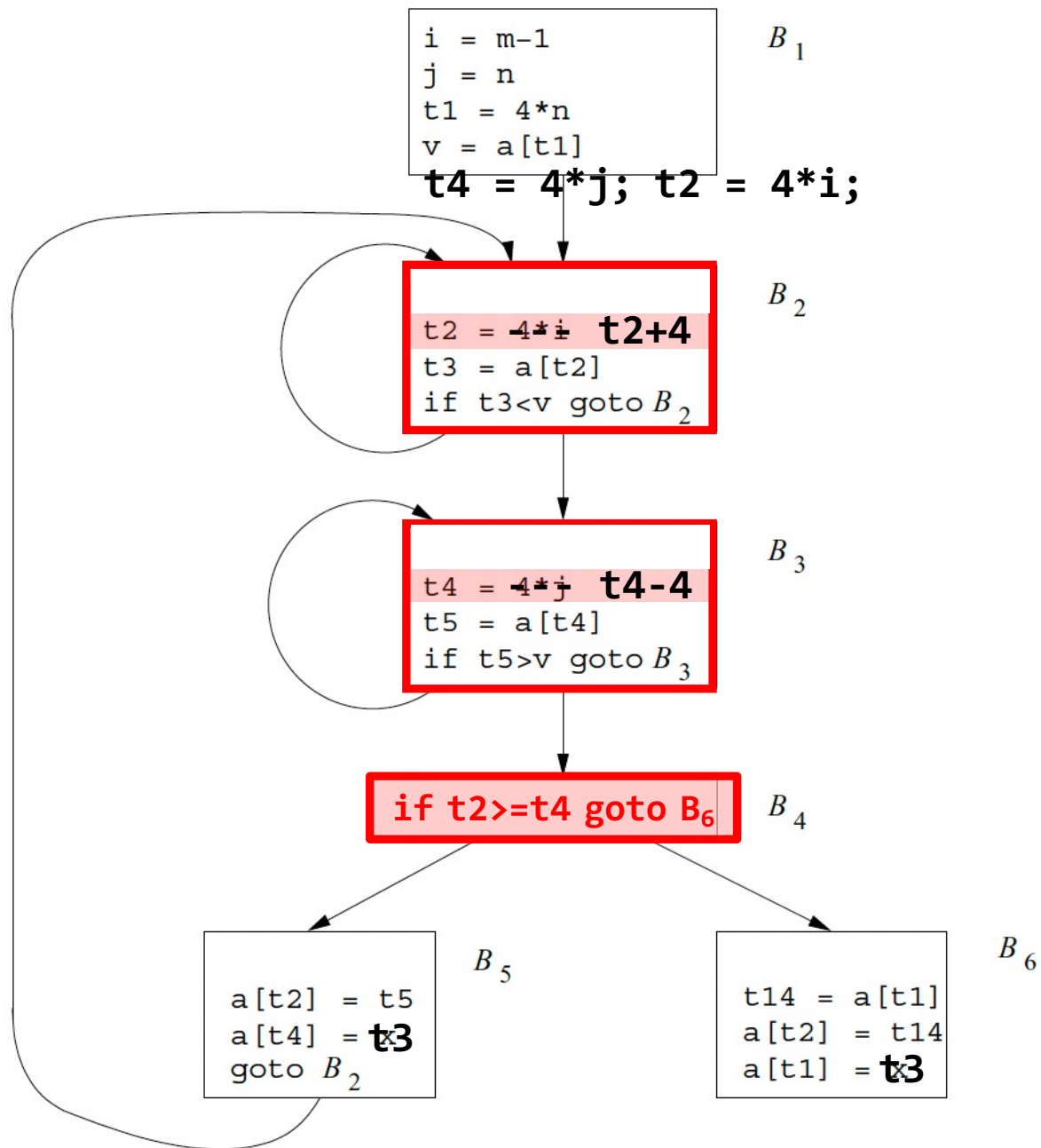
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. Strength reduction
5. Induction-variable elimination



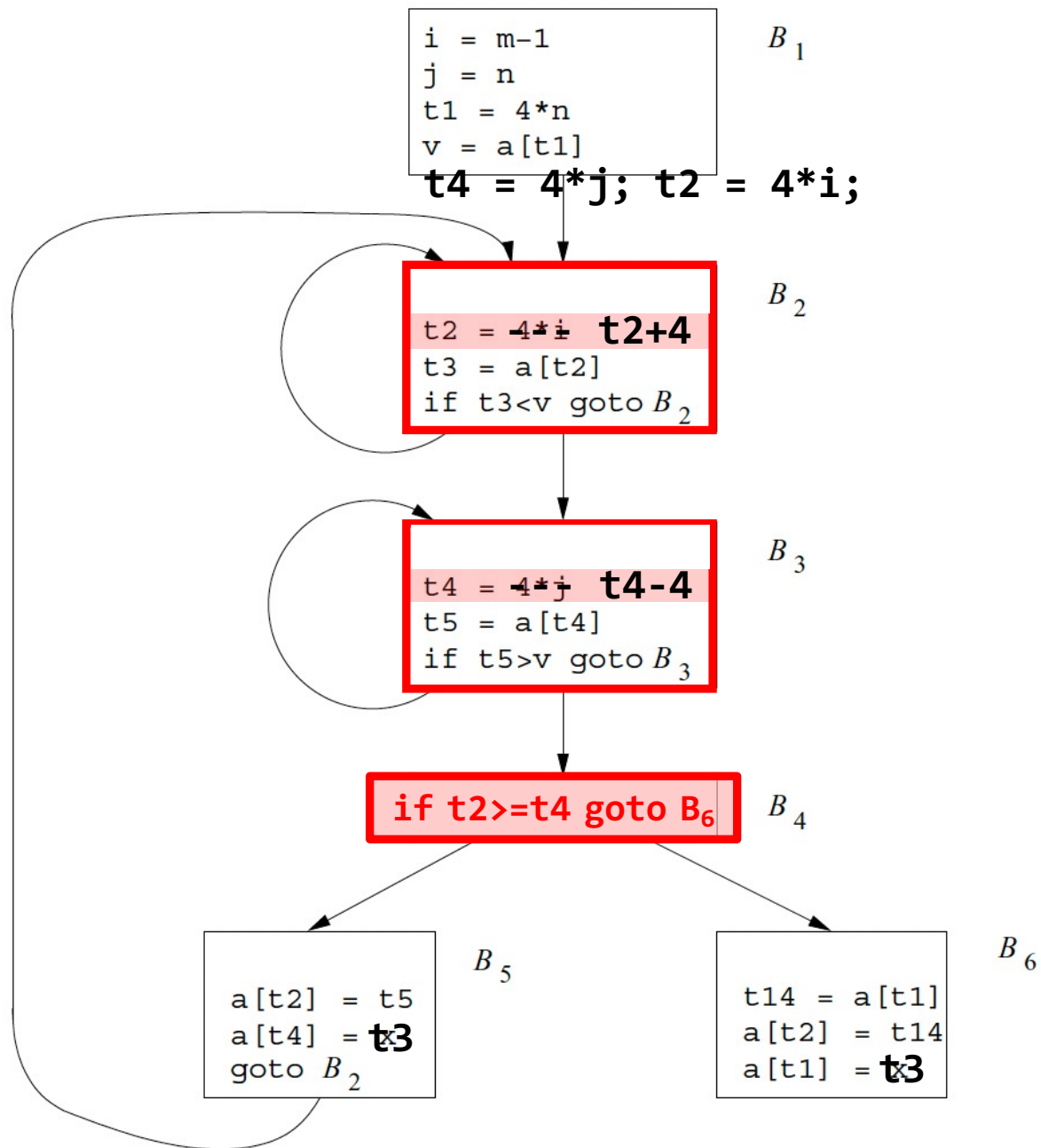
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. **Strength reduction**
5. Induction-variable elimination



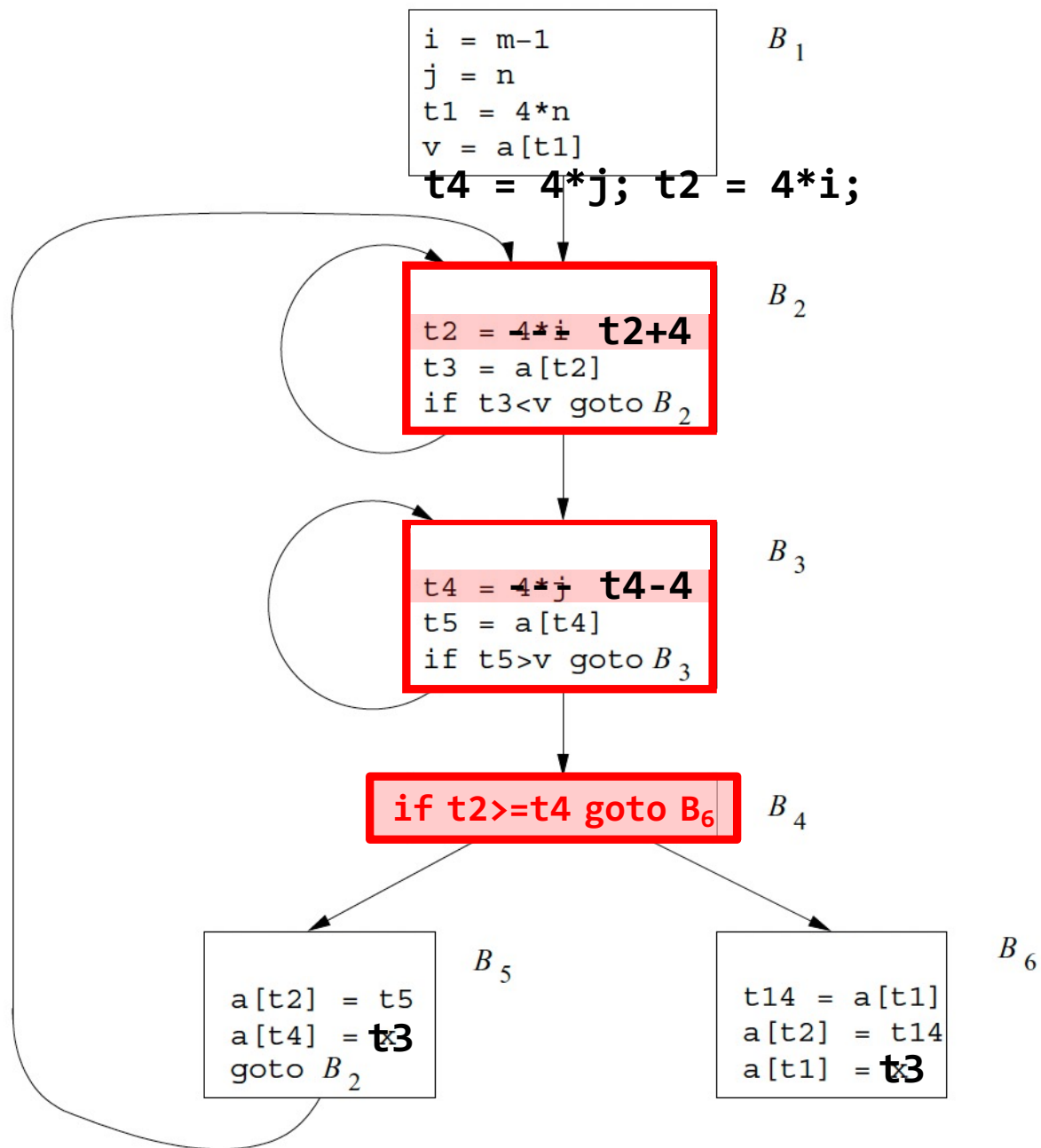
1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. **Strength reduction**
5. Induction-variable elimination



1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. **Strength reduction**
5. Induction-variable elimination



1. Local common subexpression
2. **Global common subexpression**
3. **Copy propagation**
4. **Strength reduction**
5. **Induction-variable elimination**
6.



1. Local common subexpression
2. **Global common subexpression**
3. Copy propagation
4. **Strength reduction**
5. **Induction-variable elimination**
6.

How?

PART I: DFA Scheme

Data Flow Analysis

- Describes how “data flow facts” transfer in a program, which often contains unbounded number of execution paths.

Data Flow Analysis

- Describes how “data flow facts” transfer in a program, which often contains unbounded number of execution paths.
- **Data flow fact** (or value) is an abstraction of all possible concrete program behaviors, *e.g.*,

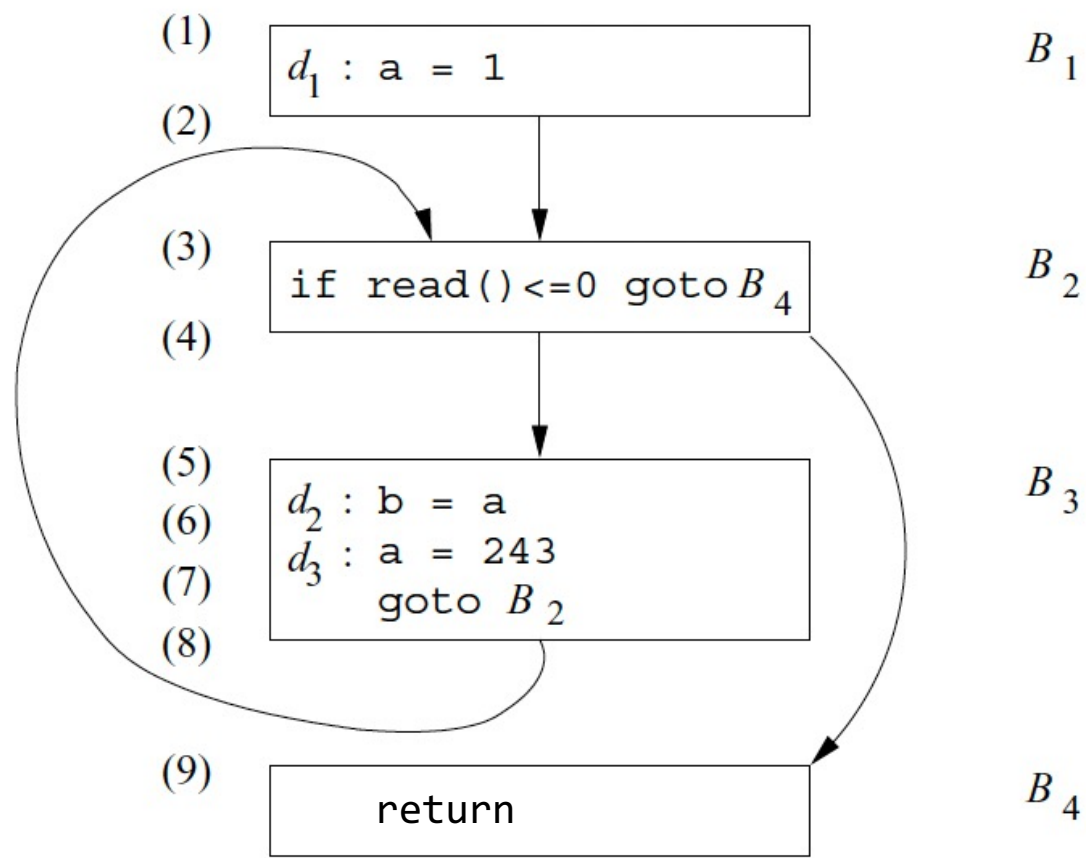
Data Flow Analysis

- Describes how “data flow facts” transfer in a program, which often contains unbounded number of execution paths.
- **Data flow fact** (or value) is an abstraction of all possible concrete program behaviors, *e.g.*,
 - A variable v may have one of the concrete values $\{ 1, 2, 4, 5, 6, \dots \}$
 - Impossible to enumerate all cases in a data flow analysis

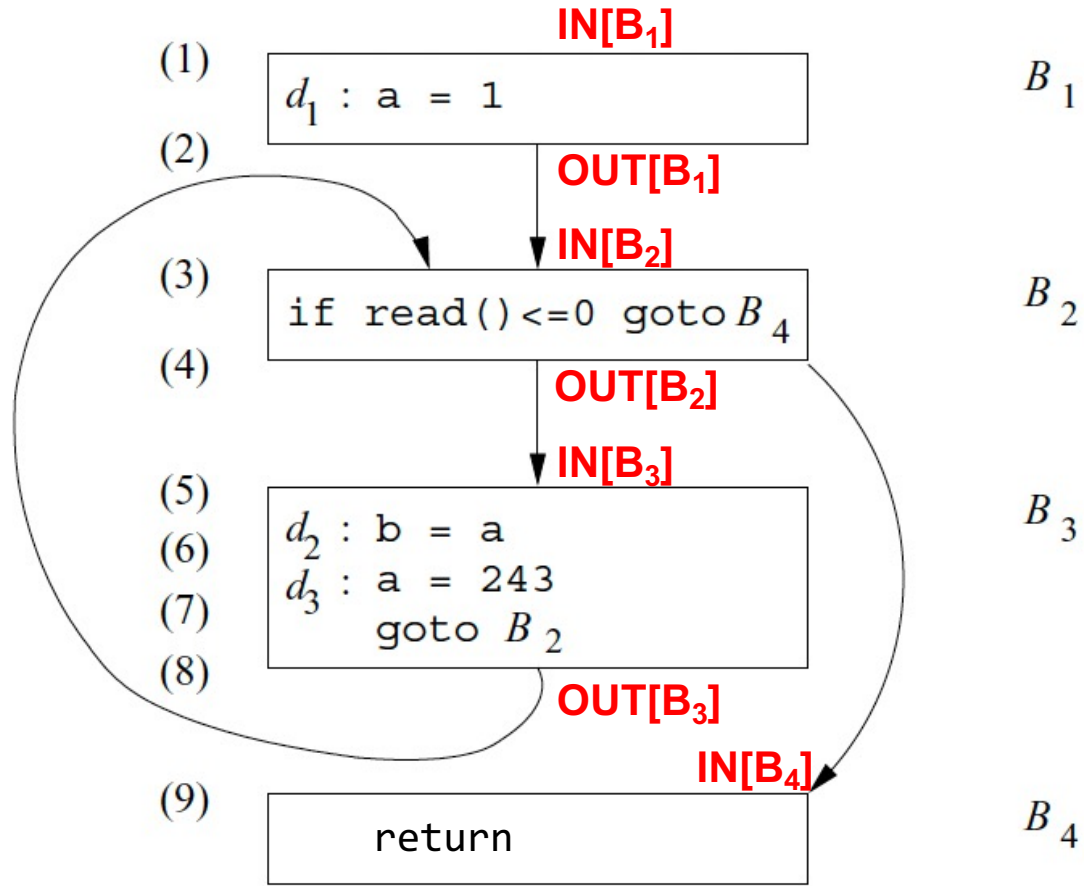
Data Flow Analysis

- Describes how “data flow facts” transfer in a program, which often contains unbounded number of execution paths.
- **Data flow fact** (or value) is an abstraction of all possible concrete program behaviors, *e.g.*,
 - A variable v may have one of the concrete values $\{ 1, 2, 4, 5, 6, \dots \}$
 - Impossible to enumerate all cases in a data flow analysis
 - **Abstraction:** $v = \text{NAC}$;
 - NAC is an abstract value of the concrete values $\{ 1, 2, 4, 5, 6, \dots \}$

Data Flow Analysis

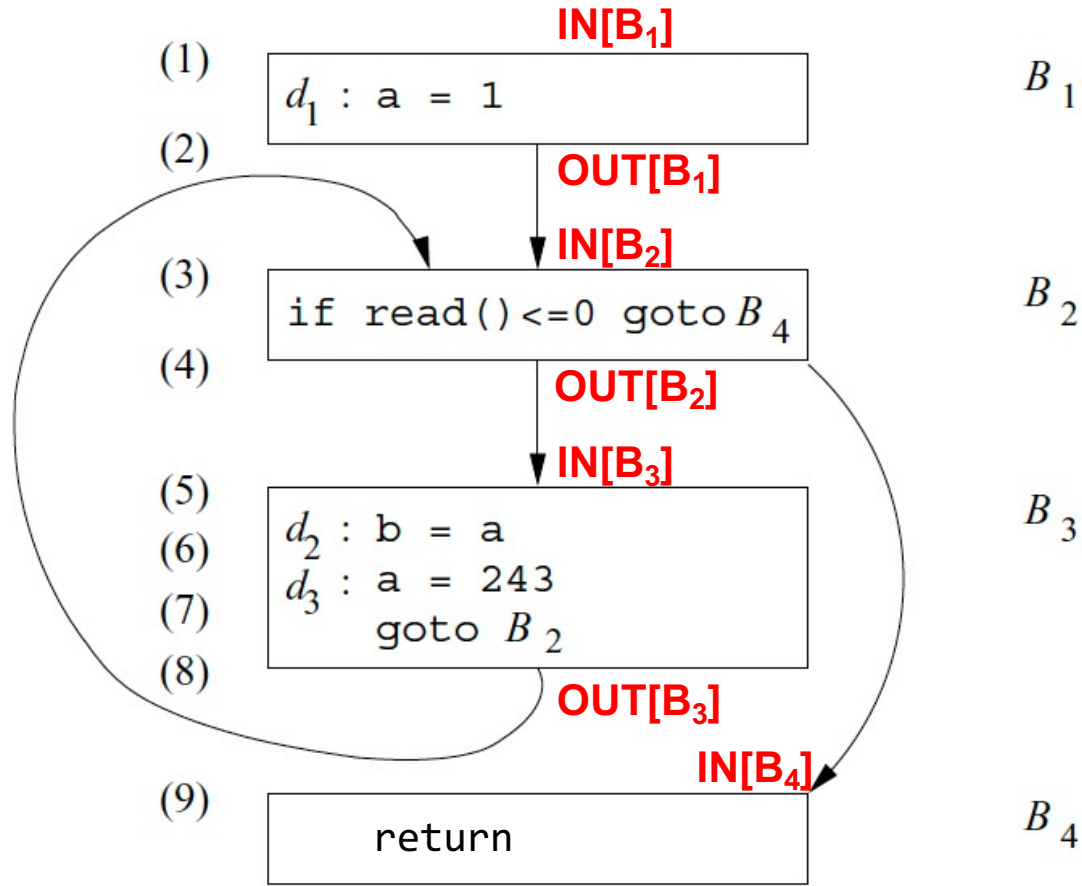


Data Flow Analysis



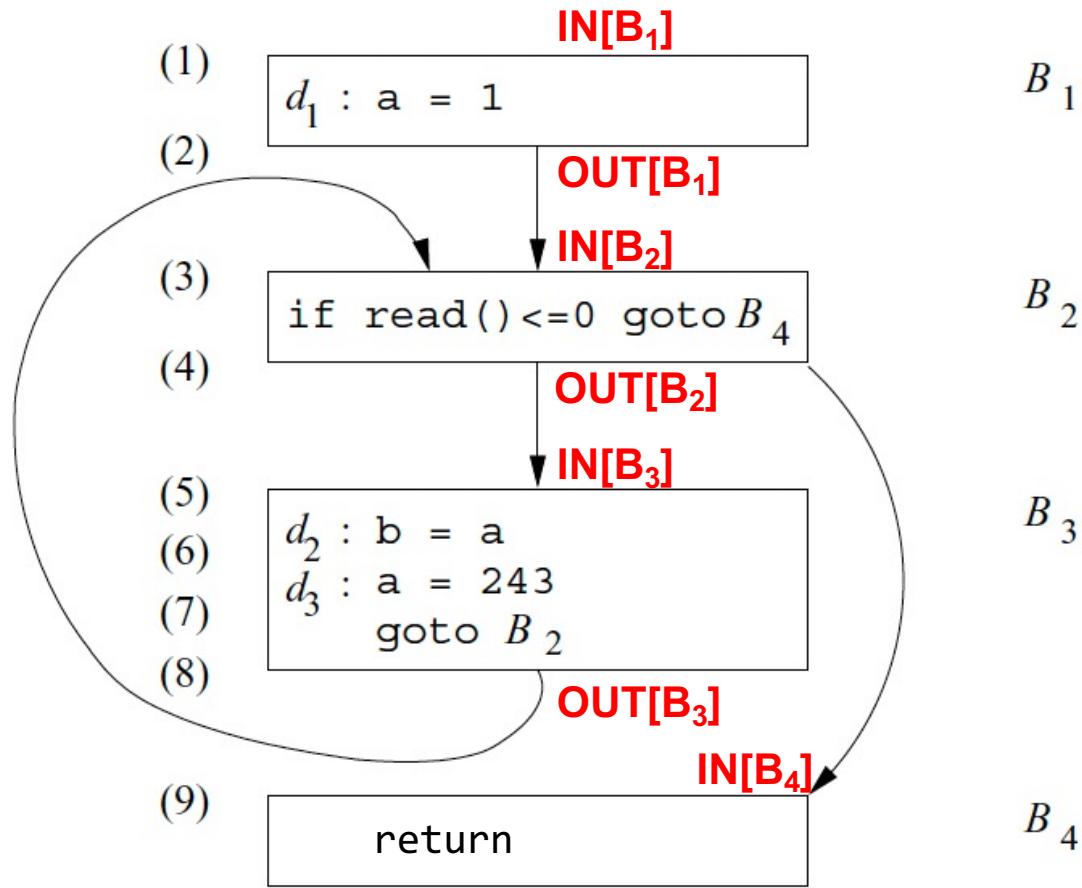
- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block

Data Flow Analysis



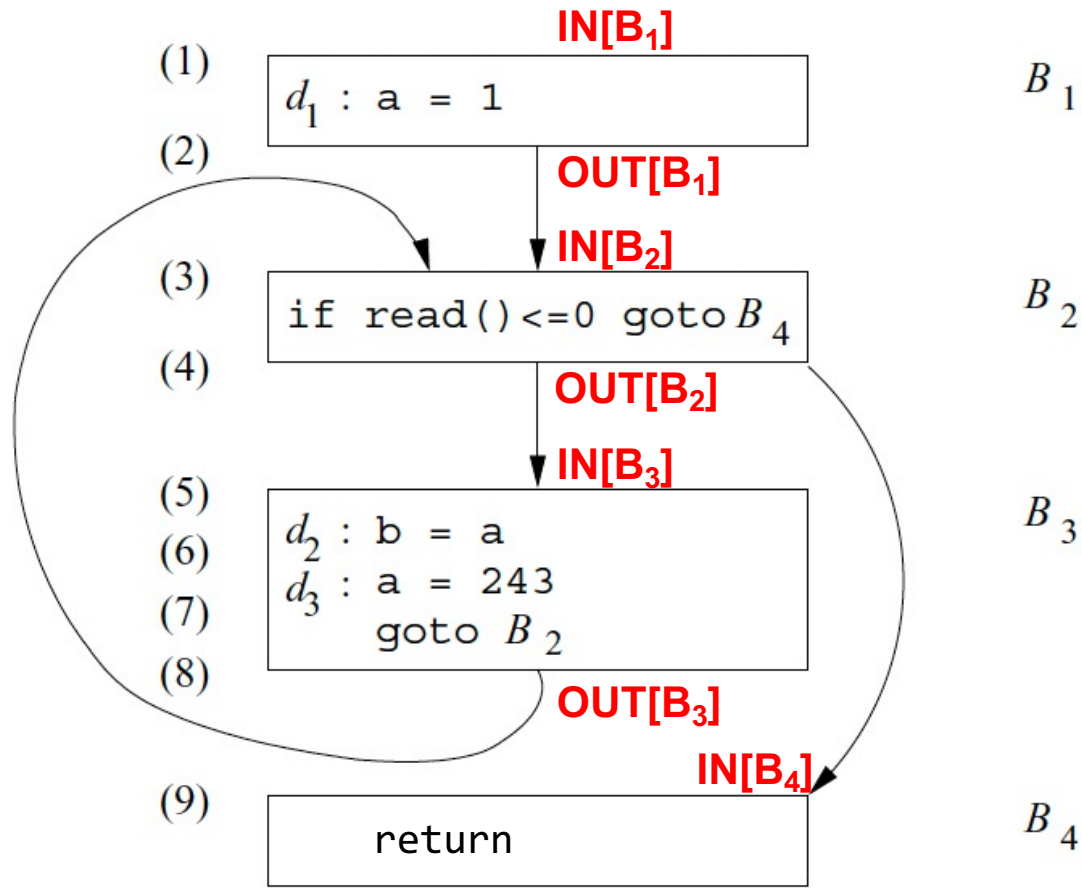
- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block
- Constraints
 - $OUT[B] = f_B(IN[B])$

Data Flow Analysis



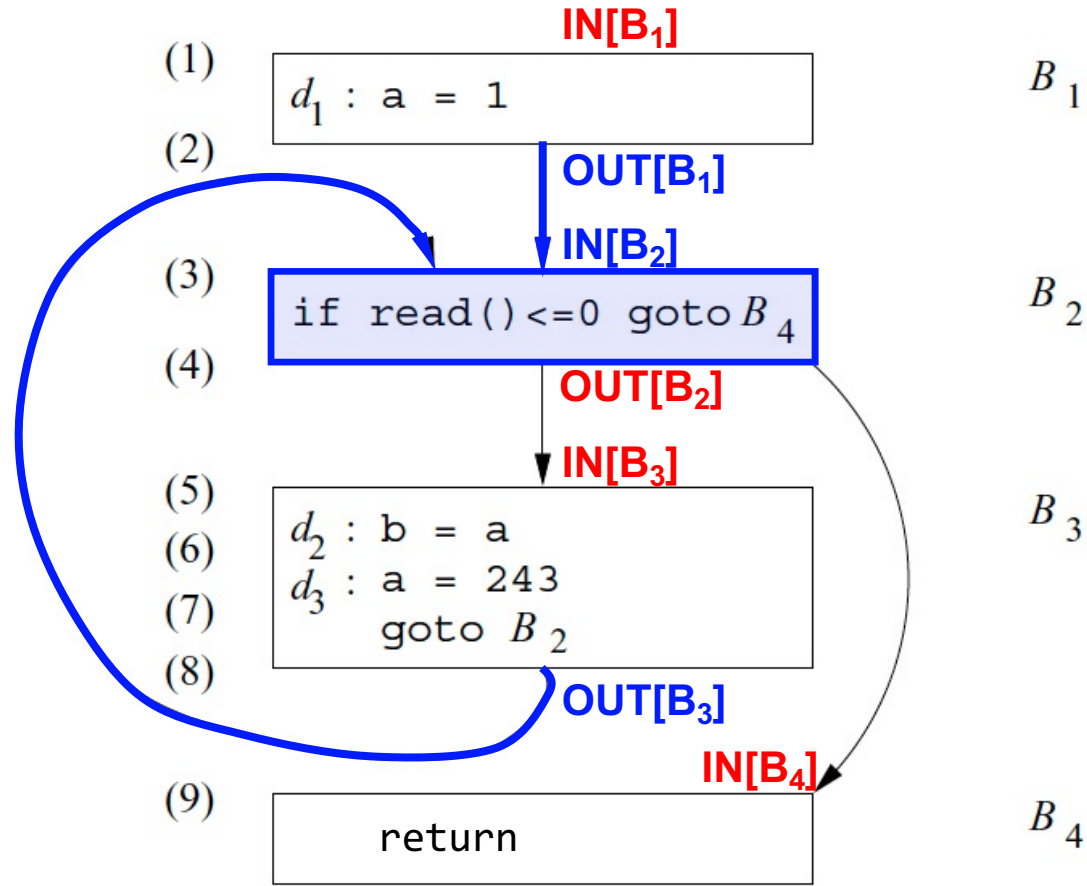
- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block
- Constraints
 - $OUT[B] = f_B(IN[B])$
 - B contains n statements $\Rightarrow$
 - $f_B = f_n \circ f_{n-1} \circ \dots \circ f_3 \circ f_2 \circ f_1$

Data Flow Analysis



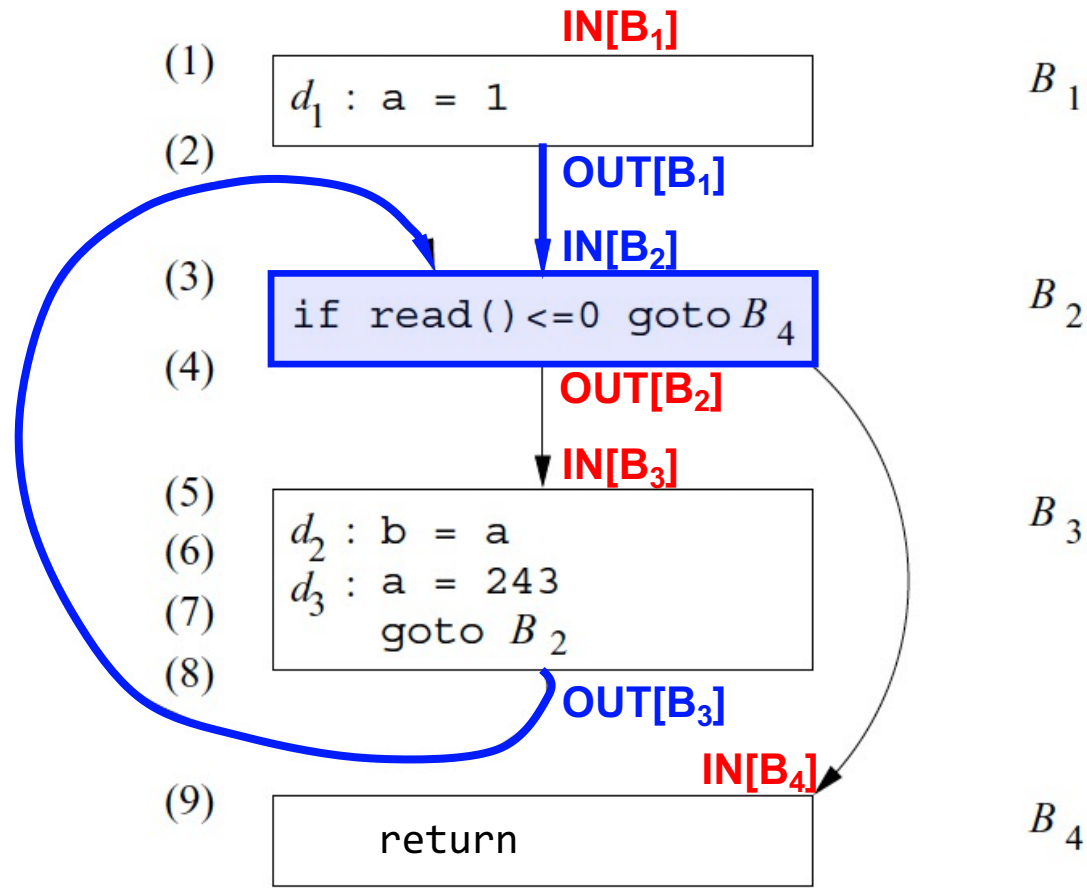
- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block
- Constraints
 - $OUT[B] = f_B(IN[B])$
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]$

Data Flow Analysis



- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block
- Constraints
 - $OUT[B] = f_B(IN[B])$
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]$

Data Flow Analysis



- $IN[B]/OUT[B]$
 - Set of data flow facts before and after a block
- Constraints
 - $OUT[B] = \mathbf{f}_B(IN[B])$
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]$
- $\Rightarrow$ Data Flow Equations

Data Flow Analysis

- To perform data flow analysis, define
 - transfer function f for each statement/block
 - merge function $\wedge$ at the joint point of multiple paths

Data Flow Analysis

- To perform data flow analysis, define
 - transfer function f for each statement/block
 - merge function $\wedge$ at the joint point of multiple paths
 - the **initial** set of data flow facts for each block

Data Flow Analysis

- To perform data flow analysis, define
 - transfer function f for each statement/block
 - merge function $\wedge$ at the joint point of multiple paths
 - the **initial** set of data flow facts for each block
- Data flow analysis produces
 - $IN[B]/OUT[B]$ that include data flow facts at a program point

Data Flow Analysis

- To perform data flow analysis, define
 - transfer function f for each statement/block
 - merge function $\wedge$ at the joint point of multiple paths
 - the **initial** set of data flow facts for each block
- Data flow analysis produces
 - $IN[B]/OUT[B]$ that include data flow facts at a program point
 - The resulting data flow facts are always true during program execution

The Worklist Algorithm

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
- **EndWhile**

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - **// do something over B**
- **EndWhile**

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - **// do something over B**
 - **// add some blocks back to the worklist if necessary**
- **EndWhile**

The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
 - **// add some blocks back to the worklist if necessary**
- **EndWhile**

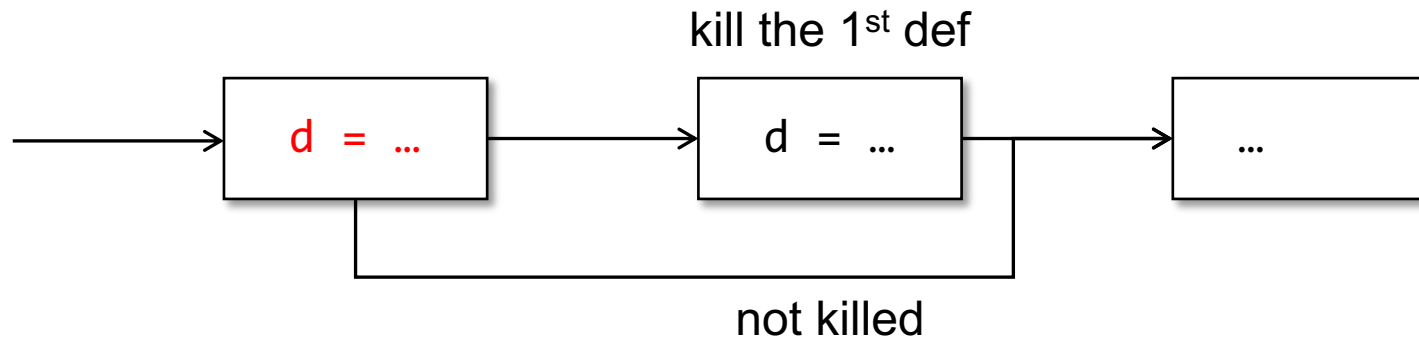
The Worklist Algorithm

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
 - **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Reaching Definitions

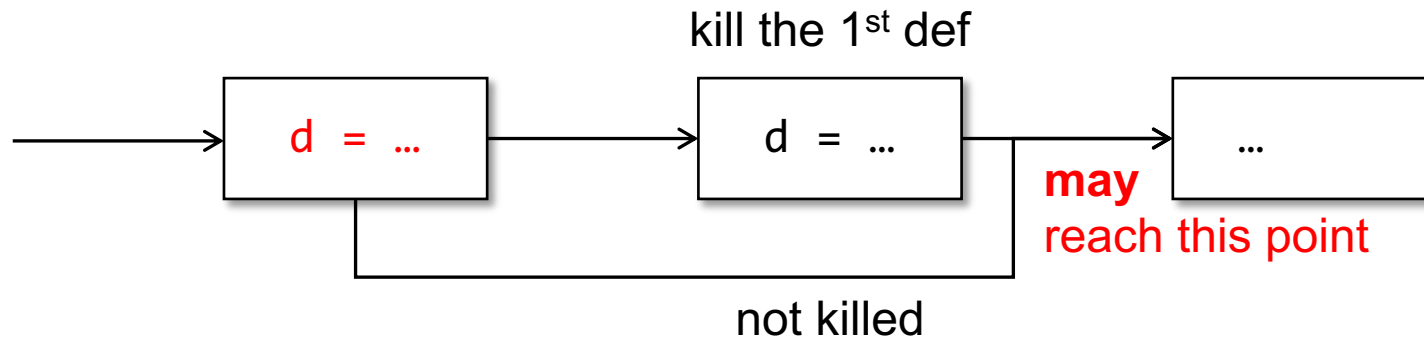
Reaching Definitions

- A definition d **may** reach a program point, if **there is a path from d to the program point** such that **d is not killed along the path**.



Reaching Definitions

- A definition d **may** reach a program point, if **there is a path from d to the program point** such that **d is not killed along the path**.



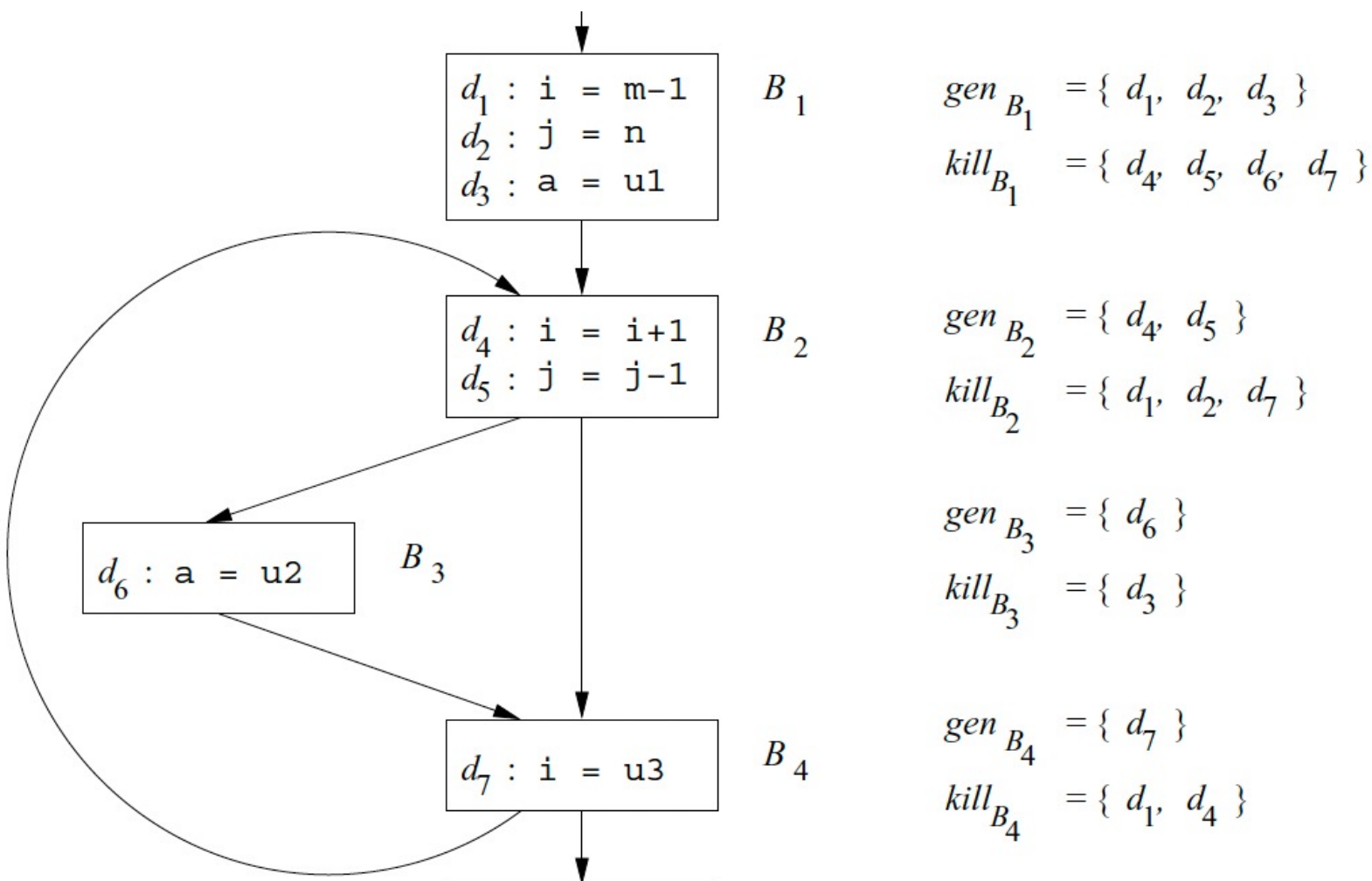
Reaching Definitions

- A definition d may reach a program point, if there is a path from d to the program point such that d is not killed along the path.
- $OUT[B] = f_B(IN[B])$
- $IN[B] = \bigwedge_{P \in preds(B)} OUT[P]$

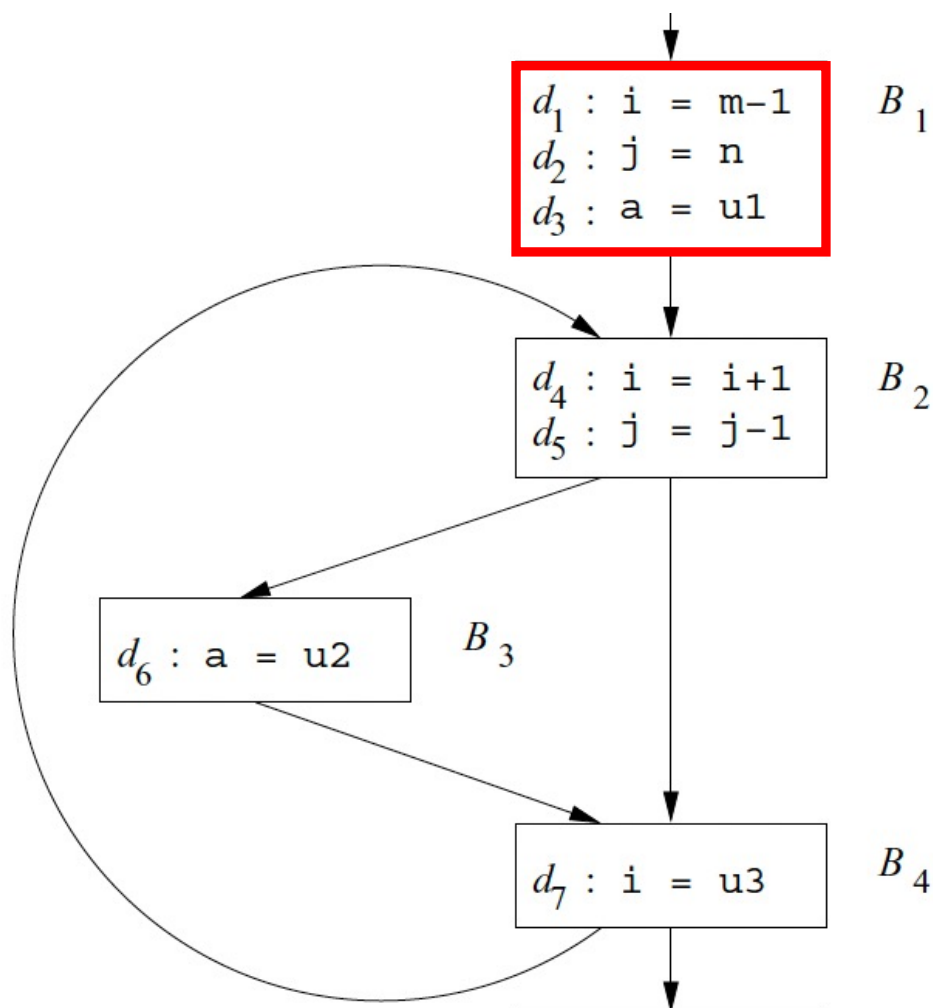
Reaching Definitions

- A definition d may reach a program point, if there is a path from d to the program point such that d is not killed along the path.
- $OUT[B] = f_B(IN[B]) = gen_B \cup (IN[B] - kill_B)$
- $IN[B] = \bigwedge_{P \in preds(B)} OUT[P] = \bigcup_{P \in preds(B)} OUT[P]$

Reaching Definitions



Reaching Definitions



$$gen_{B_1} = \{ d_1, d_2, d_3 \}$$

$$kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$$

$$gen_{B_2} = \{ d_4, d_5 \}$$

$$kill_{B_2} = \{ d_1, d_2, d_7 \}$$

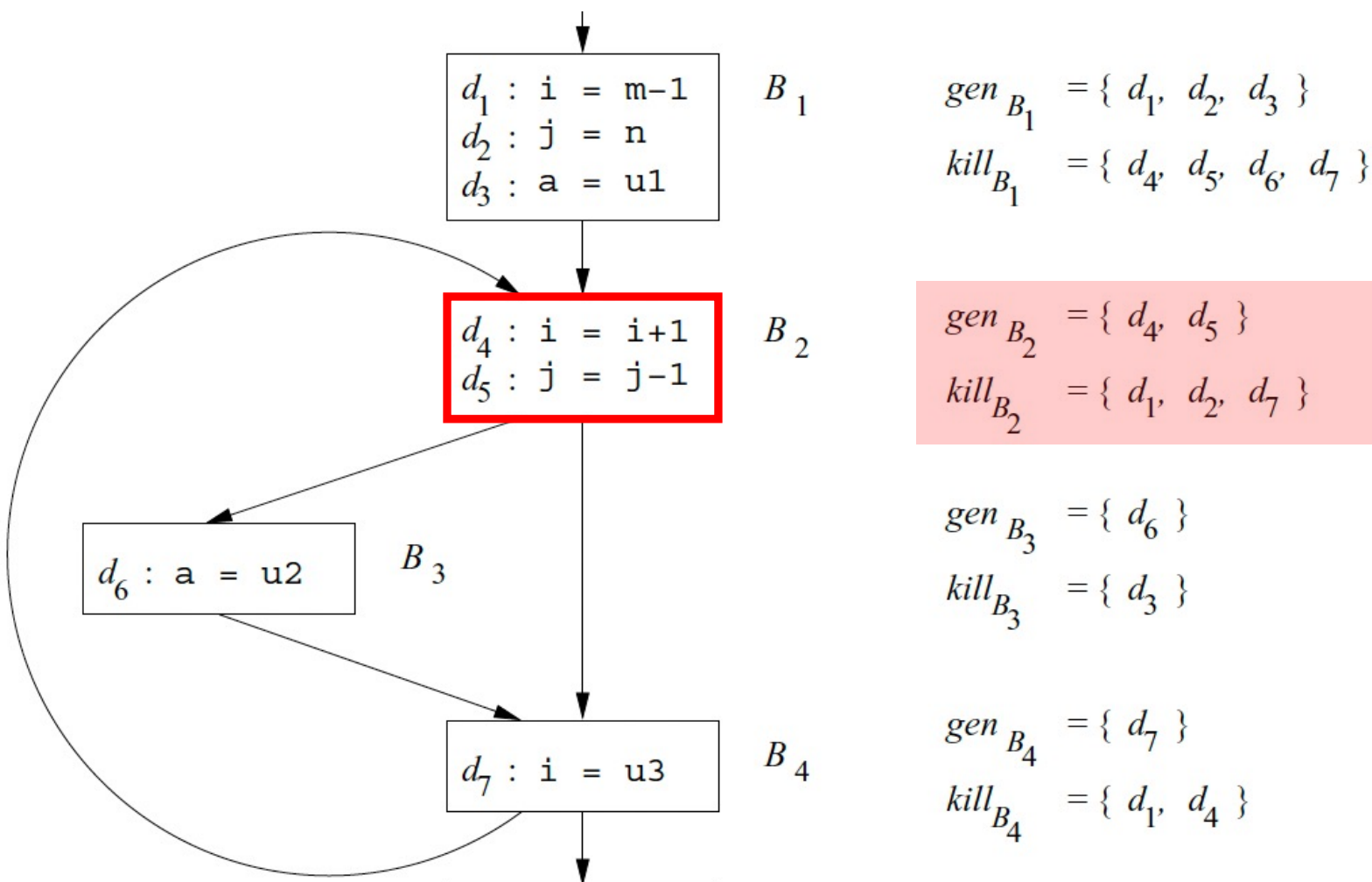
$$gen_{B_3} = \{ d_6 \}$$

$$kill_{B_3} = \{ d_3 \}$$

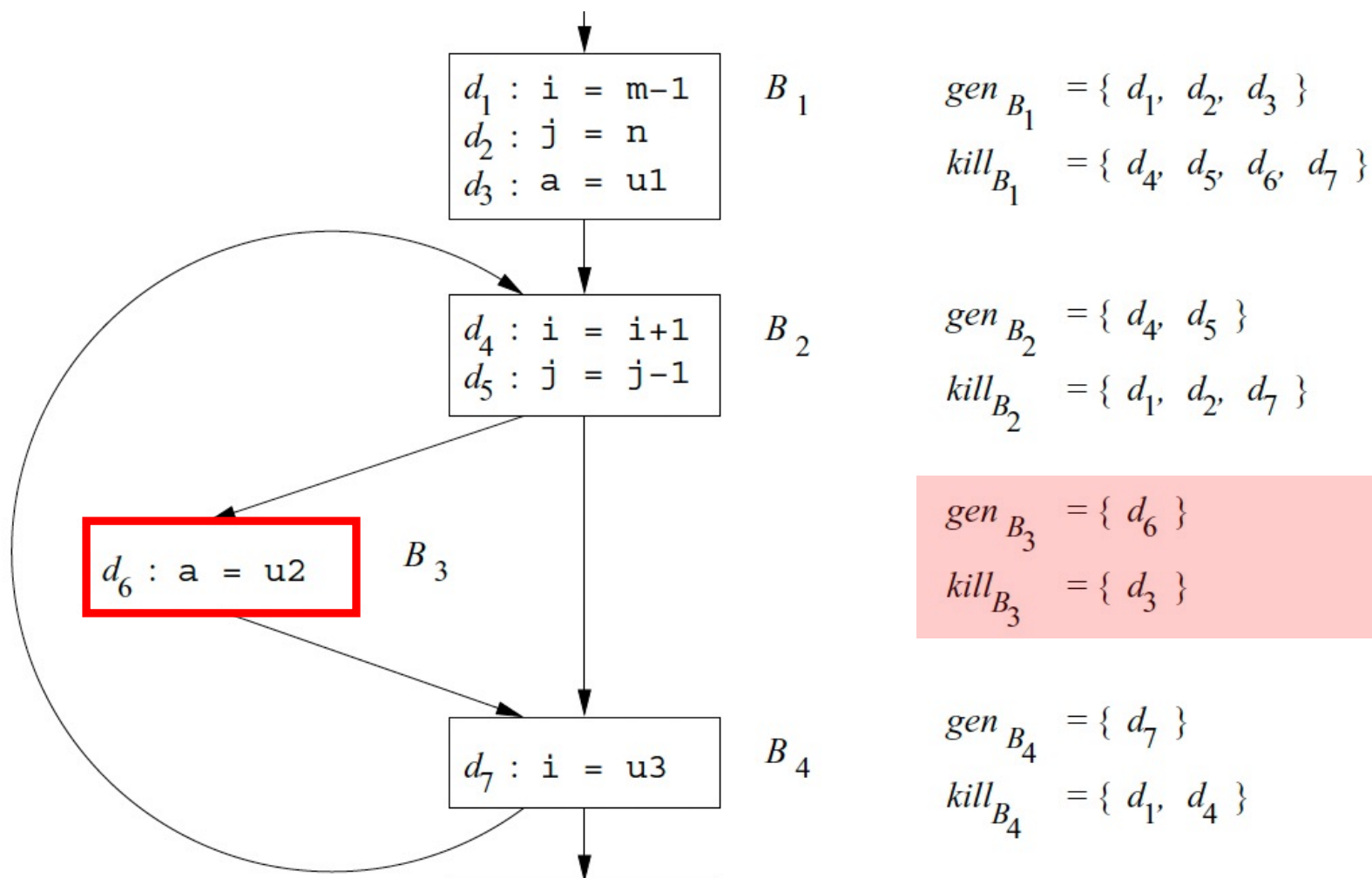
$$gen_{B_4} = \{ d_7 \}$$

$$kill_{B_4} = \{ d_1, d_4 \}$$

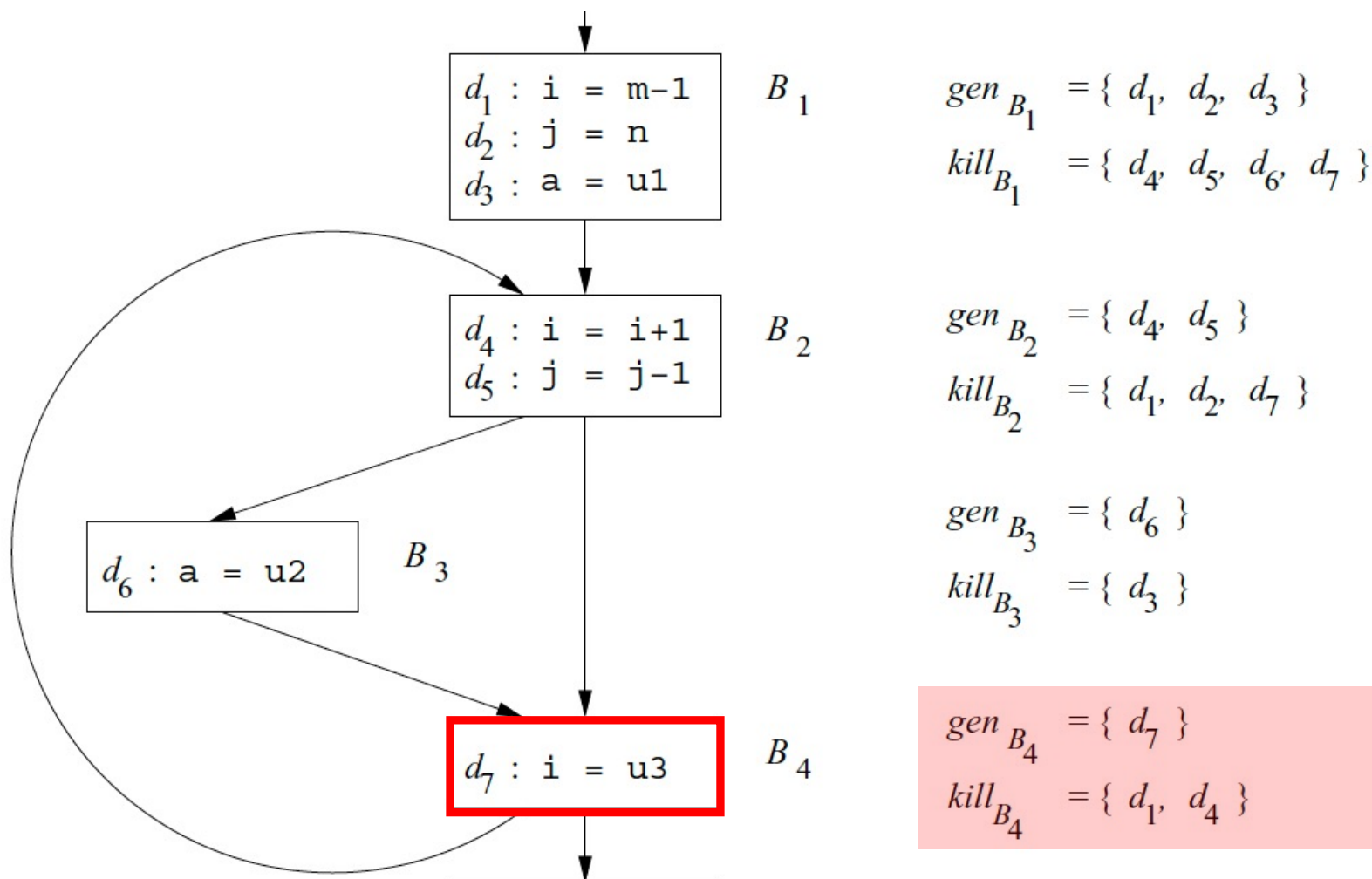
Reaching Definitions



Reaching Definitions



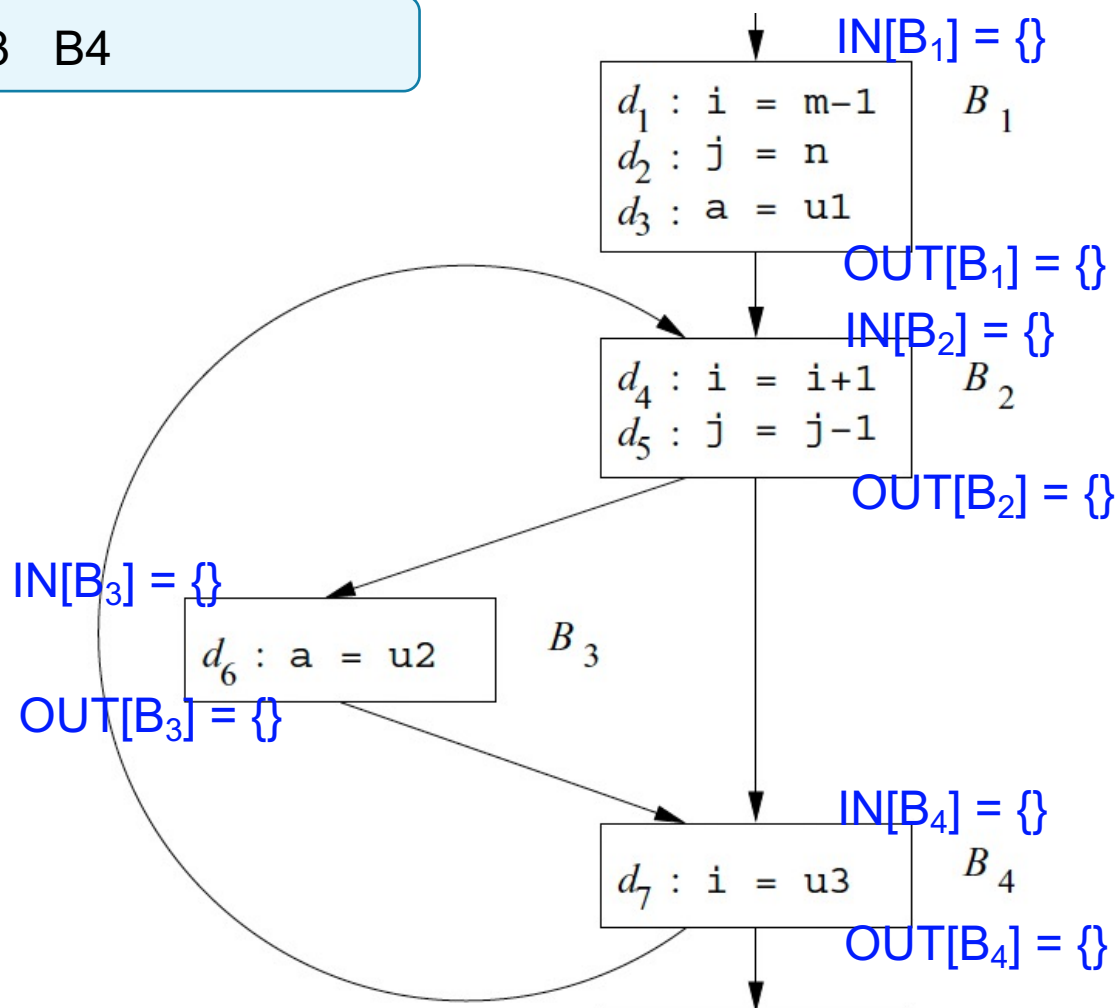
Reaching Definitions



Reaching Definitions

B1 B2 B3 B4

Worklist



$$gen_{B_1} = \{ d_1, d_2, d_3 \}$$

$$kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$$

$$gen_{B_2} = \{ d_4, d_5 \}$$

$$kill_{B_2} = \{ d_1, d_2, d_7 \}$$

$$gen_{B_3} = \{ d_6 \}$$

$$kill_{B_3} = \{ d_3 \}$$

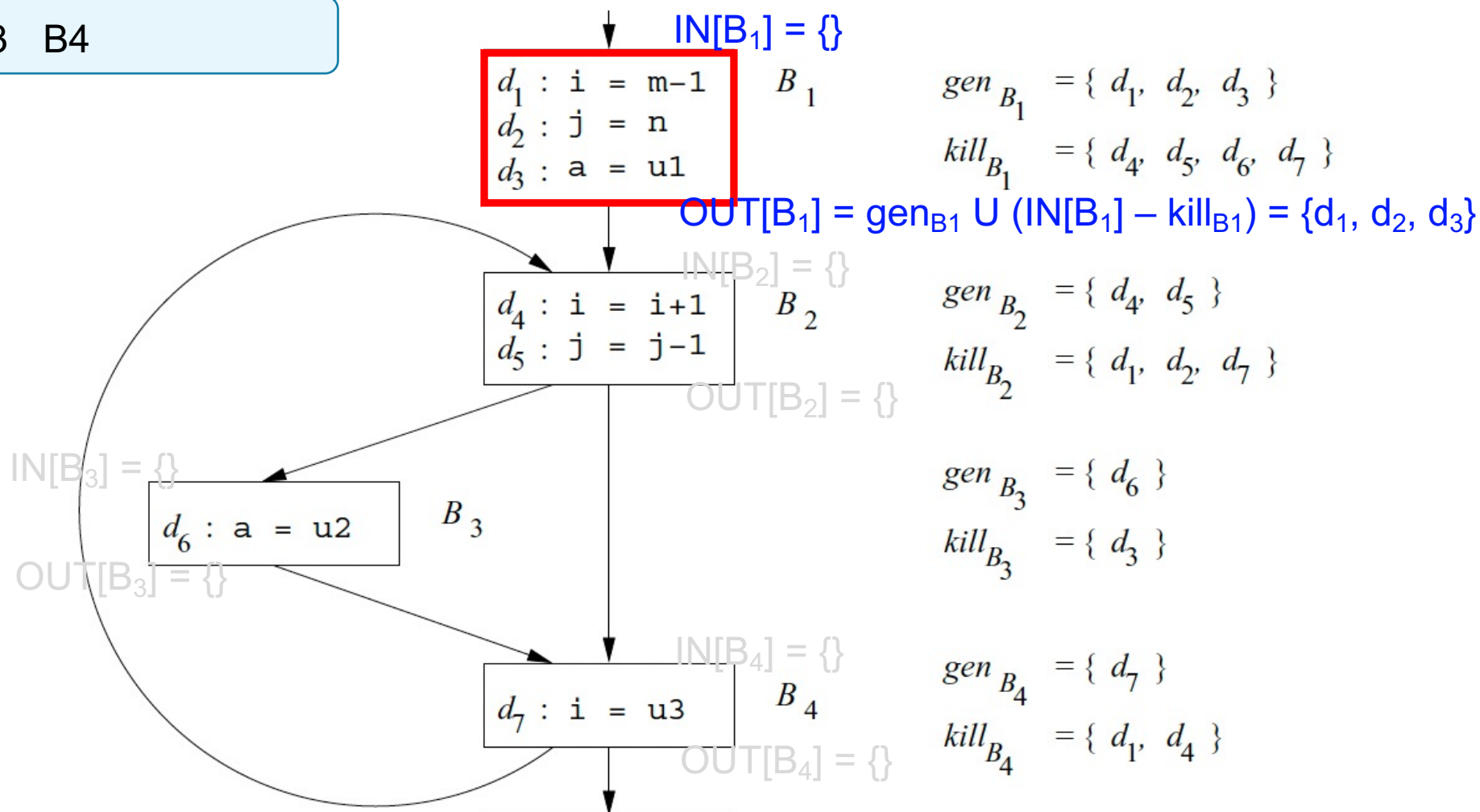
$$gen_{B_4} = \{ d_7 \}$$

$$kill_{B_4} = \{ d_1, d_4 \}$$

Reaching Definitions

B1 B2 B3 B4

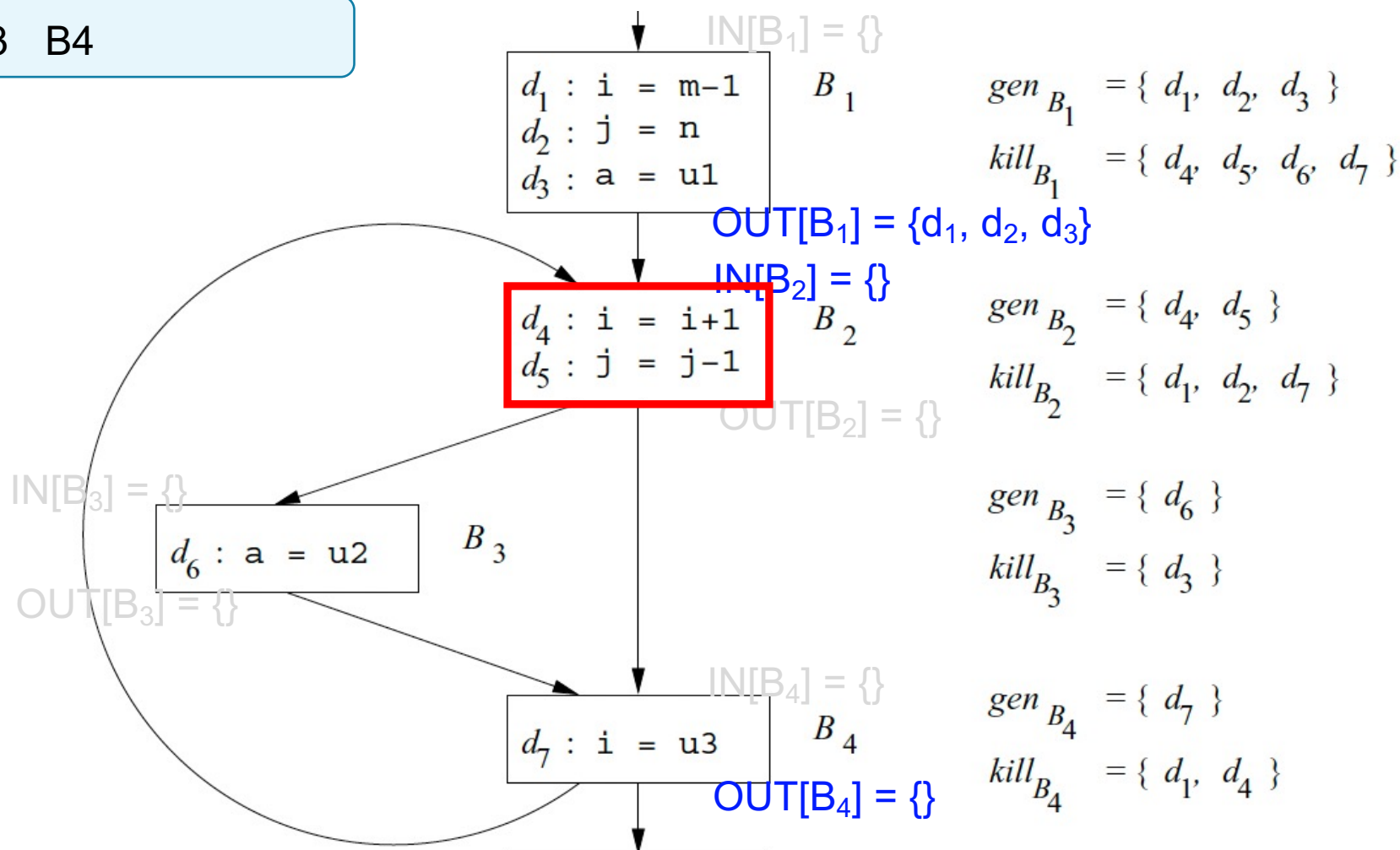
Worklist



Reaching Definitions

B1 B2 B3 B4

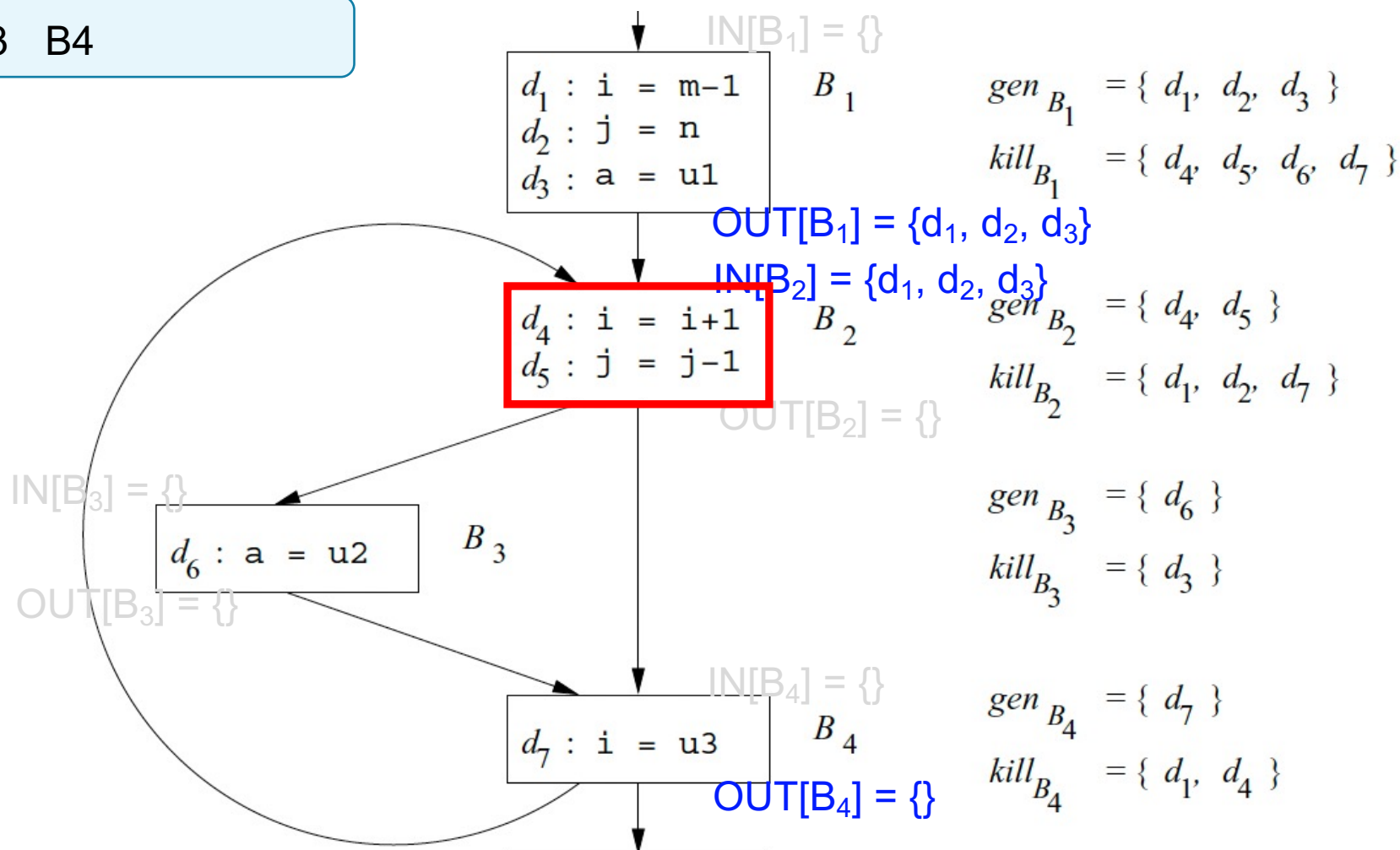
Worklist



Reaching Definitions

B1 B2 B3 B4

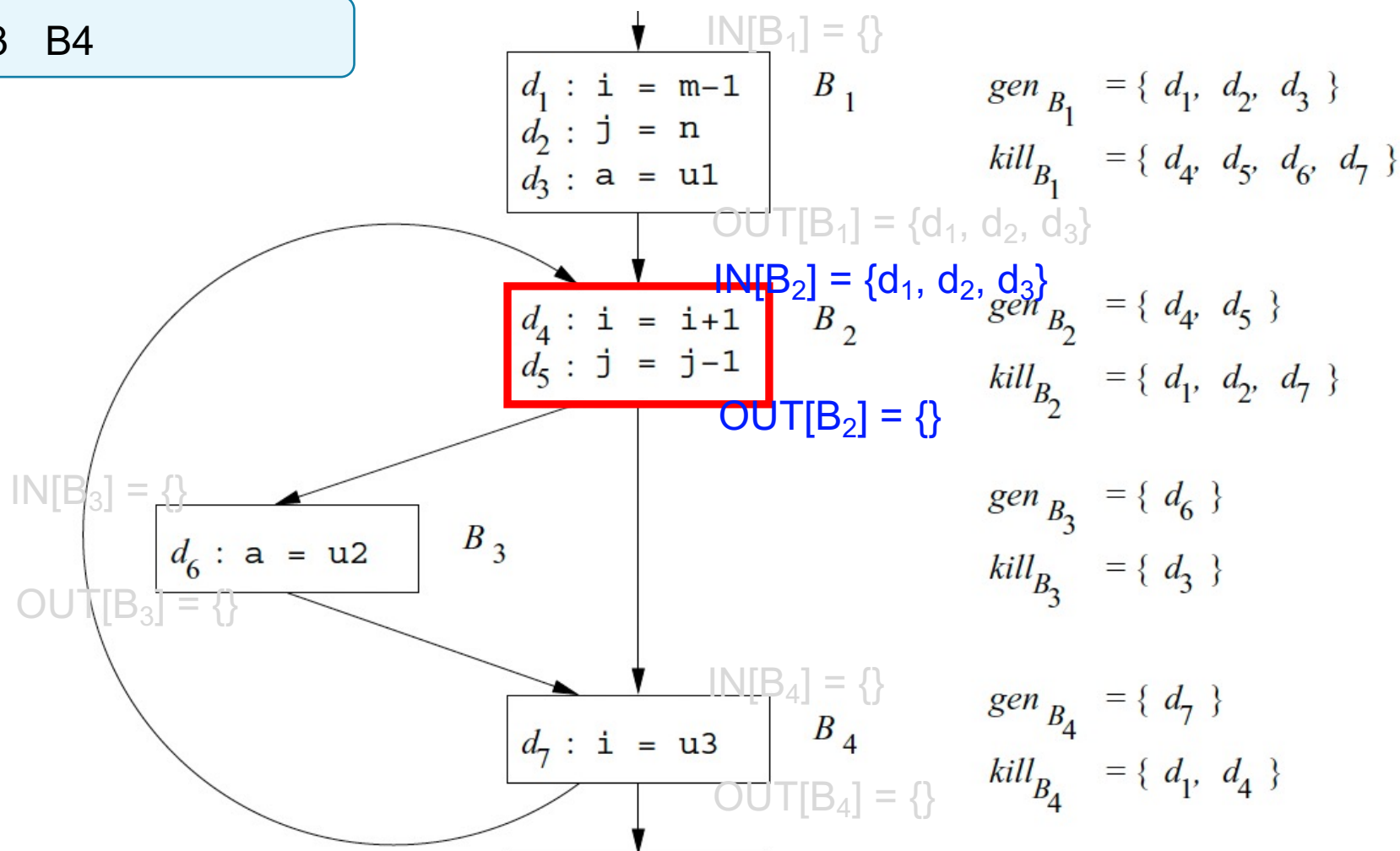
Worklist



Reaching Definitions

B1 B2 B3 B4

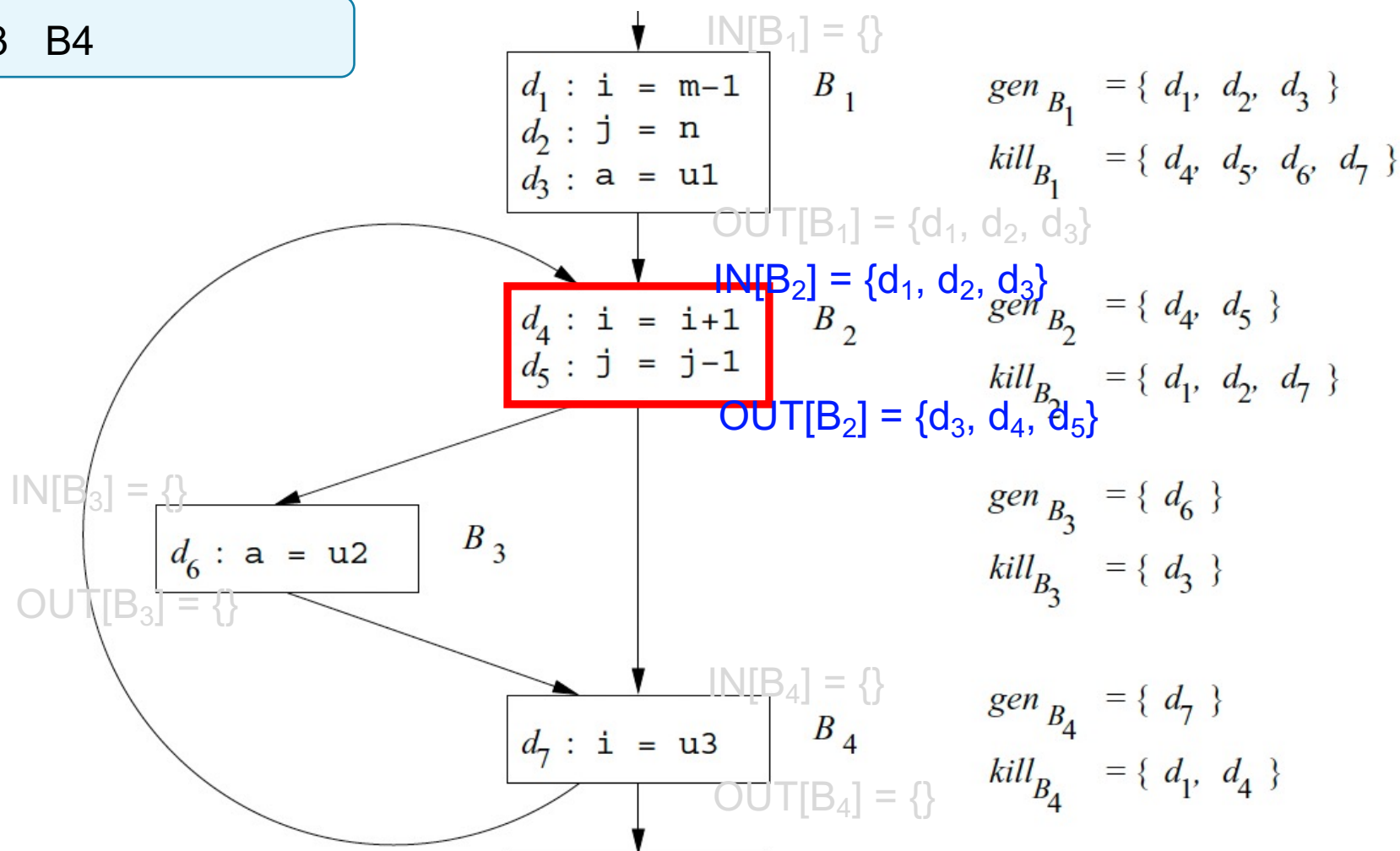
Worklist



Reaching Definitions

B1 B2 B3 B4

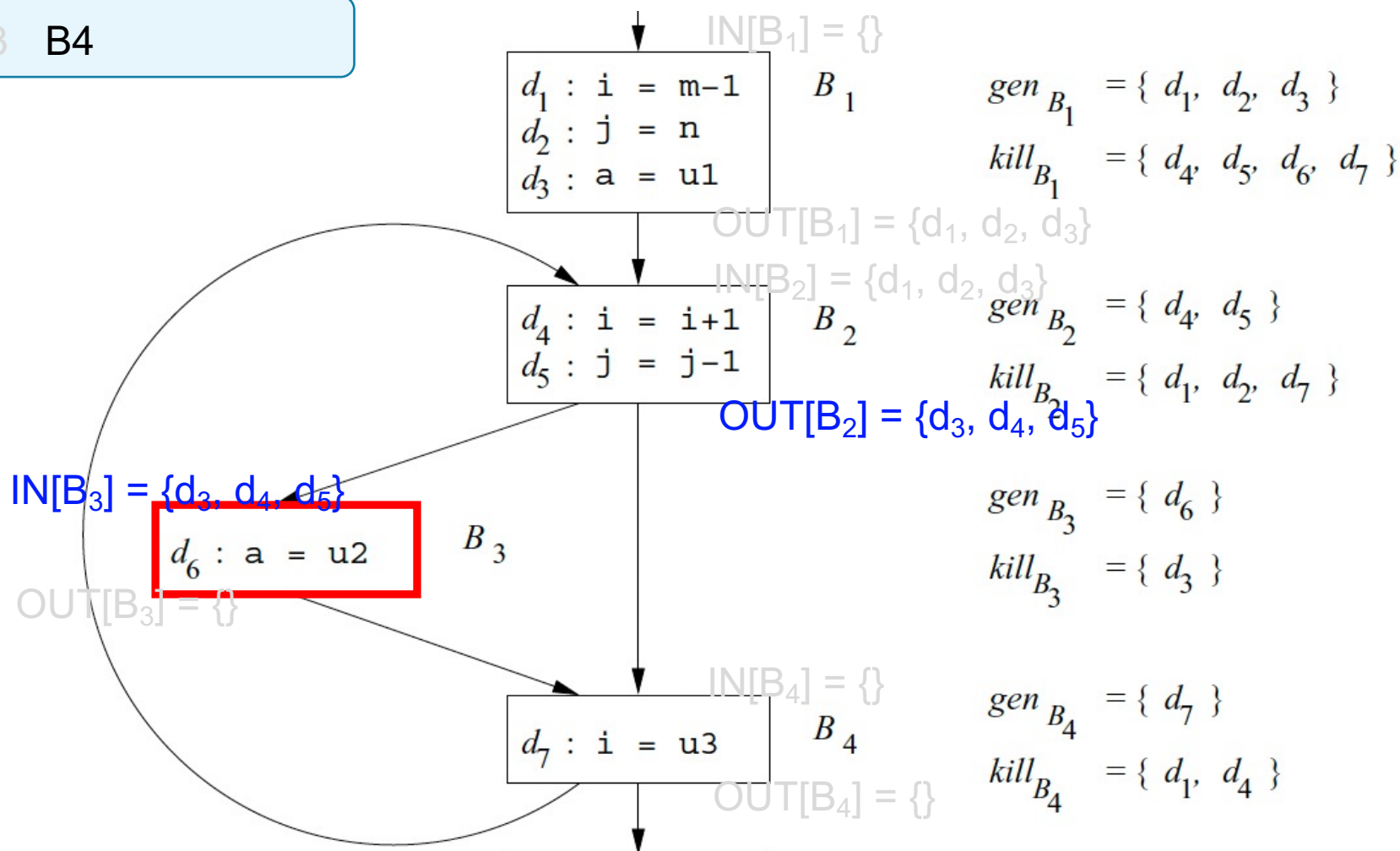
Worklist



Reaching Definitions

B1 B2 B3 B4

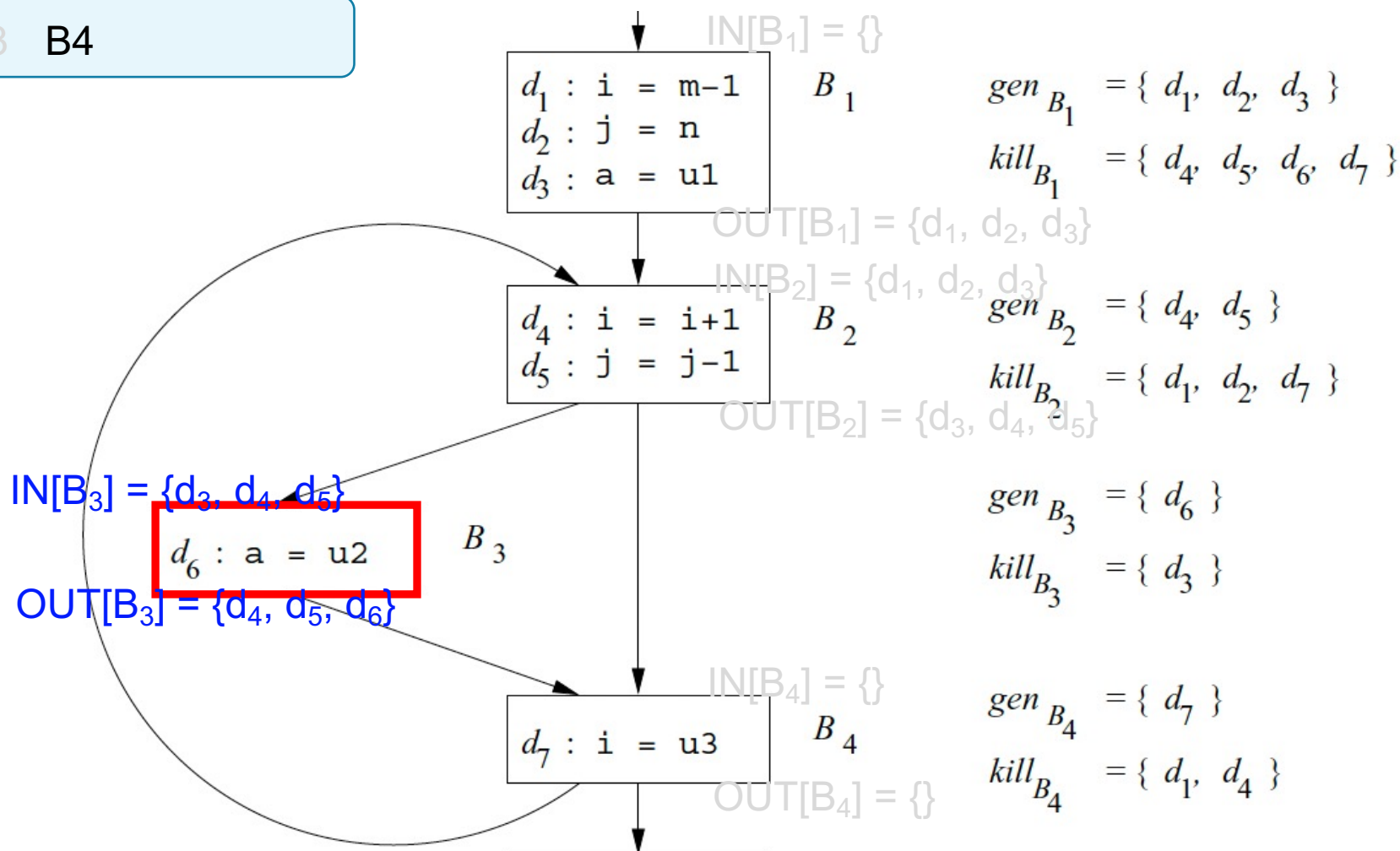
Worklist



Reaching Definitions

B1 B2 B3 B4

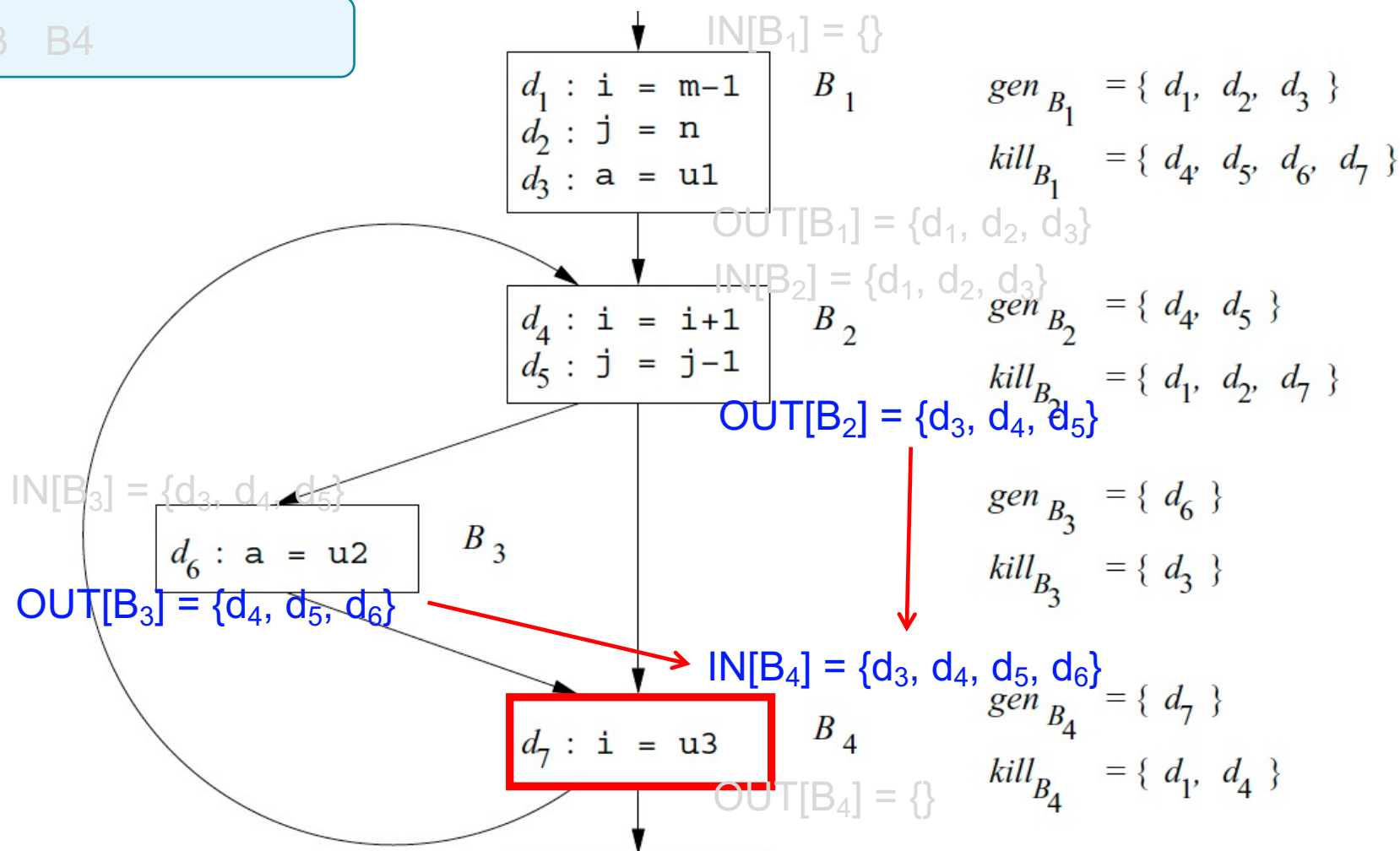
Worklist



Reaching Definitions

B1 B2 B3 B4

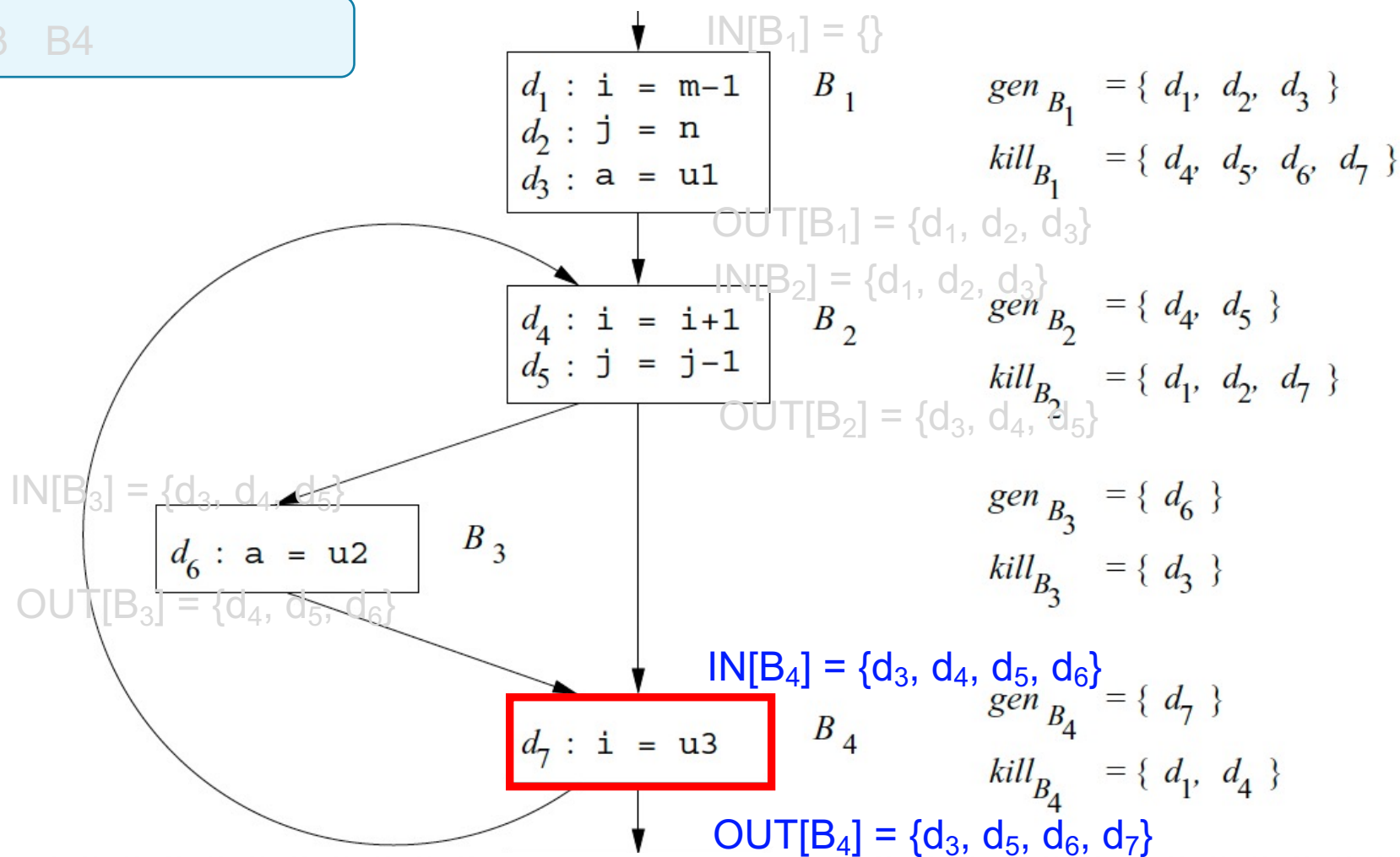
Worklist



Reaching Definitions

B1 B2 B3 B4

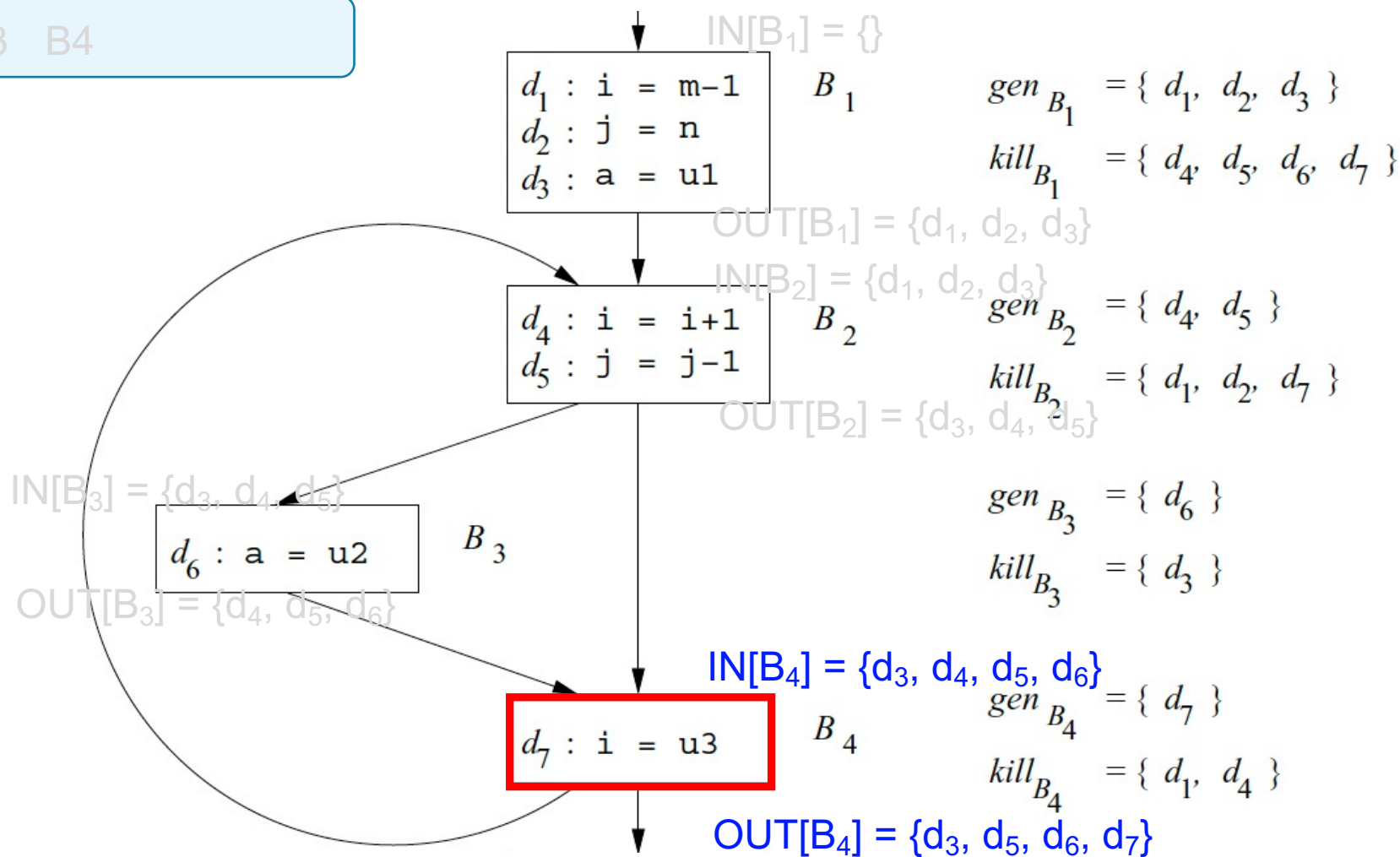
Worklist



Reaching Definitions

B1 B2 B3 B4

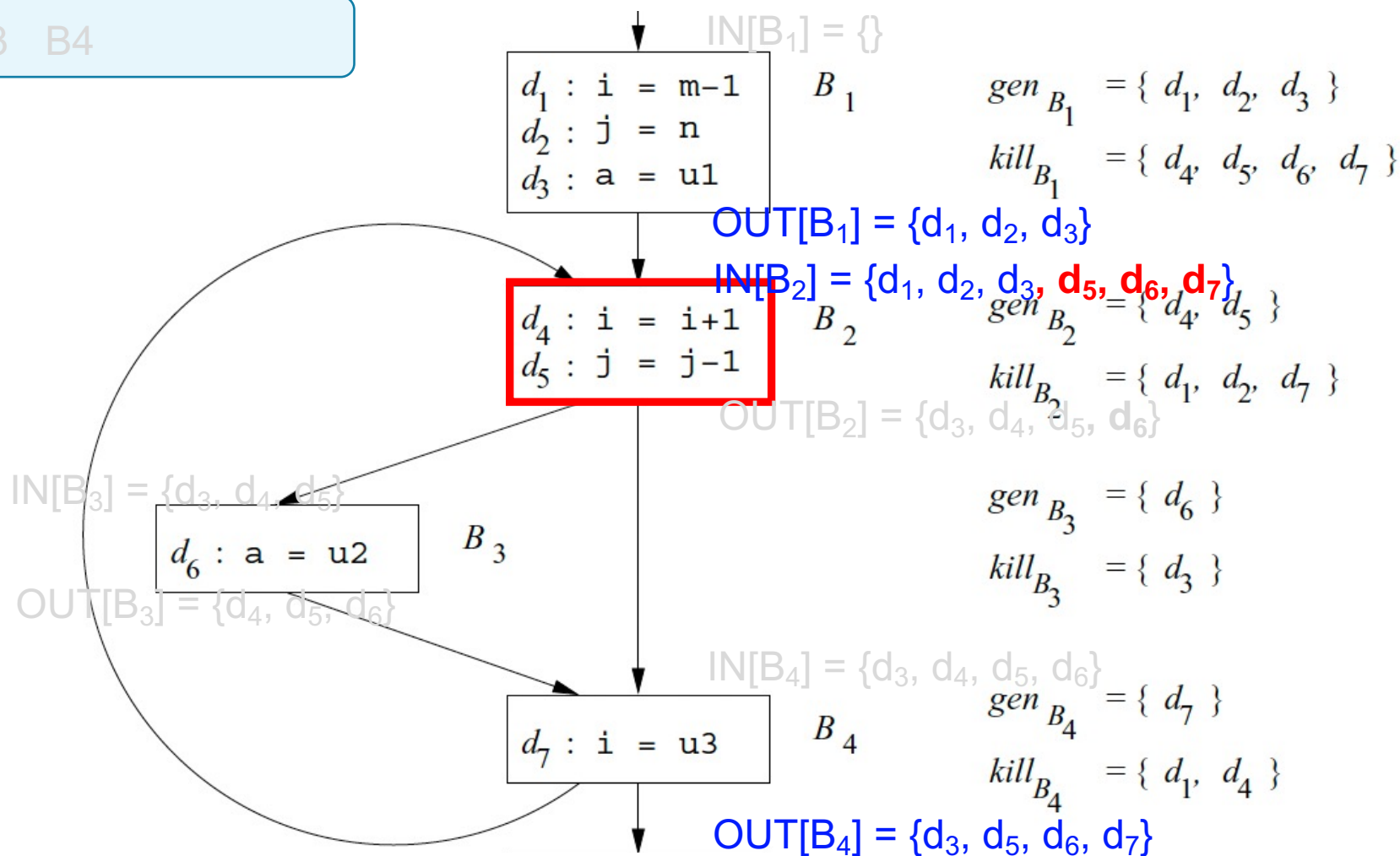
Worklist



Reaching Definitions

B1 B2 B3 B4

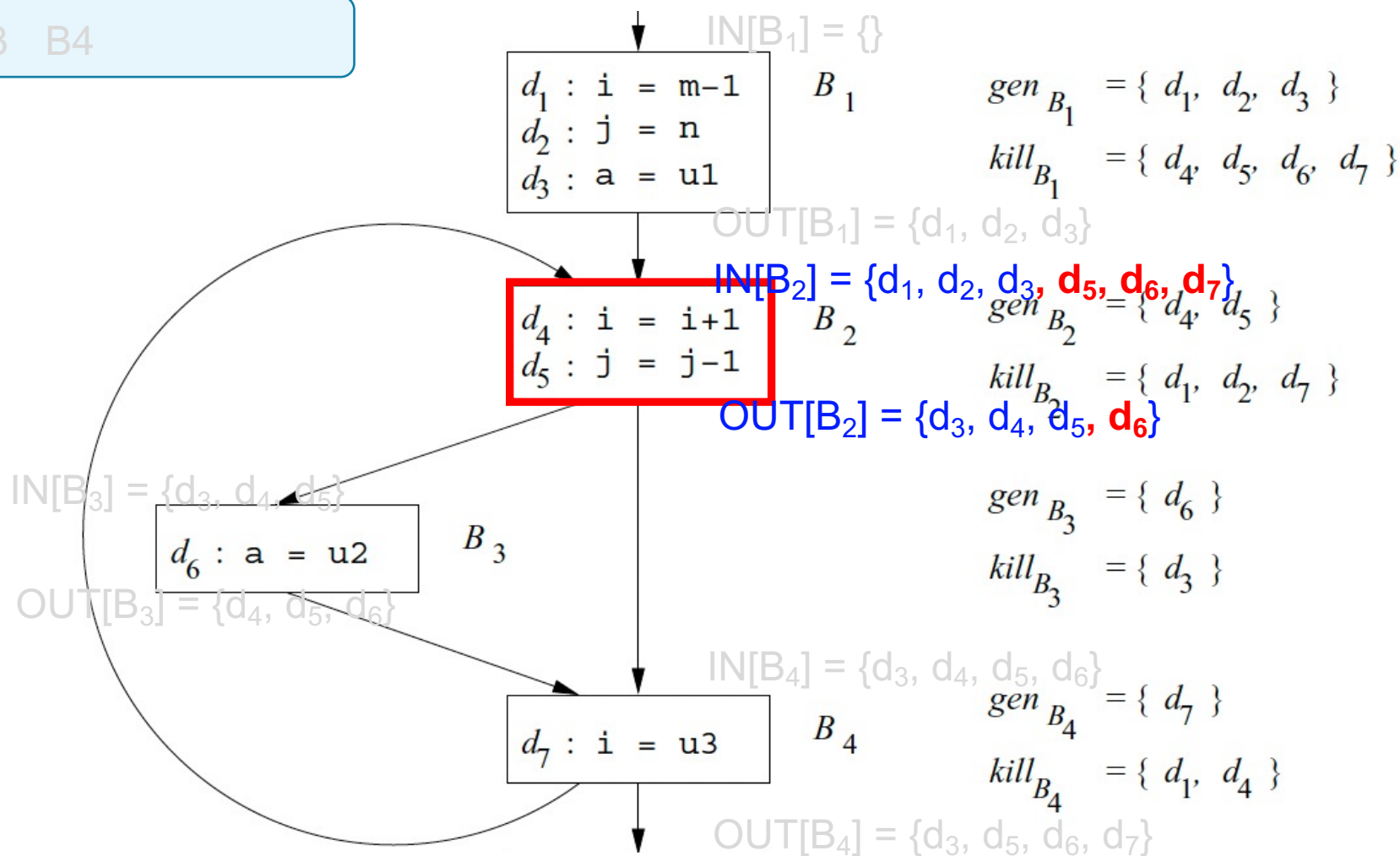
Worklist



Reaching Definitions

B1 B2 B3 B4

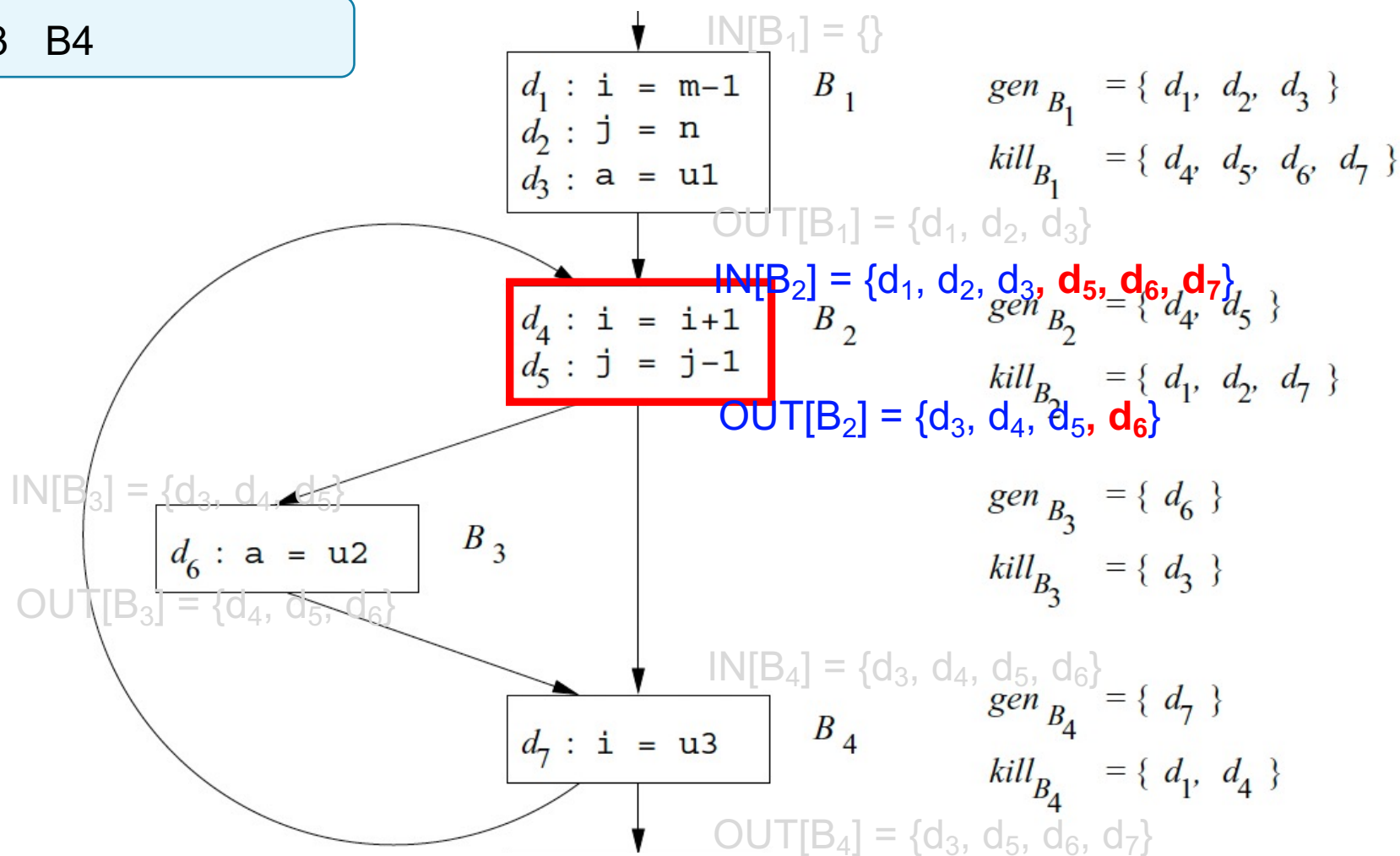
Worklist



Reaching Definitions

B1 B2 B3 B4

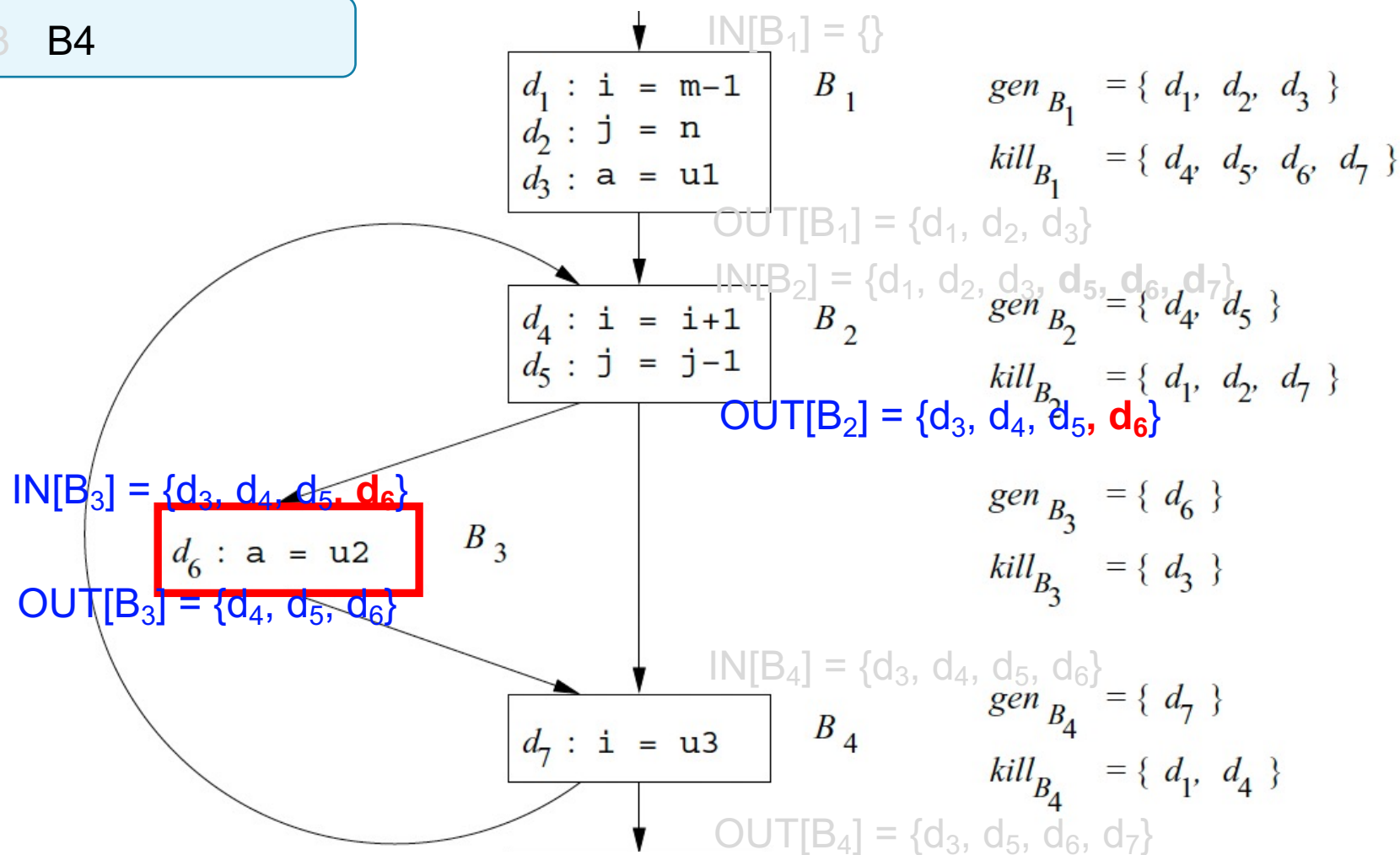
Worklist



Reaching Definitions

B1 B2 B3 B4

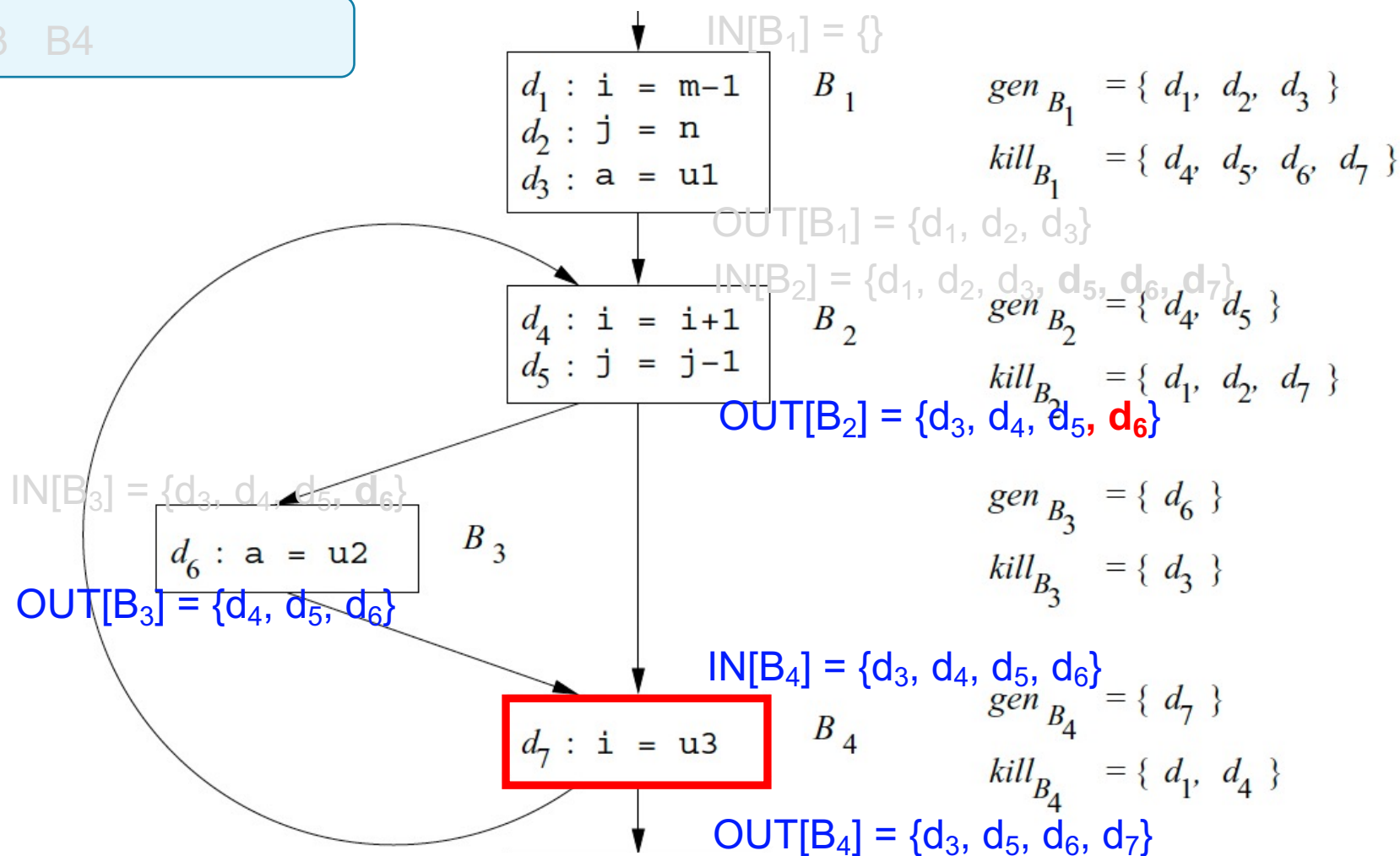
Worklist



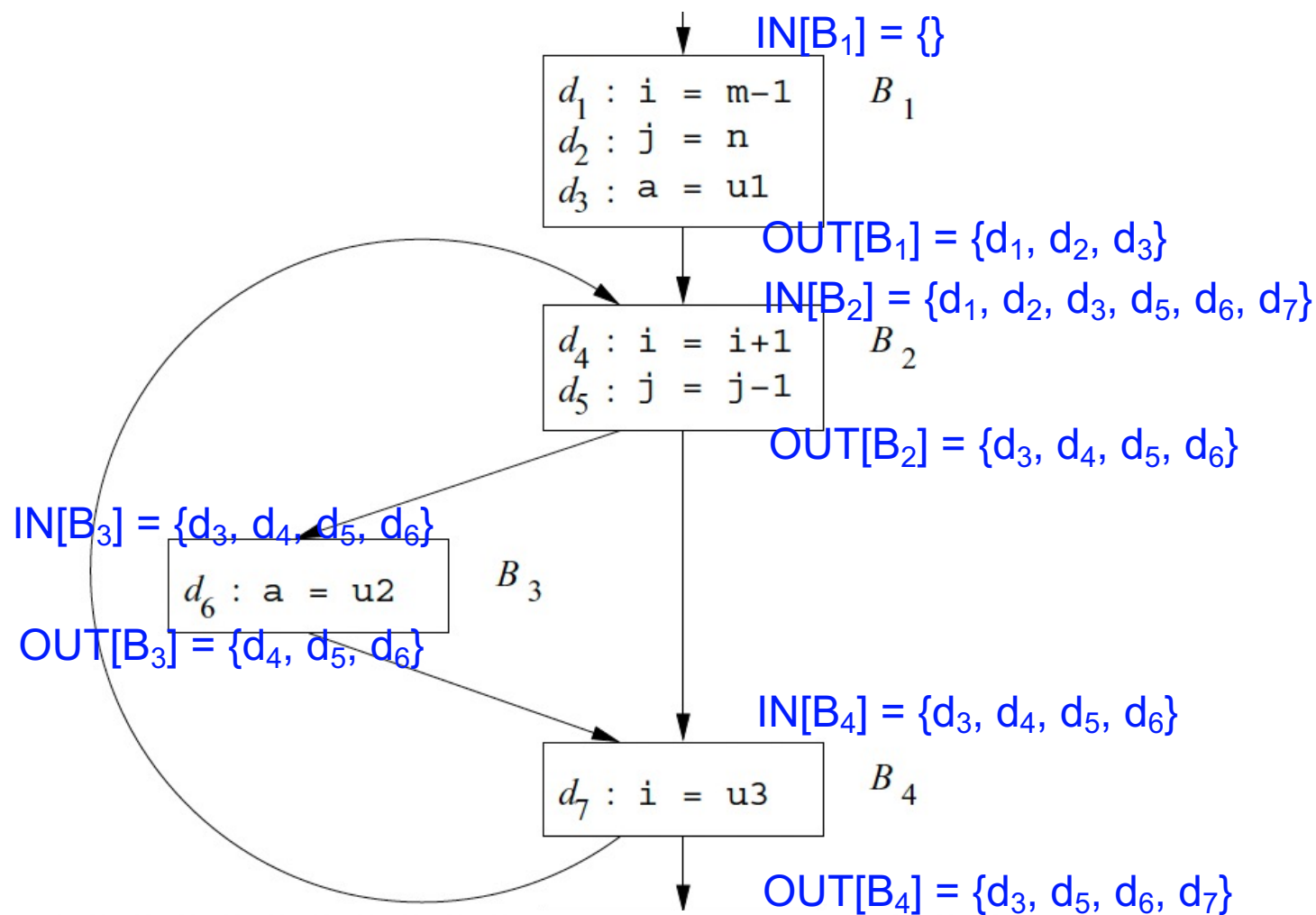
Reaching Definitions

B1 B2 B3 B4

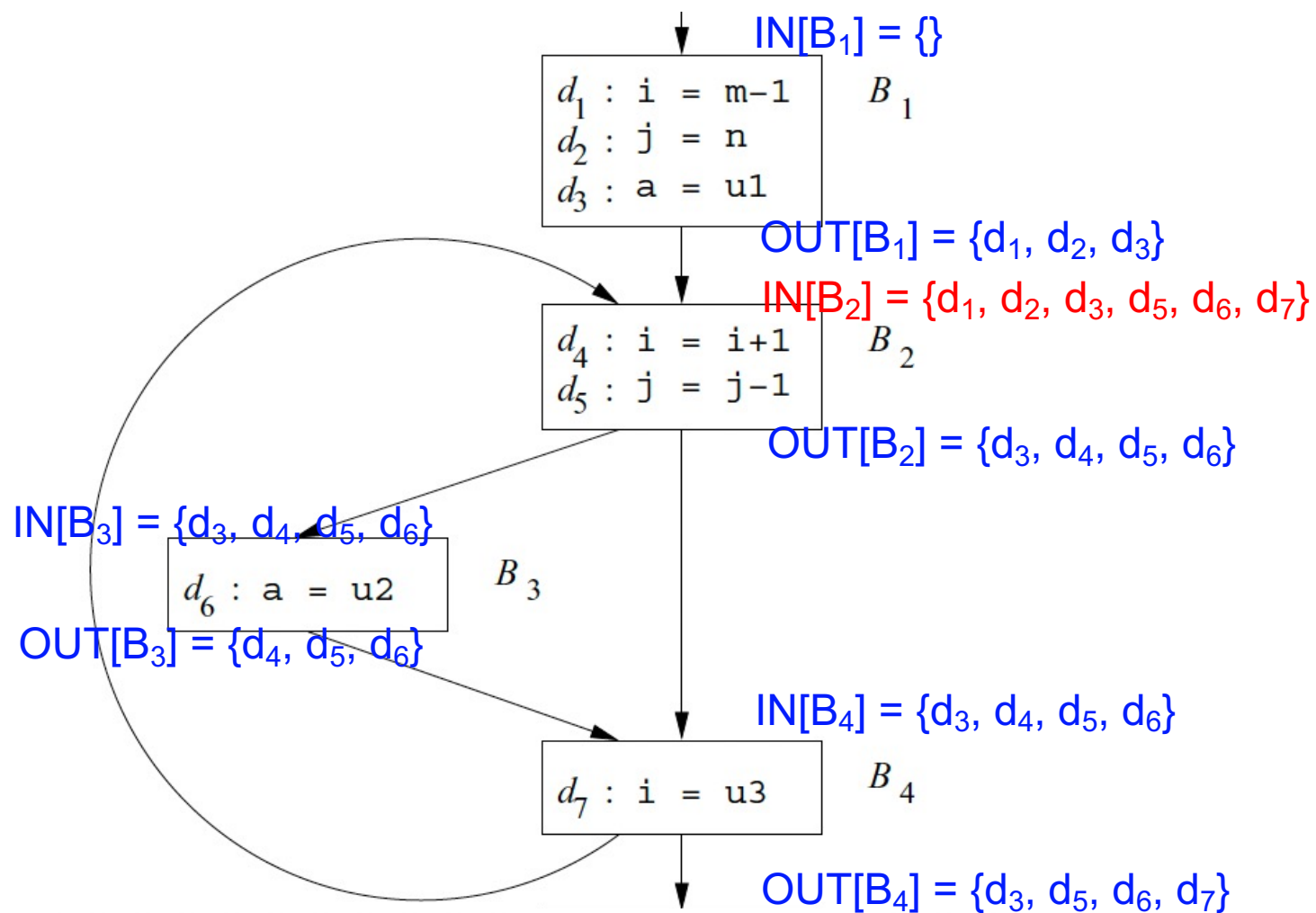
Worklist



Reaching Definitions

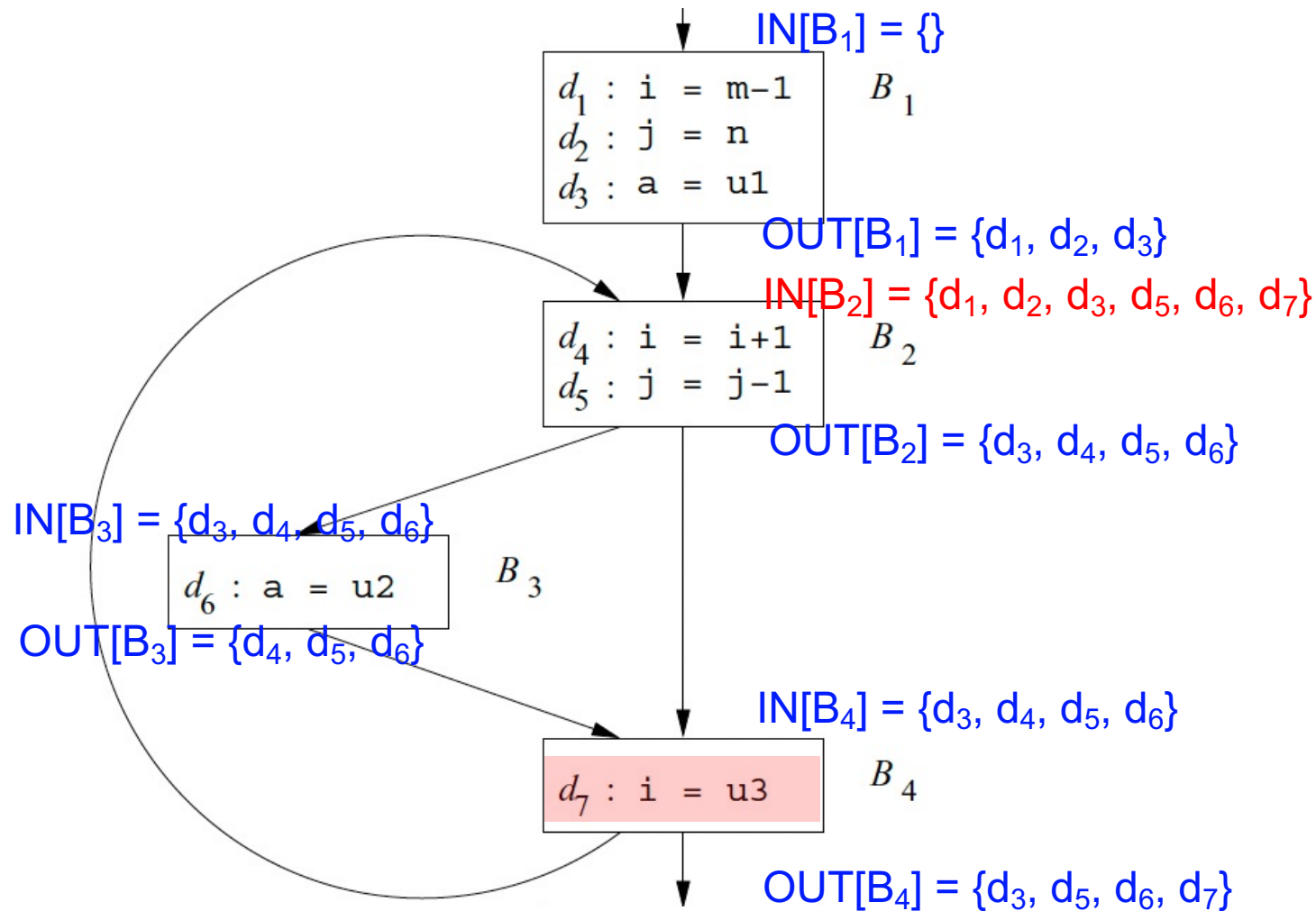


Reaching Definitions



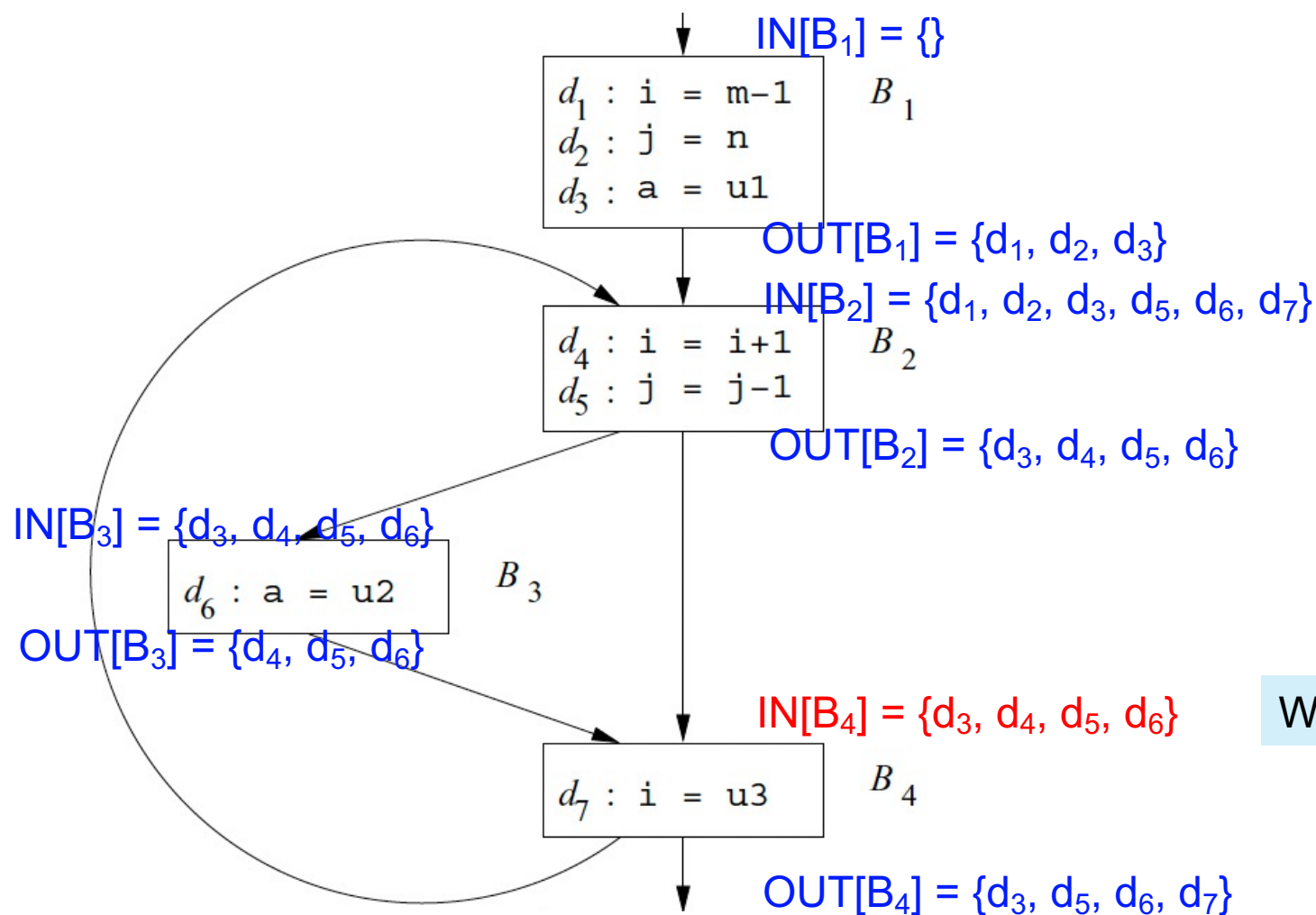
Why cannot d_4 reach here?

Reaching Definitions



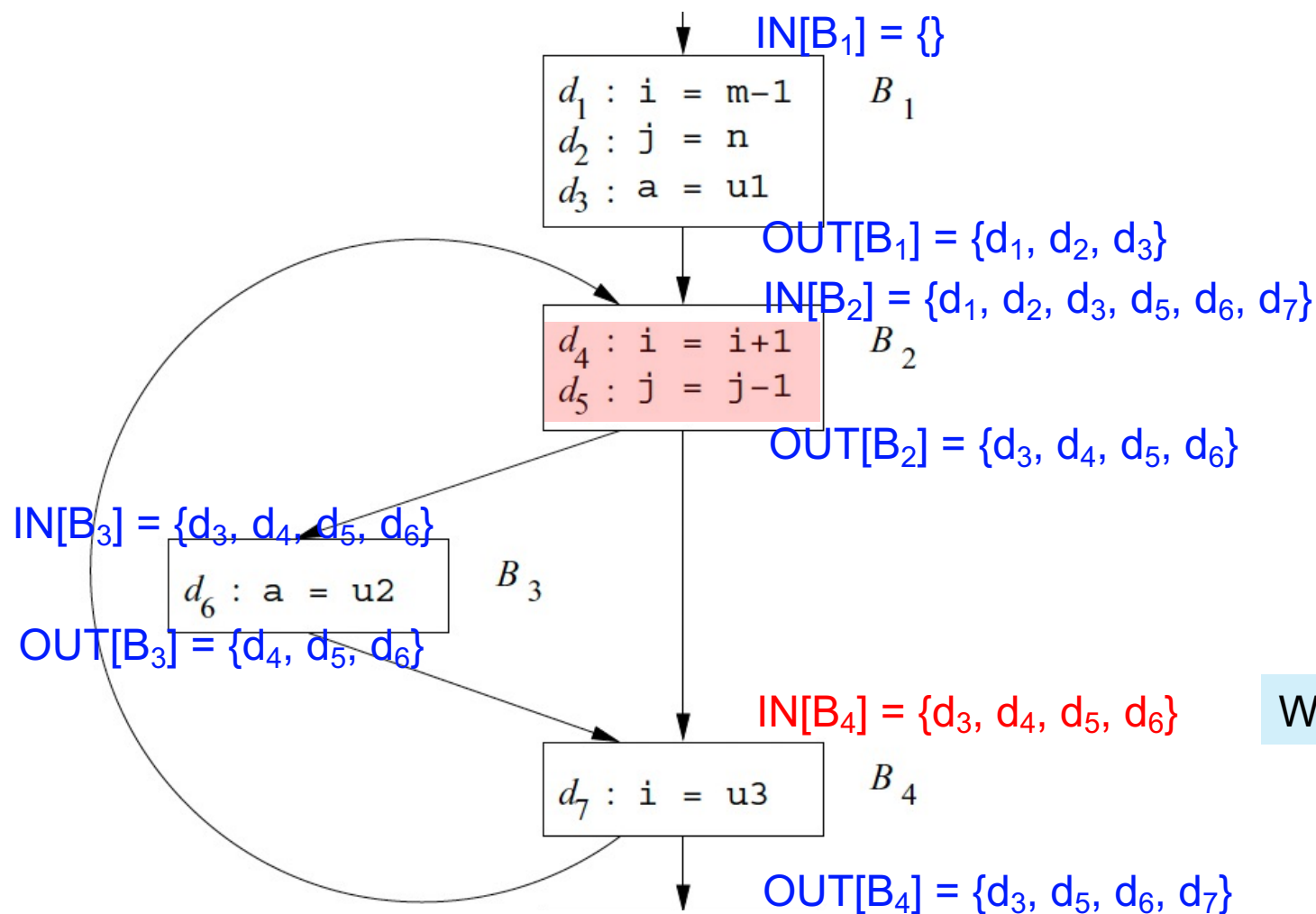
Why cannot d_4 reach here?

Reaching Definitions



Why cannot d_1, d_2, d_7 reach here?

Reaching Definitions



Why cannot d_1, d_2, d_7 reach here?

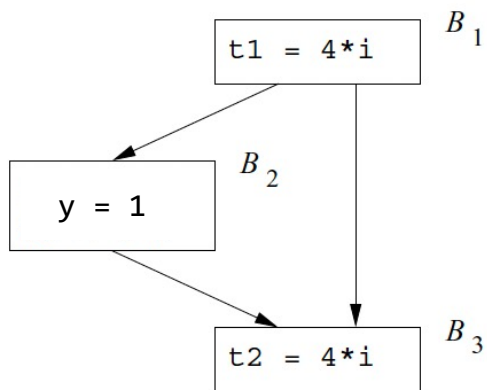
Available Expressions

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

Available Expressions

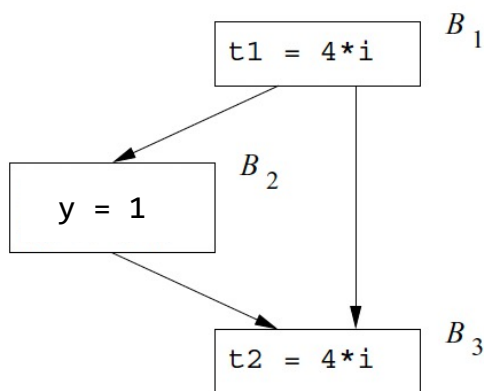
- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .



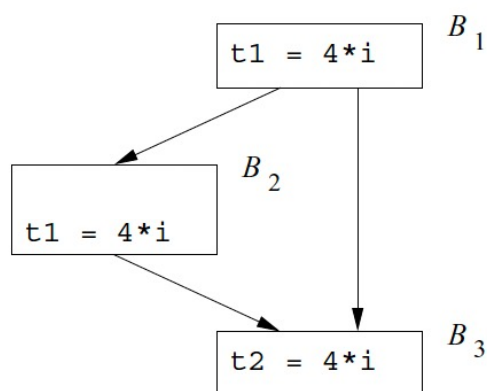
$4 * i$ is available before B_3

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .



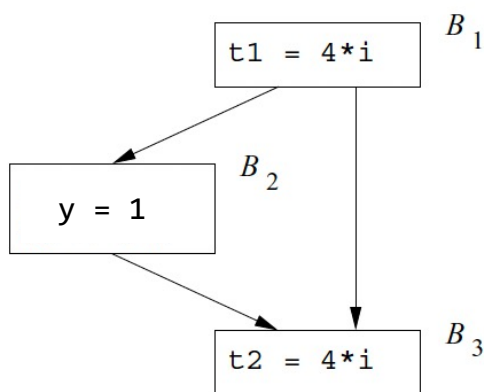
$4 * i$ is available before B_3



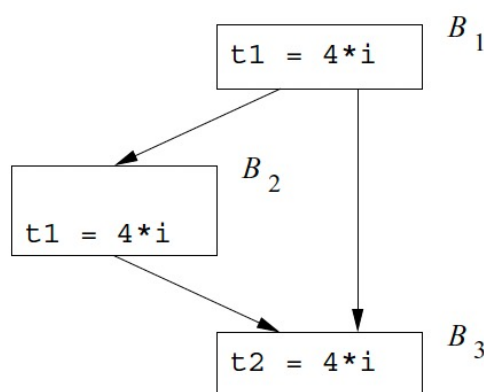
$4 * i$ is available before B_3

Available Expressions

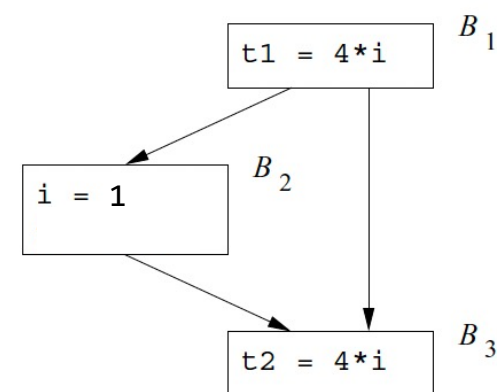
- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .



$4 * i$ is available before B_3



$4 * i$ is available before B_3



$4 * i$ is **NOT** available before B_3

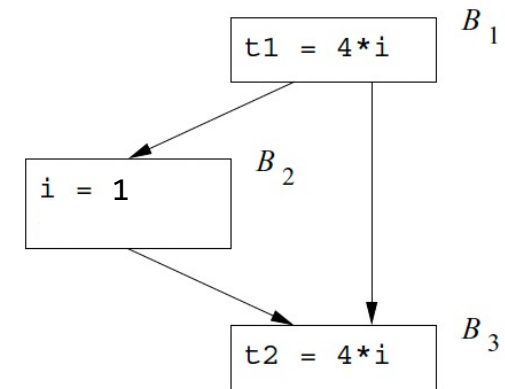
Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$



$4 * i$ is **NOT** available before B_3

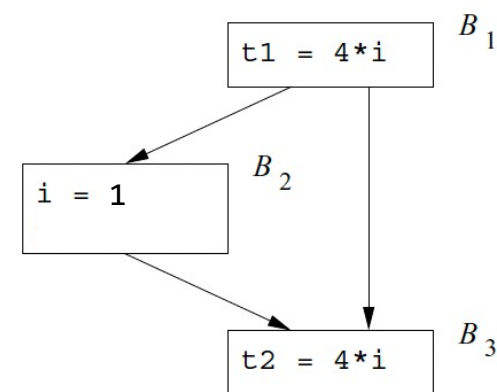
Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$



$4 * i$ is **NOT** available before B_3

Available Expressions

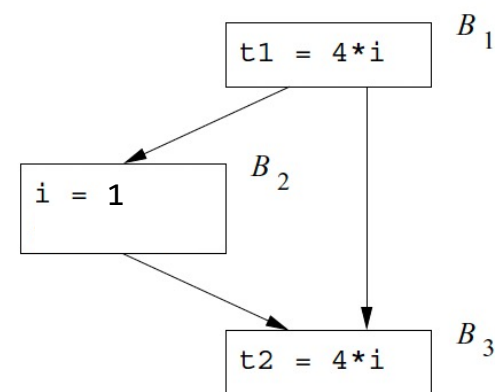
- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Why not $\cup$?



$4 * i$ is **NOT** available before B_3

Available Expressions

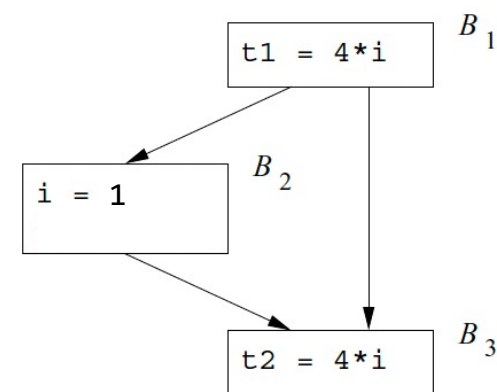
- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Why not $\cup$?



$4 * i$ is **NOT** available before B_3

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Why is $b+c$ not included?

Available Expressions

- An expression $x + y$ is available at a point p if
 - every path from the entry node to p evaluates $x + y$, and
 - after the last such evaluation prior to reaching p , there are no subsequent assignments to x or y .

- The equations

$$\text{OUT}[B] = e_gen_B \cup (\text{IN}[B] - e_kill_B)$$

$$\text{IN}[B] = \bigcap_{P \text{ a predecessor of } B} \text{OUT}[P].$$

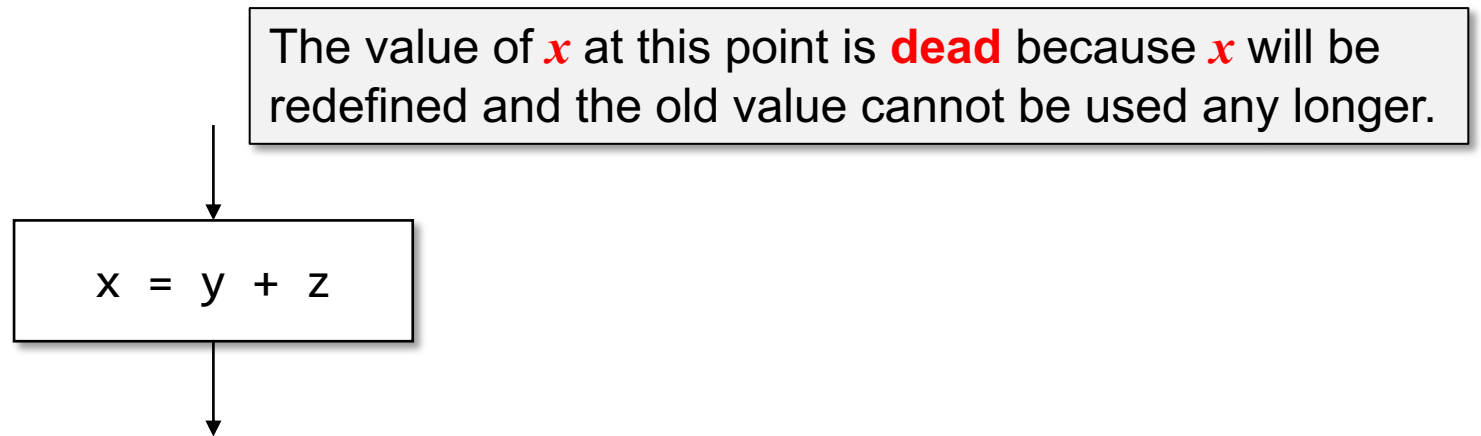
Statement	Available Expressions
	$\emptyset$
$a = b + c$	$\{b + c\}$
$b = a - d$	$\{a - d\}$
$c = b + c$	$\{a - d\}$
$d = a - d$	$\emptyset$

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .



Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .

The values of y , z at this point are **live** as we can use them in the next statement.

The value of x at this point is **dead** because x will be redefined and the old value cannot be used any longer.

$x = y + z$

Live Variable Analysis

- **Application:** Register Allocation.
- A dead value
 - After a variable is revised, its previous value (in the register) is dead
 - We need to reload its value from the memory to the register

Live Variable Analysis

- **Application:** Register Allocation.
- A dead value
 - After a variable is revised, its previous value (in the register) is dead
 - We need to reload its value from the memory to the register
- A live value
 - A variable is not re-defined
 - After loading to a register, the value in the register is valid
 - No need to reload the value from the memory

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$\text{IN}[B] = \text{use}_B \cup (\text{OUT}[B] - \text{def}_B)$$

$$\text{OUT}[B] = \bigcup_{S \text{ a successor of } B} \text{IN}[S]$$

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$\text{IN}[B] = \text{use}_B \cup (\text{OUT}[B] - \text{def}_B)$$

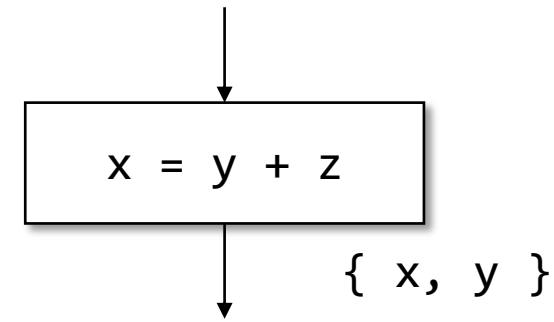
$$\text{OUT}[B] = \bigcup_{S \text{ a successor of } B} \text{IN}[S]$$

Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

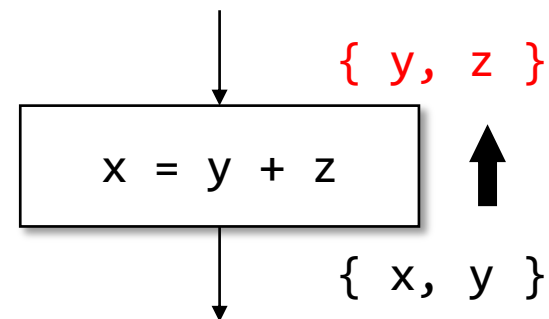


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

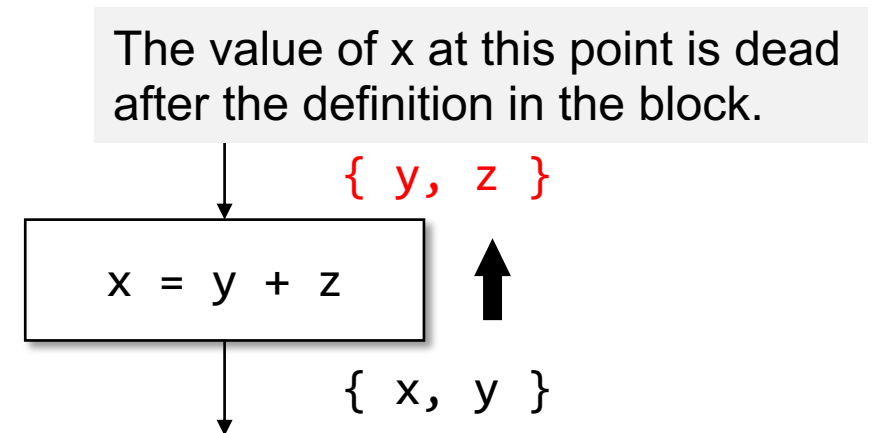


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

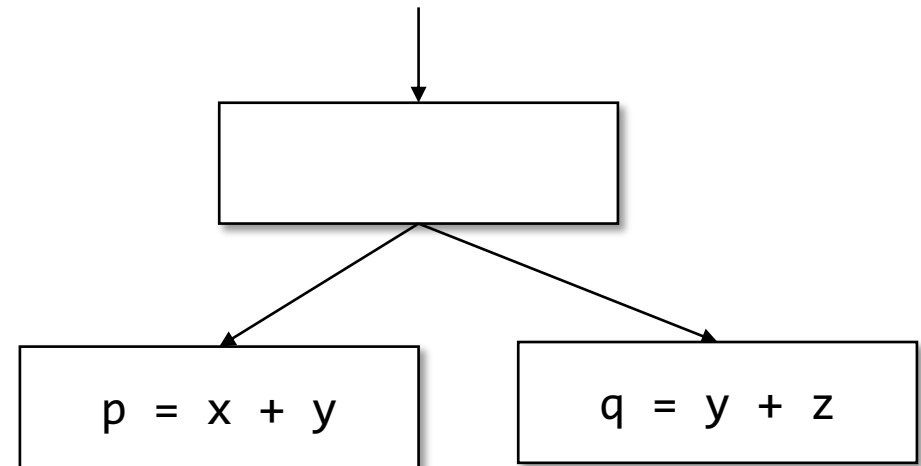


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

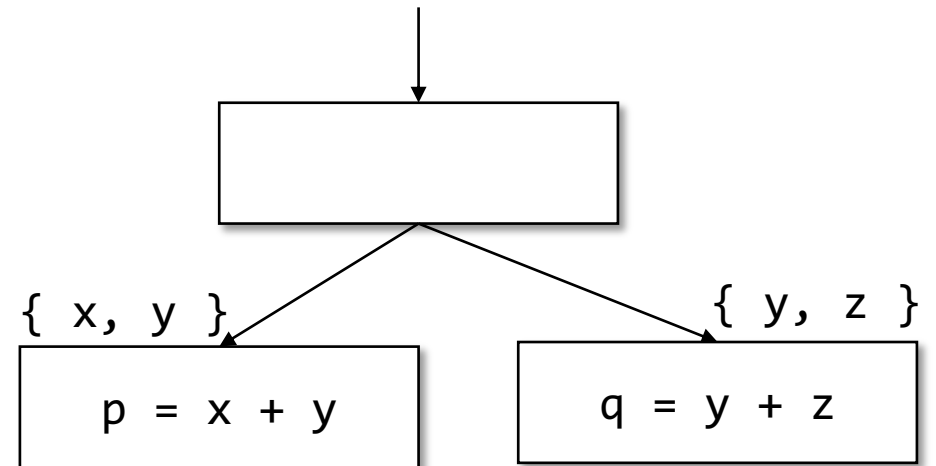


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$

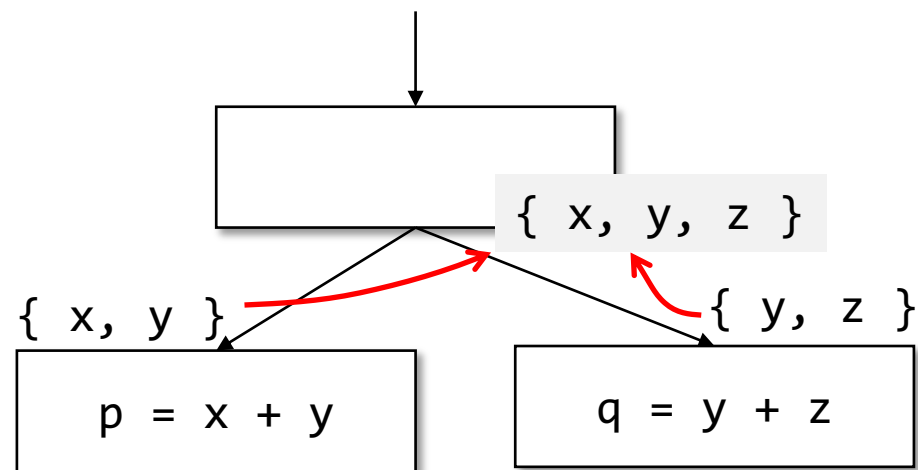


Live Variable Analysis

- We wish to know if the value of a variable x at point p can be used along the path starting from p .
- **Backward data flow analysis.**
 - The same as forward analysis except for the direction.

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S]$$



Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Summary

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

Question-1

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - **B = worklist.pop();**
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
 - **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Question-1

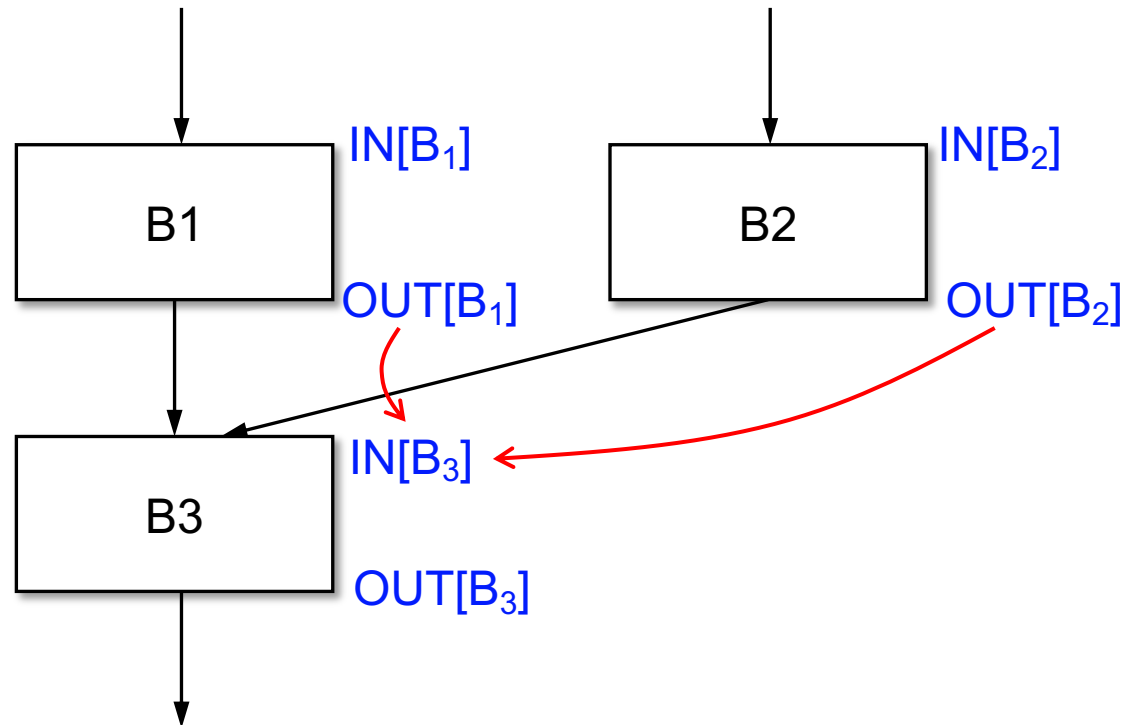
- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

What's the best order of blocks?

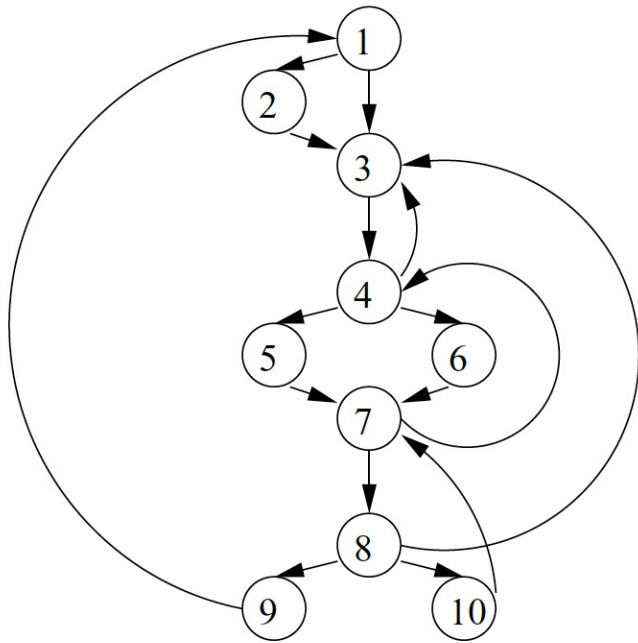
- **While** (!worklist.empty()) **Do**
- **B = worklist.pop();**
- $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
- **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

What's the Best Order of Blocks

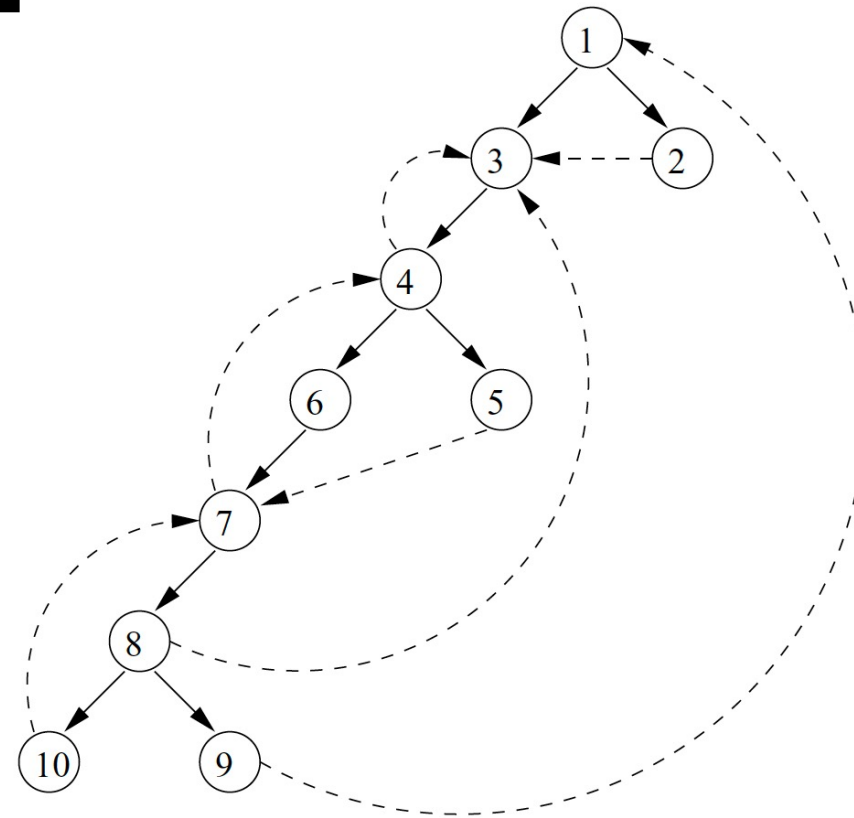
- If we analyze B3 before B1 and B2, B3 may be analyzed again after B1 and B2 are analyzed



Depth-First Order

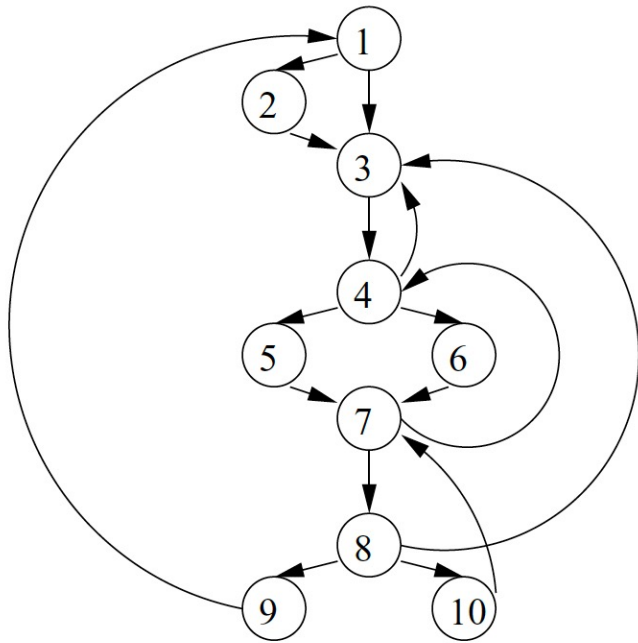


Control-Flow Graph



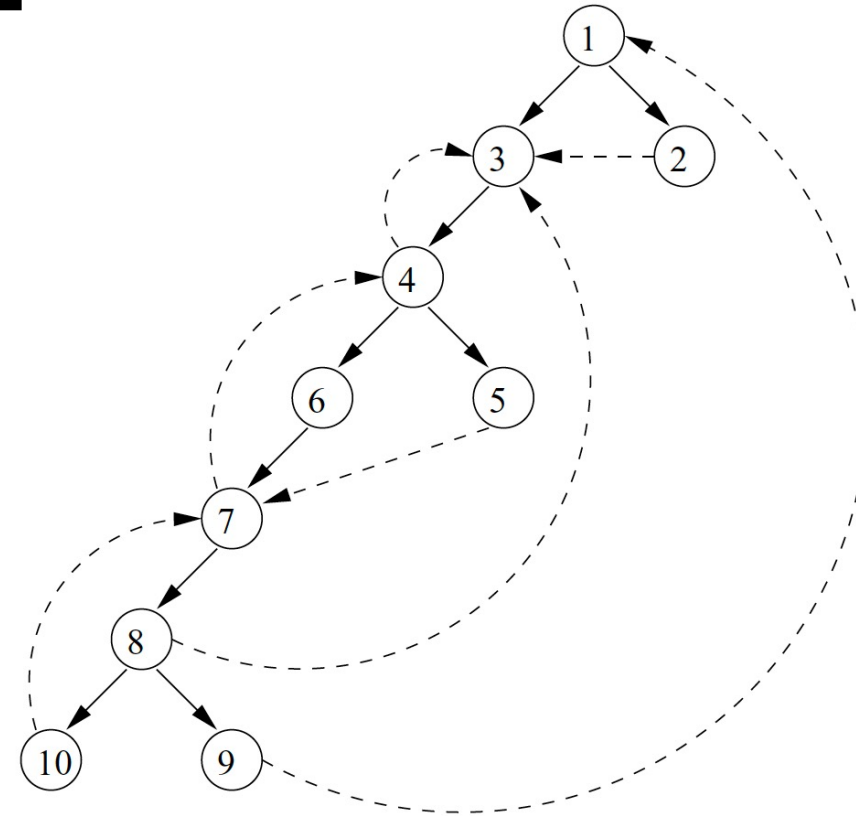
Depth-First Tree

Depth-First Order



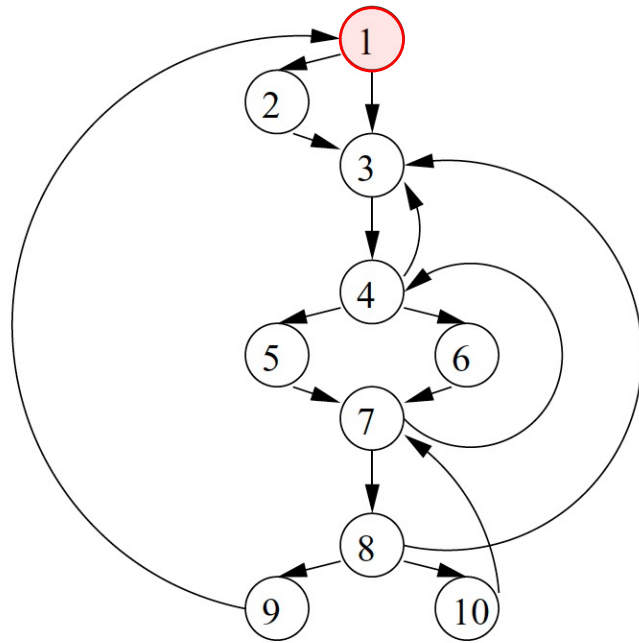
Control-Flow Graph

Depth-First Traversal to Build the Tree



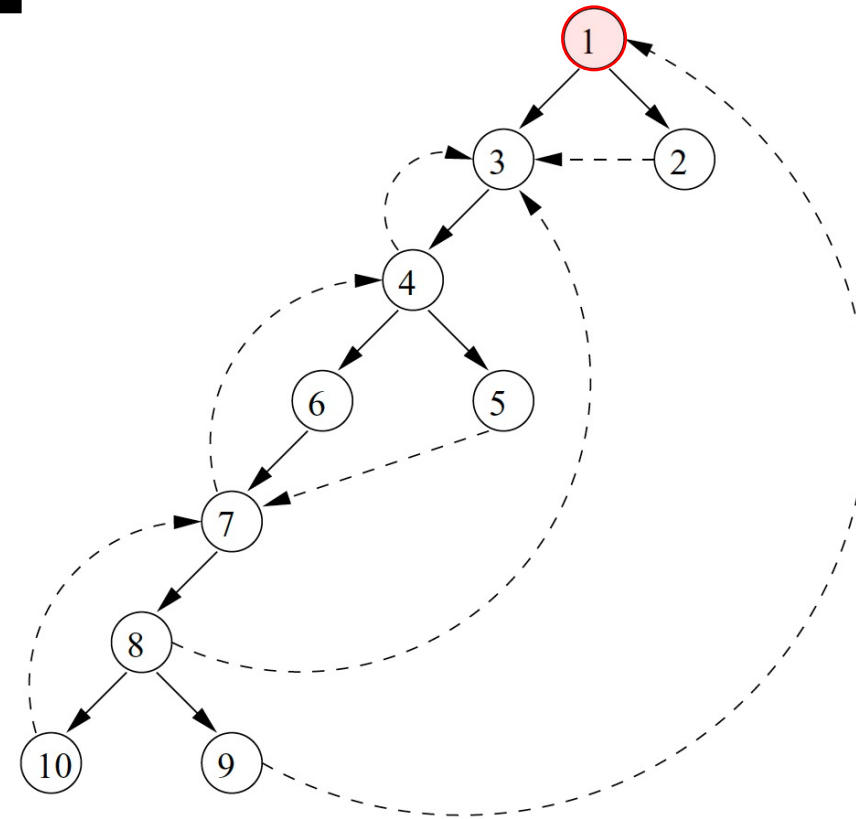
Depth-First Tree

Depth-First Order



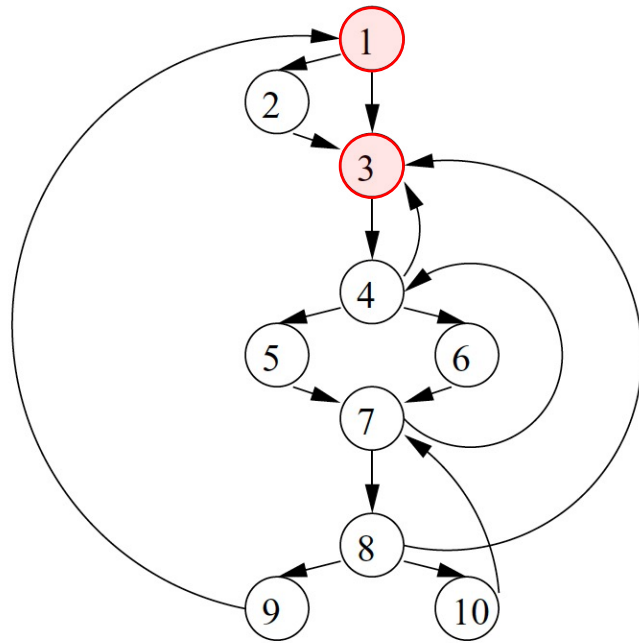
Control-Flow Graph

Depth-First Traversal to Build the Tree



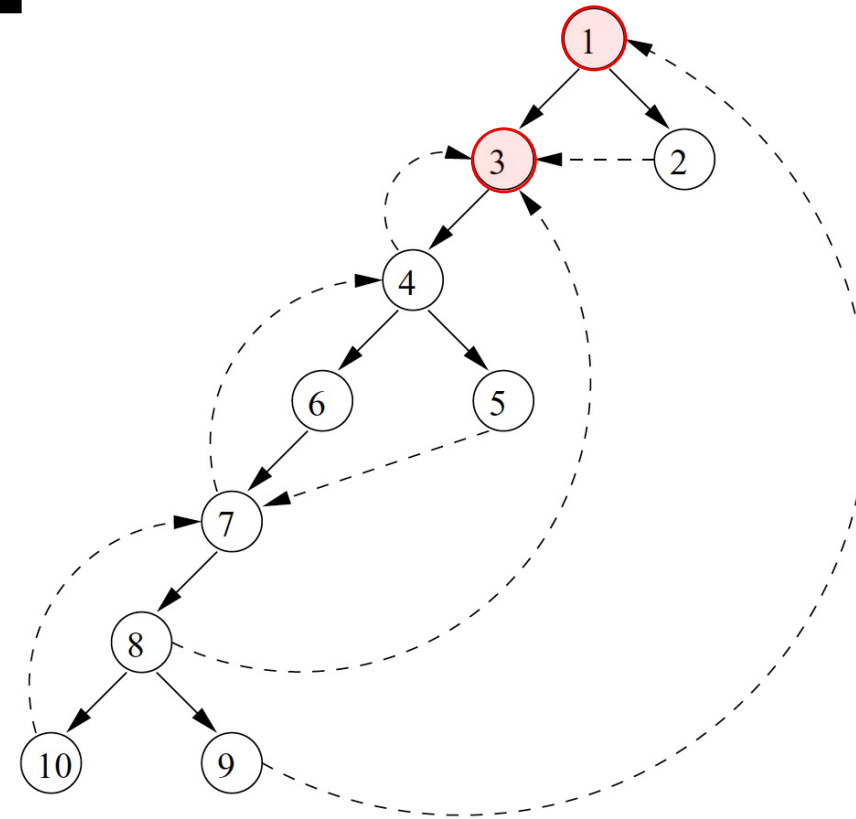
Depth-First Tree

Depth-First Order



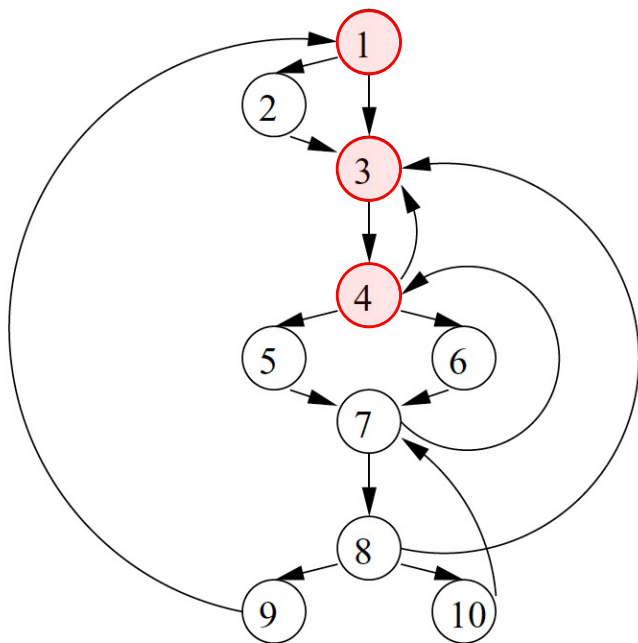
Control-Flow Graph

Depth-First Traversal to Build the Tree

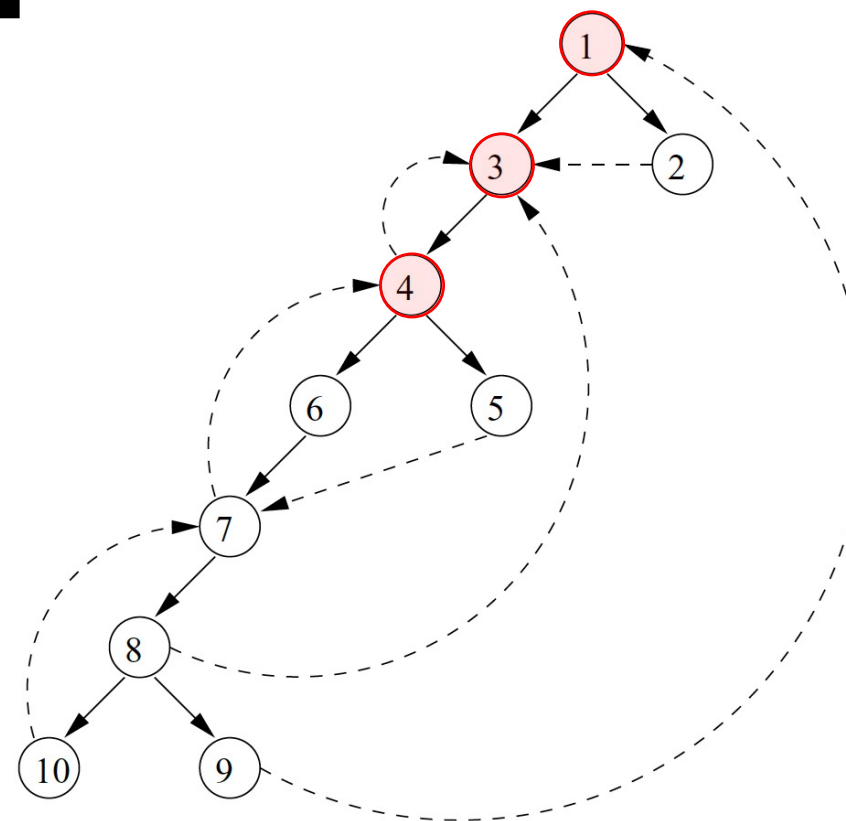


Depth-First Tree

Depth-First Order



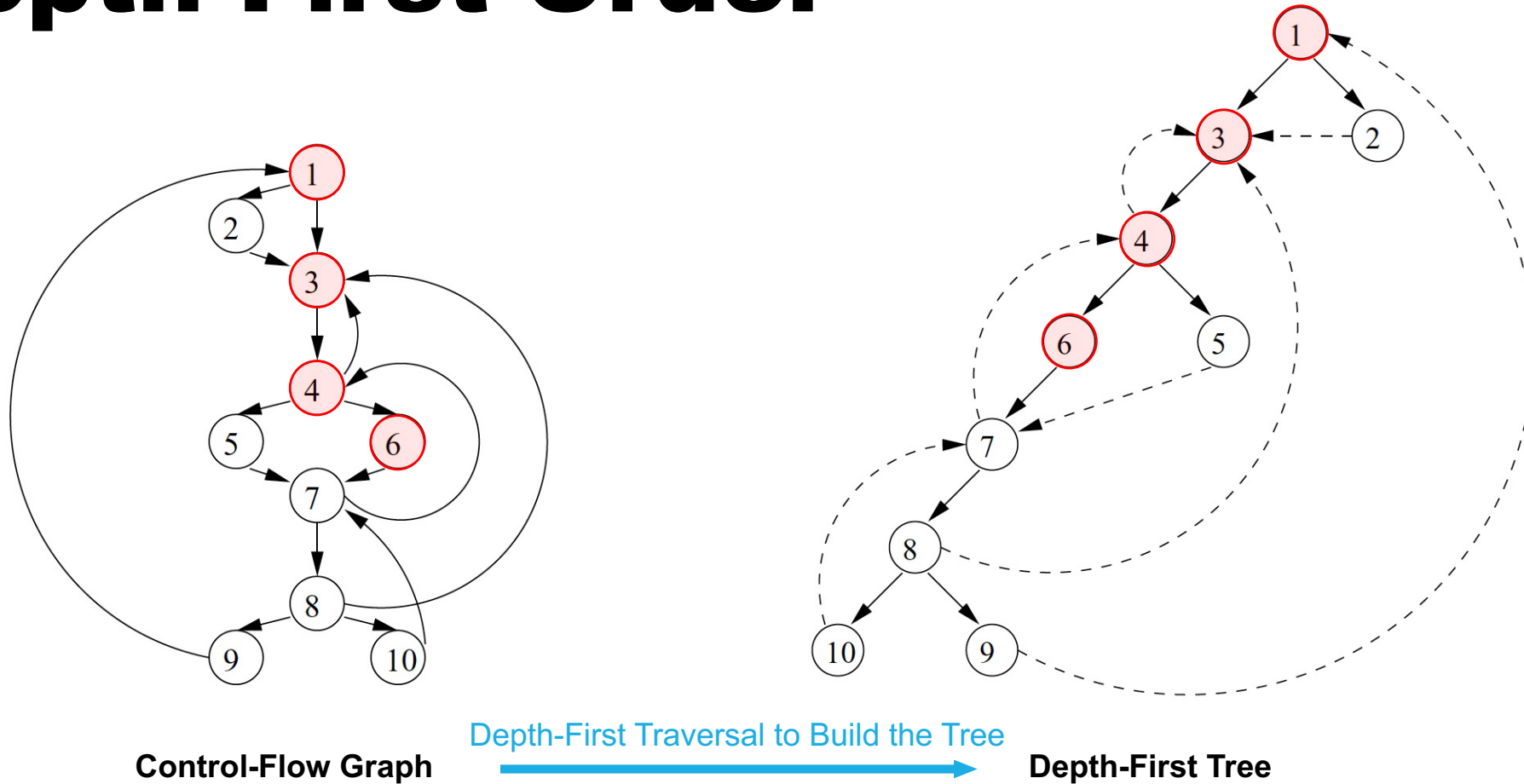
Control-Flow Graph



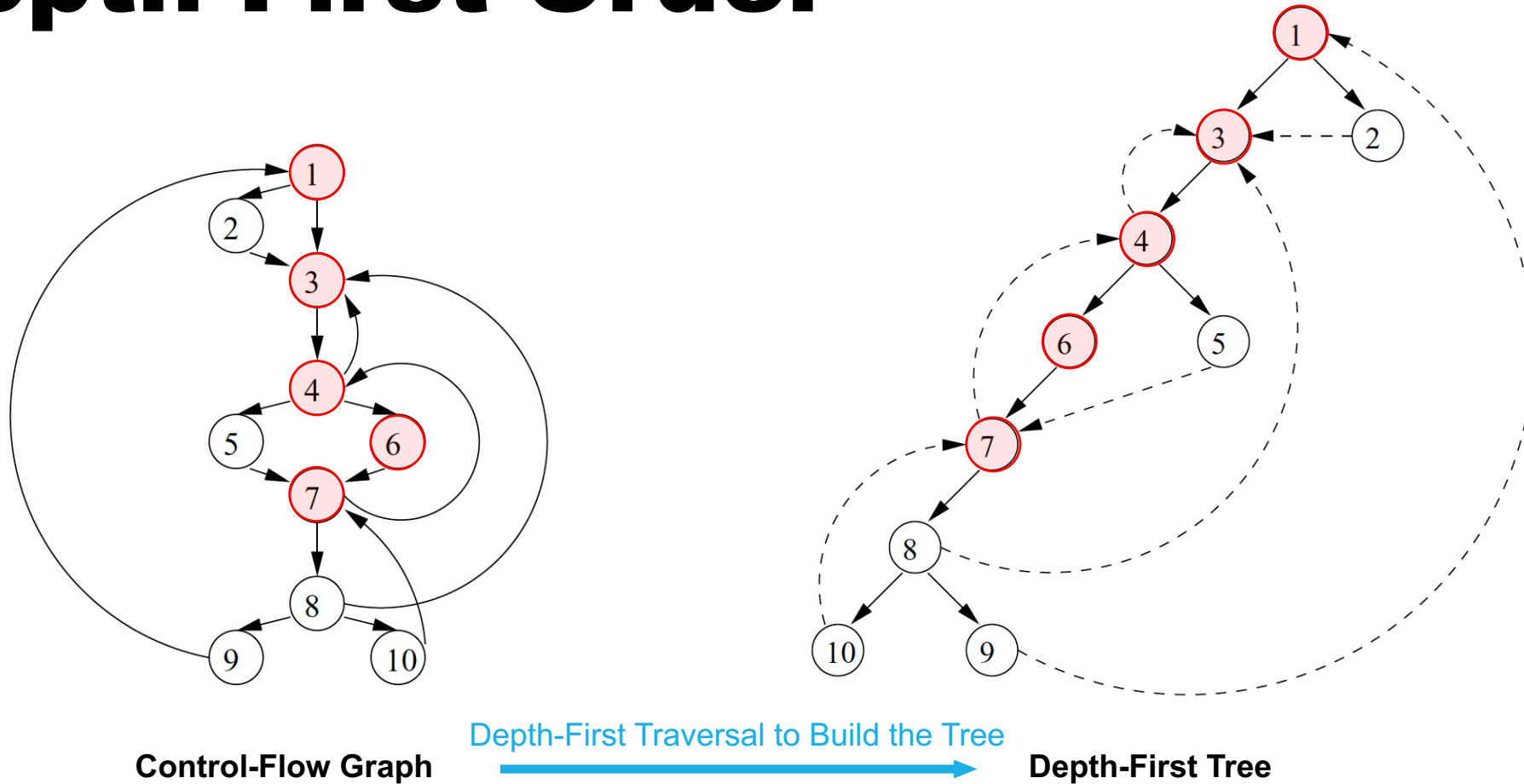
Depth-First Tree

Depth-First Traversal to Build the Tree

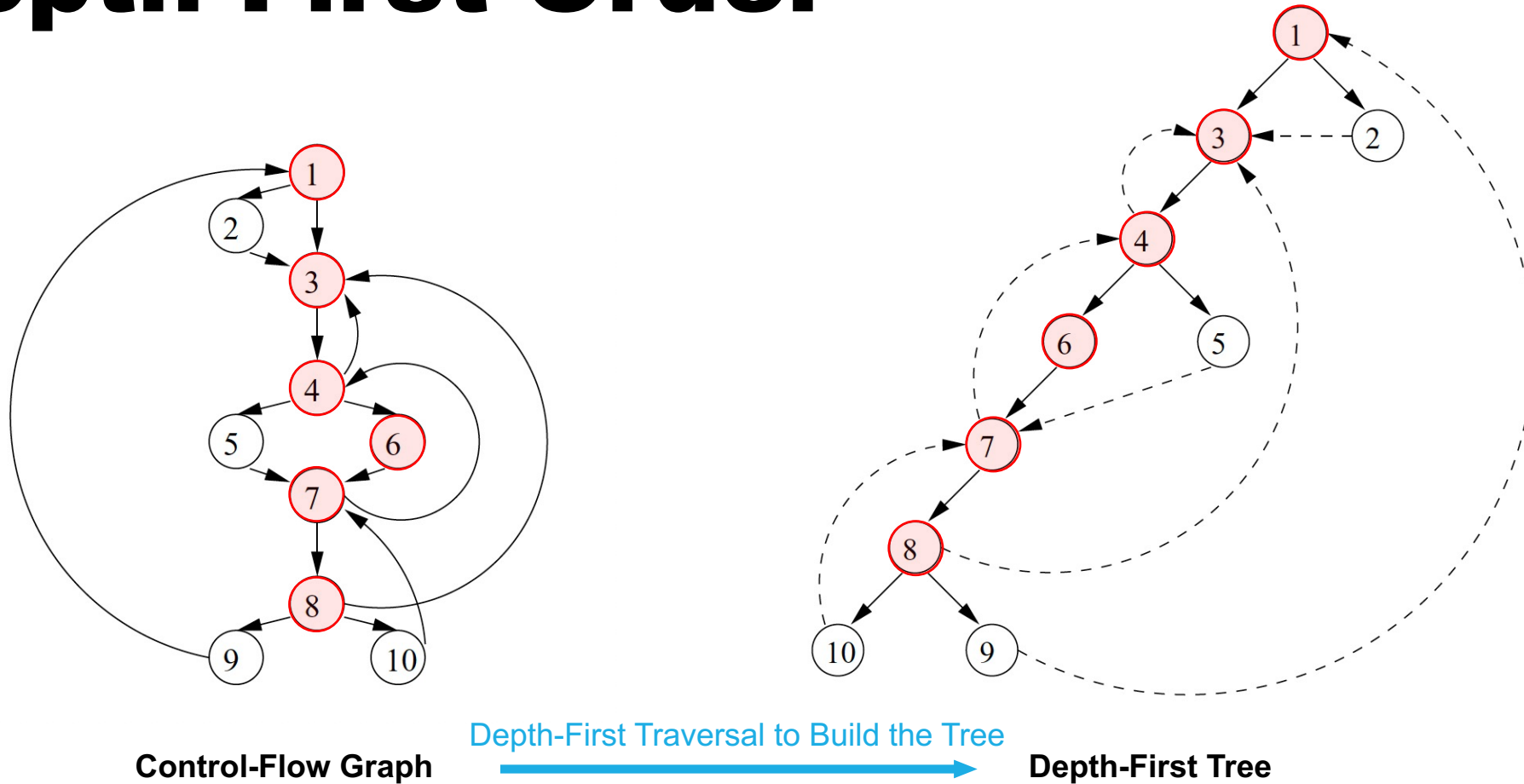
Depth-First Order



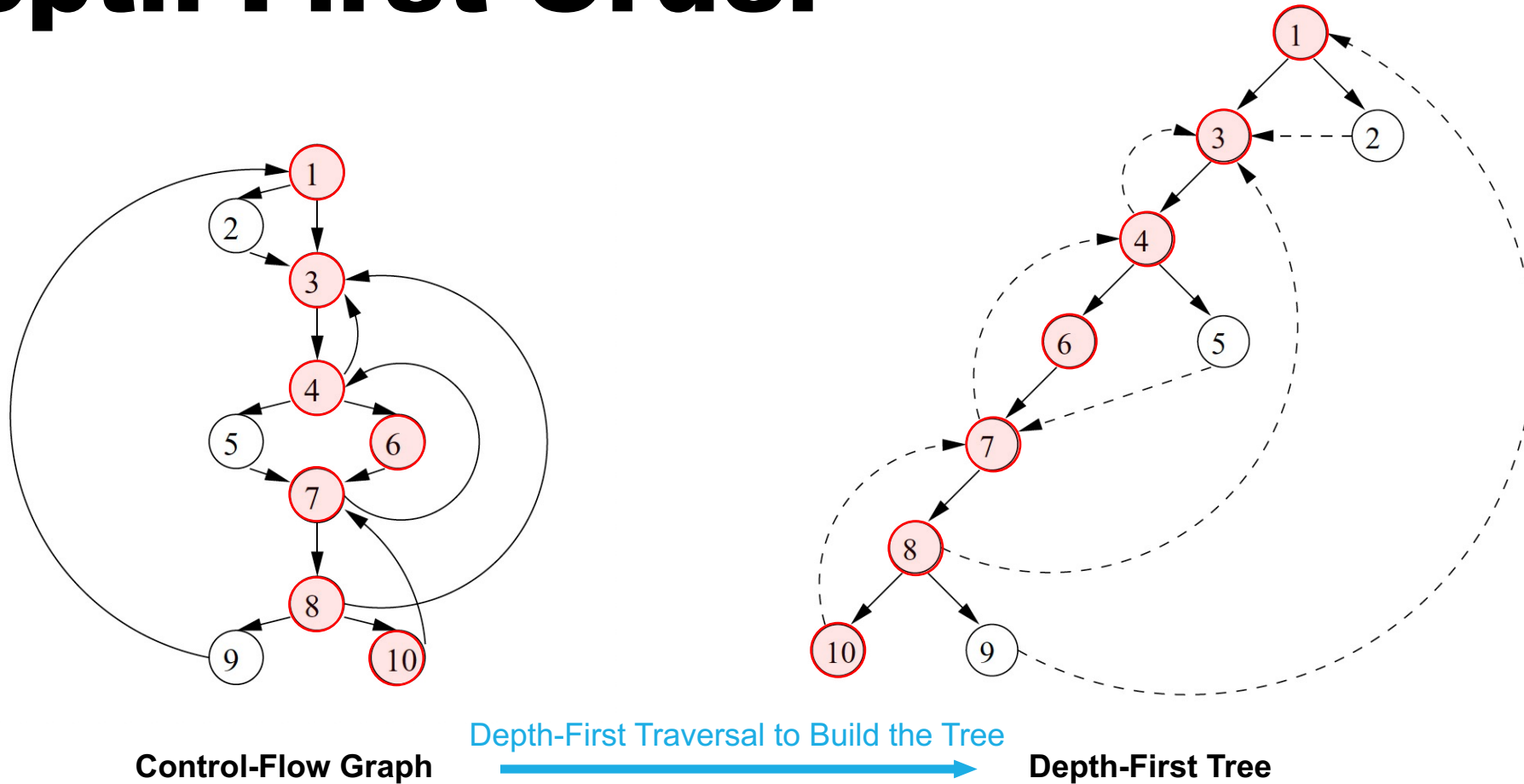
Depth-First Order



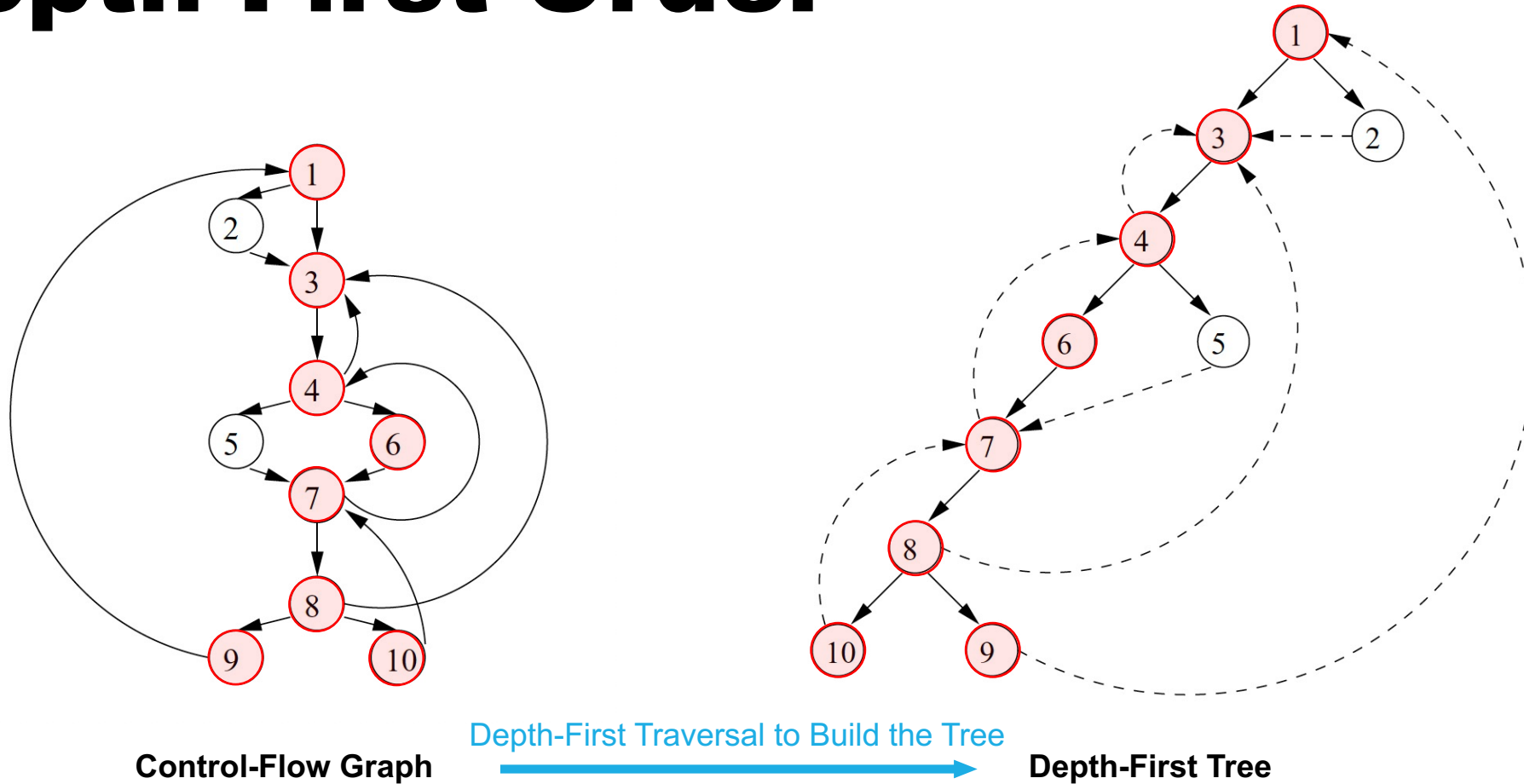
Depth-First Order



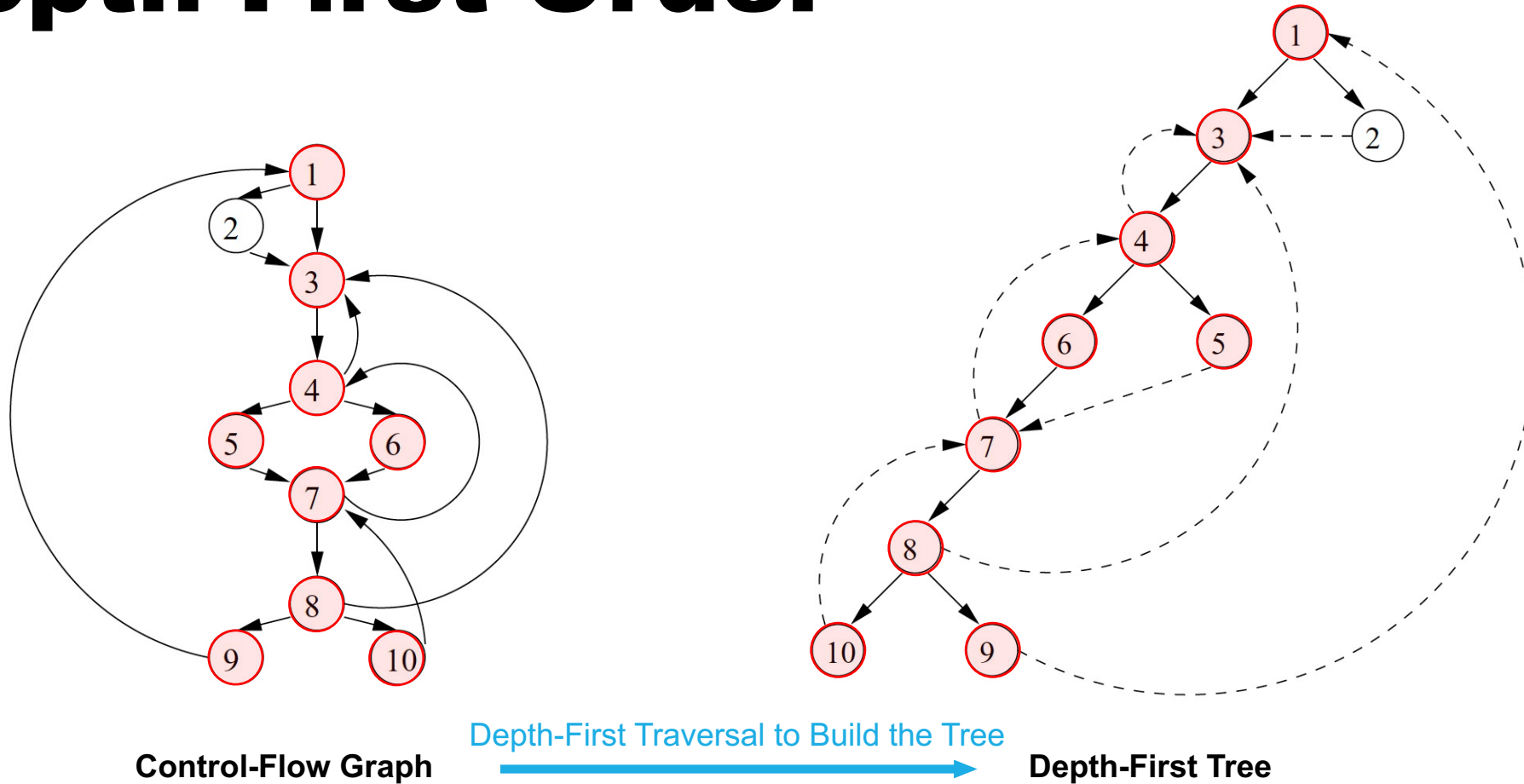
Depth-First Order



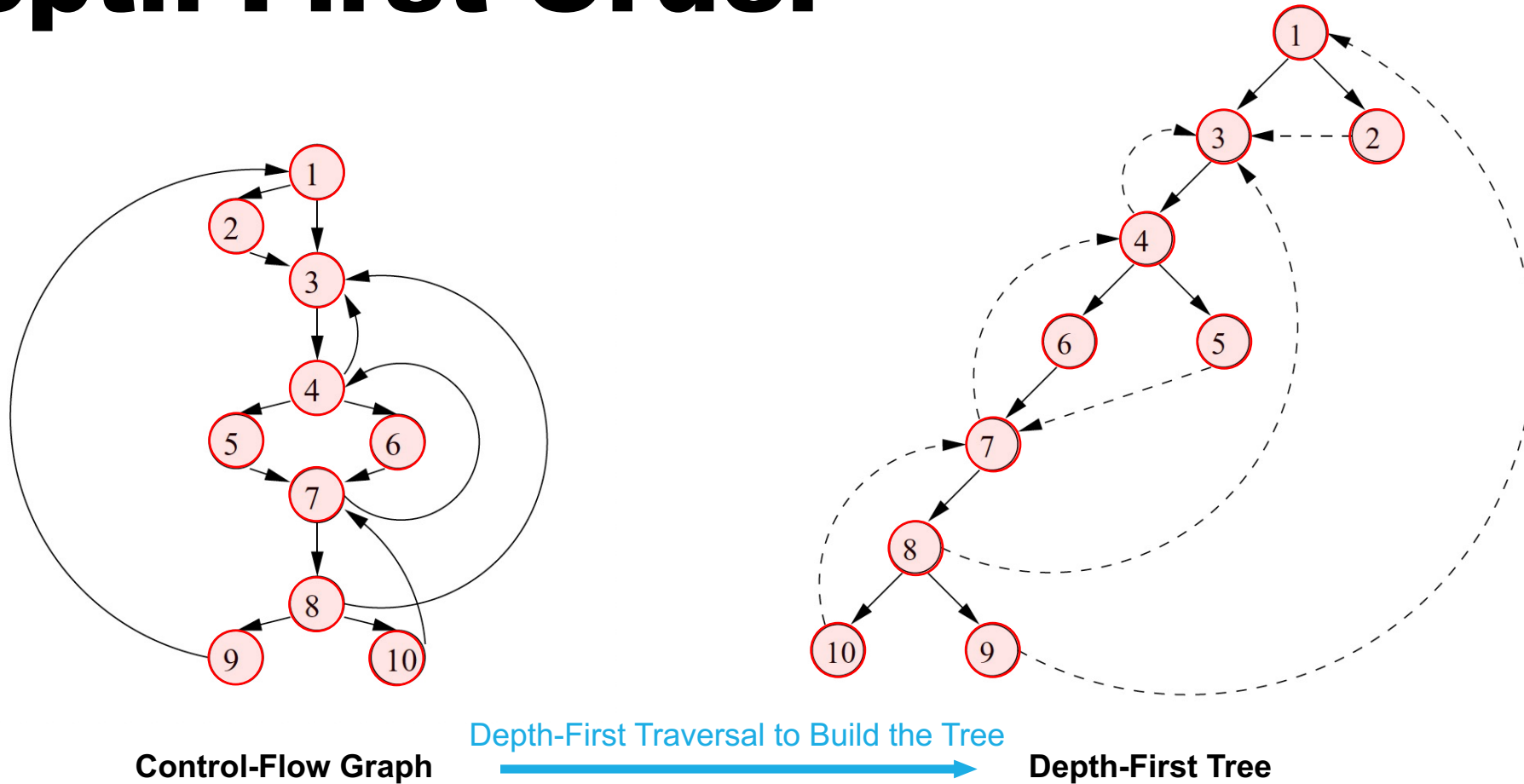
Depth-First Order



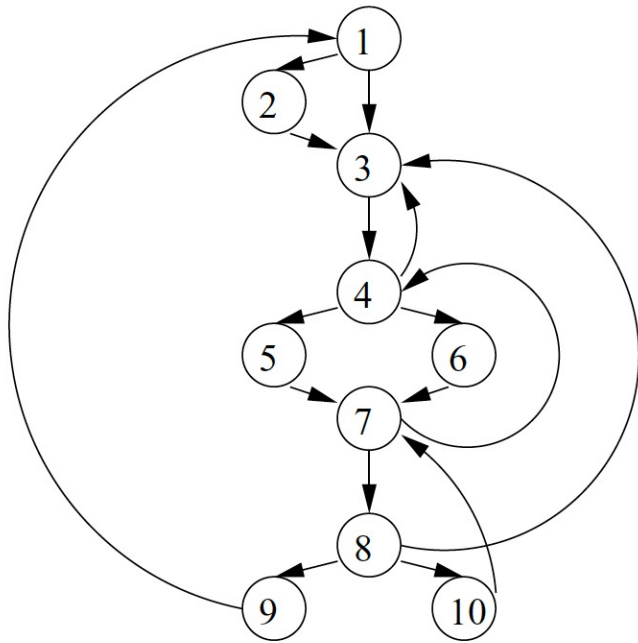
Depth-First Order



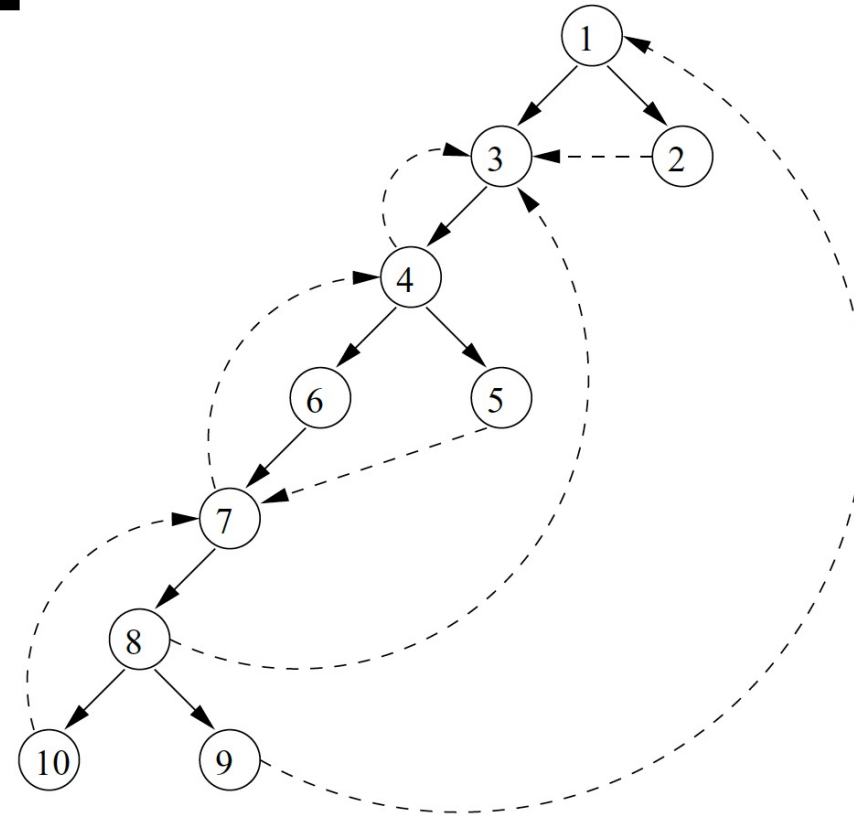
Depth-First Order



Depth-First Order



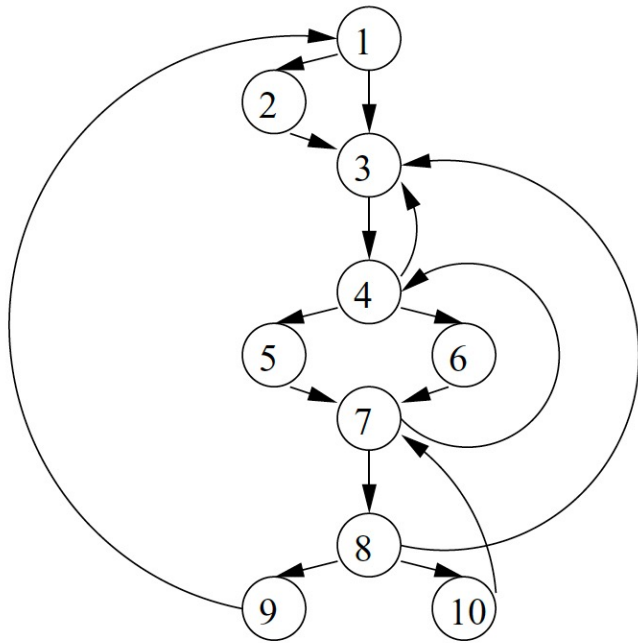
Control-Flow Graph



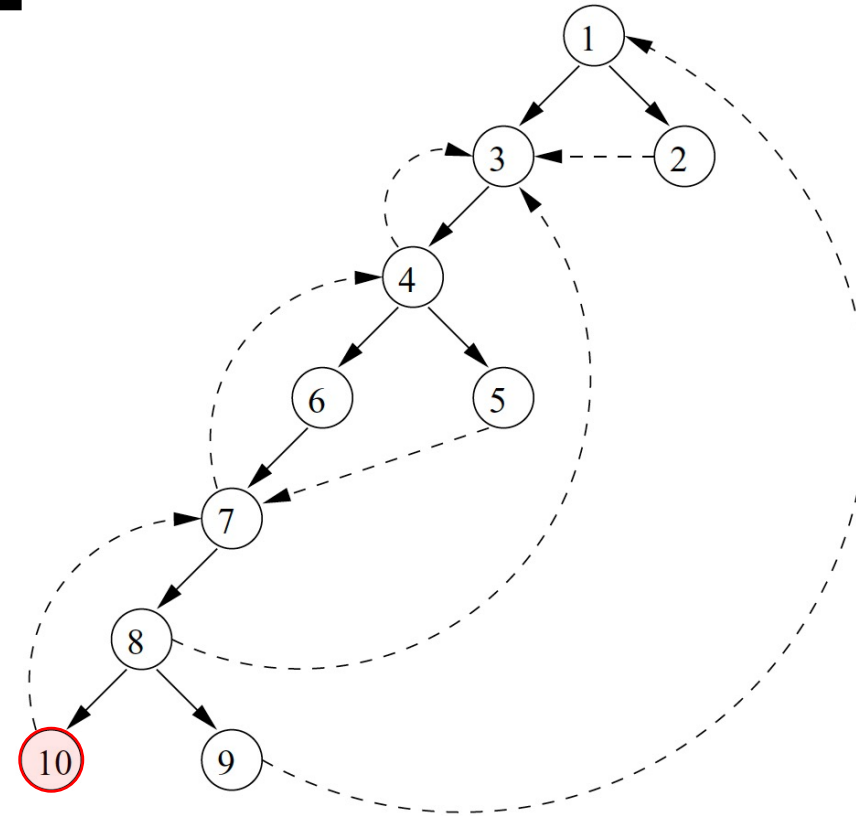
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



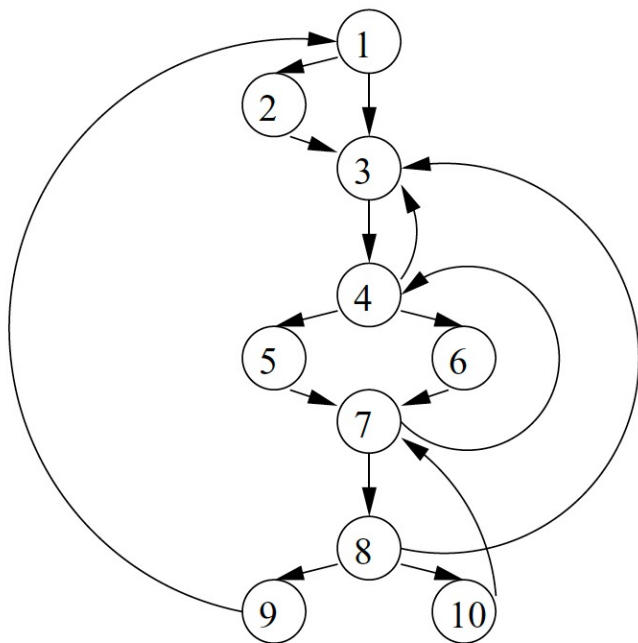
Control-Flow Graph



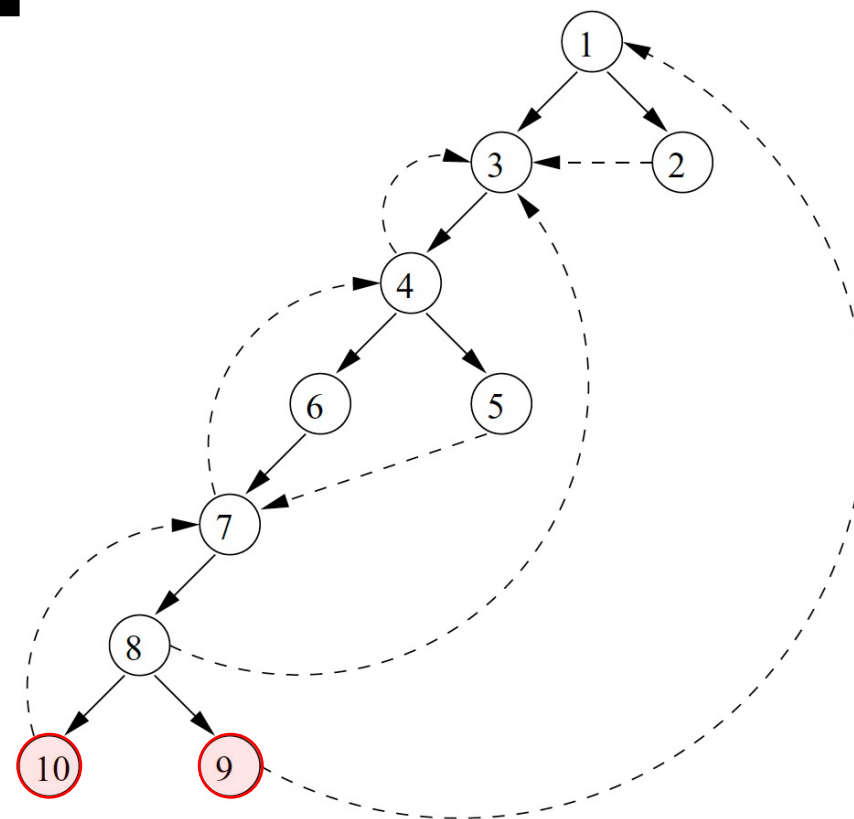
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



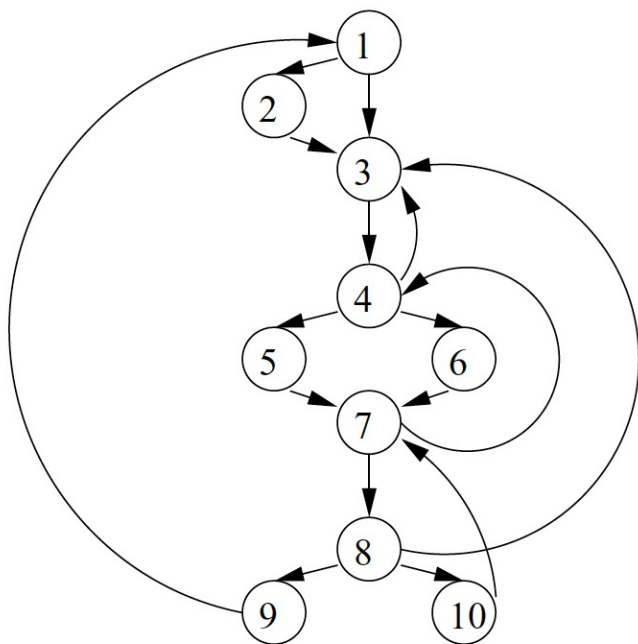
Control-Flow Graph



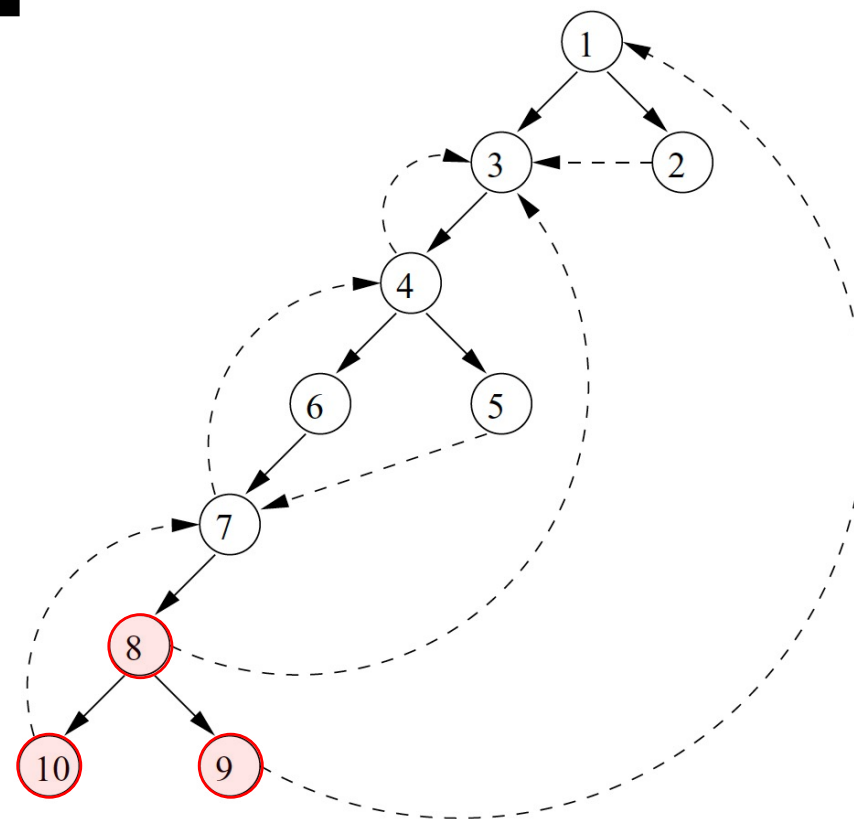
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



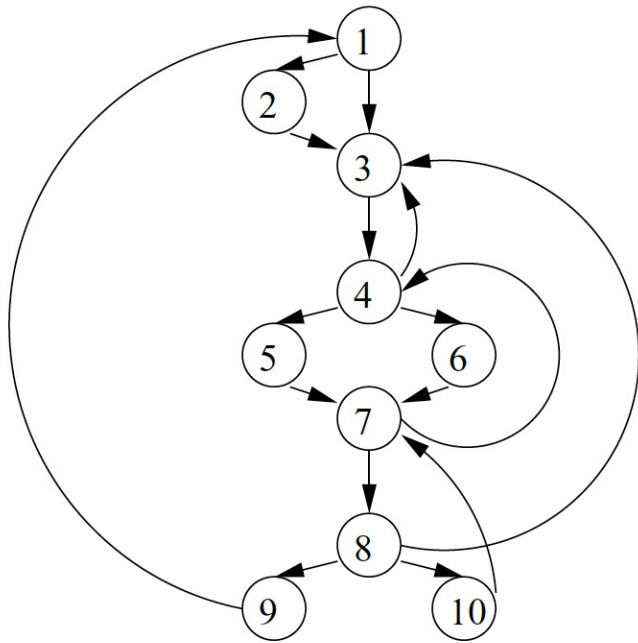
Control-Flow Graph



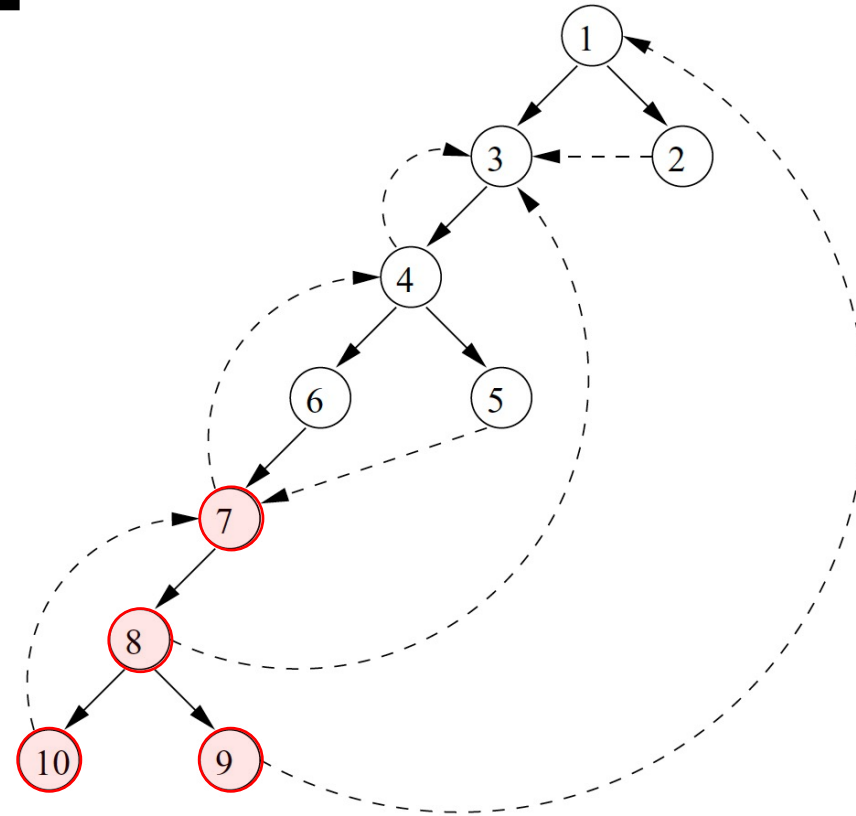
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



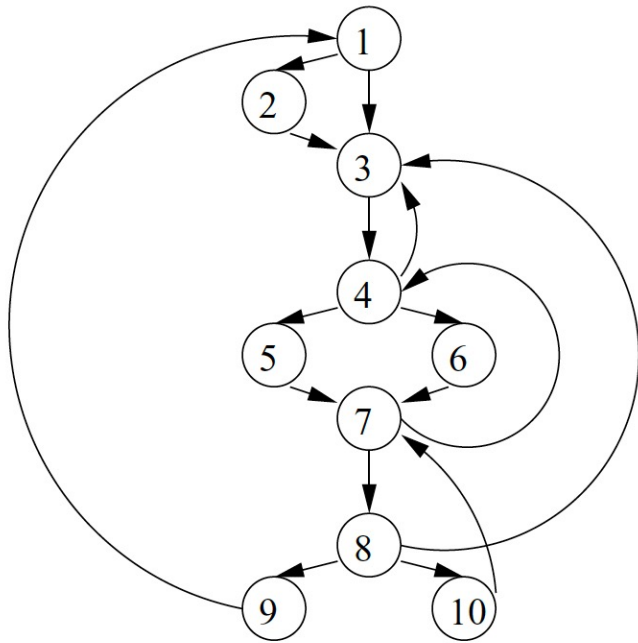
Control-Flow Graph



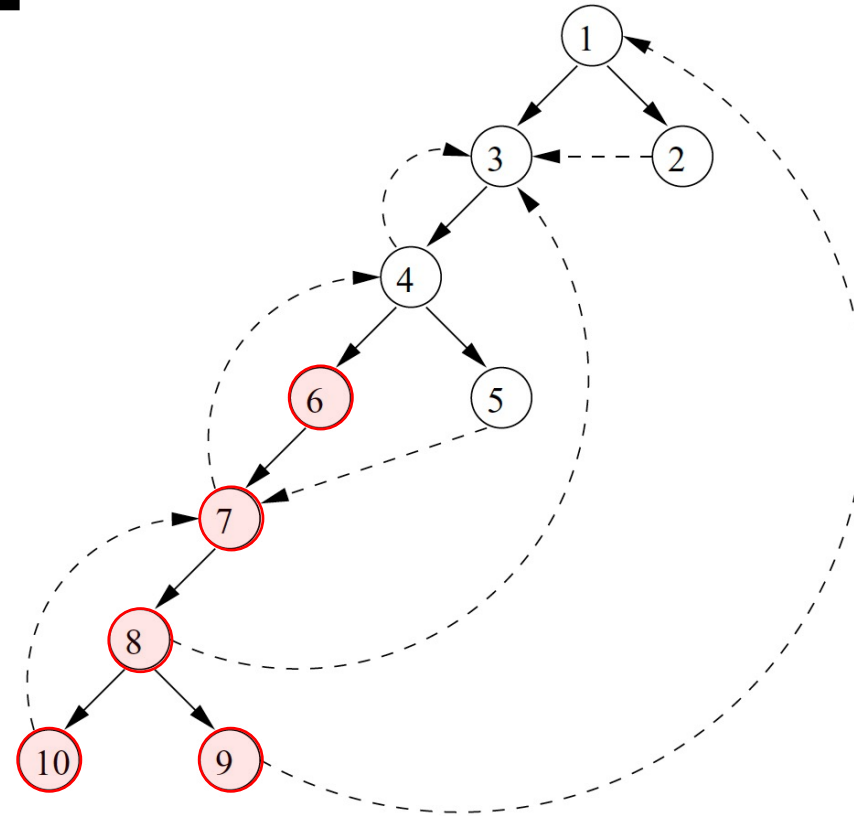
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



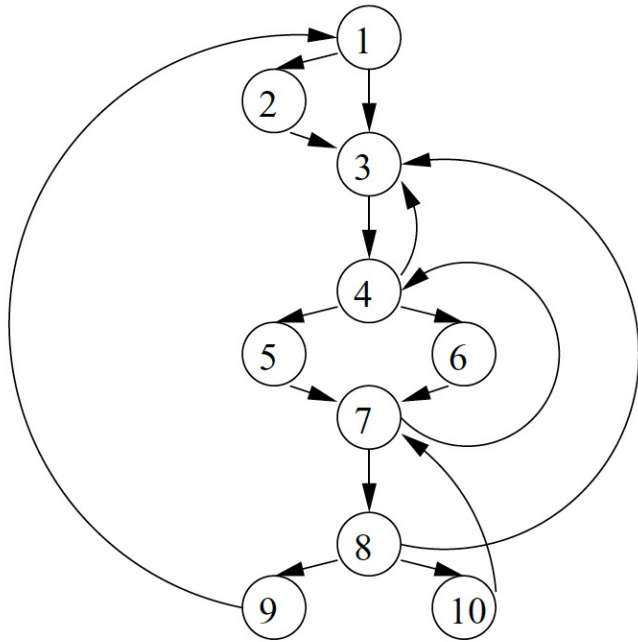
Control-Flow Graph



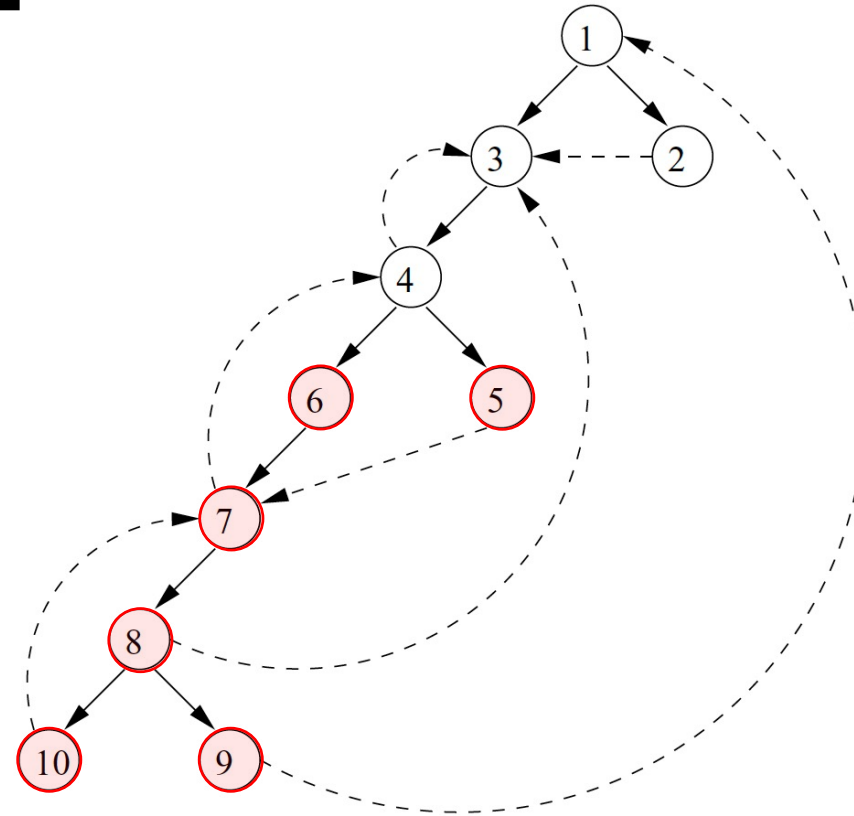
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



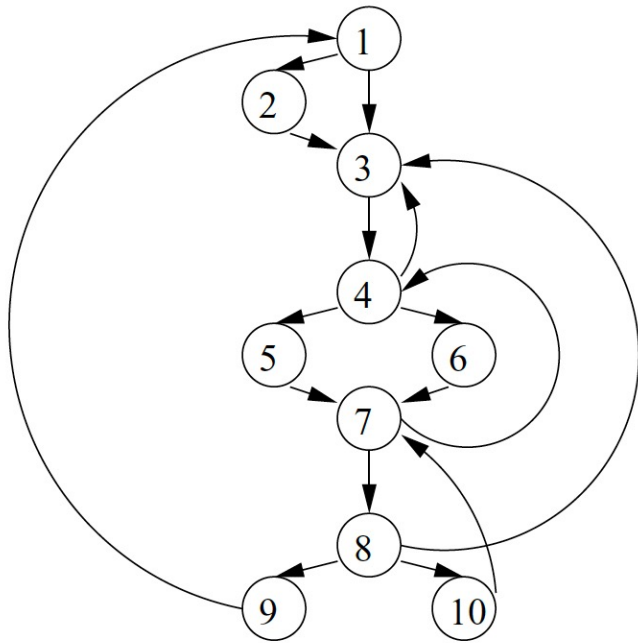
Control-Flow Graph



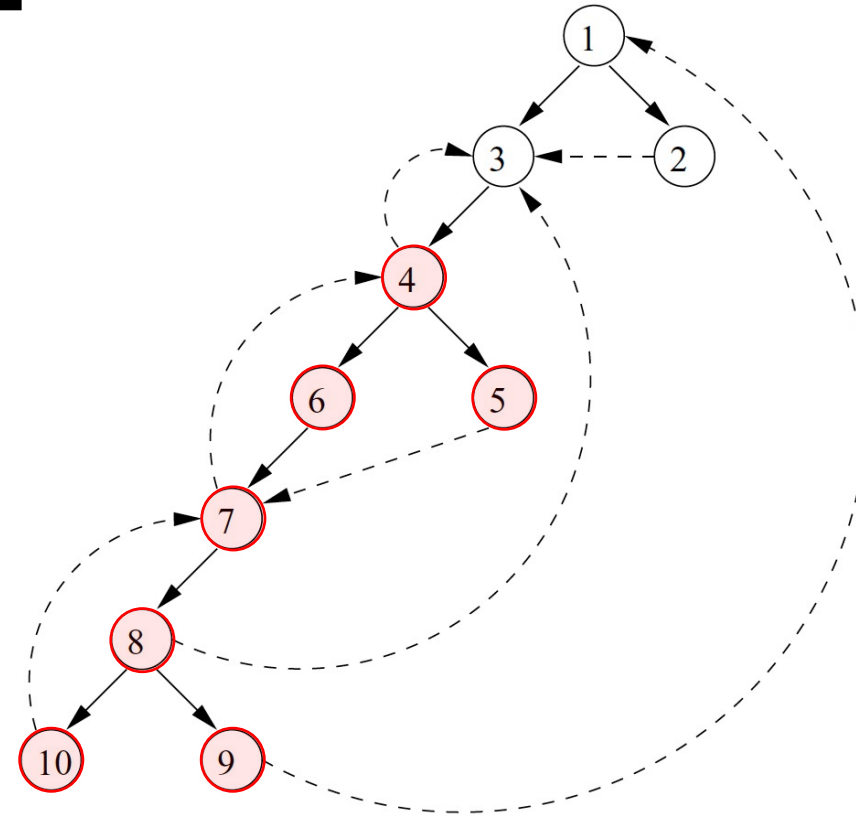
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



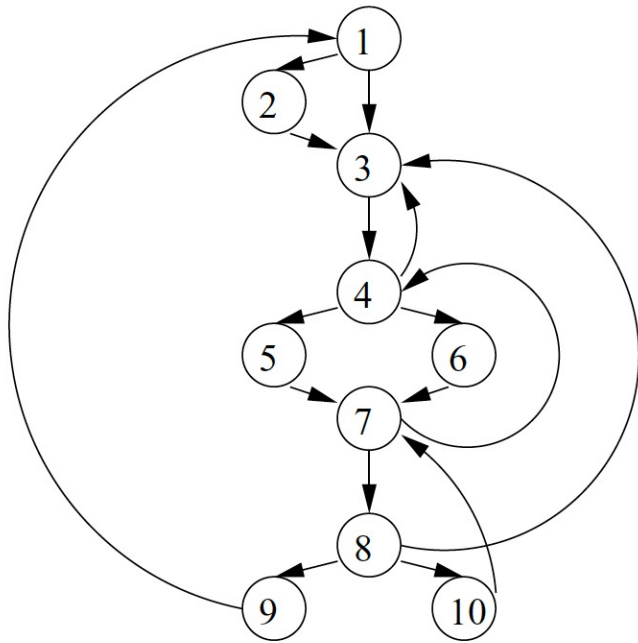
Control-Flow Graph



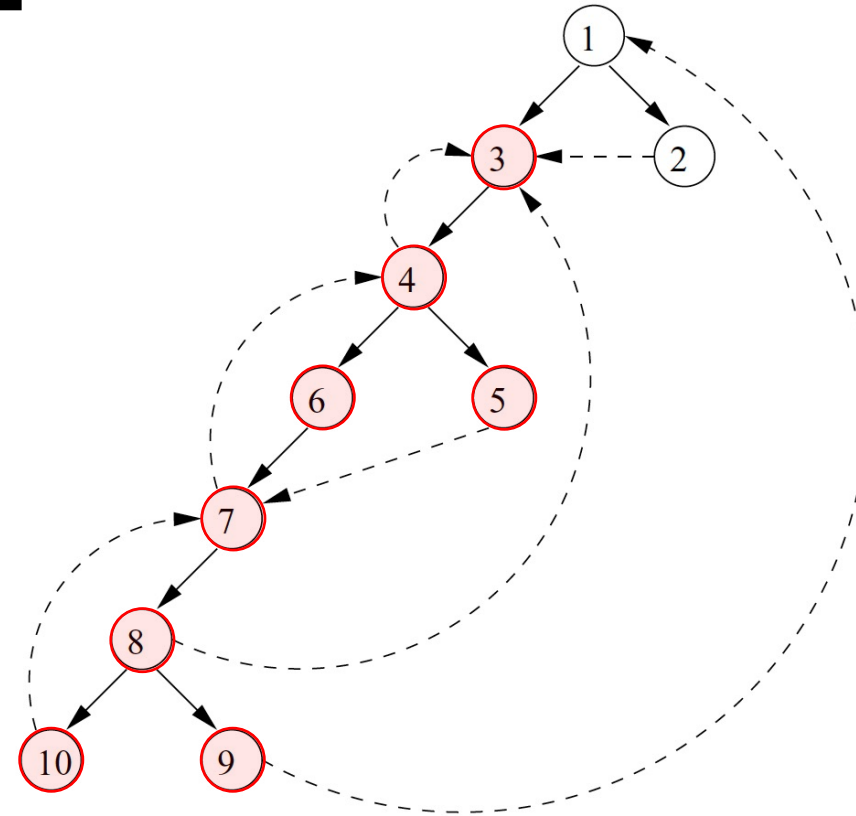
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



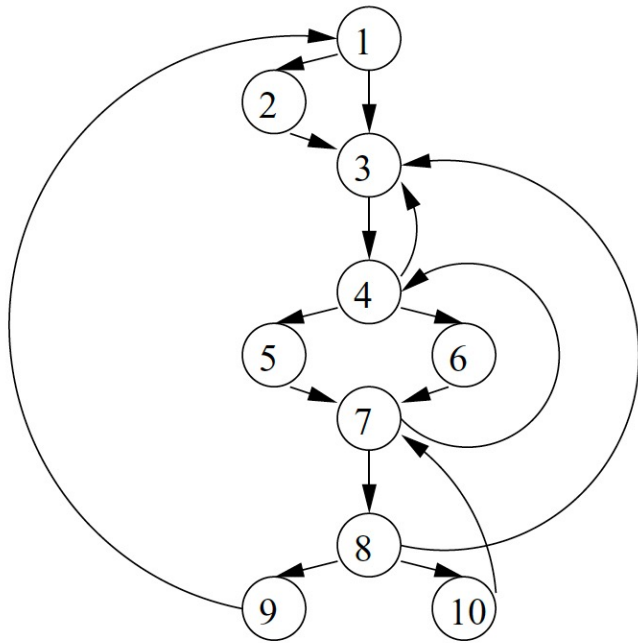
Control-Flow Graph



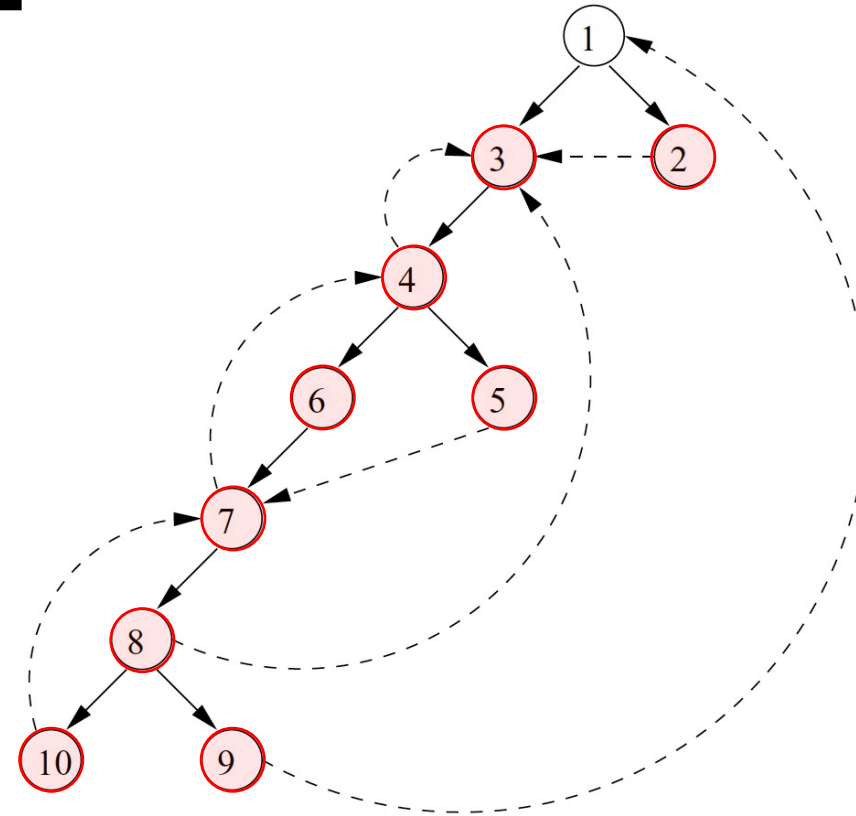
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



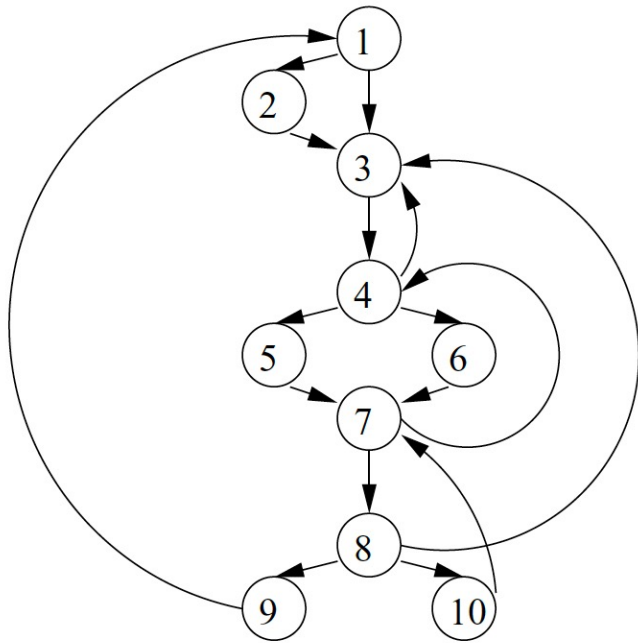
Control-Flow Graph



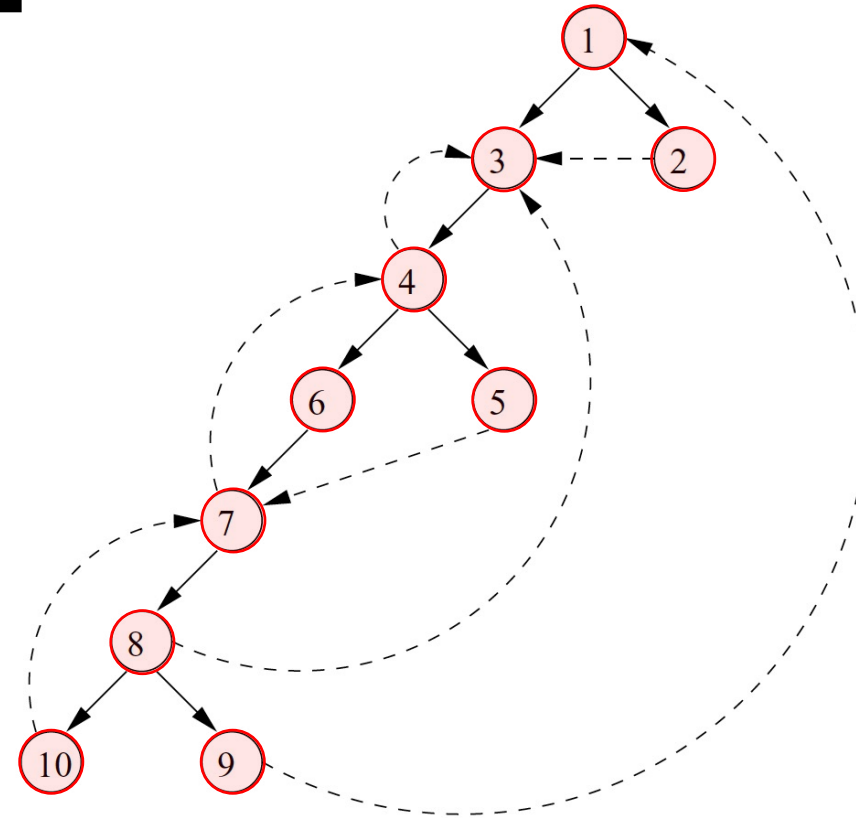
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



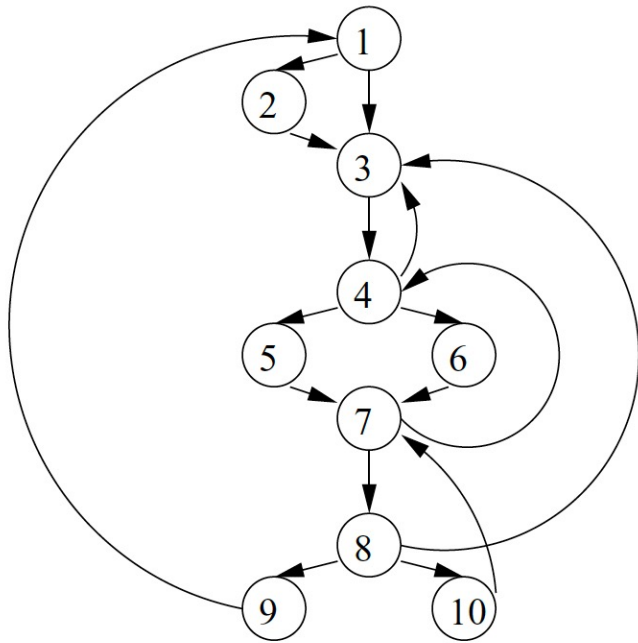
Control-Flow Graph



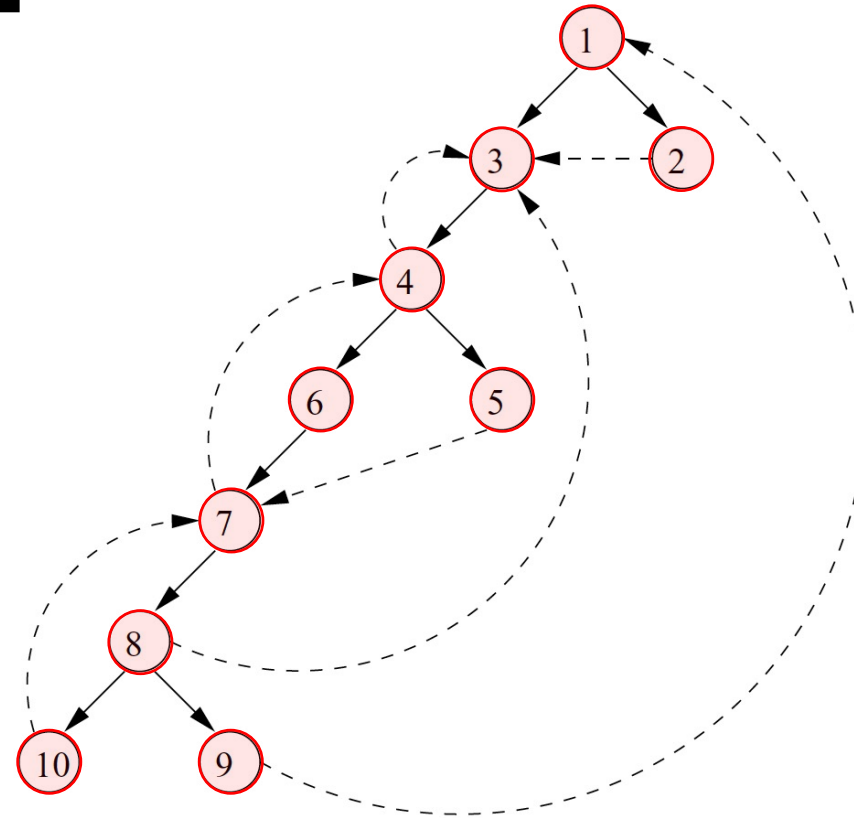
Depth-First Tree

Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Depth-First Order



Control-Flow Graph

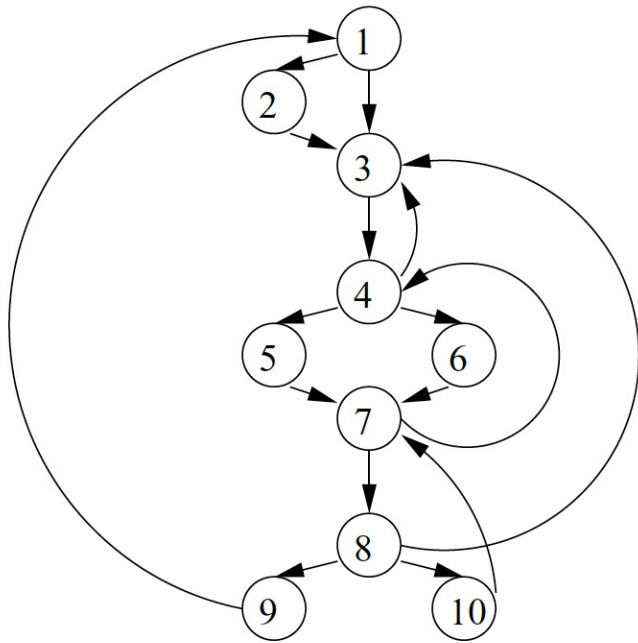


Depth-First Tree

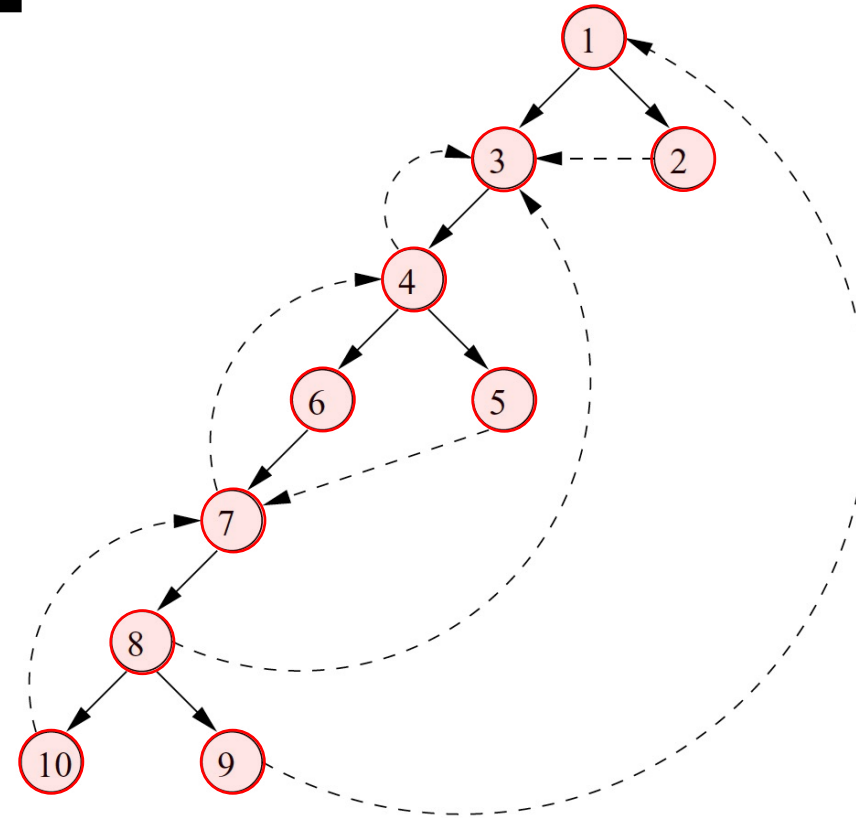
Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Post-Order of the Tree: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Depth-First Order



Control-Flow Graph



Depth-First Tree

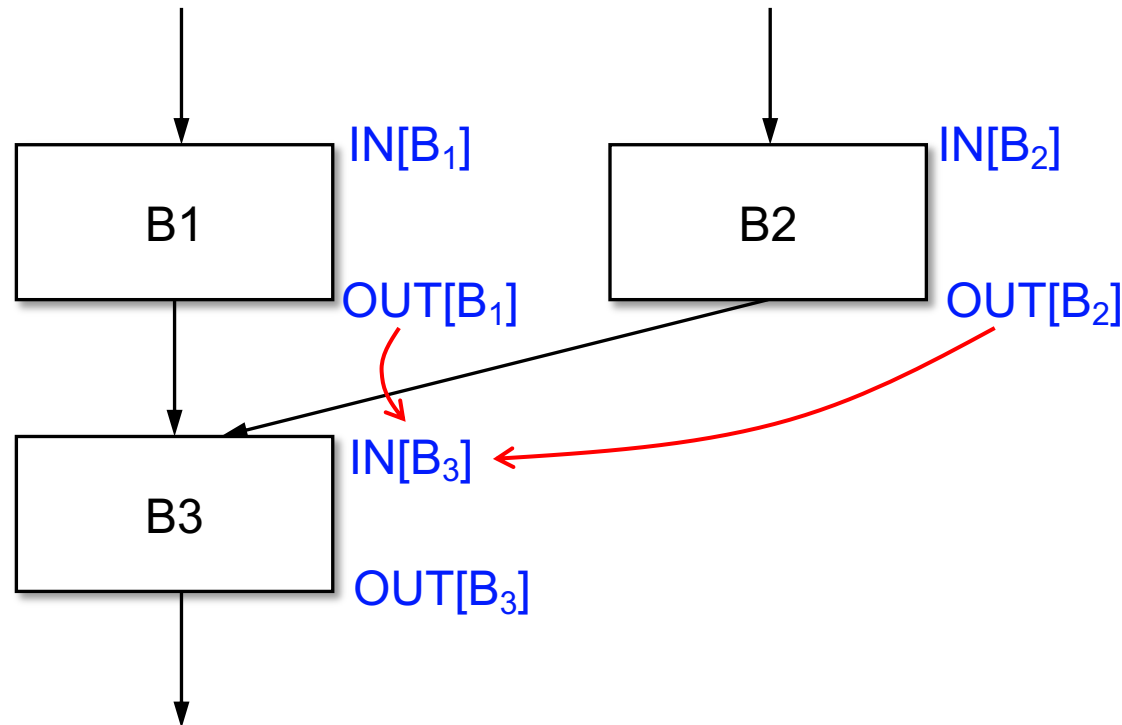
Depth-First Order is the Reversed Post-Order Sequence of the Depth-First Tree

Post-Order of the Tree: 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Depth-First Order: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

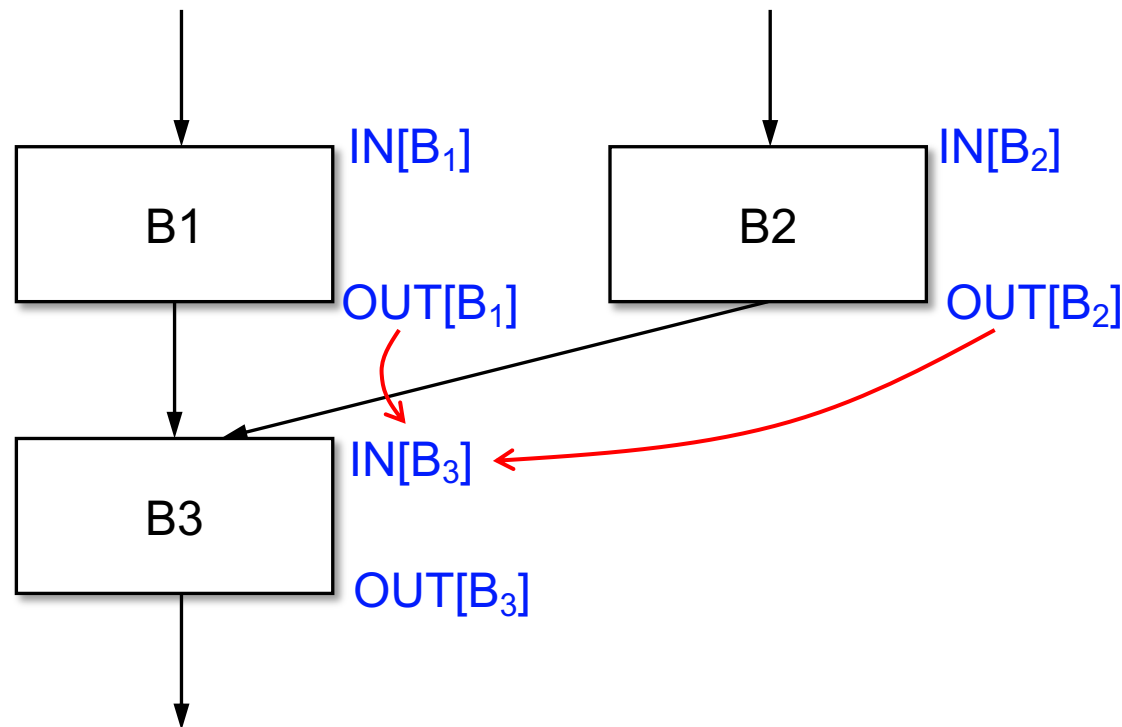
Depth-First Order

- It is exactly the topological order when the graph is acyclic



Depth-First Order

- It is exactly the topological order when the graph is acyclic
- It provides a topo-like order for cyclic graphs



Question-II

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

Can the loop terminate?

- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]$; $OUT[B] = f_B(IN[B])$
 - **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Question-II

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;

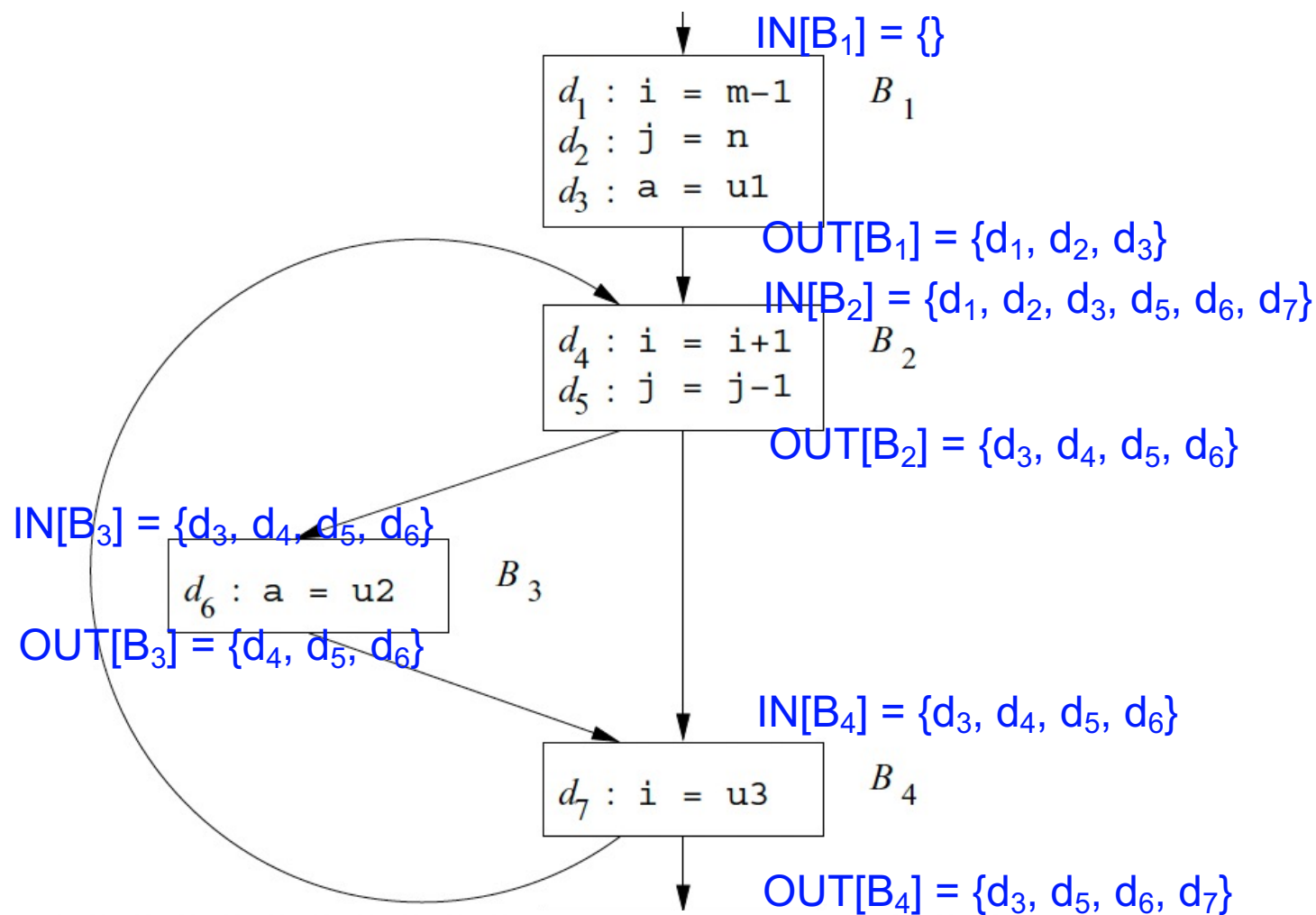
Can the loop terminate?

- **While** (!worklist.empty()) **Do**

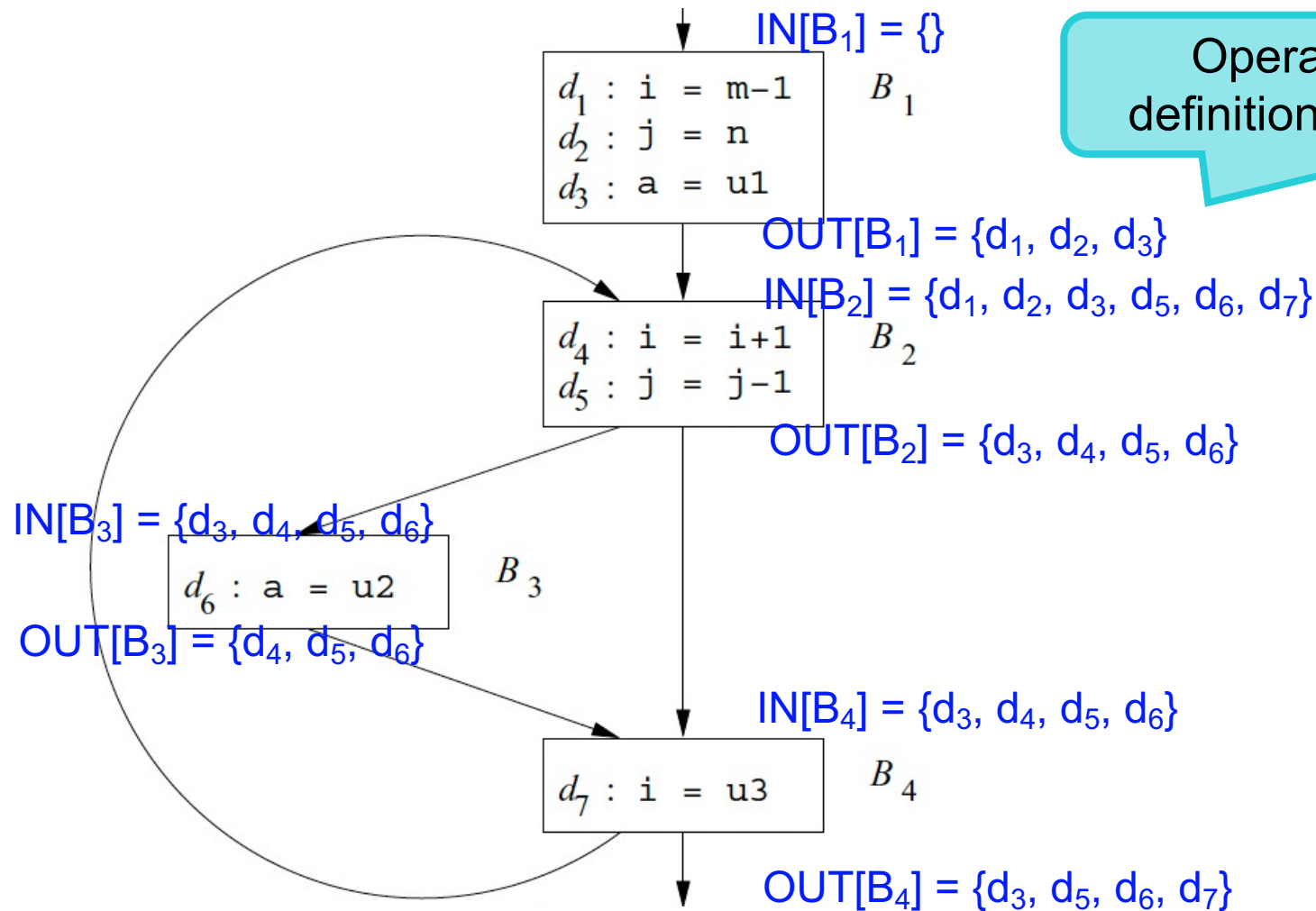
Terminable if for each block B:
IN[B] and OUT[B] is **monotonically**
increased or decreased to a limit.

- B = worklist.pop();
- $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
- **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

Recall: Reaching Definitions

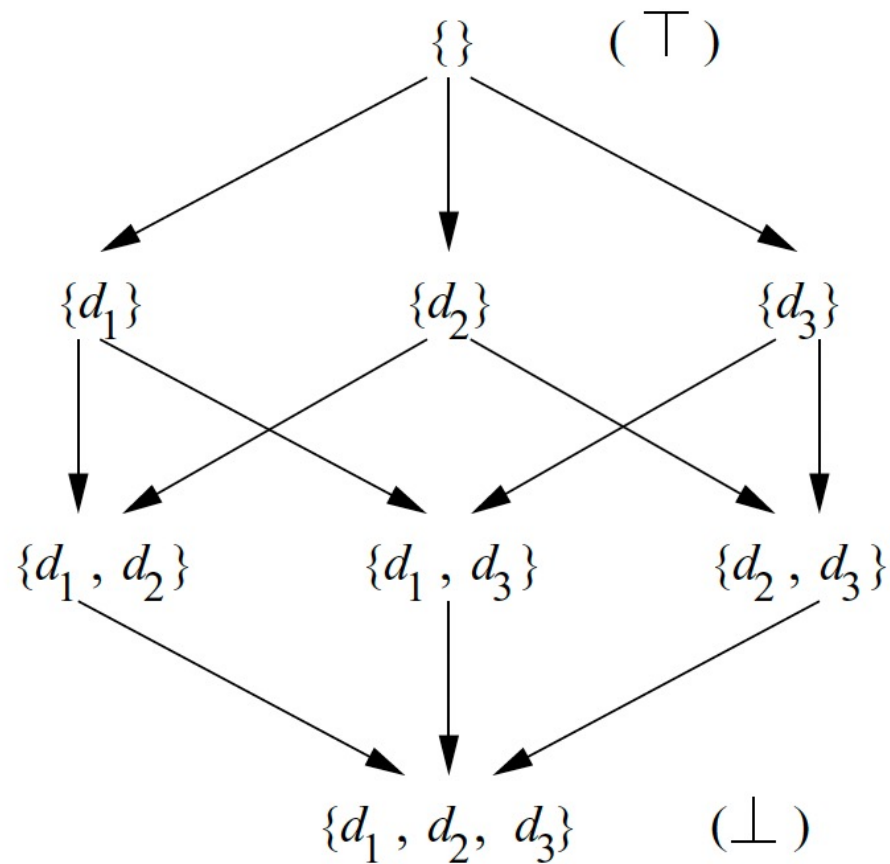


Recall: Reaching Definitions

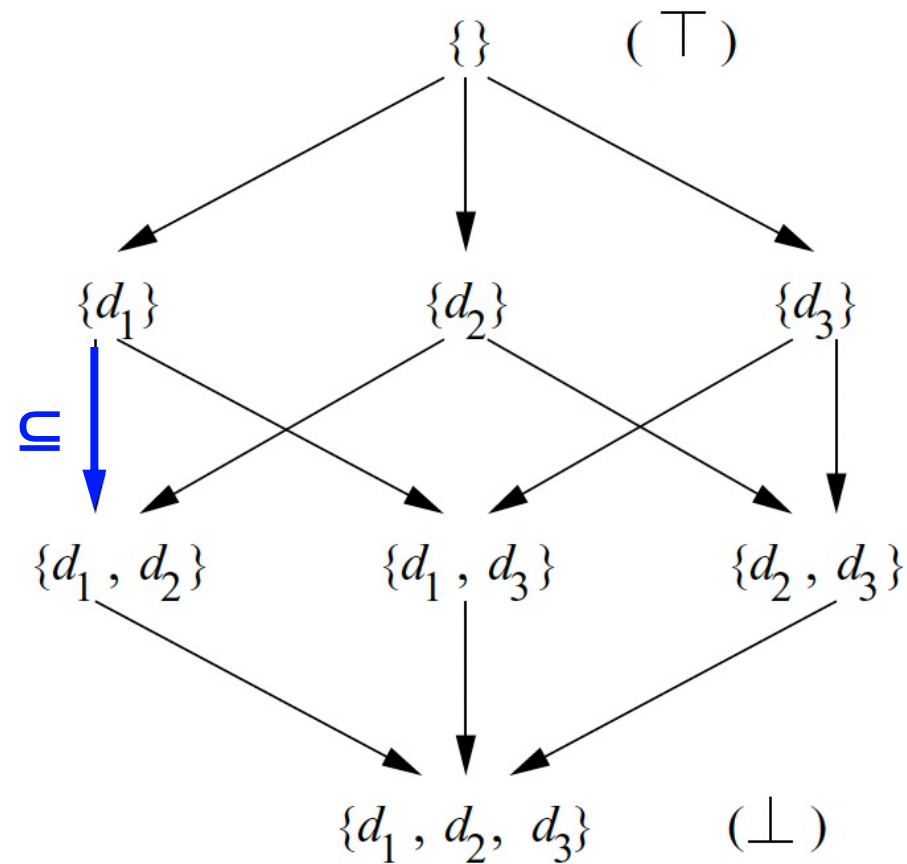


Operations over subsets of definitions, forming a semi-lattice

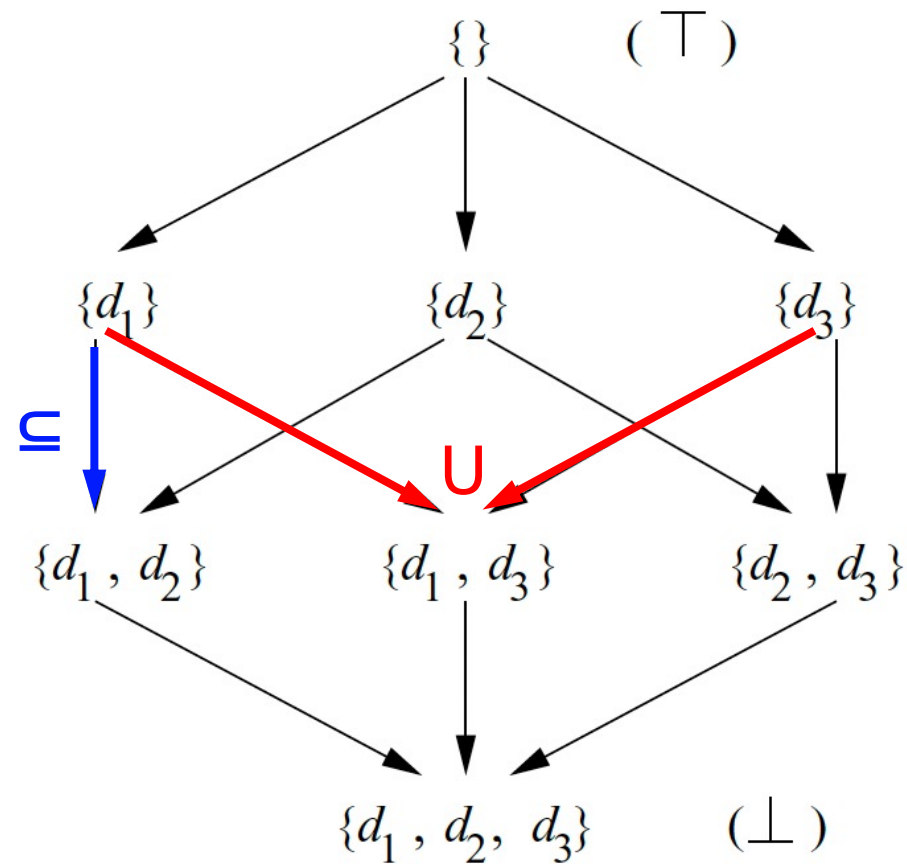
Semi-Lattice



Semi-Lattice

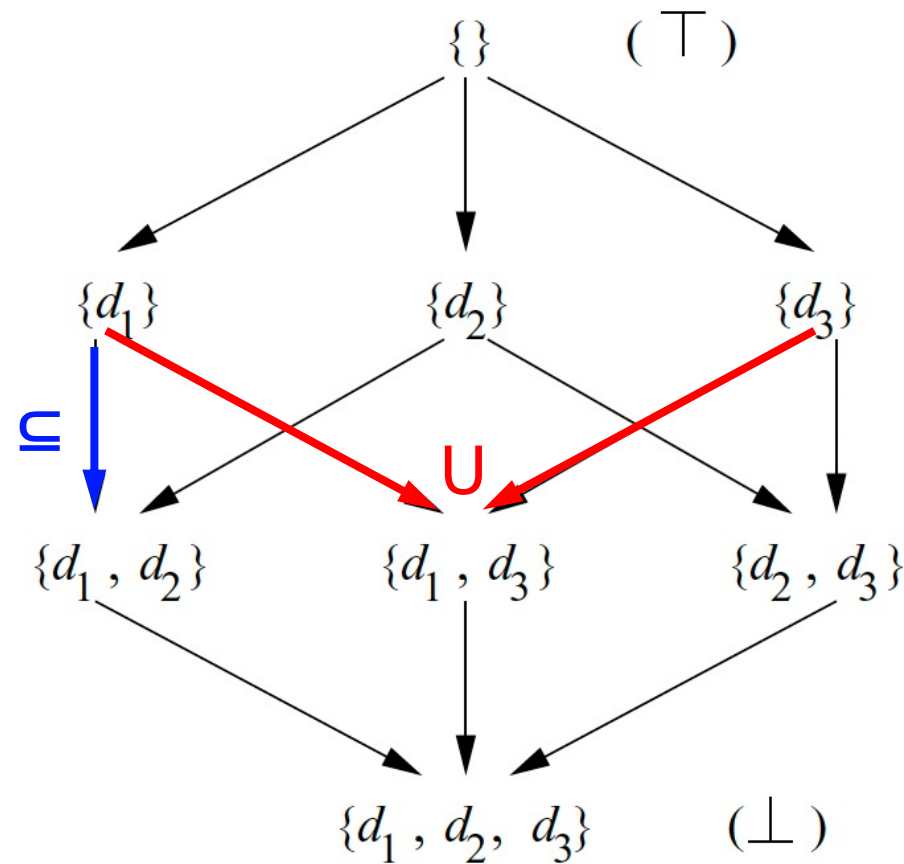


Semi-Lattice



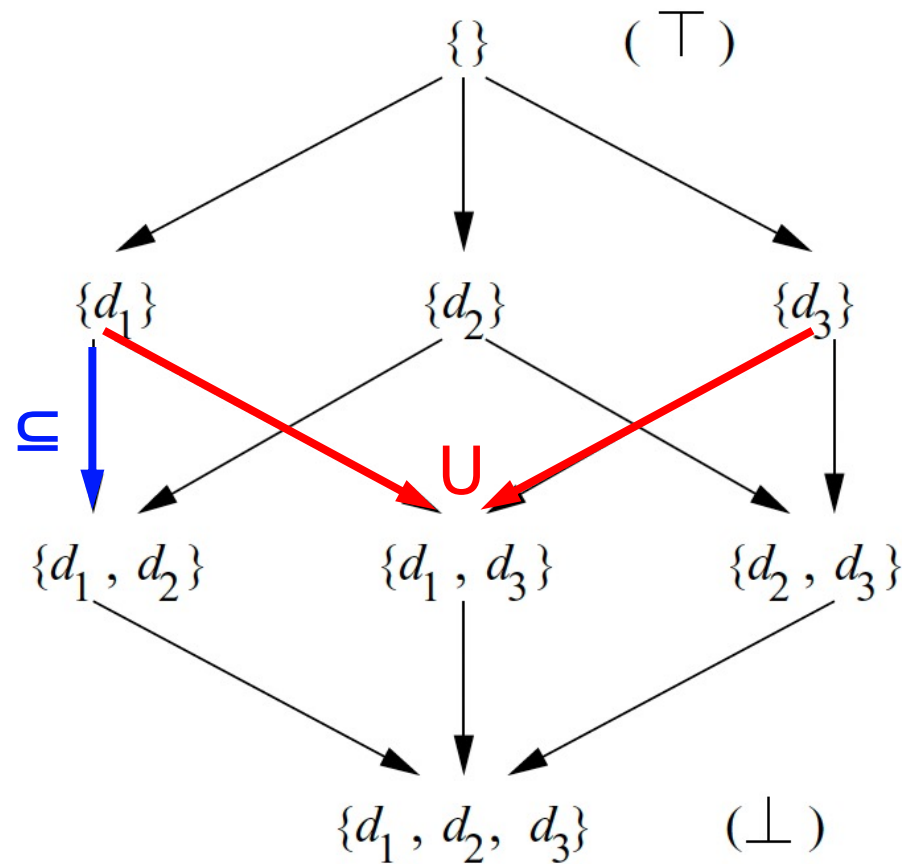
Semi-Lattice

- A semilattice is a set V with a meet operation $\wedge$, such that



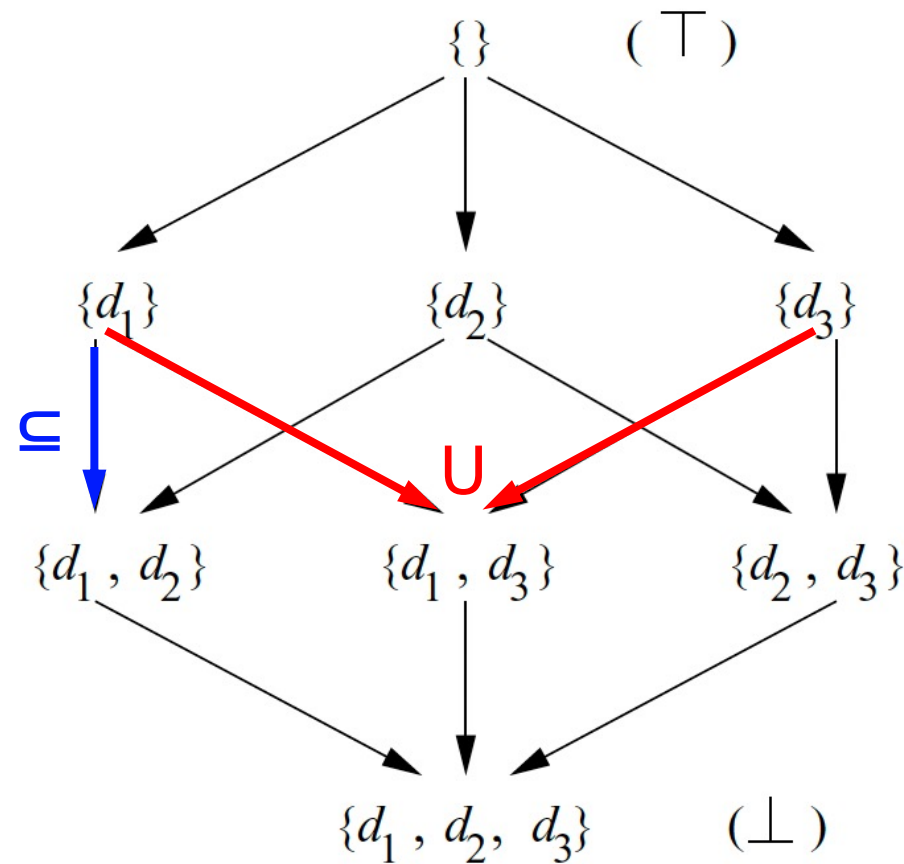
Semi-Lattice

- A semilattice is a set V with a meet operation $\wedge$, such that
- (1) $x \wedge x = x$
- (2) $x \wedge y = y \wedge x$
- (3) $x \wedge (y \wedge z) = (x \wedge y) \wedge z$
- (4) $x \wedge \top = x$
- (5) $\perp \wedge x = \perp$



Semi-Lattice & Monotonicity

- Data Flow Equations
 - $IN[B] = \bigcup_{P \in \text{preds}(B)} OUT[P]$
 - $OUT[B] = f_B(IN[B])$

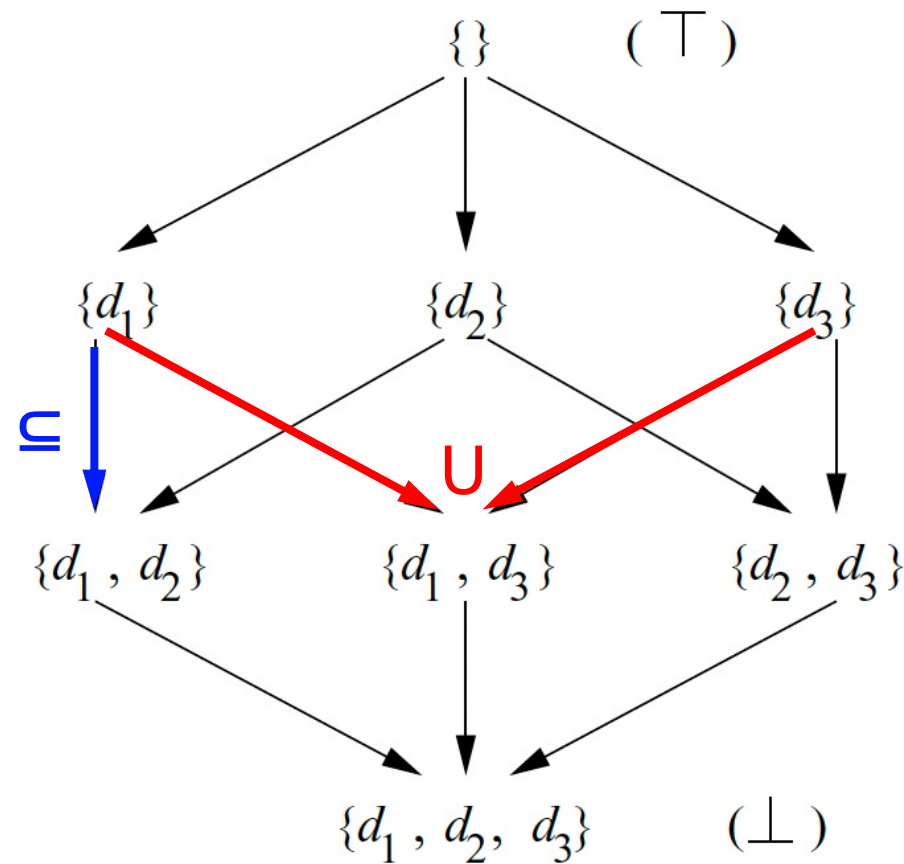


Semi-Lattice & Monotonicity

- Data Flow Equations

- $$\text{IN}[B] = \bigcup_{P \in \text{preds}(B)} \text{OUT}[P]$$

- $$\text{OUT}[B] = f_B(\text{IN}[B])$$

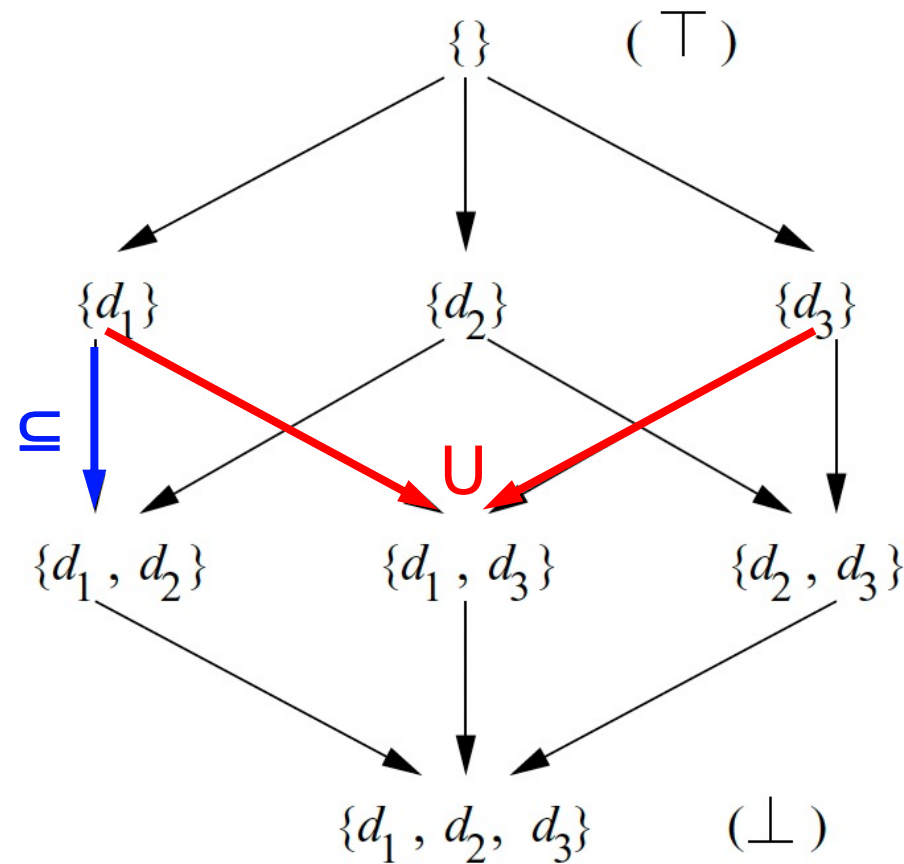


Semi-Lattice & Monotonicity

- Data Flow Equations

- $$\text{IN}[B] = \bigcup_{P \in \text{preds}(B)} \text{OUT}[P]$$

- $$\text{OUT}[B] = f_B(\text{IN}[B])$$

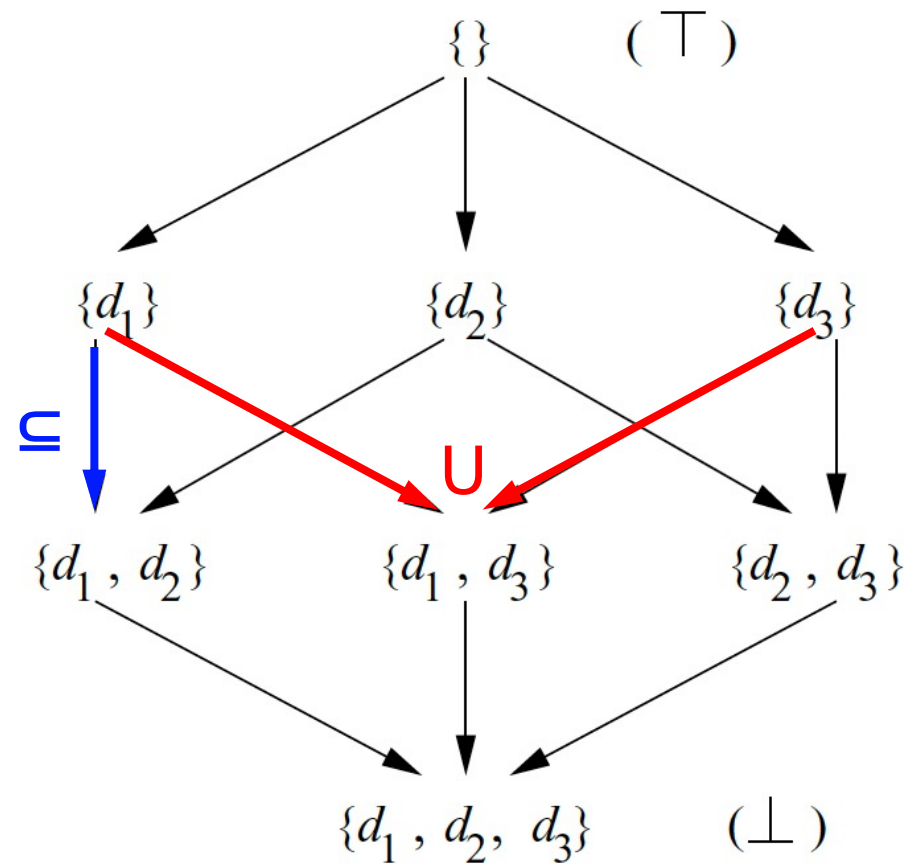


Semi-Lattice & Monotonicity

- Data Flow Equations

- $IN[B] = \bigcup_{P \in \text{preds}(B)} OUT[P]$

- $OUT[B] = f_B(IN[B])$

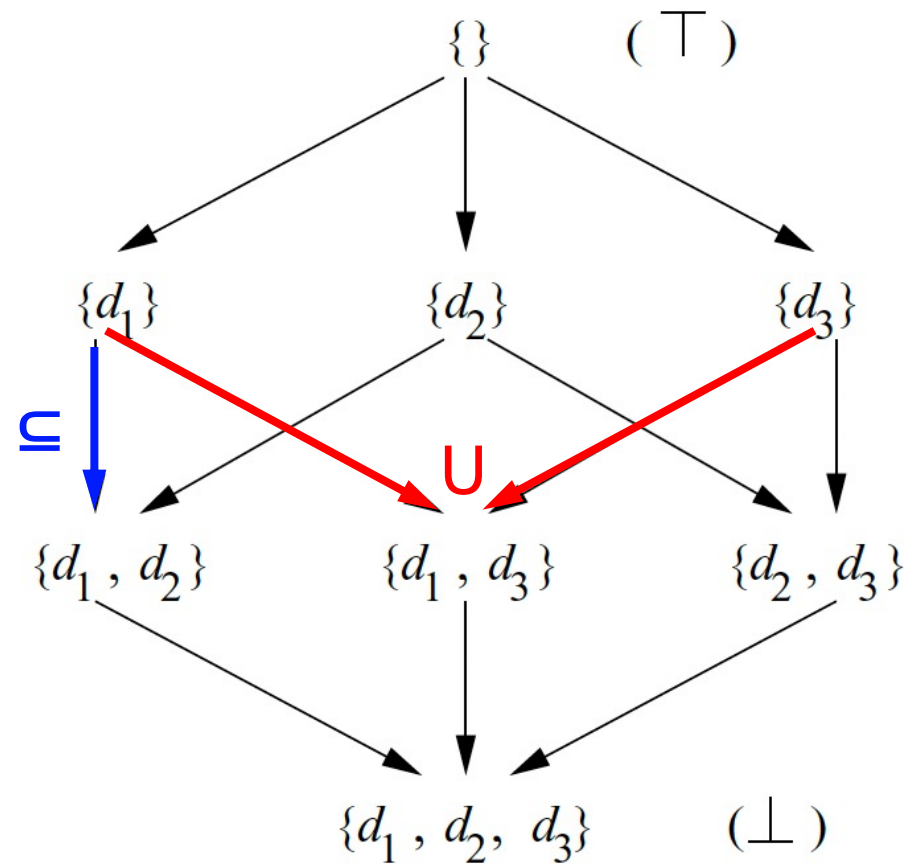


Semi-Lattice & Monotonicity

- Data Flow Equations

- $$\text{IN}[B] = \bigcup_{P \in \text{preds}(B)} \text{OUT}[P]$$

- $$\text{OUT}[B] = f_B(\text{IN}[B])$$



Semi-Lattice & Monotonicity

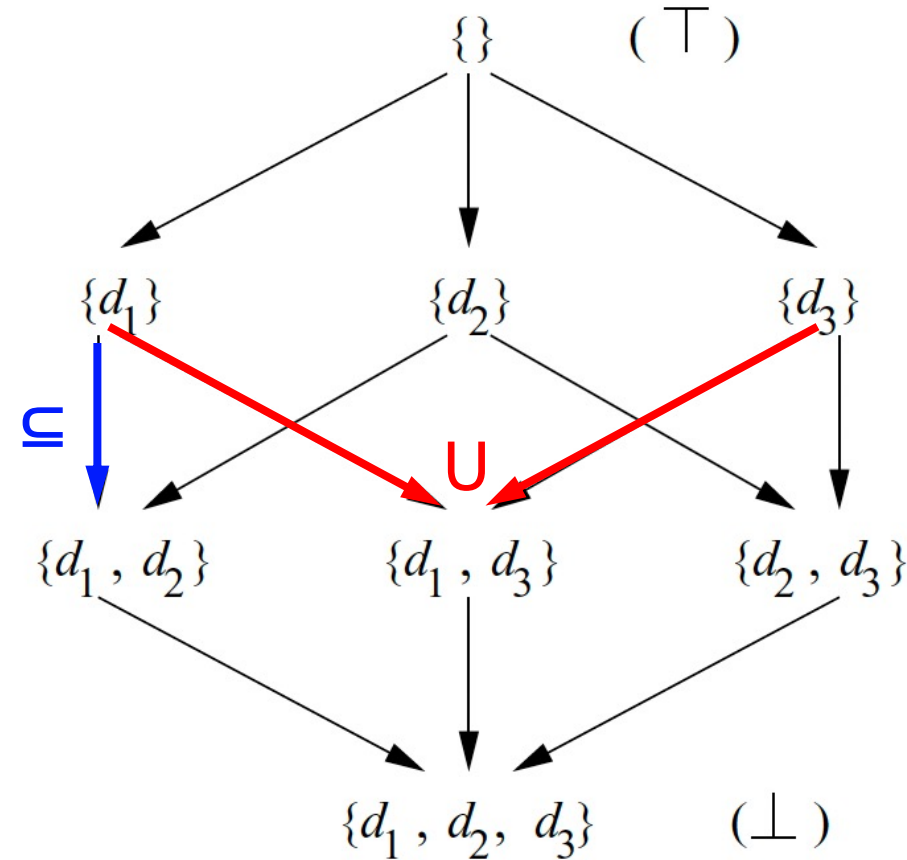
- Data Flow Equations

- $IN[B] = \bigcup_{P \in \text{preds}(B)} OUT[P]$ ↗

- $OUT[B] = f_B(IN[B])$ ↗?

Monotonicity:

- $x \subseteq y$
- $\Rightarrow \text{gen} \cup (x - \text{kill}) \subseteq \text{gen} \cup (y - \text{kill})$
- $\Rightarrow f_B(x) \subseteq f_B(y)$



Semi-Lattice & Monotonicity

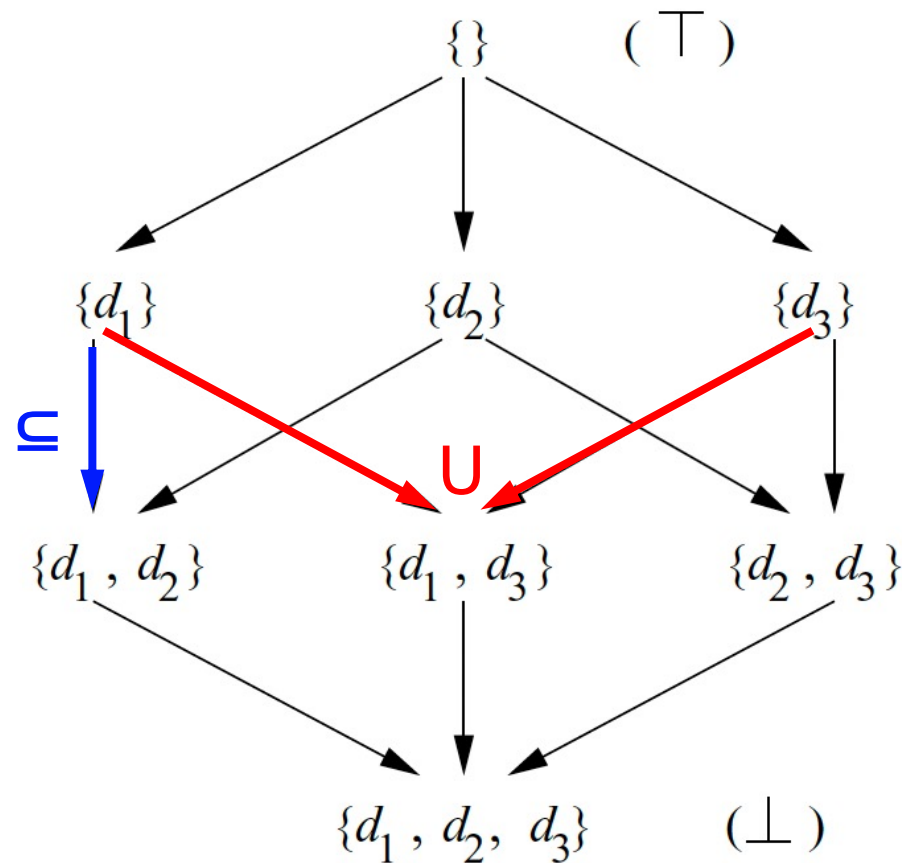
- Data Flow Equations

- $IN[B] = \bigcup_{P \in \text{preds}(B)} OUT[P]$ ↗

- $OUT[B] = f_B(IN[B])$ ↗

Monotonicity:

- $x \subseteq y$
- $\Rightarrow \text{gen} \cup (x - \text{kill}) \subseteq \text{gen} \cup (y - \text{kill})$
- $\Rightarrow f_B(x) \subseteq f_B(y)$



Semi-Lattice & Monotonicity

- Data Flow Equations

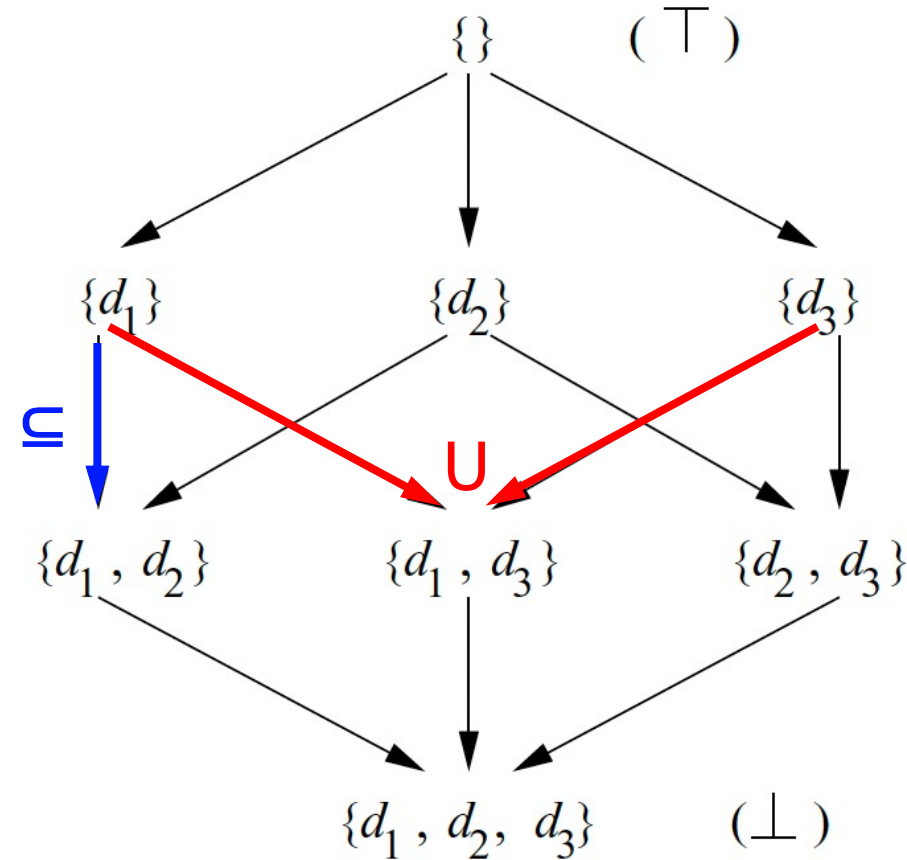
- $IN[B] = \bigcup_{P \in \text{preds}(B)} OUT[P]$ ↗

- $OUT[B] = f_B(IN[B])$ ↗

Monotonicity:

- $x \subseteq y$
- $\Rightarrow \text{gen} \cup (x - \text{kill}) \subseteq \text{gen} \cup (y - \text{kill})$
- $\Rightarrow f_B(x) \subseteq f_B(y)$

The semi-lattice has a finite height.



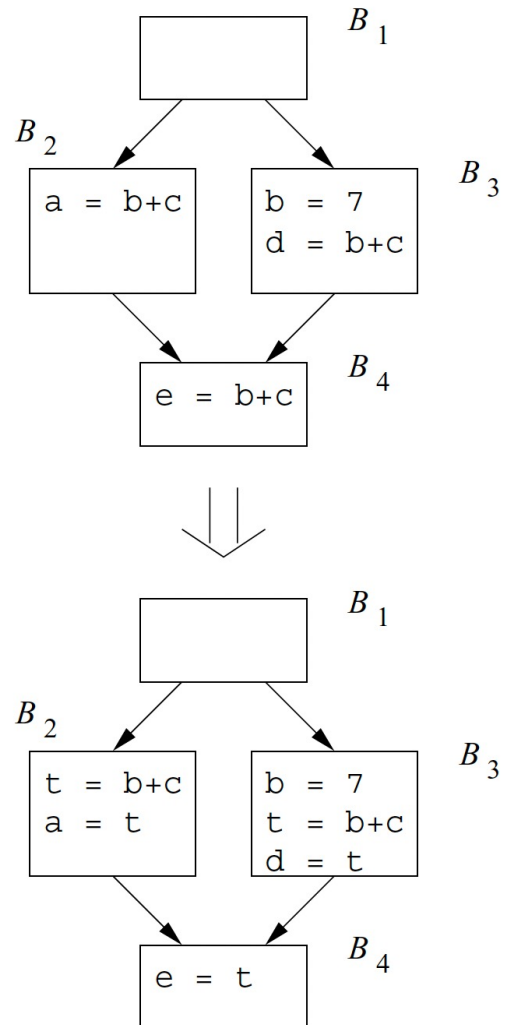
Question-II

- **ForEach** basic block B: initialize IN[B] and OUT[B]; **EndFor**
- worklist = set of all basic blocks;
- **While** (!worklist.empty()) **Do**
 - B = worklist.pop();
 - $IN[B] = \bigwedge_{P \in \text{preds}(B)} OUT[P]; \quad OUT[B] = f_B(IN[B])$
 - **If** OUT[B] changed: worklist.push(all B's successors); **EndIf**
- **EndWhile**

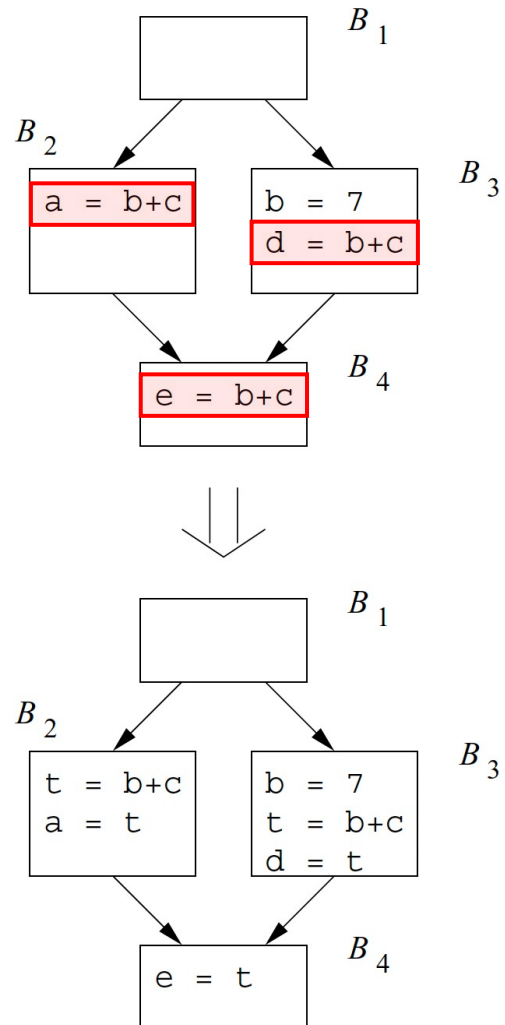
If the semilattice of the framework is monotone and of finite height, then the algorithm is guaranteed to converge.

PART II: Partial Redundancy

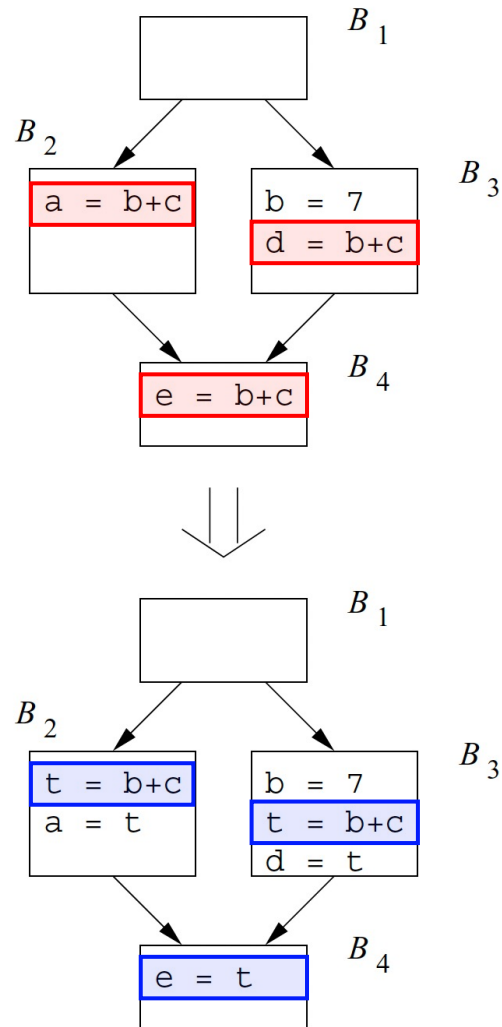
Sources of Redundancy



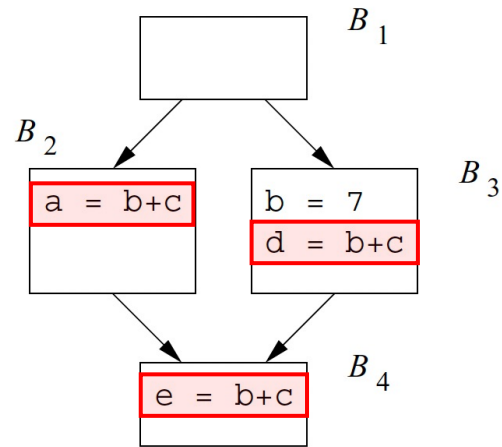
Sources of Redundancy



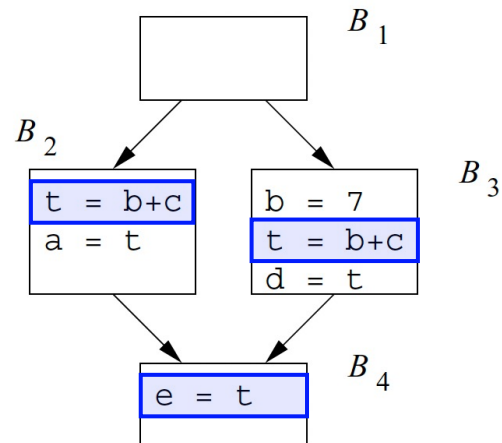
Sources of Redundancy



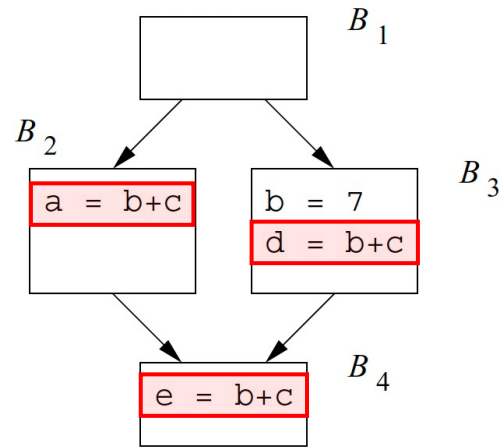
Sources of Redundancy



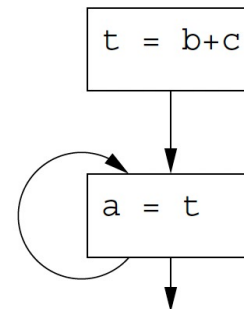
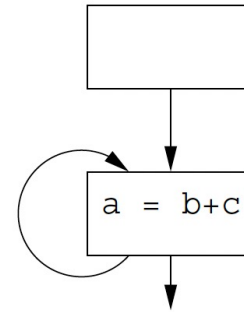
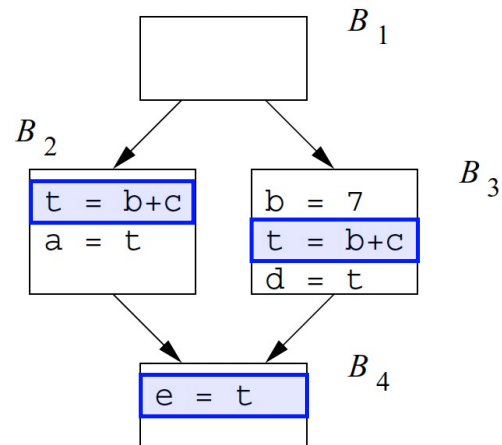
1. Common Expression



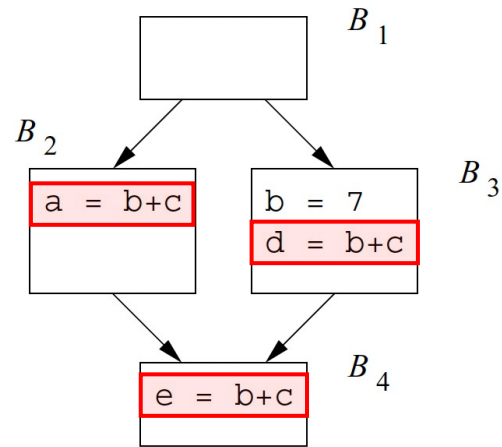
Sources of Redundancy



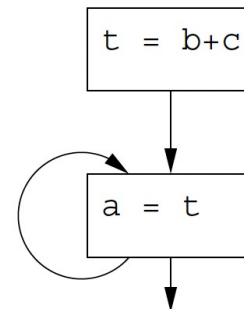
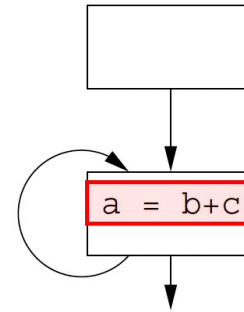
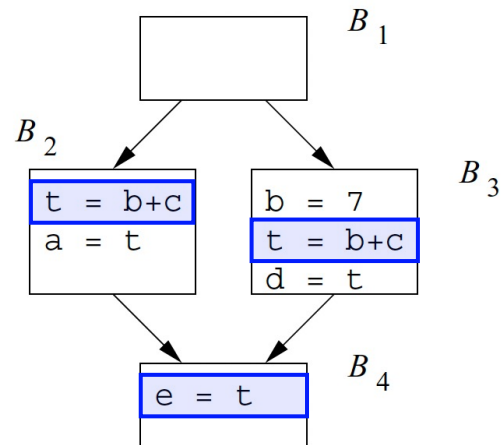
1. Common Expression



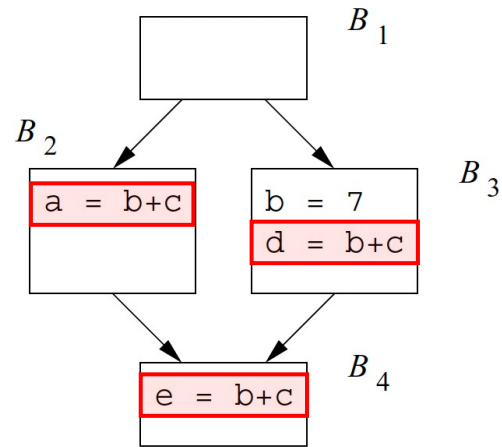
Sources of Redundancy



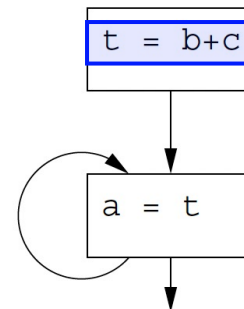
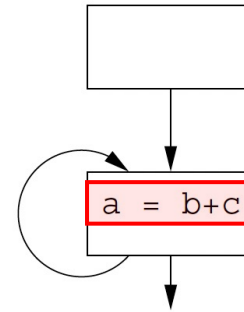
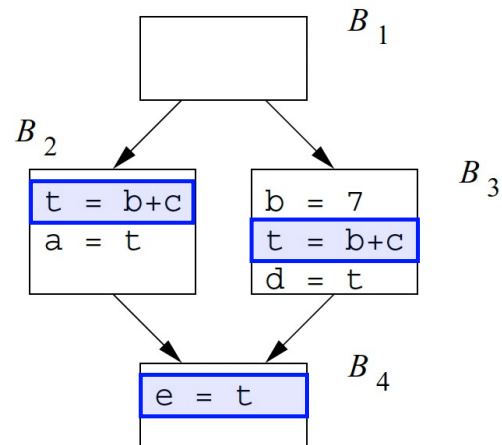
1. Common Expression



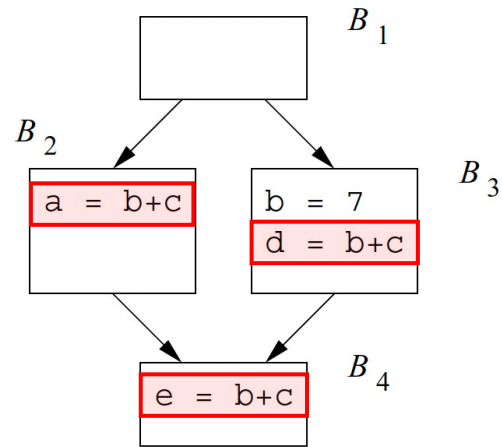
Sources of Redundancy



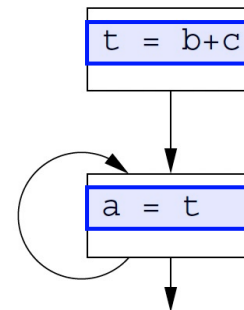
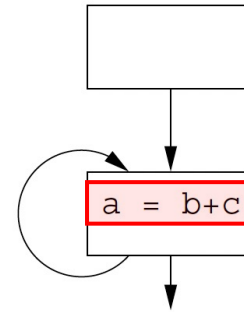
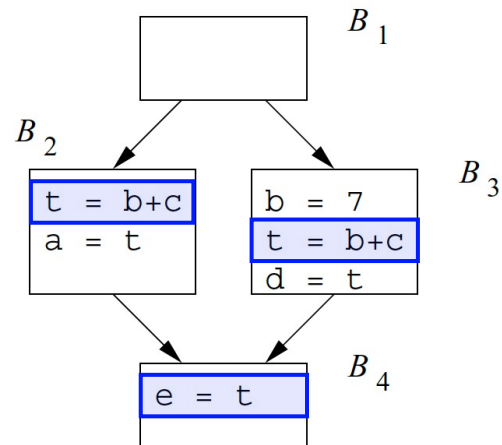
1. Common Expression



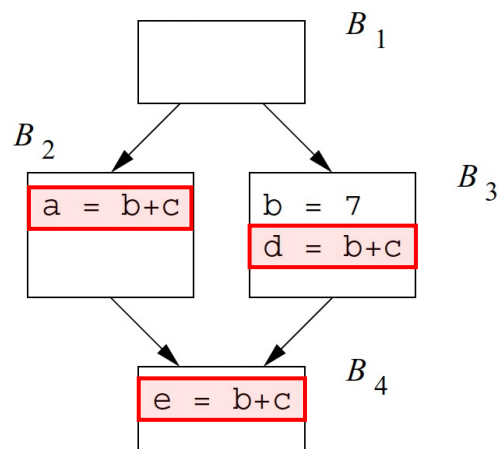
Sources of Redundancy



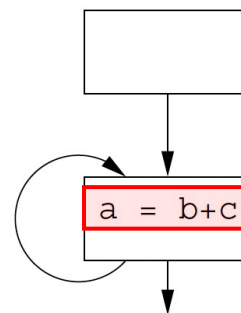
1. Common Expression



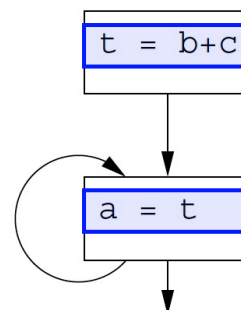
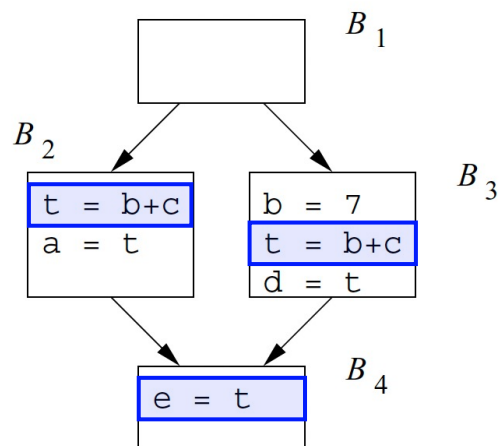
Sources of Redundancy



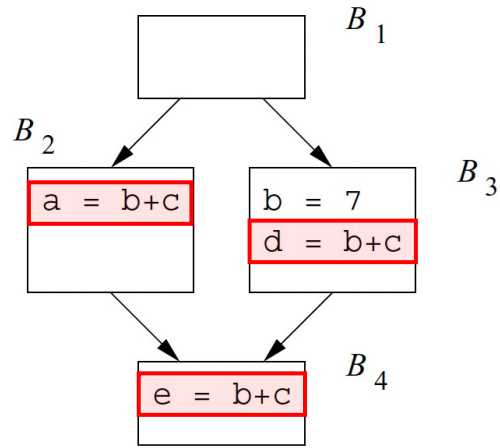
1. Common Expression



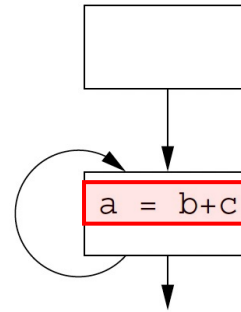
2. Loop Invariant Motion



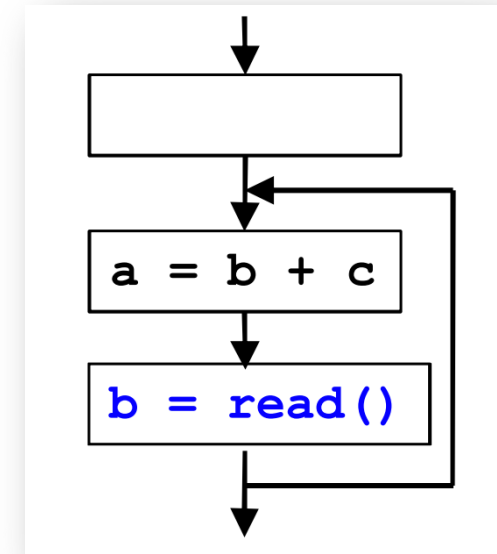
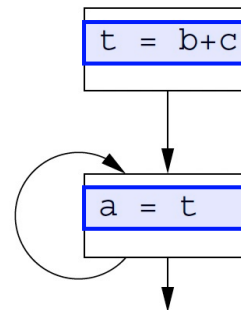
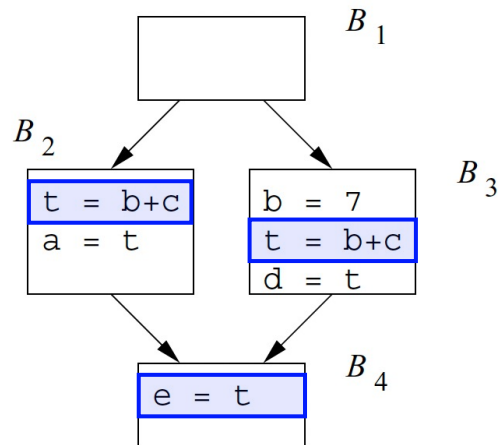
Sources of Redundancy



1. Common Expression

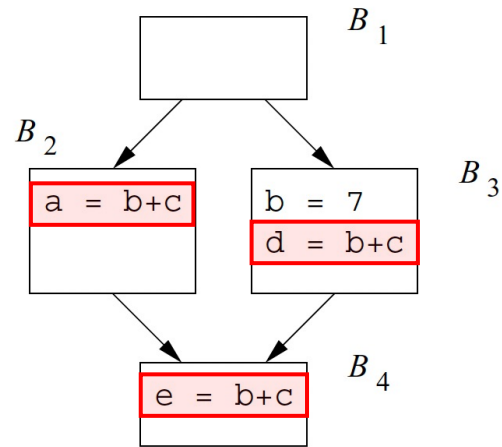


2. Loop Invariant Motion

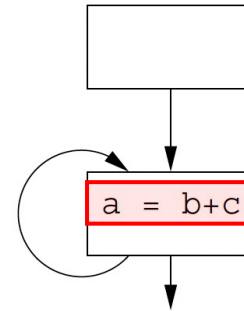


Can `b+c` be moved out?

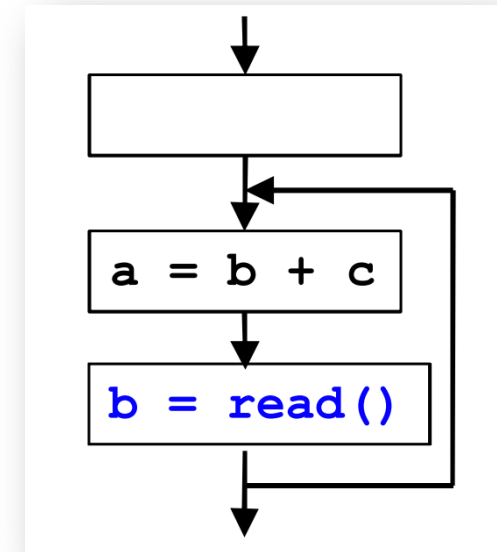
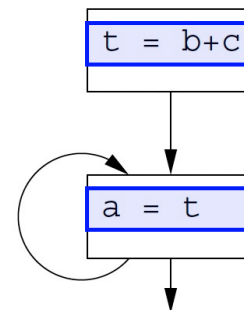
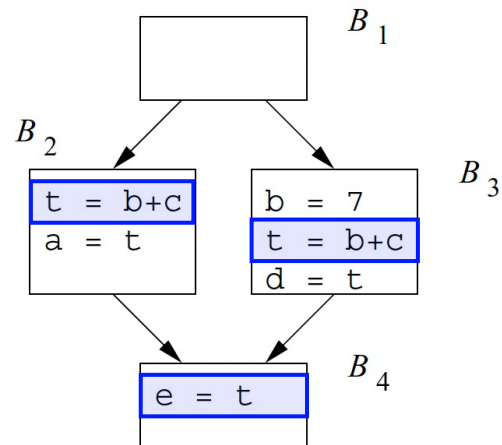
Sources of Redundancy



1. Common Expression



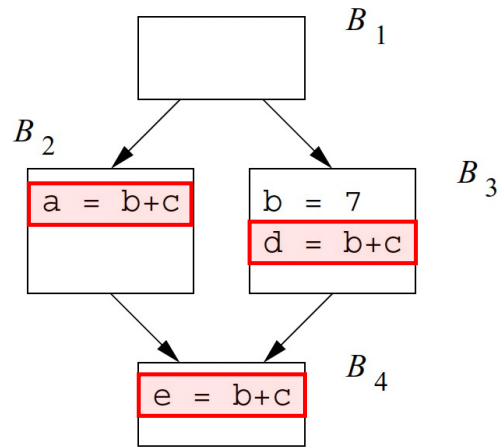
2. Loop Invariant Motion



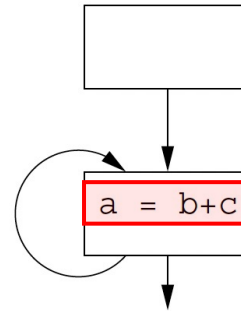
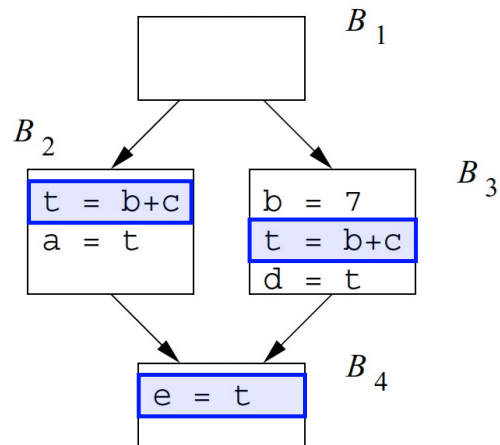
Can `b+c` be moved out?

No!

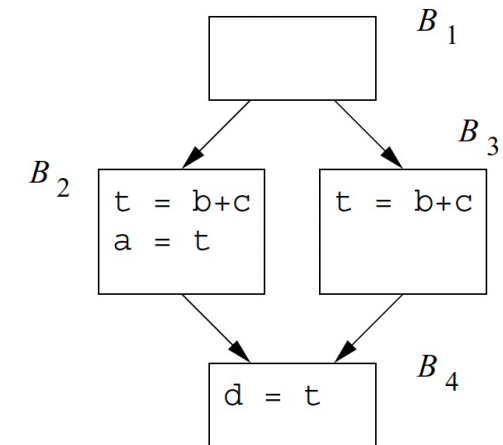
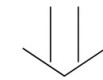
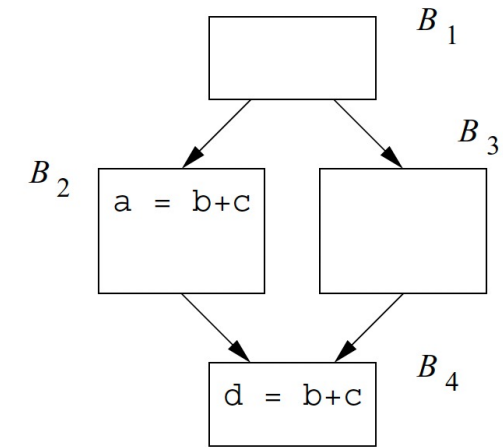
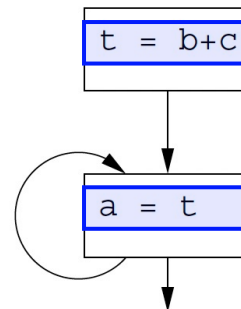
Sources of Redundancy



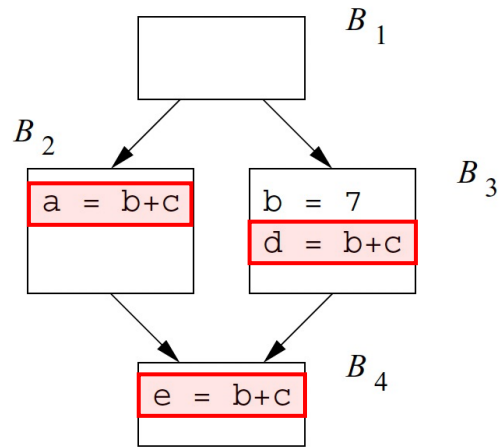
1. Common Expression



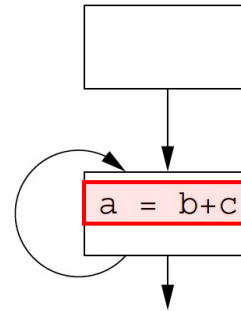
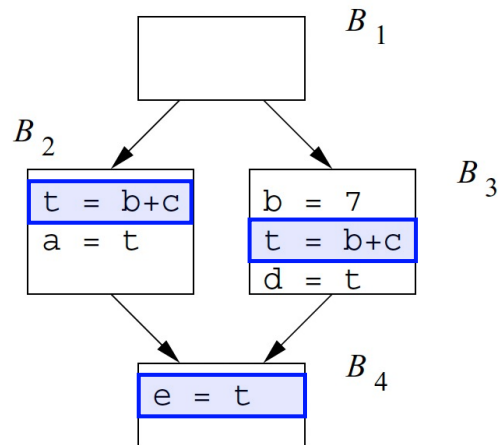
2. Loop Invariant Motion



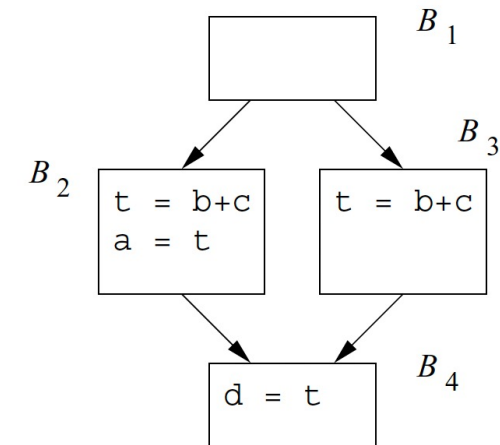
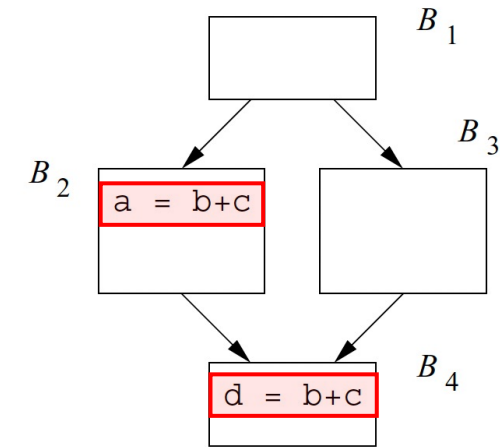
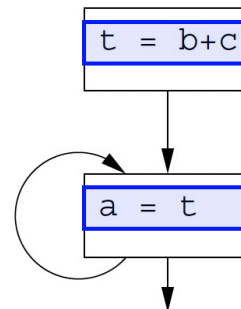
Sources of Redundancy



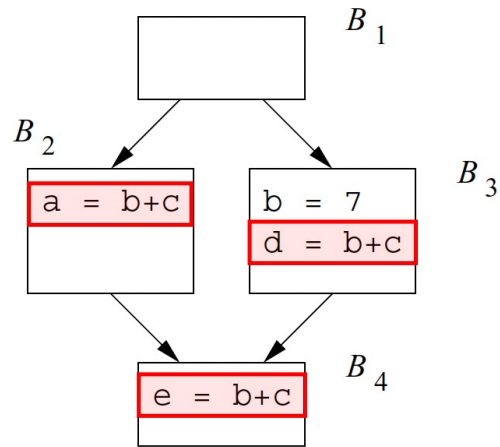
1. Common Expression



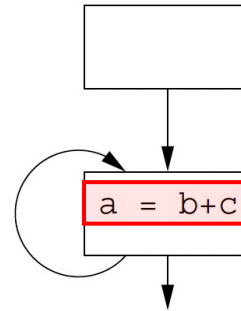
2. Loop Invariant Motion



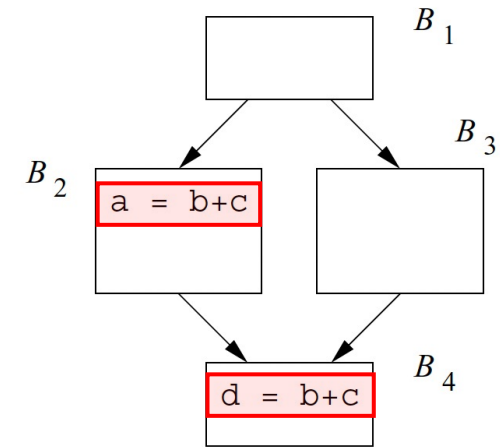
Sources of Redundancy



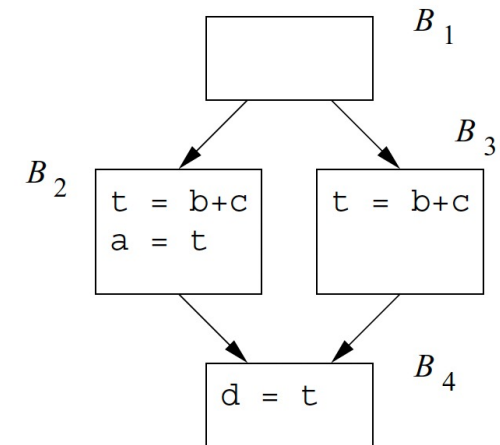
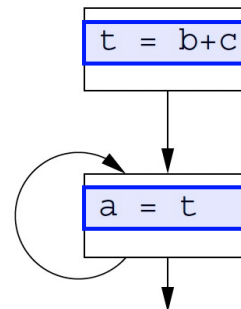
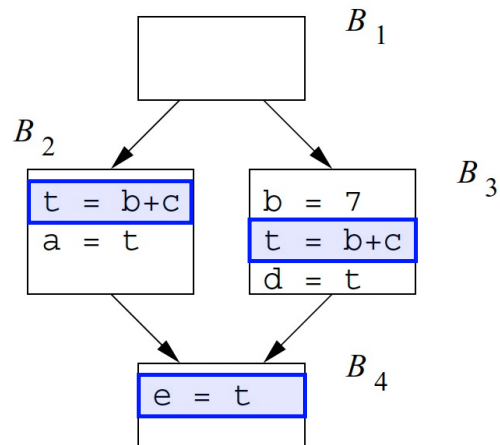
1. Common Expression



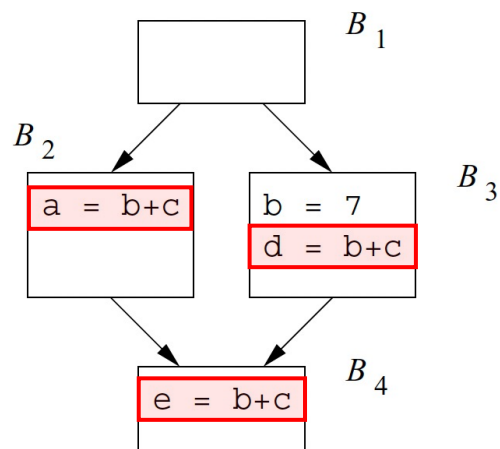
2. Loop Invariant Motion



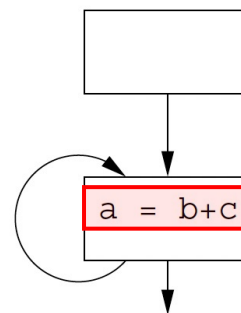
3. Partial Redundancy



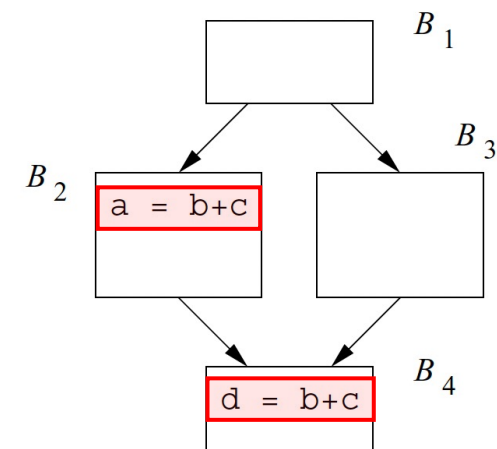
Sources of Redundancy



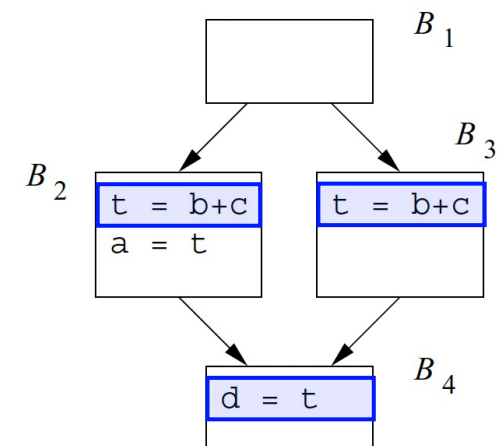
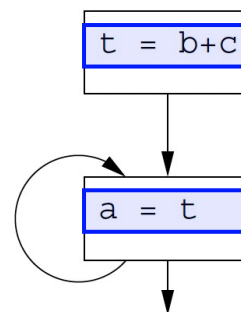
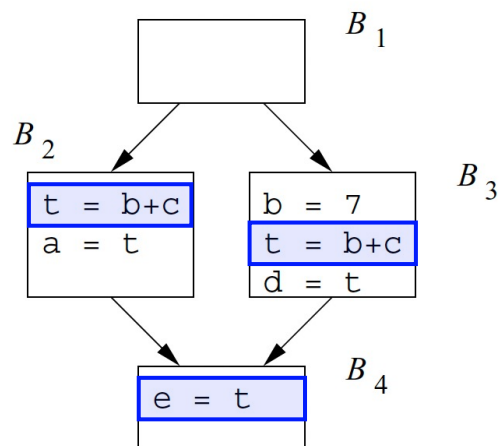
1. Common Expression



2. Loop Invariant Motion



3. Partial Redundancy



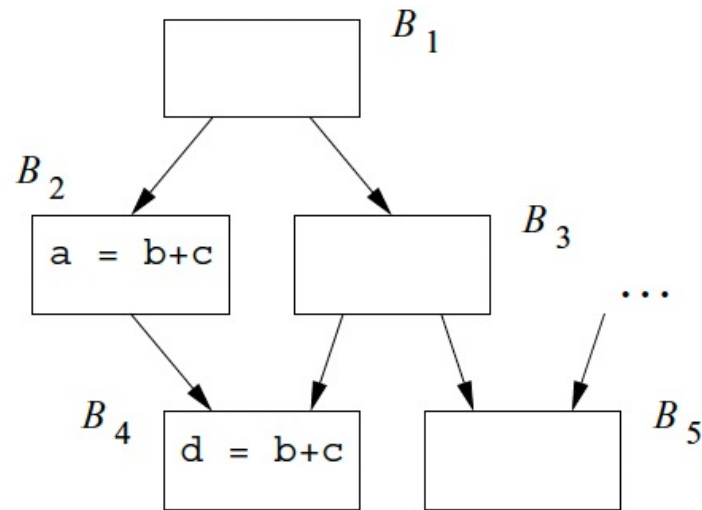
Eliminating All Redundancy?

Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.

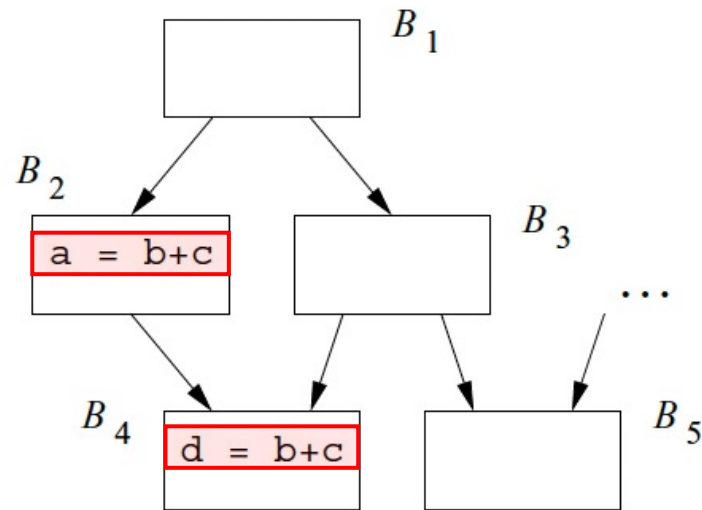
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



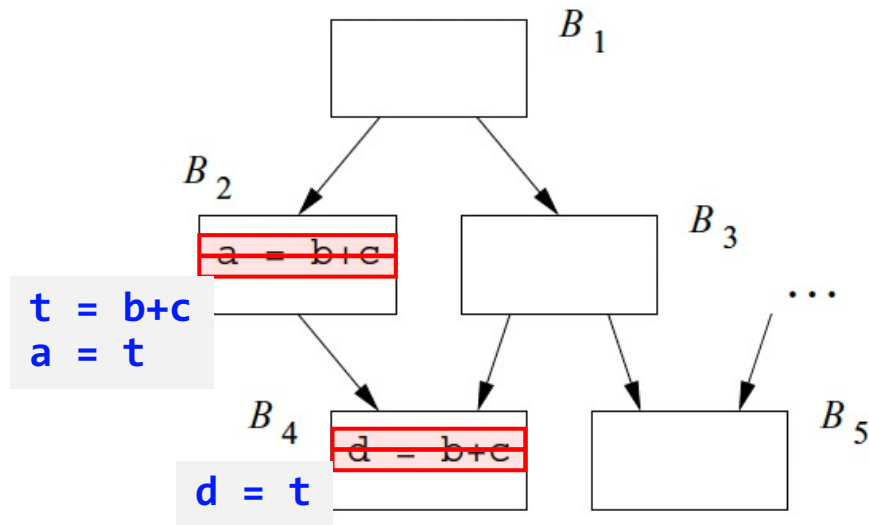
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



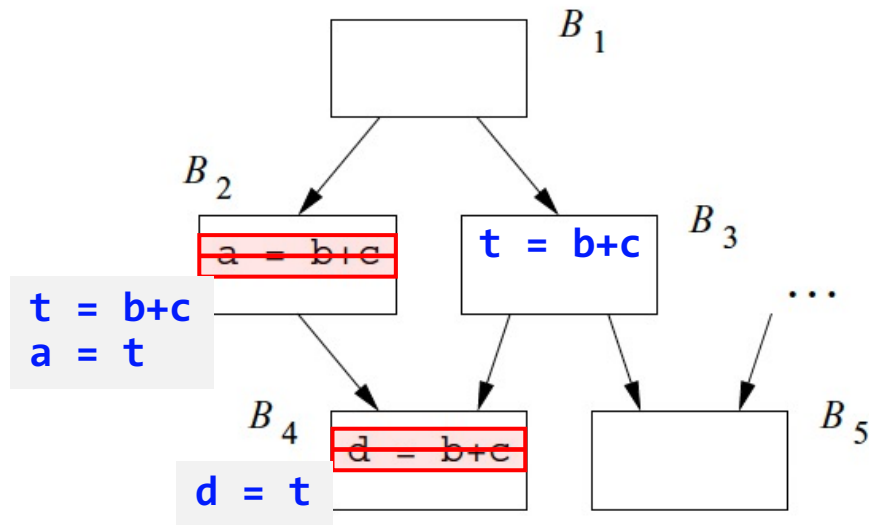
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



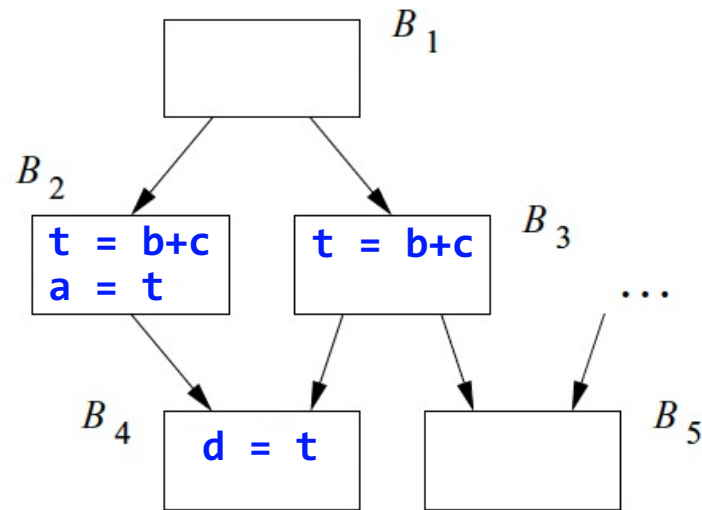
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



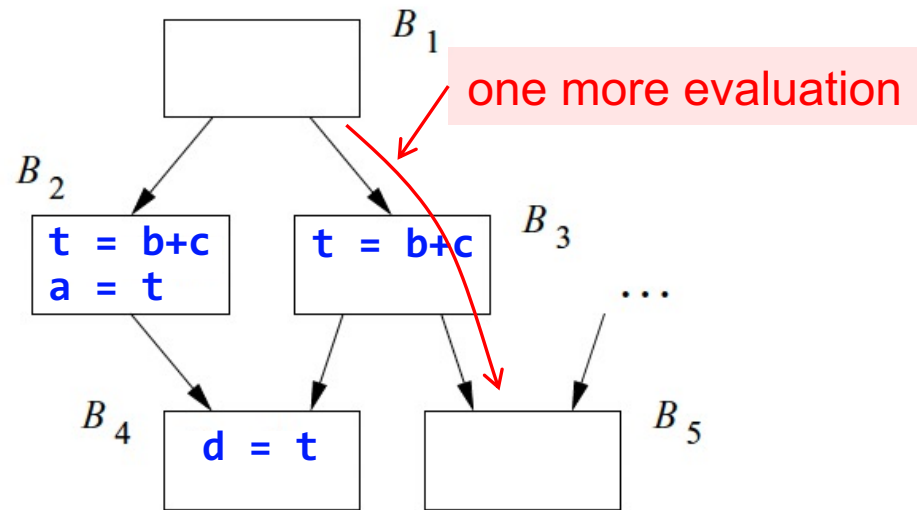
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



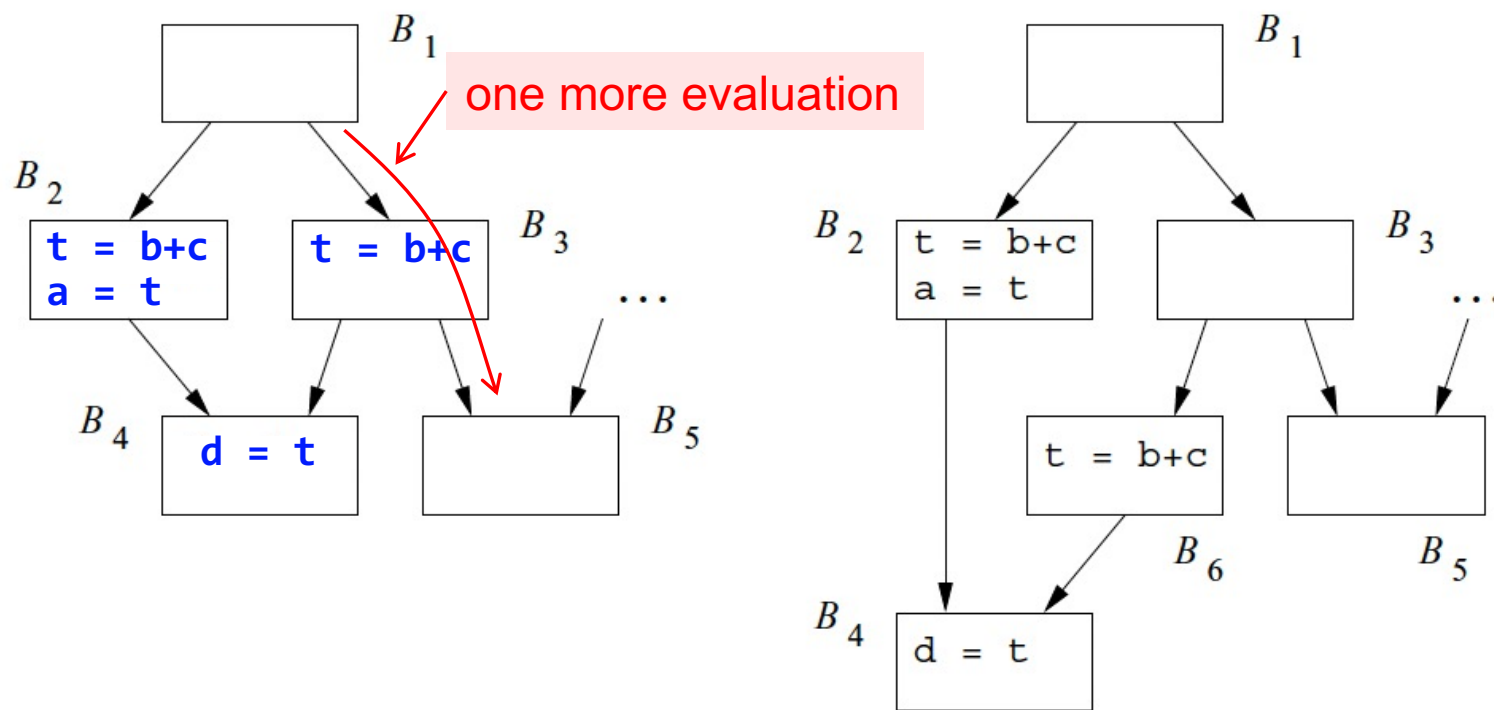
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



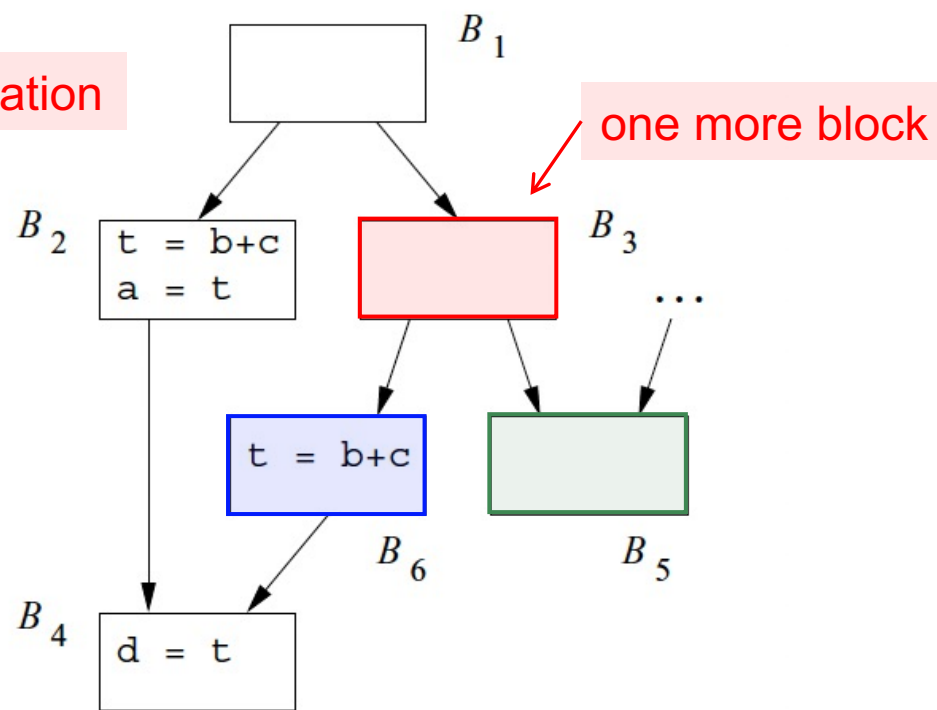
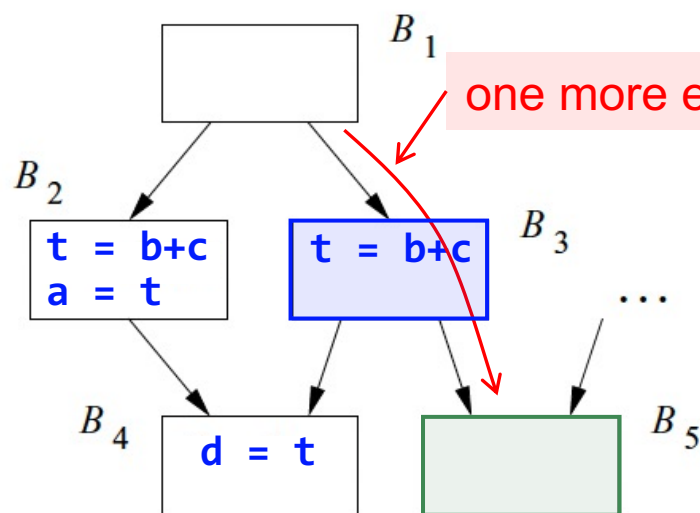
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



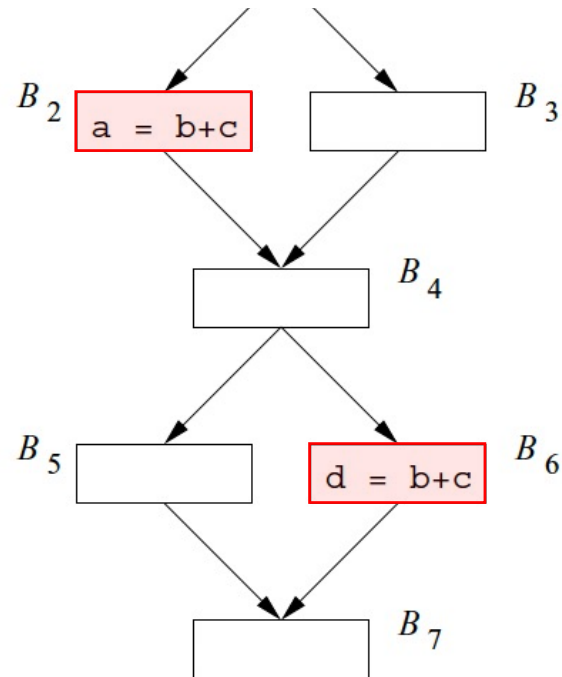
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



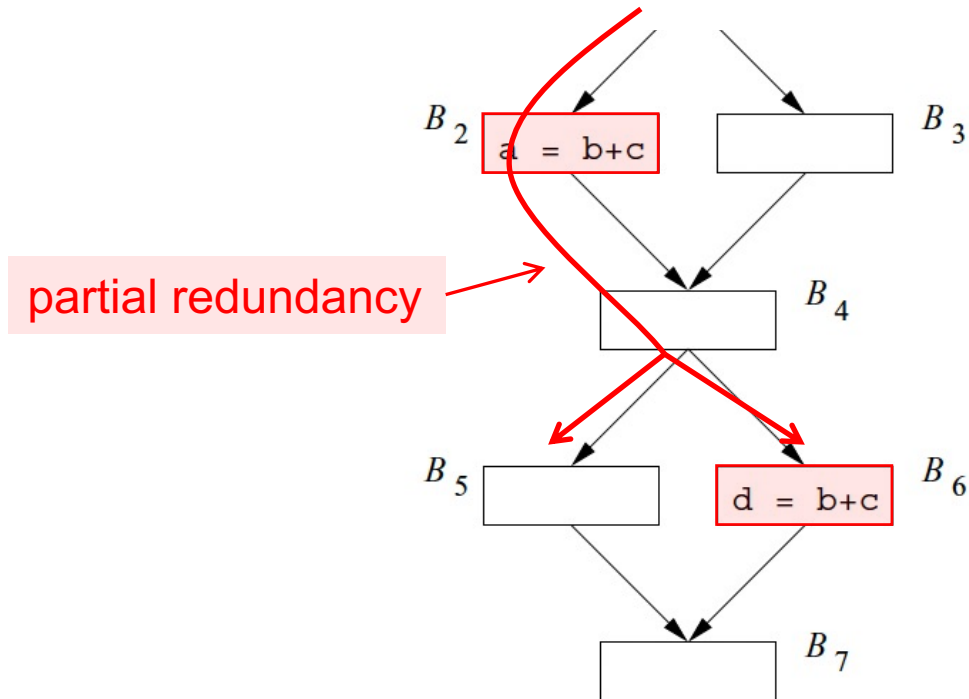
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



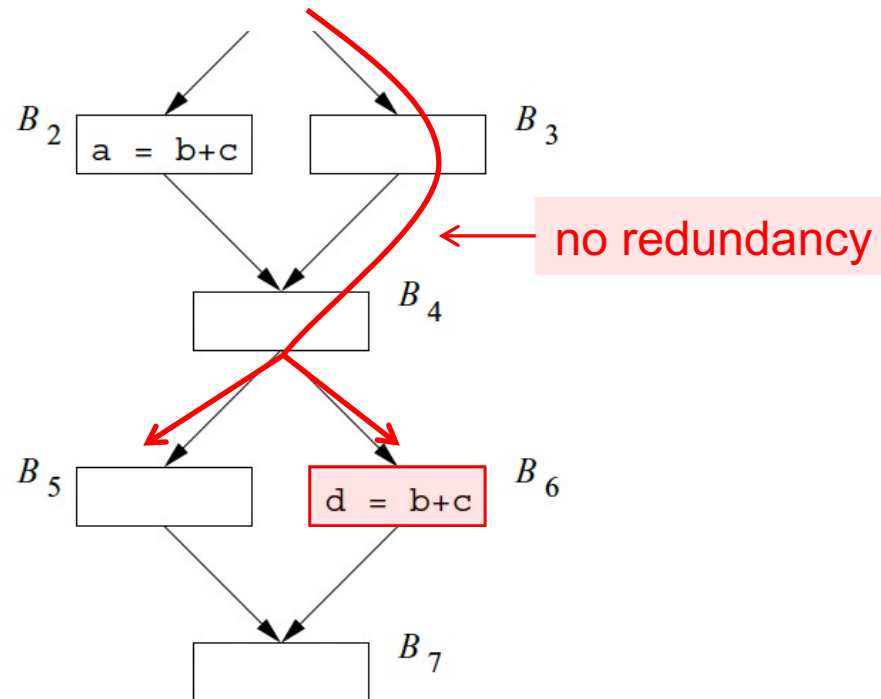
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



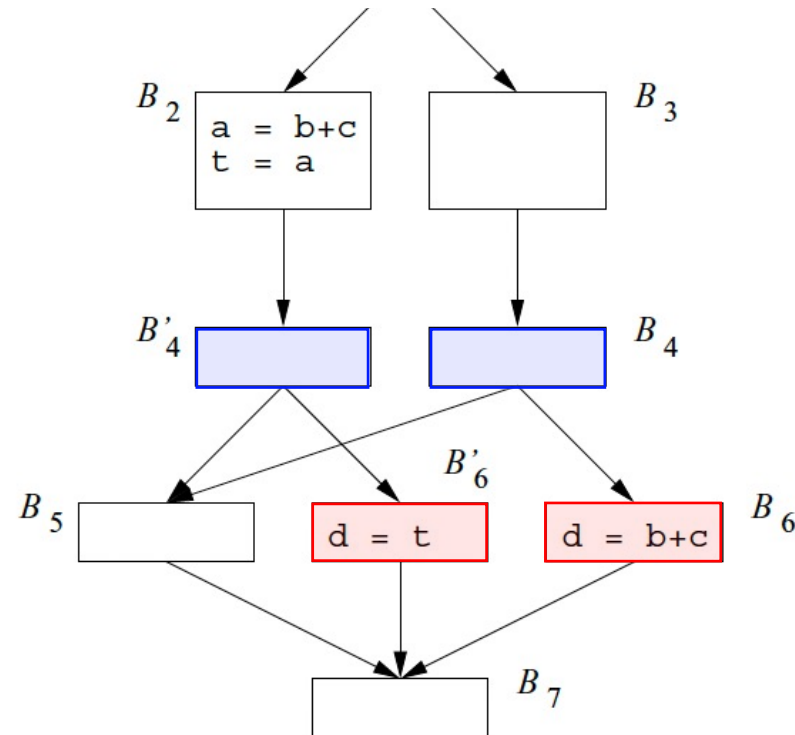
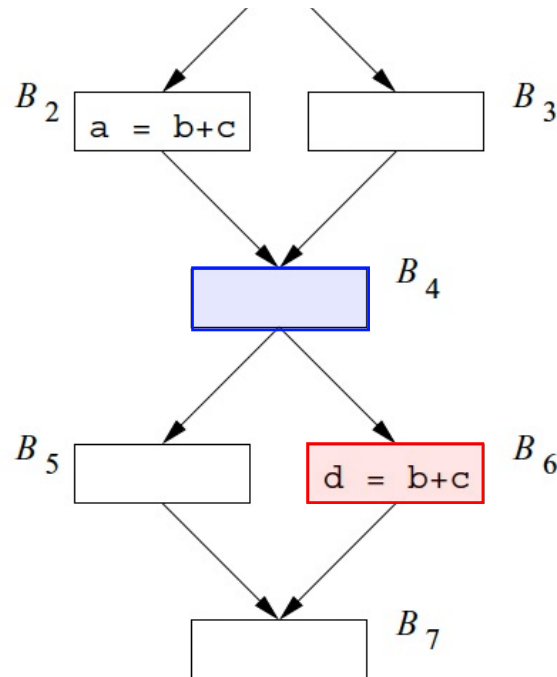
Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



Eliminating All Redundancy?

- Not Possible!
- Unless we are allowed to create new blocks or duplicate code.



The Lazy Code Motion Problem

- All redundant computations of expressions that can be eliminated without code duplication are eliminated.

The Lazy Code Motion Problem

- All redundant computations of expressions that can be eliminated without code duplication are eliminated.
- The optimized program does not perform any computation that is not in the original program execution.

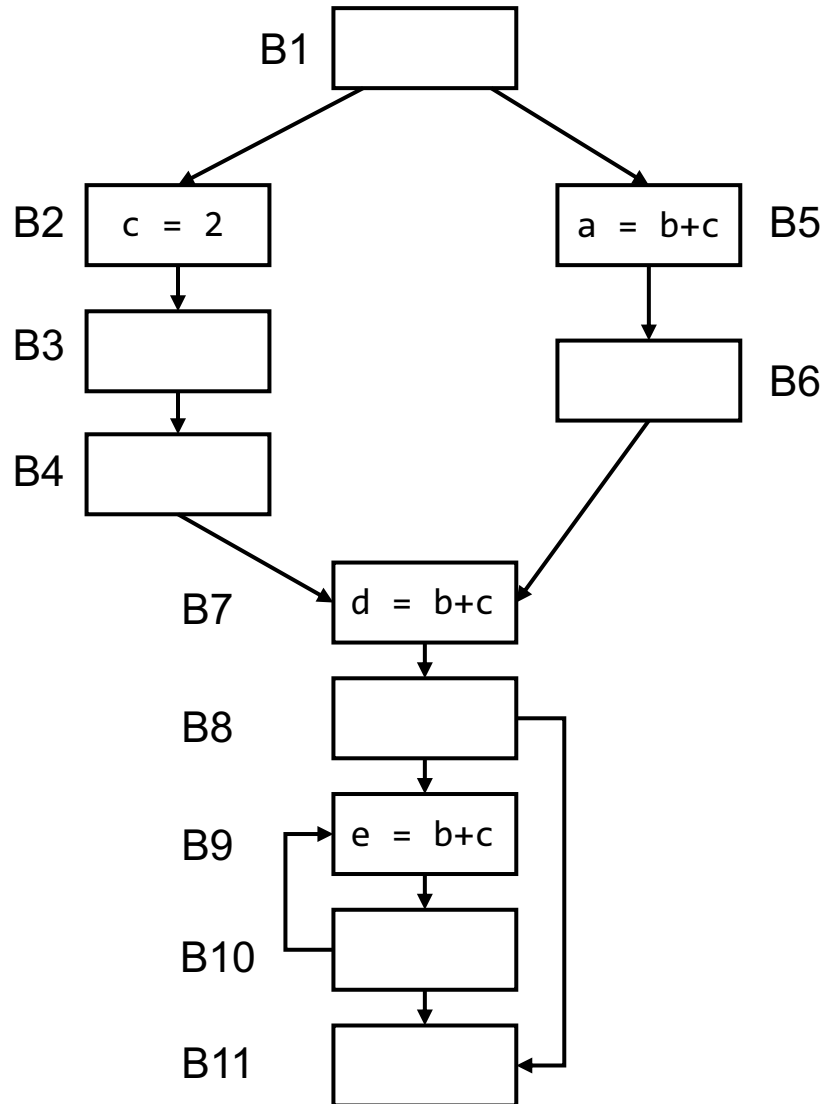
The Lazy Code Motion Problem

- All redundant computations of expressions that can be eliminated without code duplication are eliminated.
- The optimized program does not perform any computation that is not in the original program execution.
- Expressions are computed at the latest possible time.

The Lazy Code Motion Problem

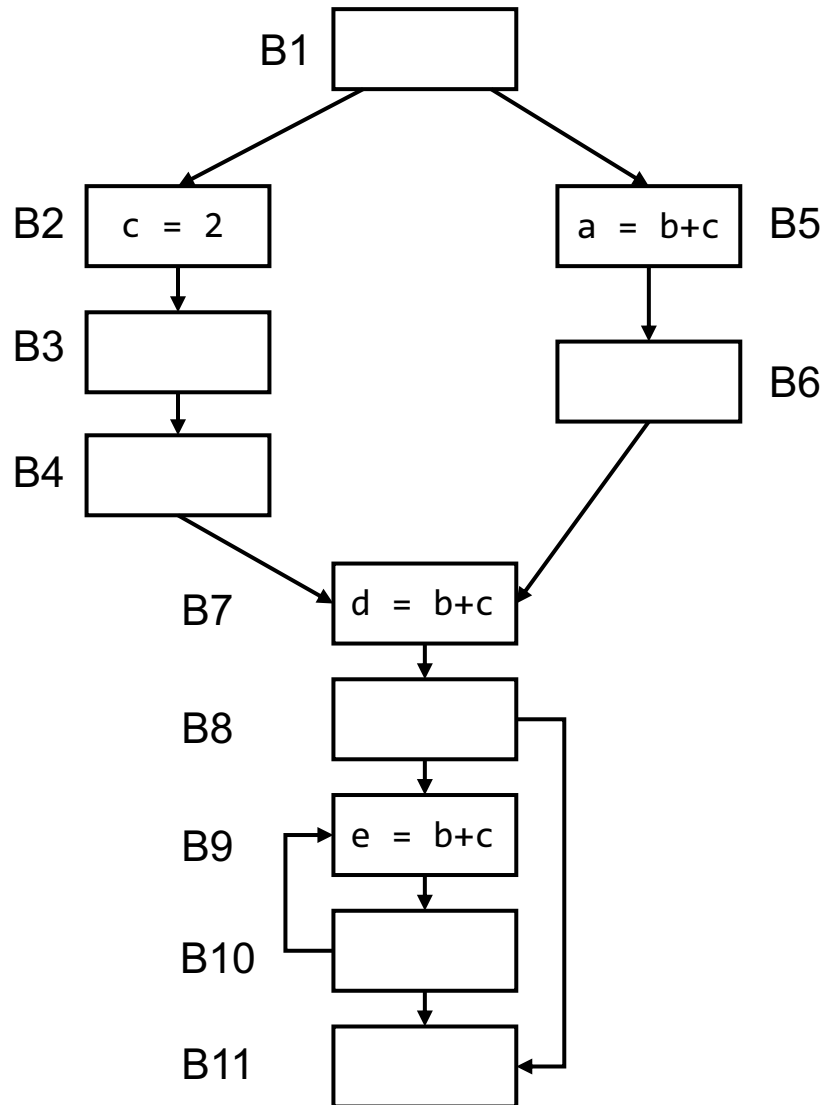
- **Step 1:** Anticipated Expressions (Backwards)
- **Step 2:** Available Expressions (Forwards)
- **Step 3:** Postponable Expressions (Forwards)
- **Step 4:** Used Expressions (Backwards)

Anticipated Expr (Backwards)



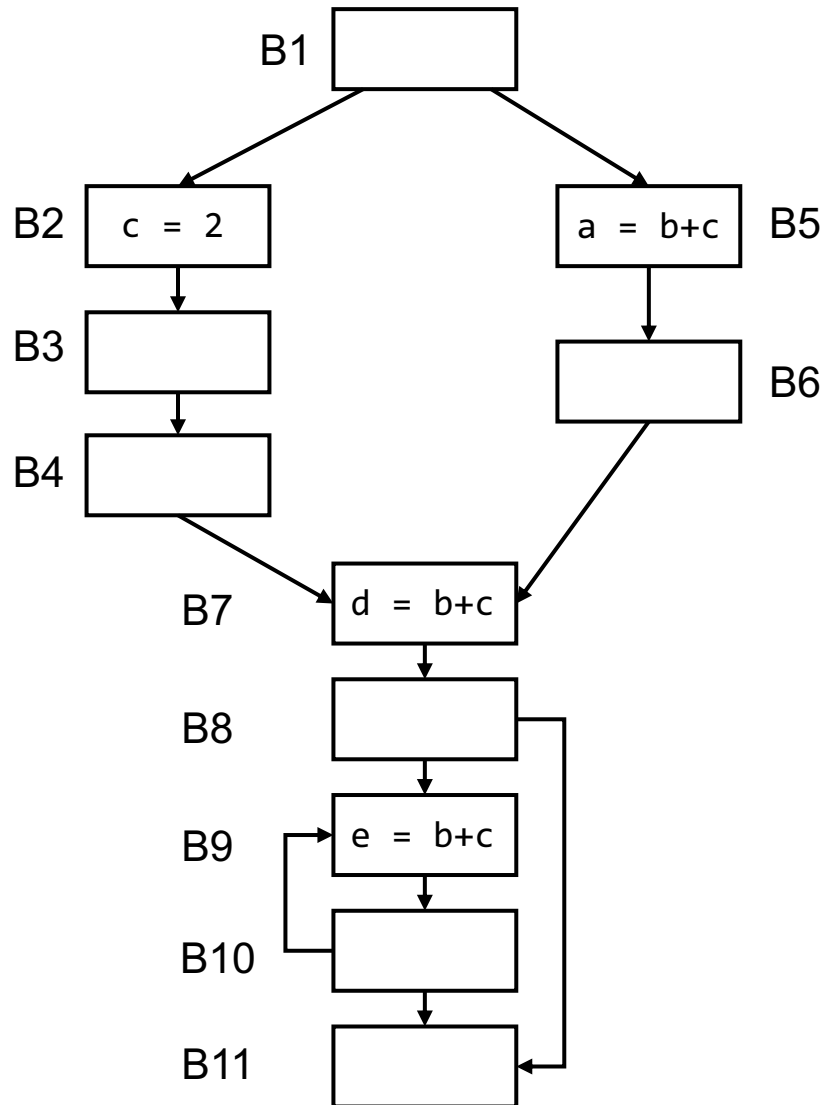
- An expression **`b+c`** is anticipated at a program point `p` if

Anticipated Expr (Backwards)



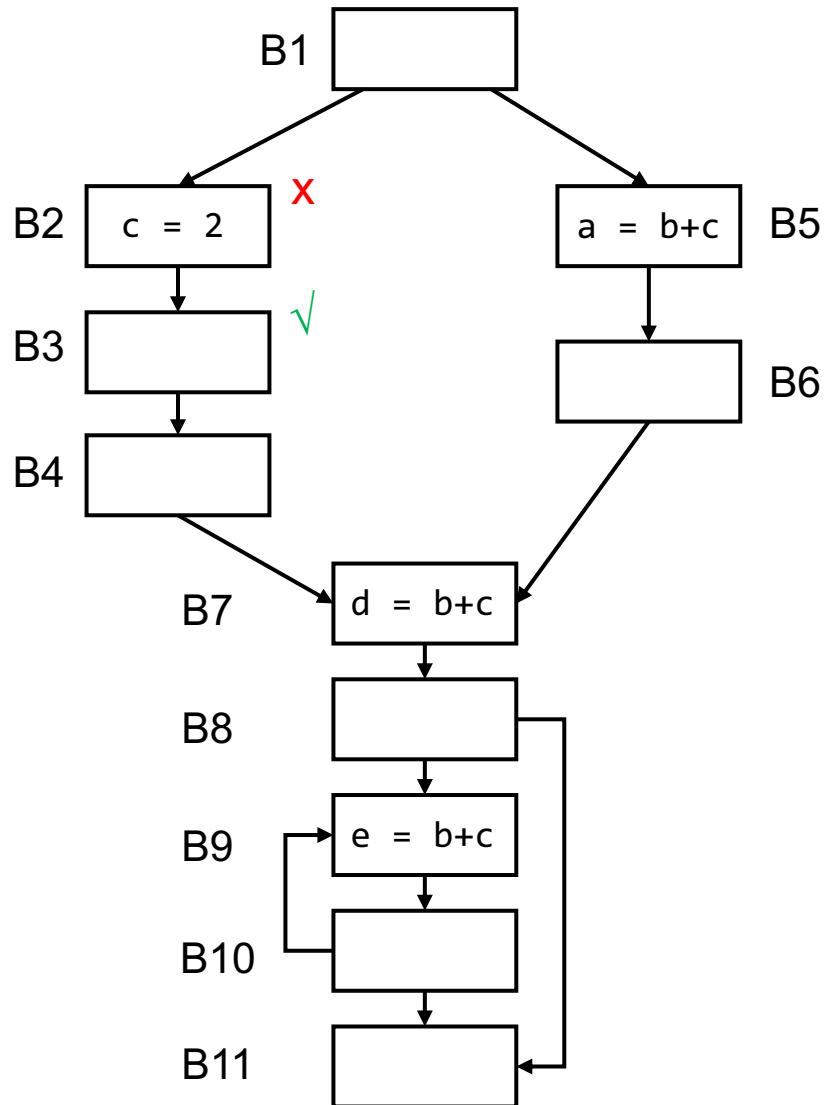
- An expression **`b+c`** is anticipated at a program point p if
 - all paths leading from point p eventually compute the expression **`b+c`**

Anticipated Expr (Backwards)



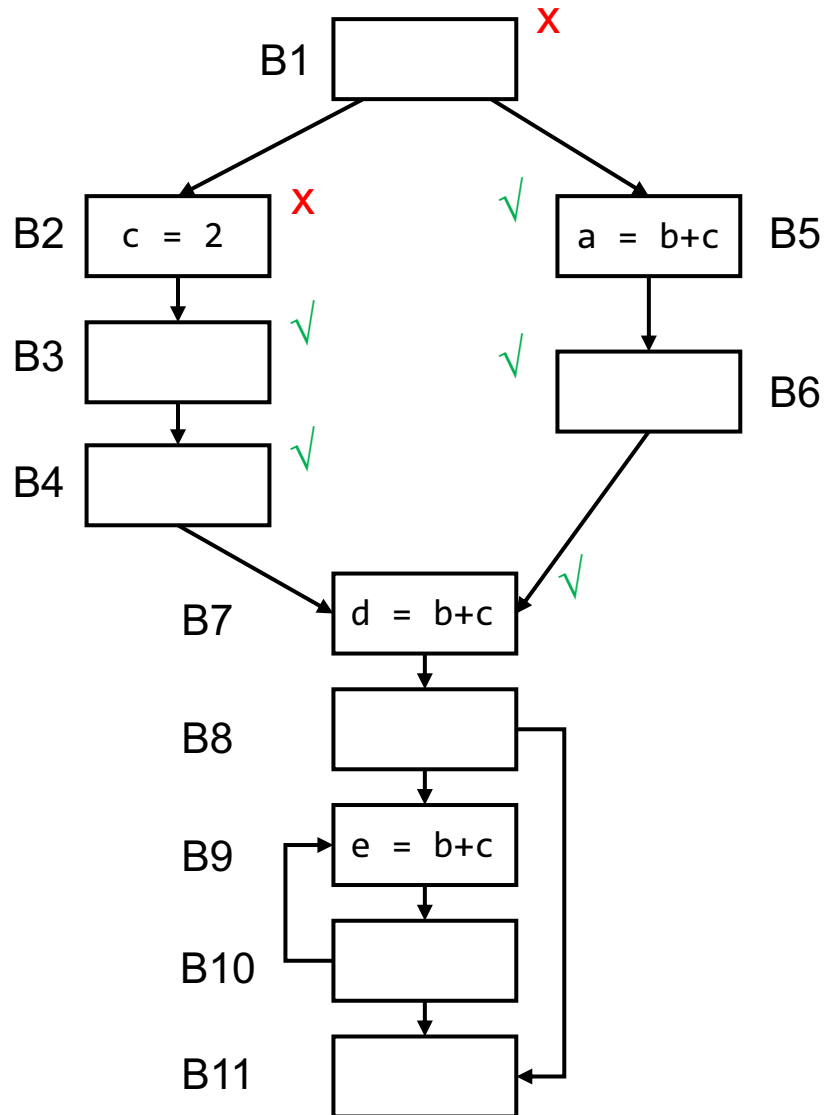
- An expression **`b+c`** is anticipated at a program point `p` if
 - all paths leading from point `p` eventually compute the expression **`b+c`** from the values of `b` and `c` that are available at that point

Anticipated Expr (Backwards)



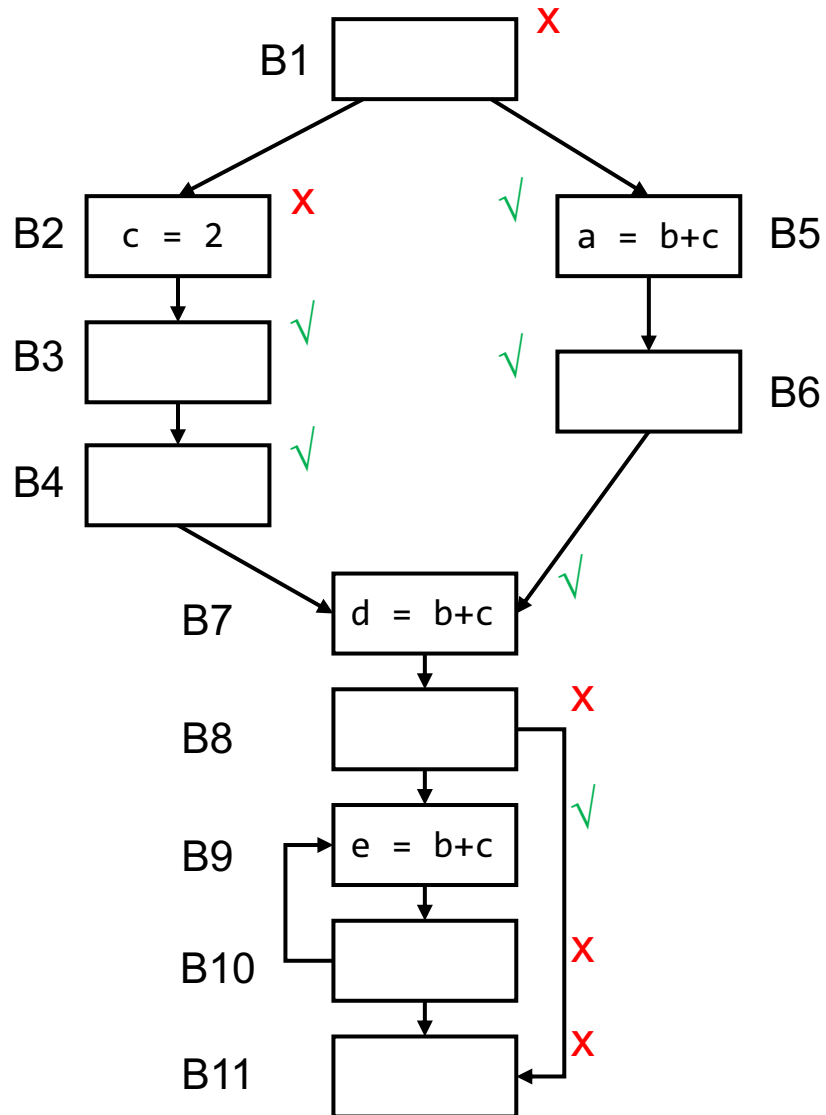
- An expression **b+c** is anticipated at a program point p if
 - all paths leading from point p eventually compute the expression **b+c** from the values of b and c that are available at that point

Anticipated Expr (Backwards)



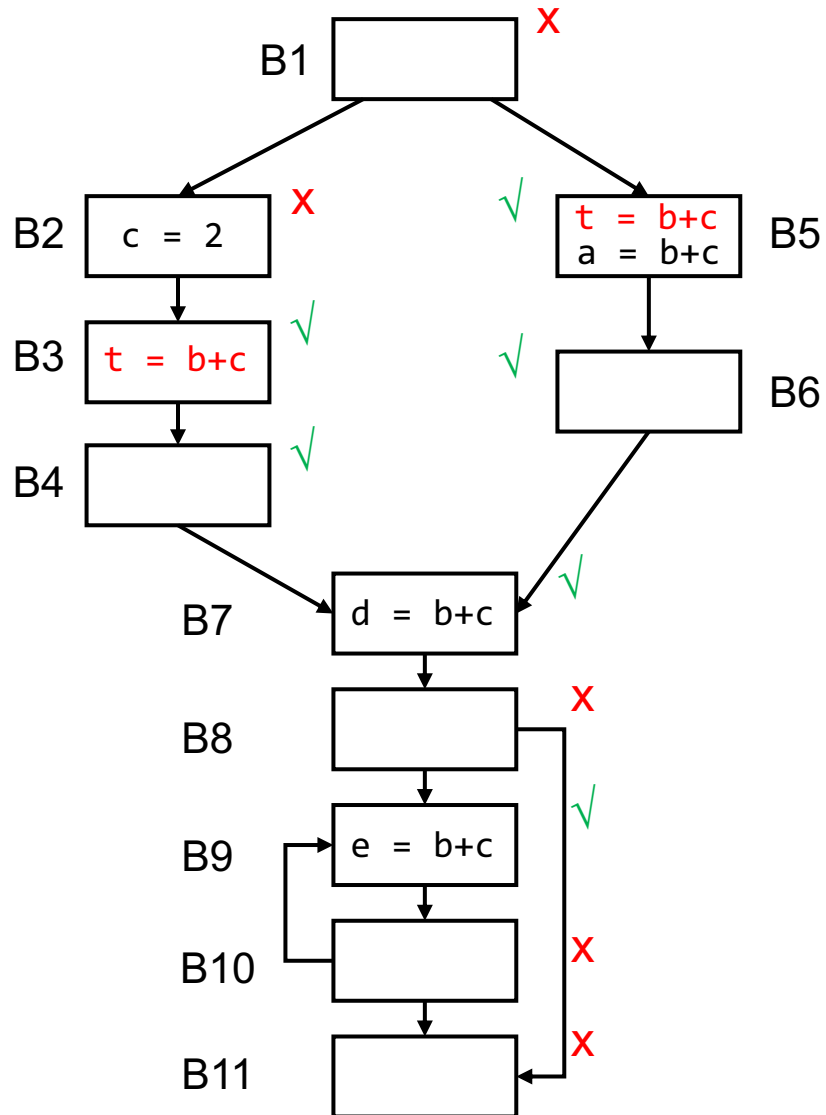
- An expression **b+c** is anticipated at a program point p if
 - all paths leading from point p eventually compute the expression **b+c** from the values of b and c that are available at that point

Anticipated Expr (Backwards)



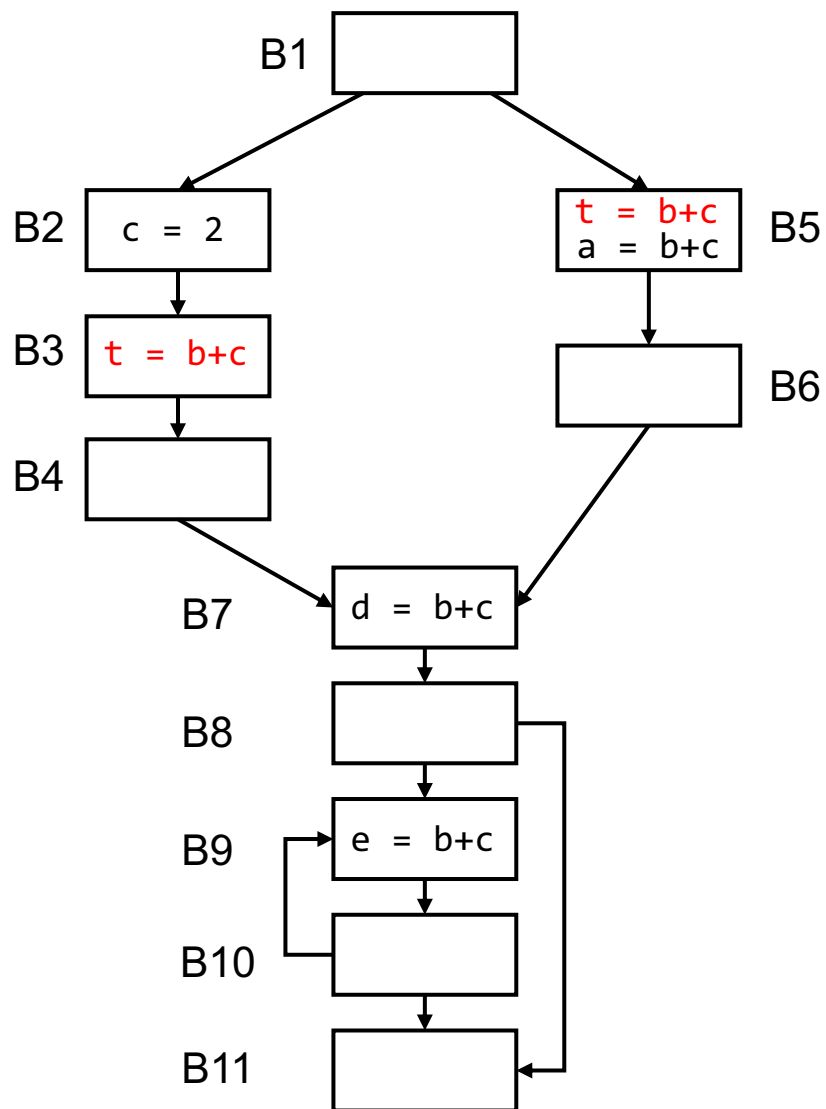
- An expression **`b+c`** is anticipated at a program point `p` if
 - all paths leading from point `p` eventually compute the expression **`b+c`** from the values of `b` and `c` that are available at that point

Anticipated Expr (Backwards)



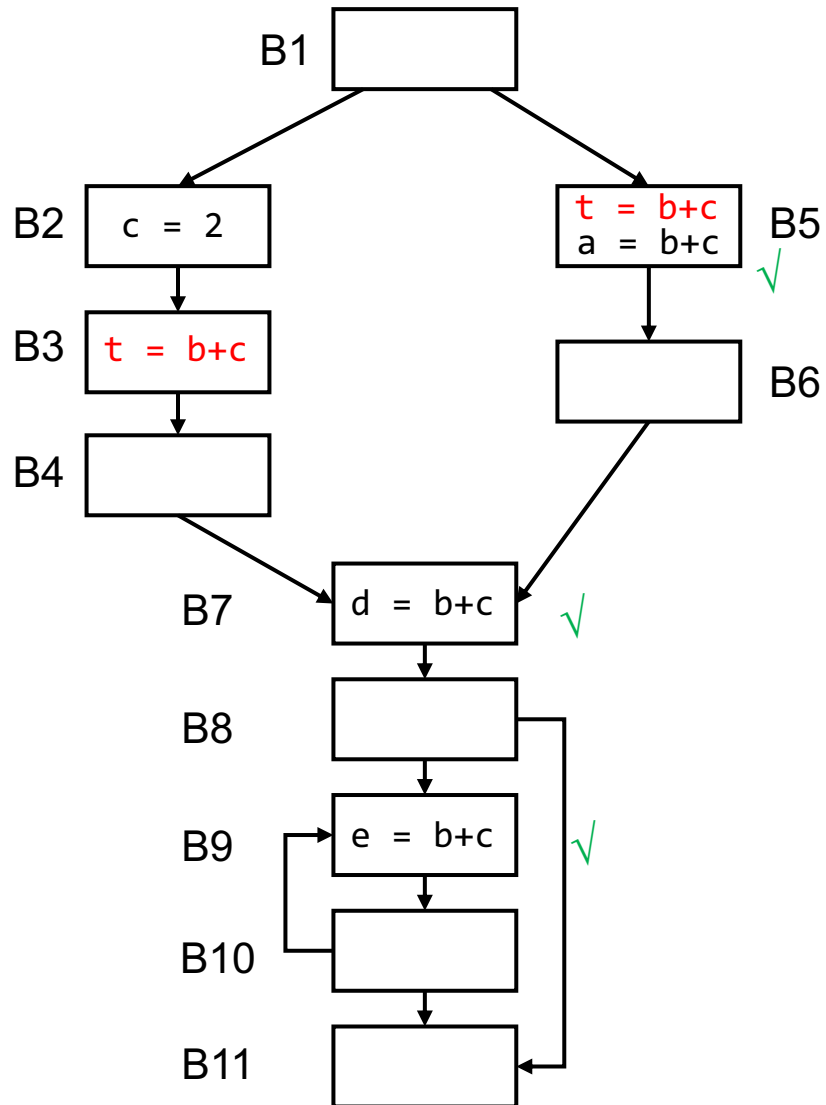
- An expression $b+c$ is anticipated at a program point p if
 - all paths leading from point p eventually compute the expression $b+c$ from the values of b and c that are available at that point

Available Expressions (Forwards)



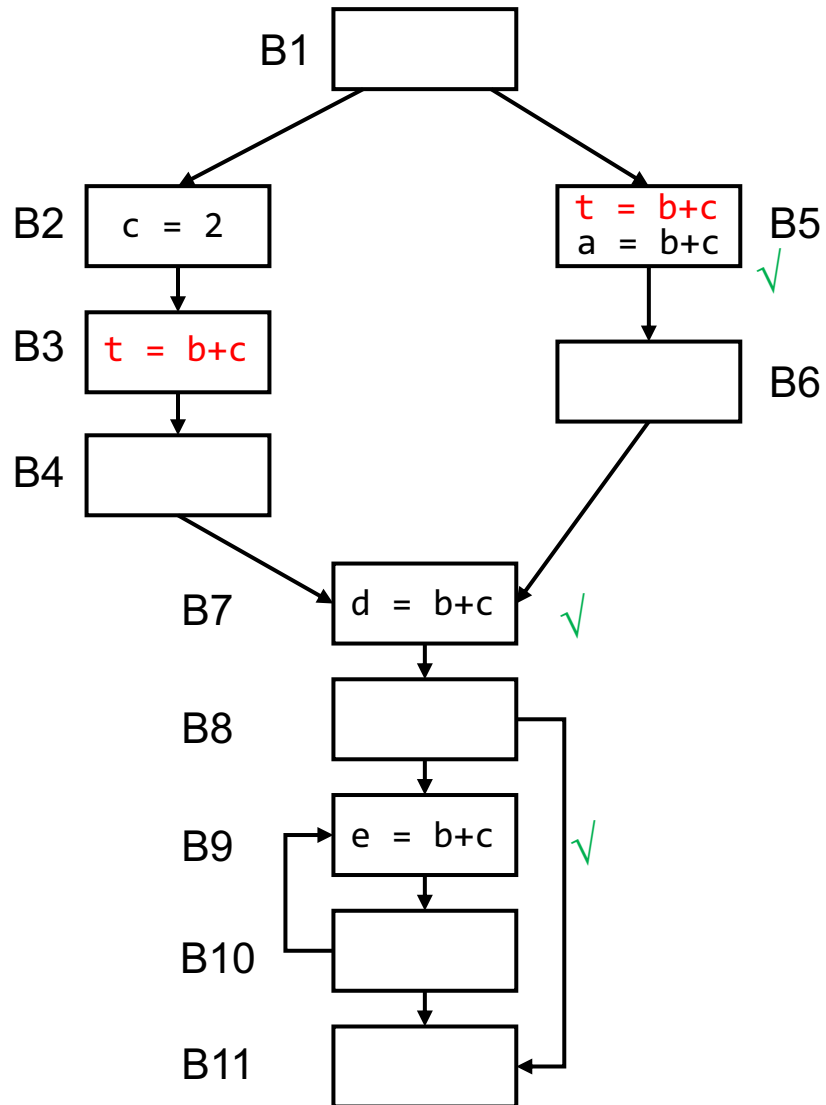
- Anticipated expr $\rightarrow$ Available expr
- An expression $b+c$ is available at program point p if it is anticipated along all paths reaching p .

Available Expressions (Forwards)



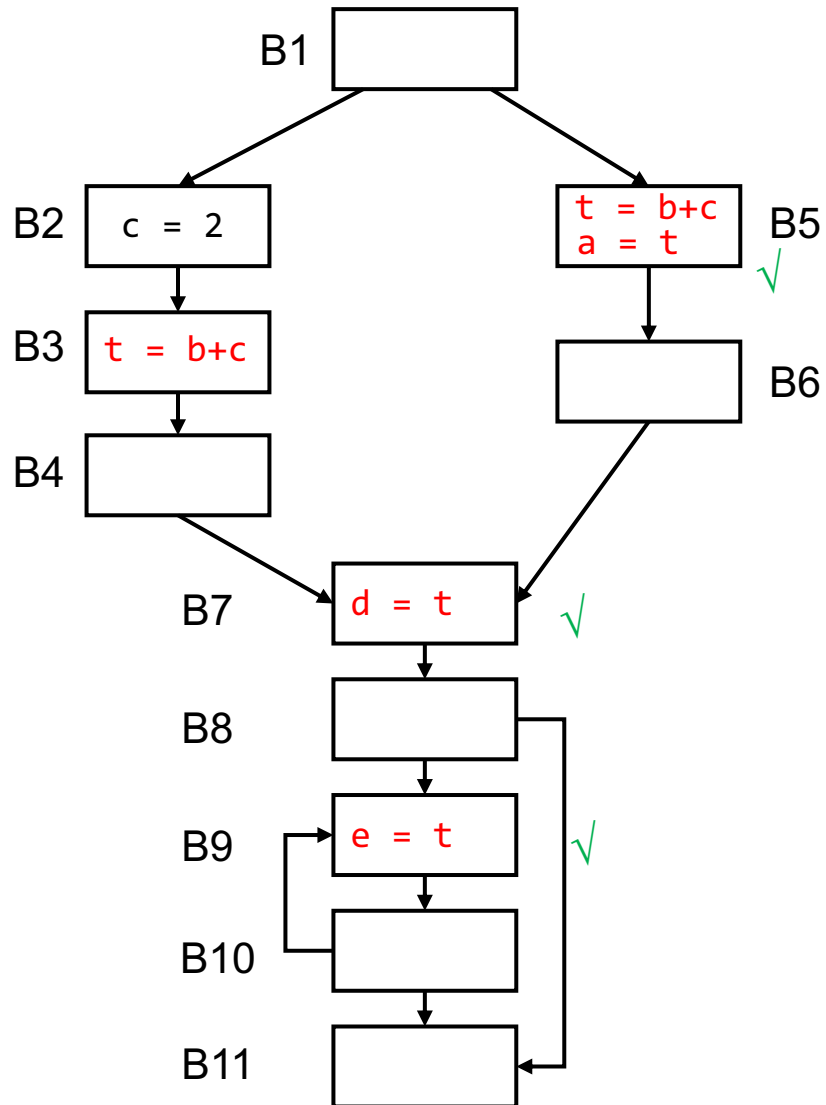
- Anticipated expr $\rightarrow$ Available expr
- An expression **`b+c`** is available at program point `p` if it is anticipated along all paths reaching `p`.

Available Expressions (Forwards)



- Anticipated expr $\rightarrow$ Available expr
- An expression `b+c` is available at program point p if it is anticipated along all paths reaching p.
- Replace `b+c` with `t`.

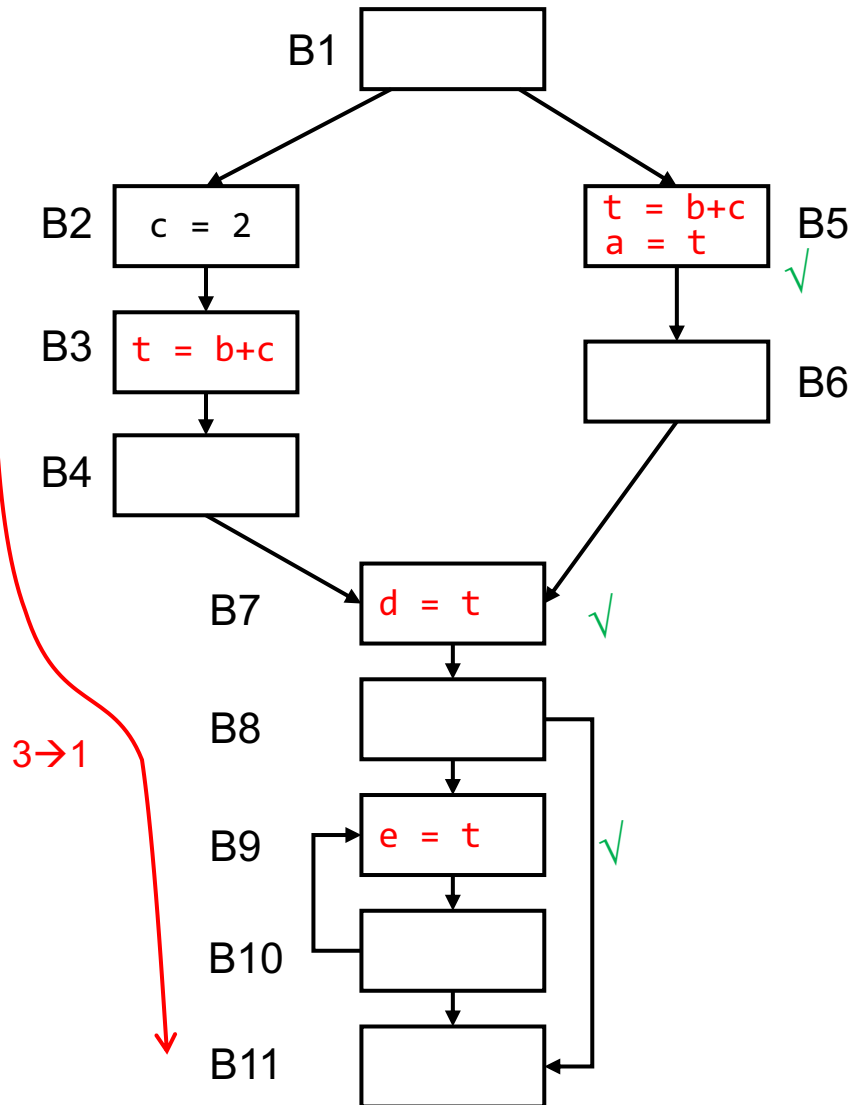
Available Expressions (Forwards)



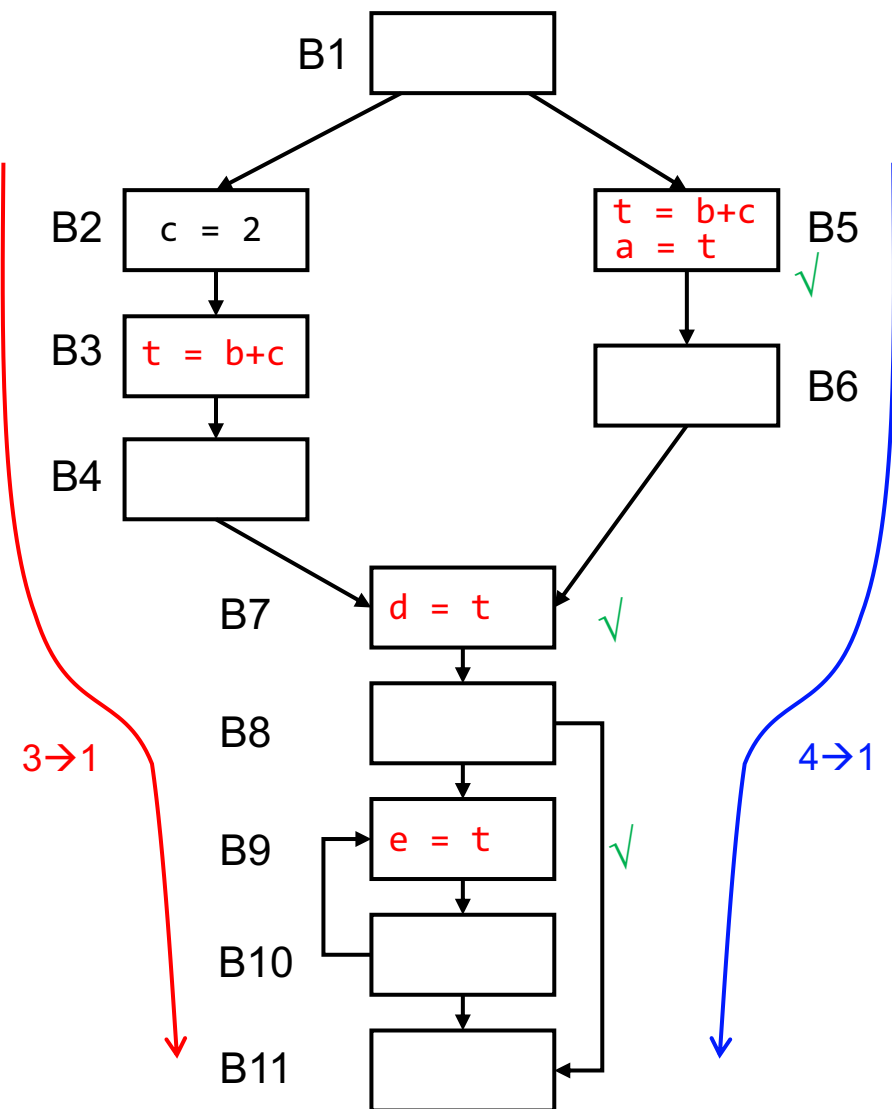
- Anticipated expr $\rightarrow$ Available expr
- An expression $b+c$ is available at program point p if it is anticipated along all paths reaching p .
- Replace $b+c$ with t .

Available Expressions (Forwards)

- Anticipated expr $\rightarrow$ Available expr
- An expression **$b+c$** is available at program point p if it is anticipated along all paths reaching p .
- Replace **$b+c$** with **t** .

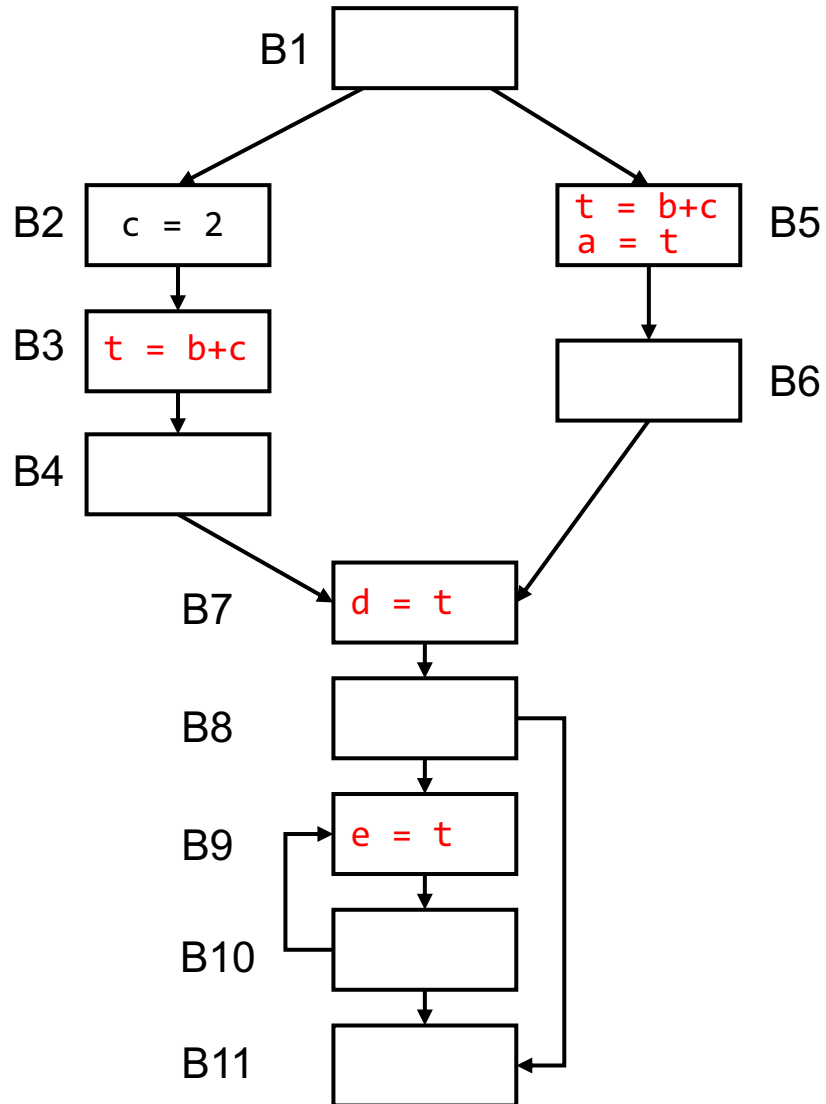


Available Expressions (Forwards)



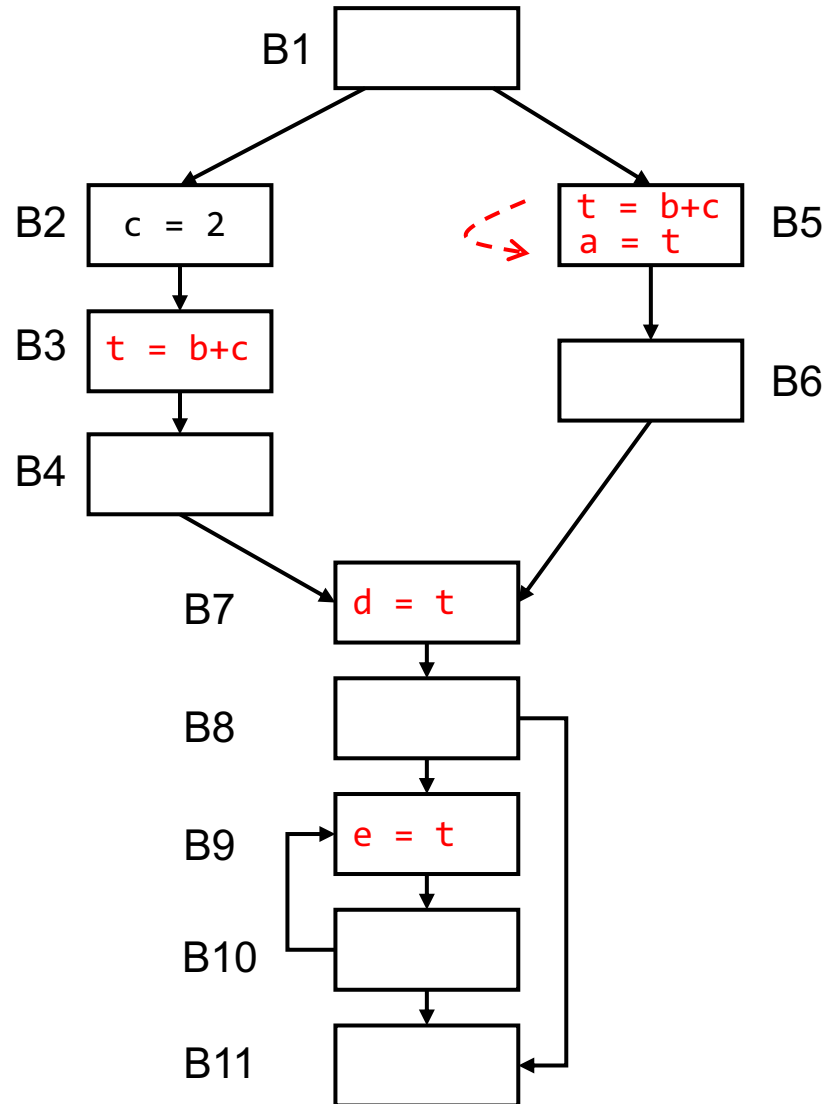
- Anticipated expr $\rightarrow$ Available expr
- An expression $b+c$ is available at program point p if it is anticipated along all paths reaching p .
- Replace $b+c$ with t .

Postponable Expr (Forwards)



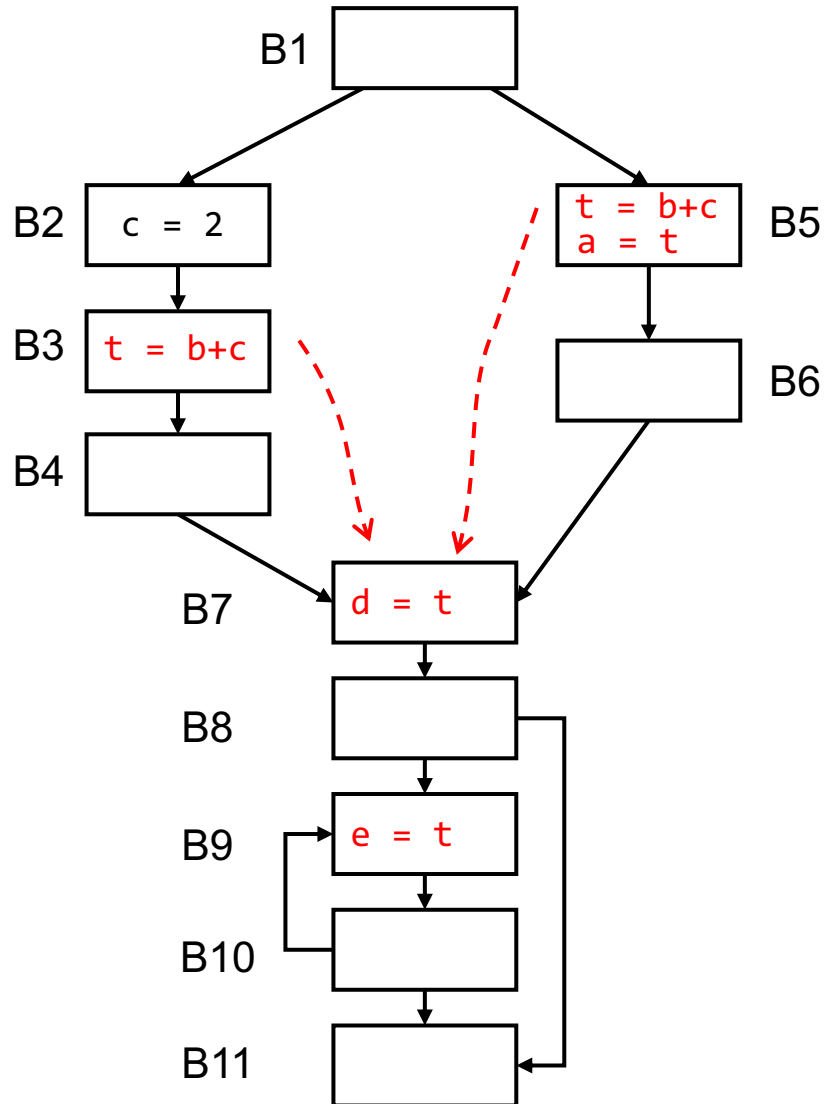
- If an expression is far from its use, what may happen?

Postponable Expr (Forwards)



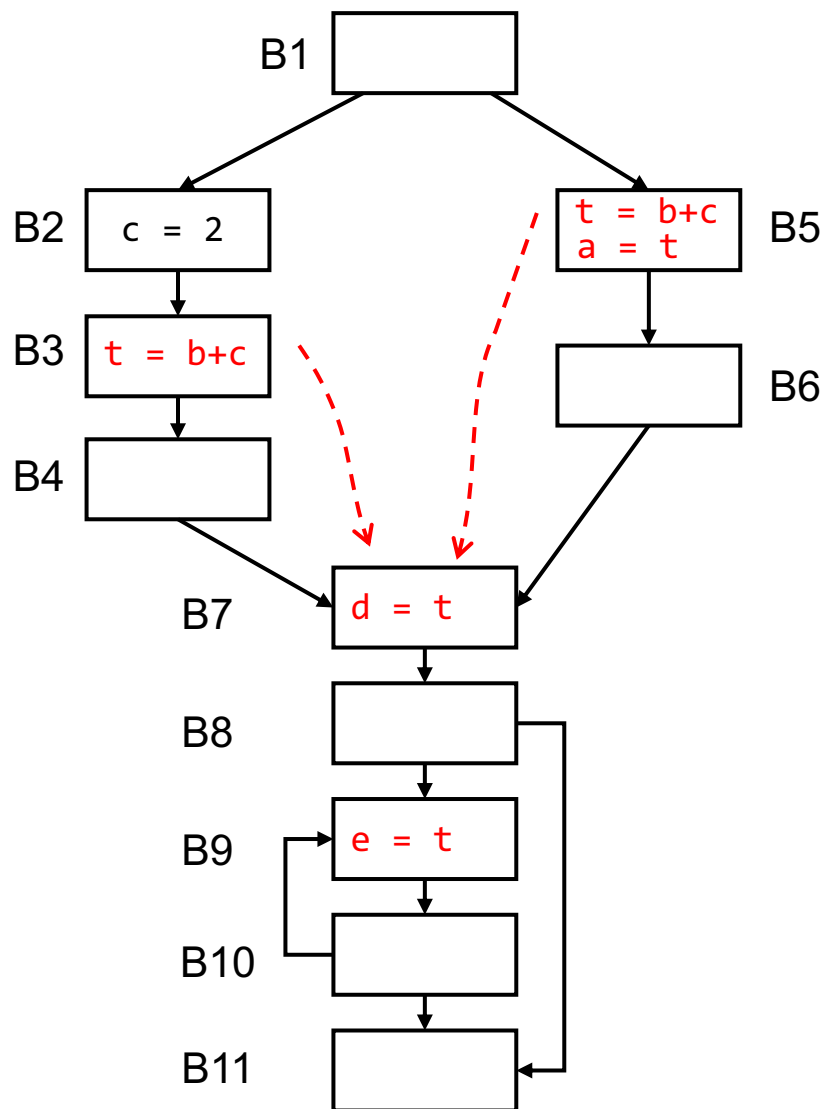
- If an expression is far from its use, what may happen?

Postponable Expr (Forwards)



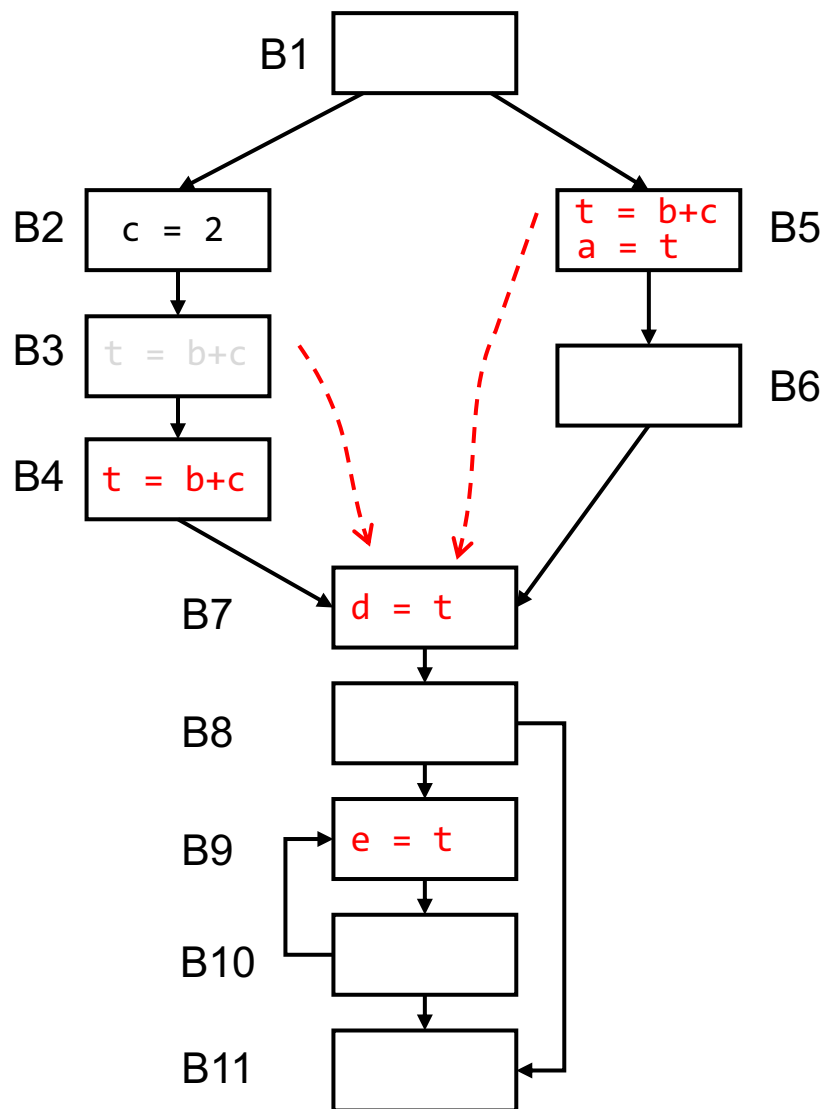
- If an expression is far from its use, what may happen?

Postponable Expr (Forwards)



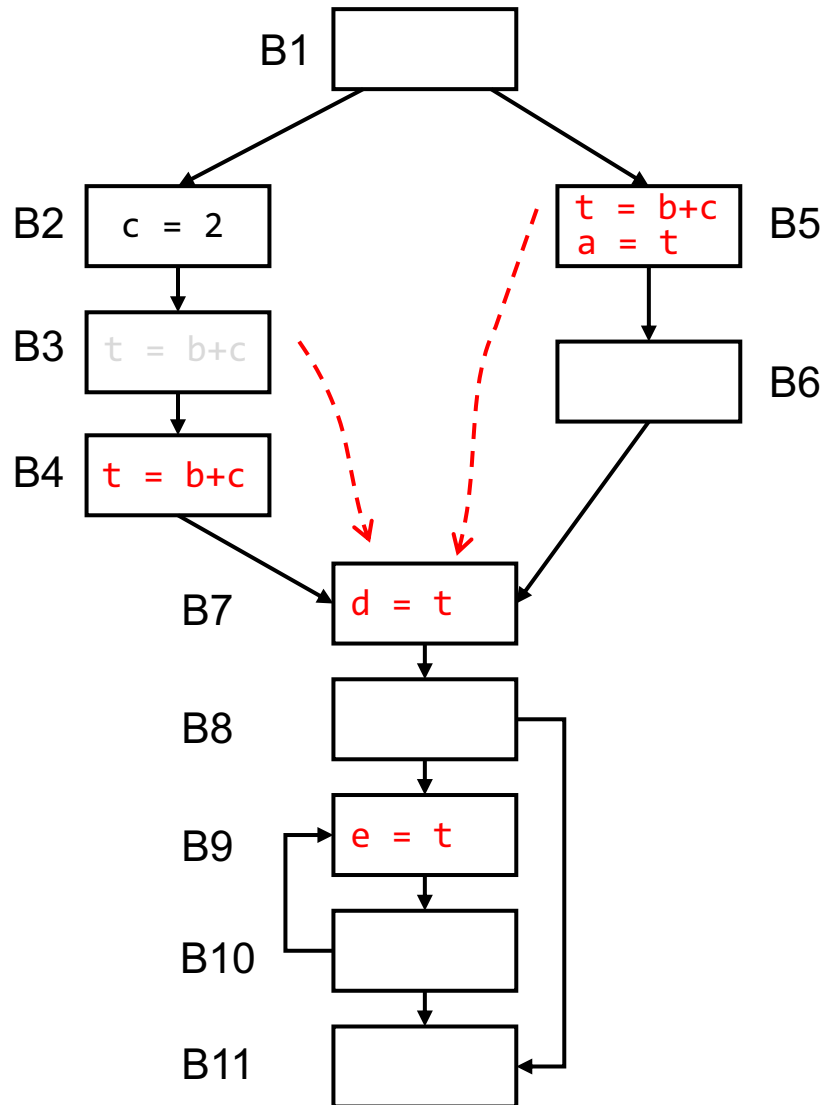
- If an expression is far from its use, what may happen?
- Why is $t = b+c$ in B3 postponable?

Postponable Expr (Forwards)



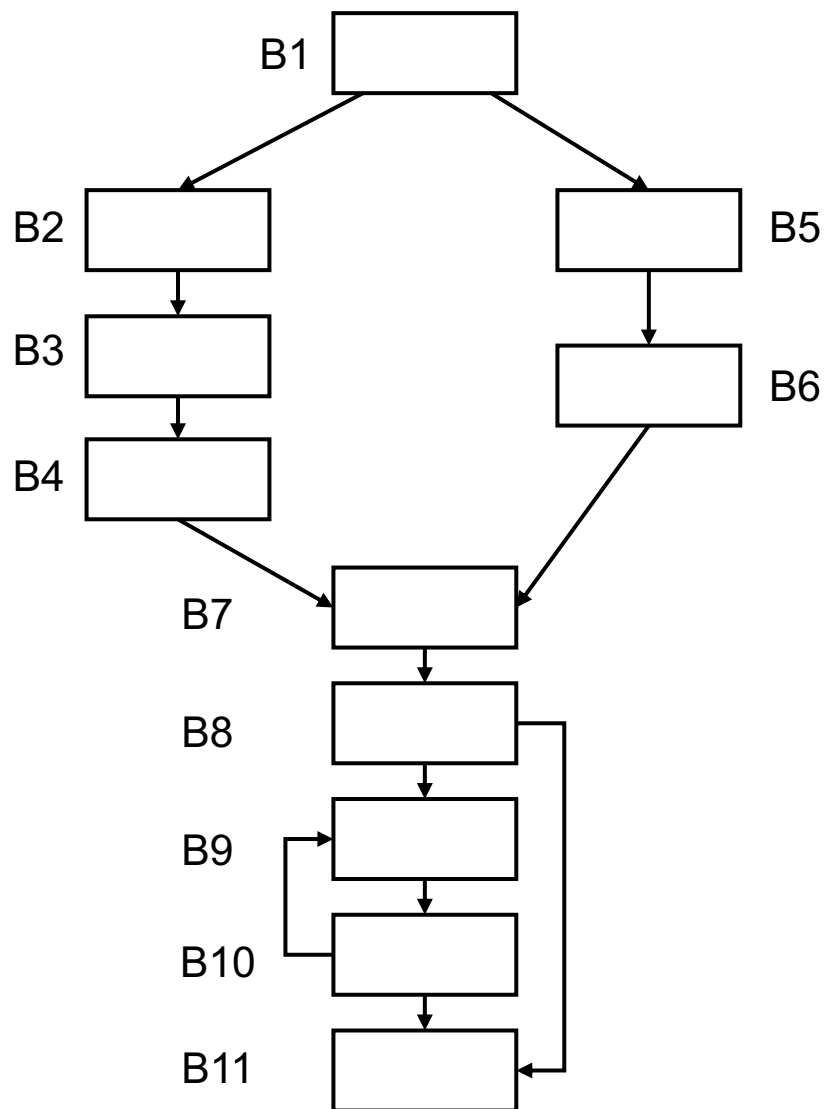
- If an expression is far from its use, what may happen?
- Why is $t = b+c$ in B3 postponable?

Postponable Expr (Forwards)



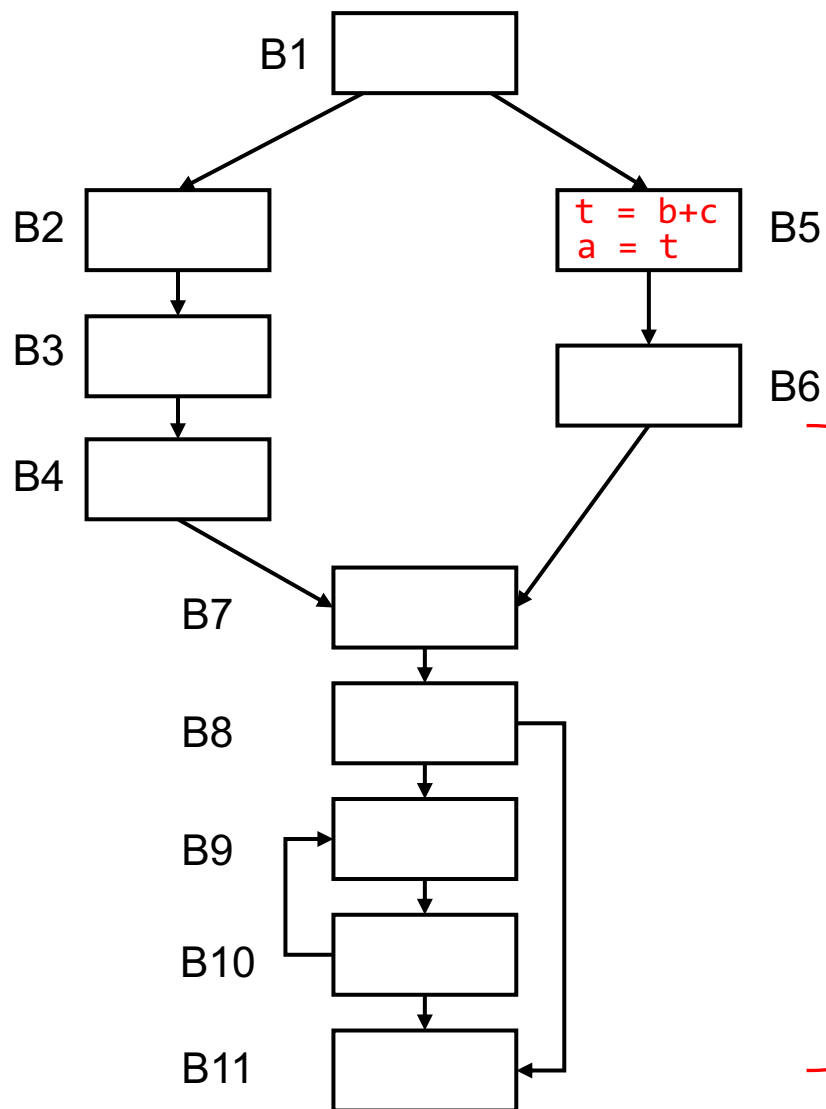
- If an expression is far from its use, what may happen?
- Why is $t = b+c$ in B3 postponable?
- Why cannot $t = b+c$ in B3 be postponed to B7?

Used Expressions (Backwards)



- A simple backward data-flow pass to collect expressions used.
- We then can eliminate temporary variables that are used only once in the program.

Used Expressions (Backwards)



- A simple backward data-flow pass to collect expressions used.
- Eliminate temporary variables that are used only once in the program.

Assume the variable t is never used from B6 to B11?

The Lazy Code Motion Problem

- **Step 1:** Anticipated Expressions (Backwards)
- **Step 2:** Available Expressions (Forwards)
- **Step 3:** Postponable Expressions (Forwards)
- **Step 4:** Used Expressions (Backwards)

Summary

	(a) Anticipated Expressions	(b) Available Expressions
Domain	Sets of expressions	Sets of expressions
Direction	Backwards	Forwards
Transfer function	$f_B(x) = e_use_B \cup (x - e_kill_B)$	$f_B(x) = (anticipated[B].in \cup x) - e_kill_B$
Boundary	$IN[EXIT] = \emptyset$	
Meet ($\wedge$)	$\cap$	
Equations	$IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{S, succ(B)} IN[S]$	
Initialization	$IN[B] = U$	

	(c) Postponable Expressions	(d) Used Expressions
Domain	Sets of expressions	Sets of expressions
Direction	Forwards	Backwards
Transfer function	$f_B(x) = (earliest[B] \cup x) - e_use_B$	$f_B(x) = (e_use_B \cup x) - latest[B]$
Boundary	$OUT[ENTRY] = \emptyset$	$IN[EXIT] = \emptyset$
Meet ($\wedge$)	$\cap$	$\cup$
Equations	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P, pred(B)} OUT[P]$	$IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{S, succ(B)} IN[S]$
Initialization	$OUT[B] = U$	$IN[B] = \emptyset$

Refer to Chapter 9, the Dragon book!

PART III: Constant Propagation

Constant Propagation

- Does a variable have a constant value at a program point?
- **Feature 1:** It has an unbounded set of possible data-flow values
- **Feature 2:** It is not distributive

Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y$

- $m(y) = \text{Constant}$
 - $\Rightarrow m(x) = \text{Constant}$
- $m(y) = \text{UNDEF}$
 - $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
 - $\Rightarrow m(x) = \text{NAC}$

Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

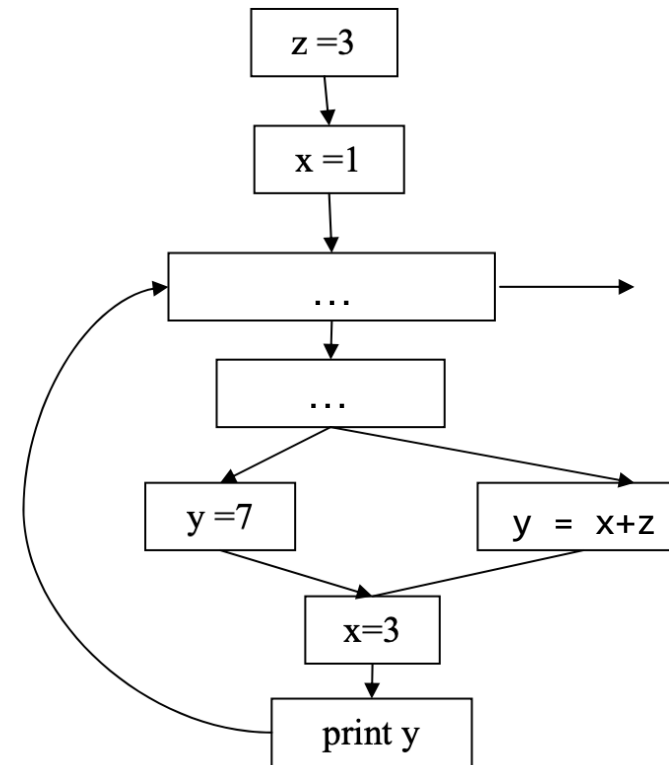
- $m(y) = \text{Constant}, m(z) = \text{Constant}$
 - $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
 - $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
 - $\Rightarrow m(x) = \text{NAC}$

Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

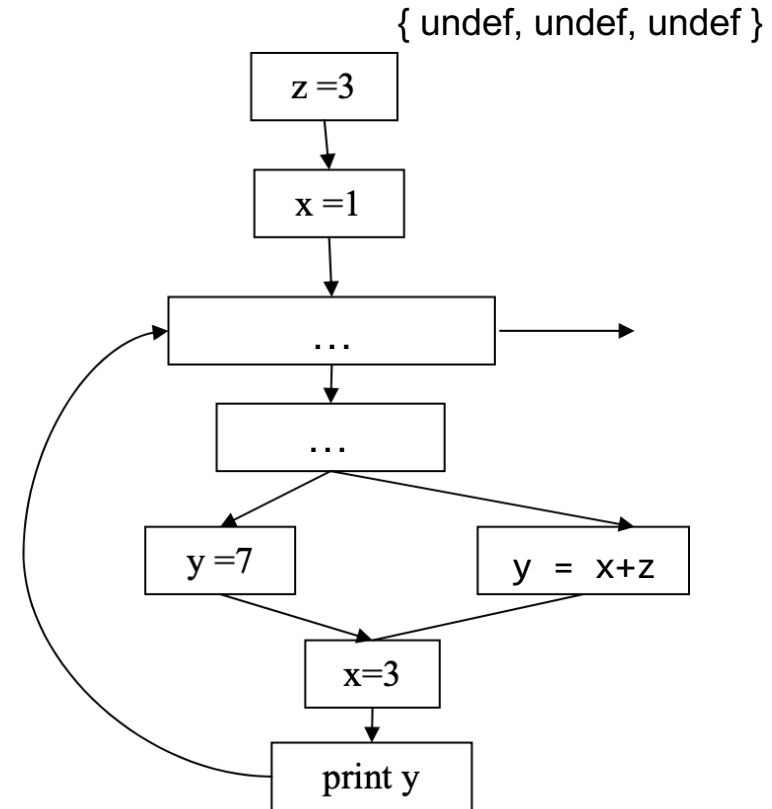


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

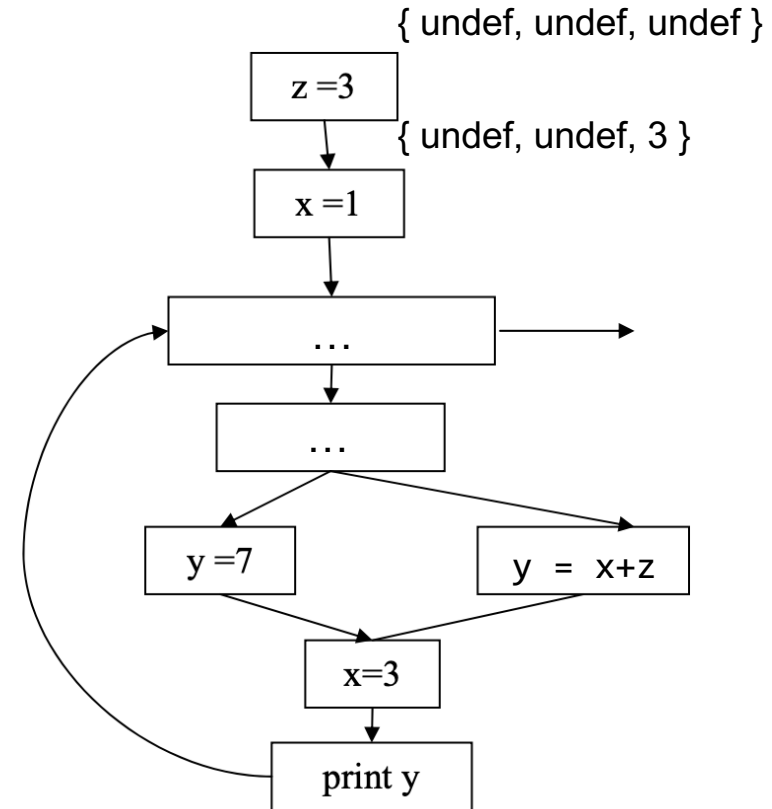


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

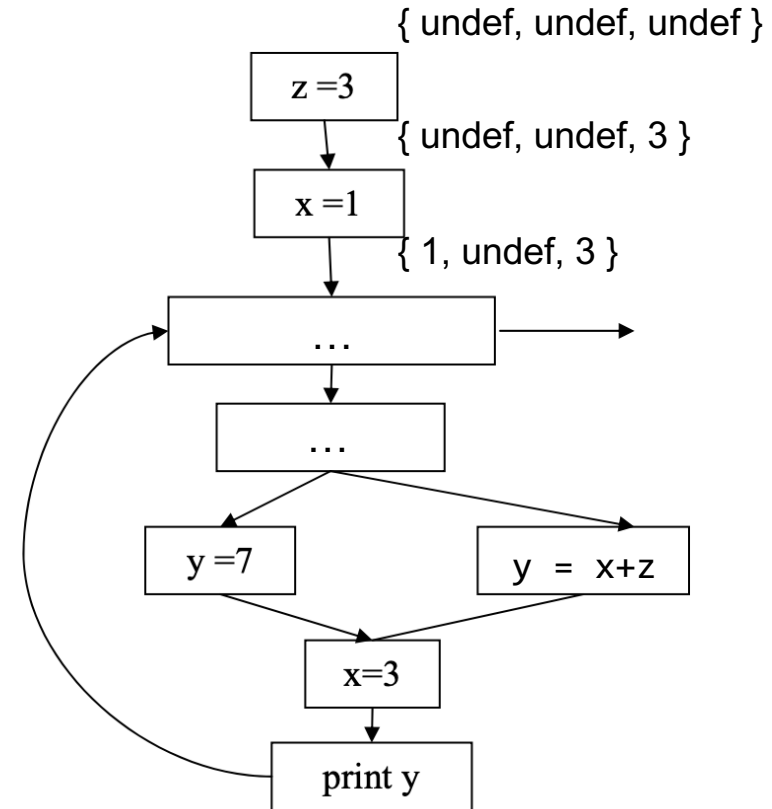


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

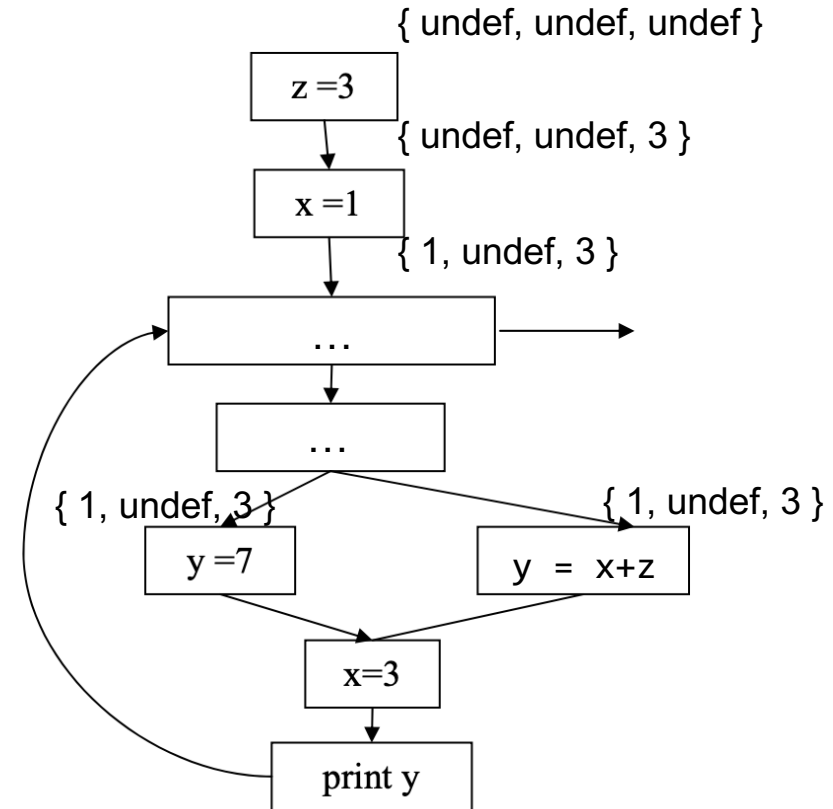


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

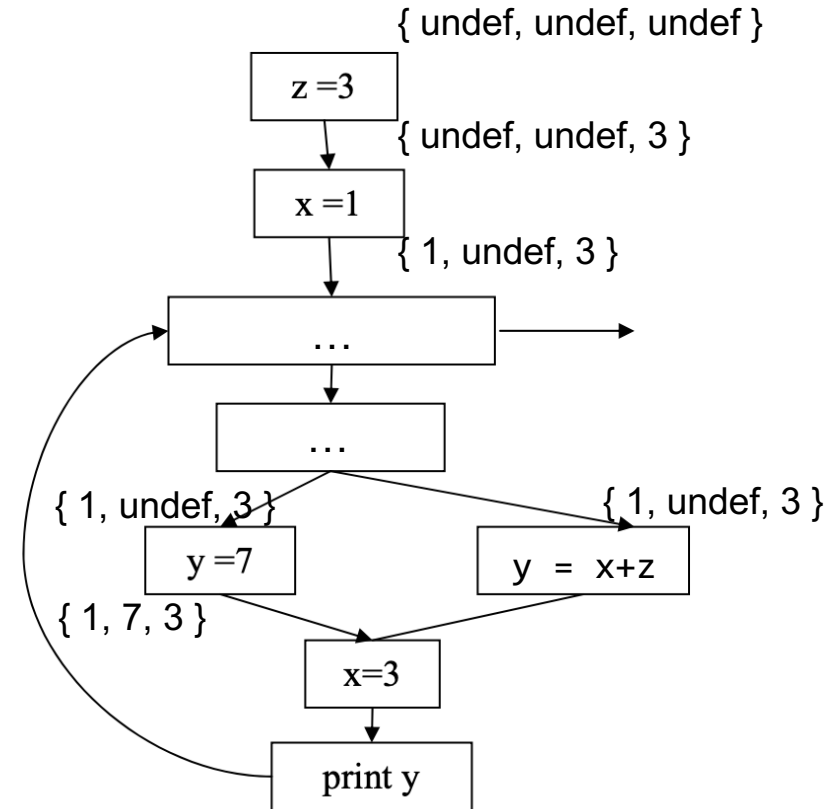


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

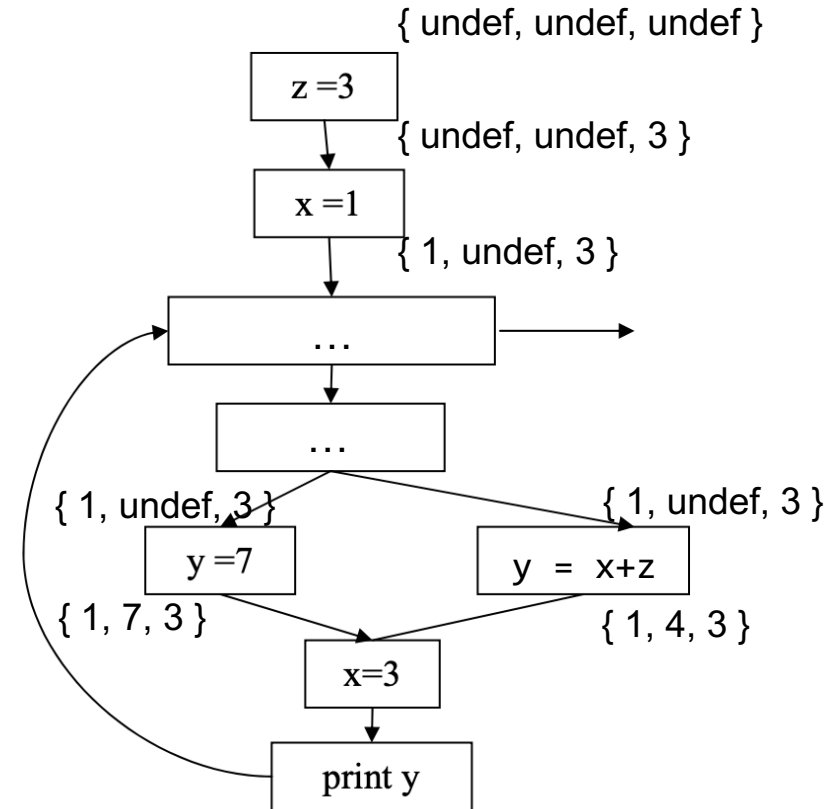


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Transfer function of $x = y + z$

- $m(y) = \text{Constant}, m(z) = \text{Constant}$
- $\Rightarrow m(x) = \text{Constant}$
- either of $m(y)$ and $m(z)$ is UNDEF
- $\Rightarrow m(x) = \text{UNDEF}$
- otherwise
- $\Rightarrow m(x) = \text{NAC}$

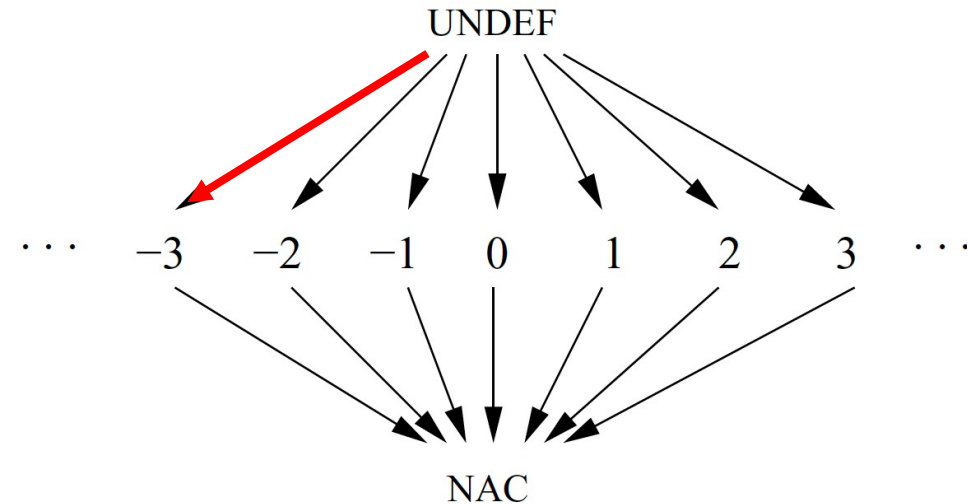


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$ **Why?**

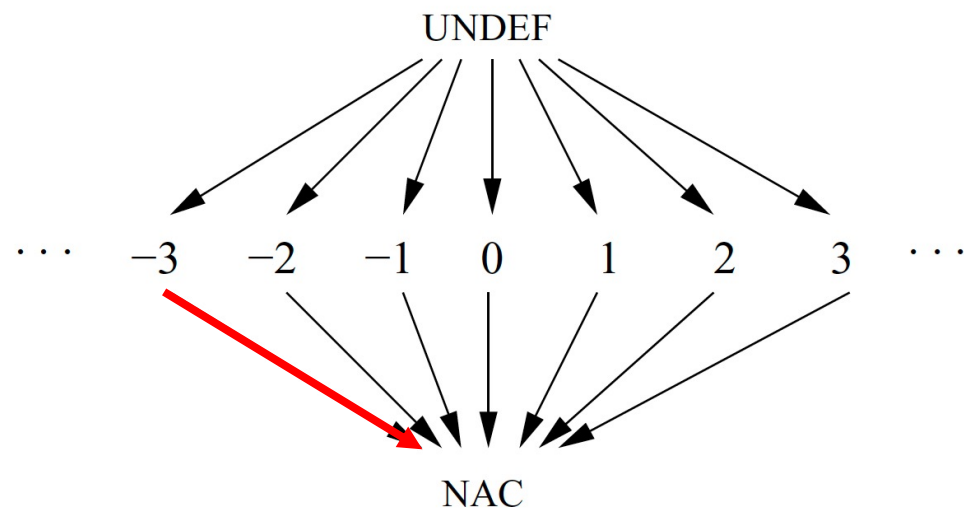


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$

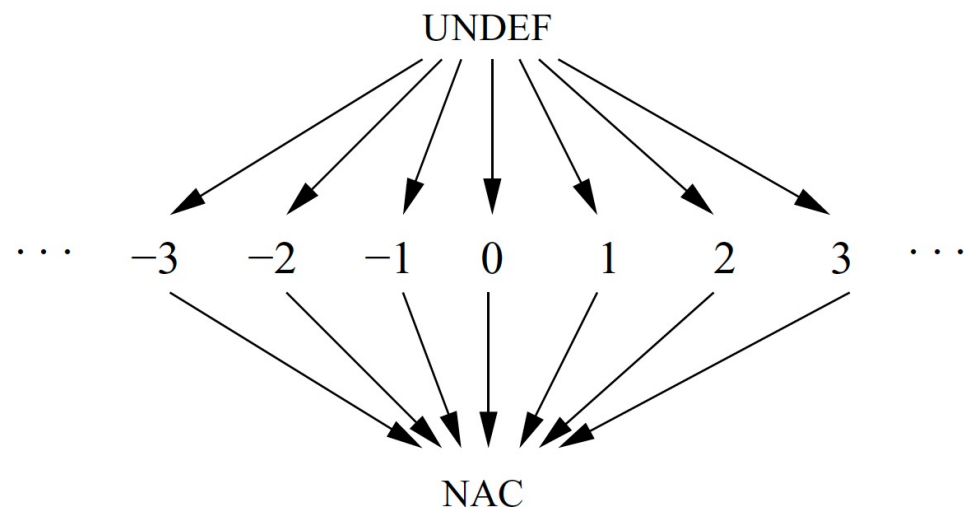


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$

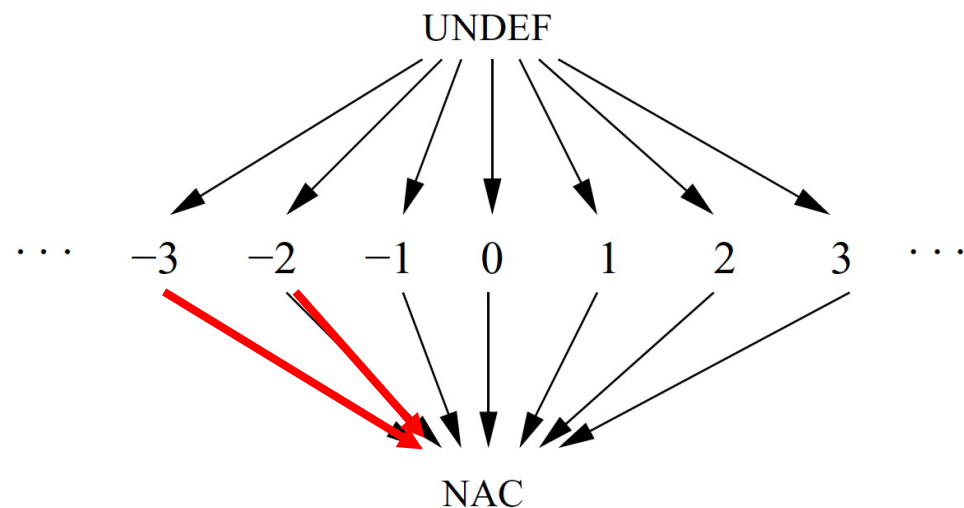


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF}, \text{Constant}, \text{NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

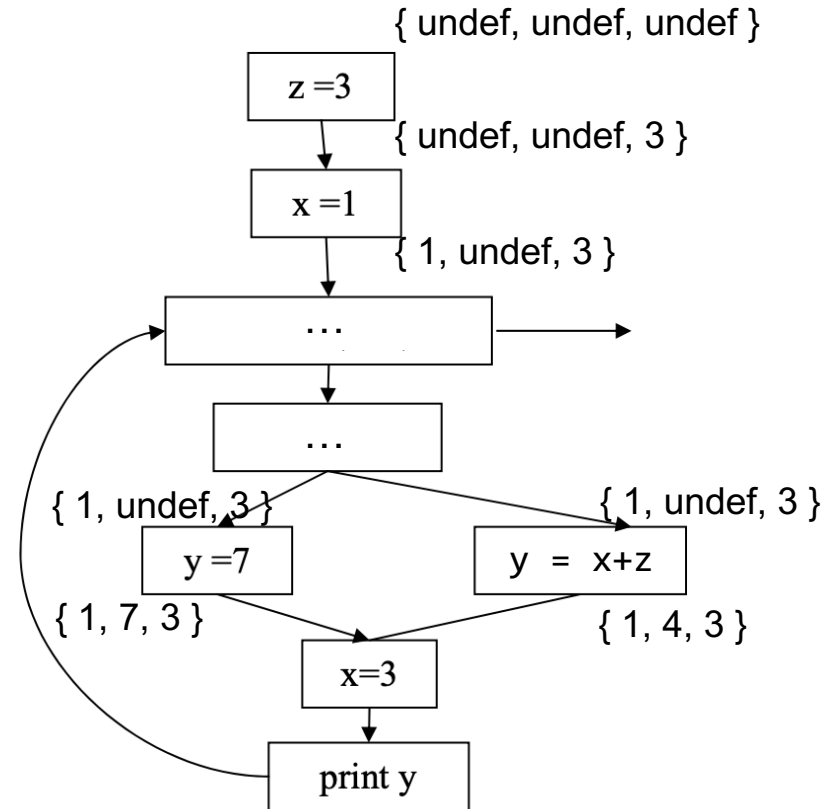


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

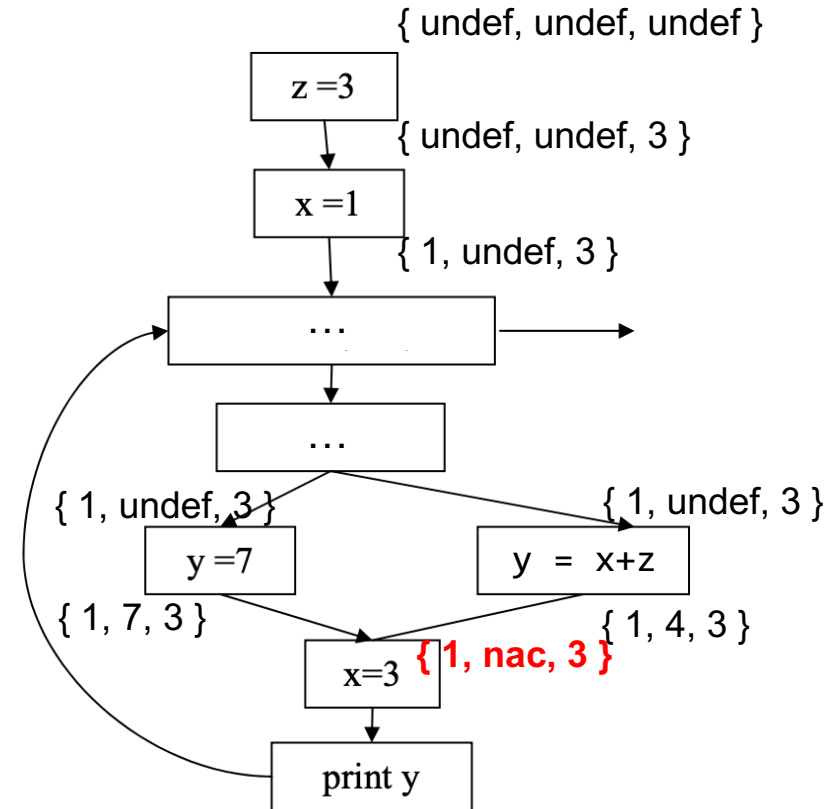


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

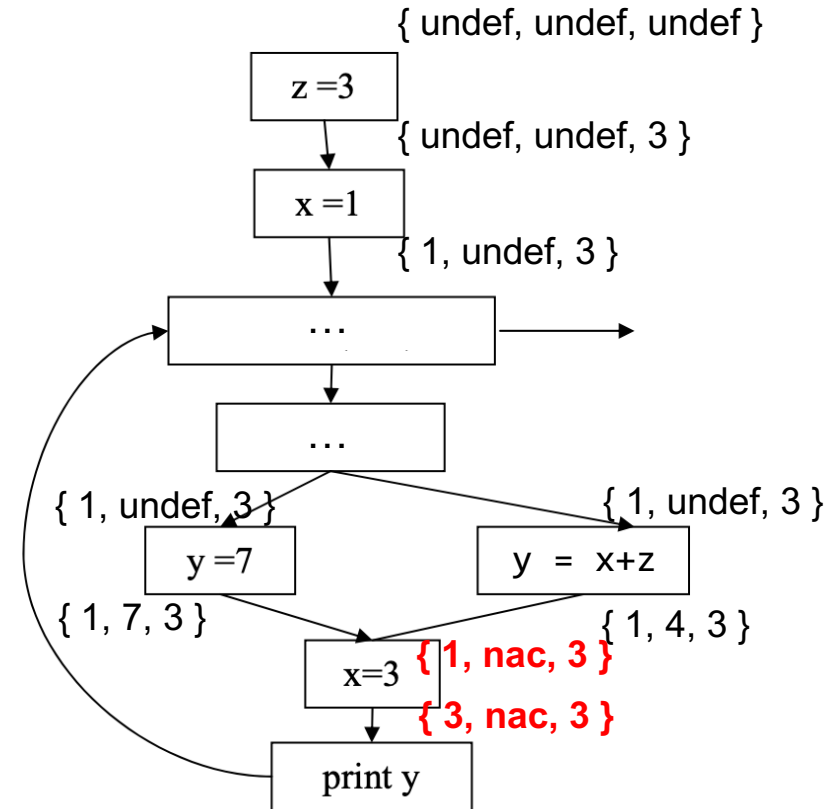


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

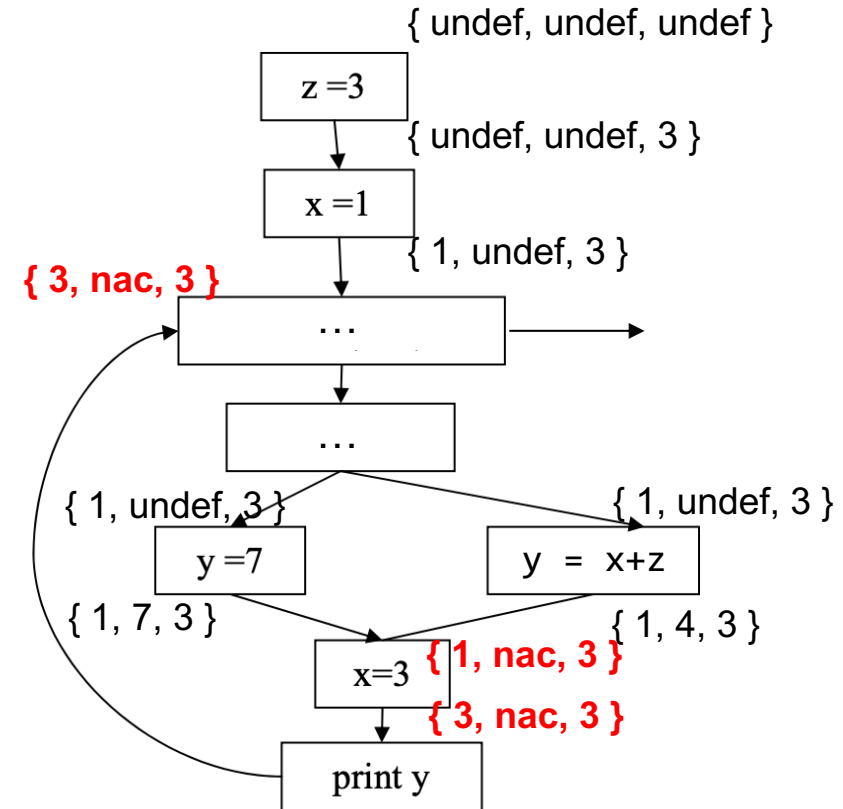


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

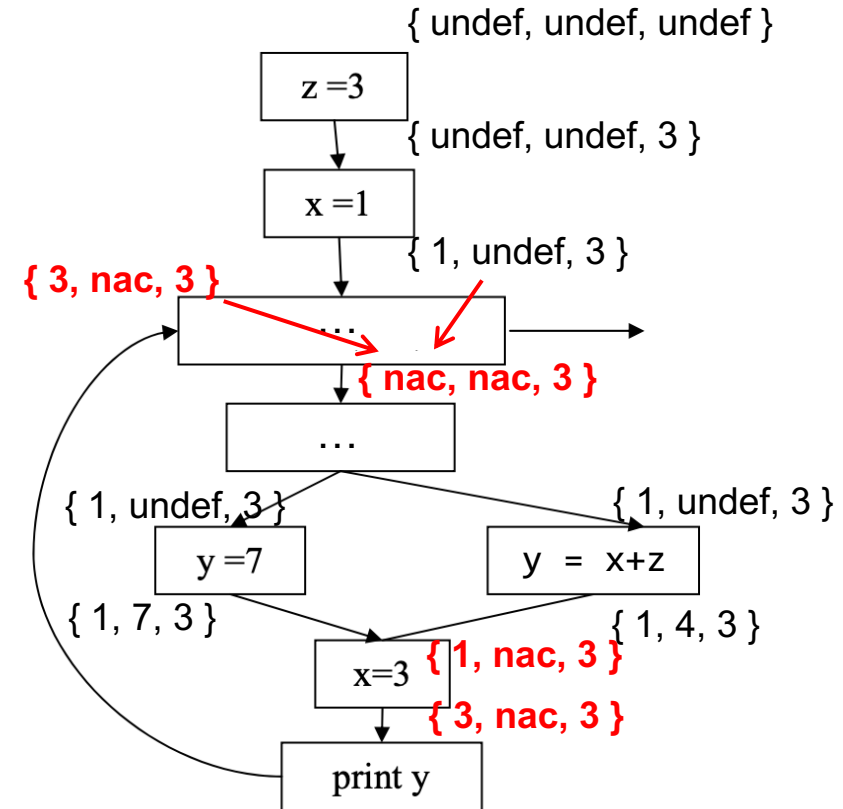


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

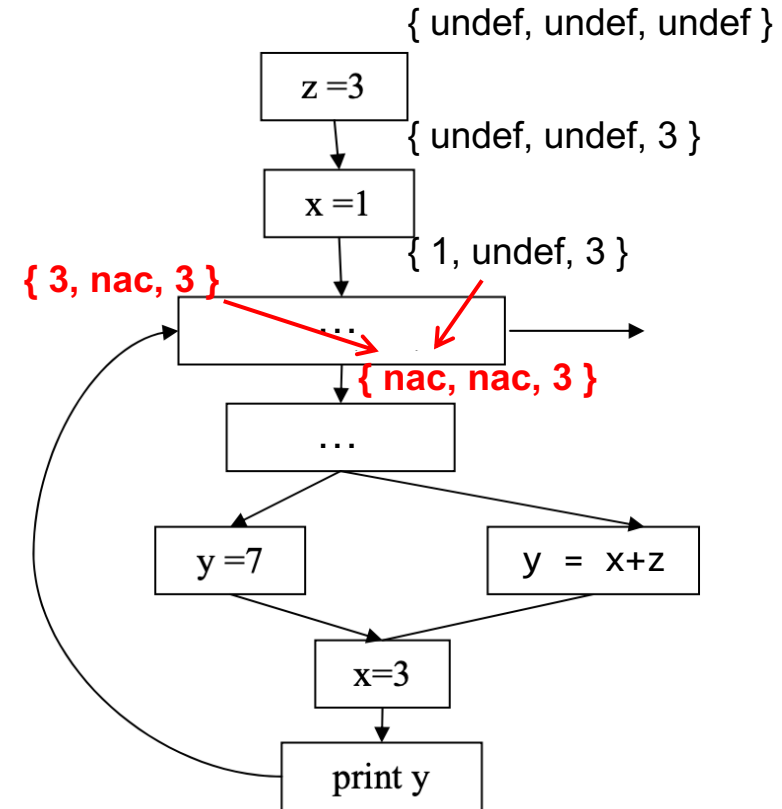


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

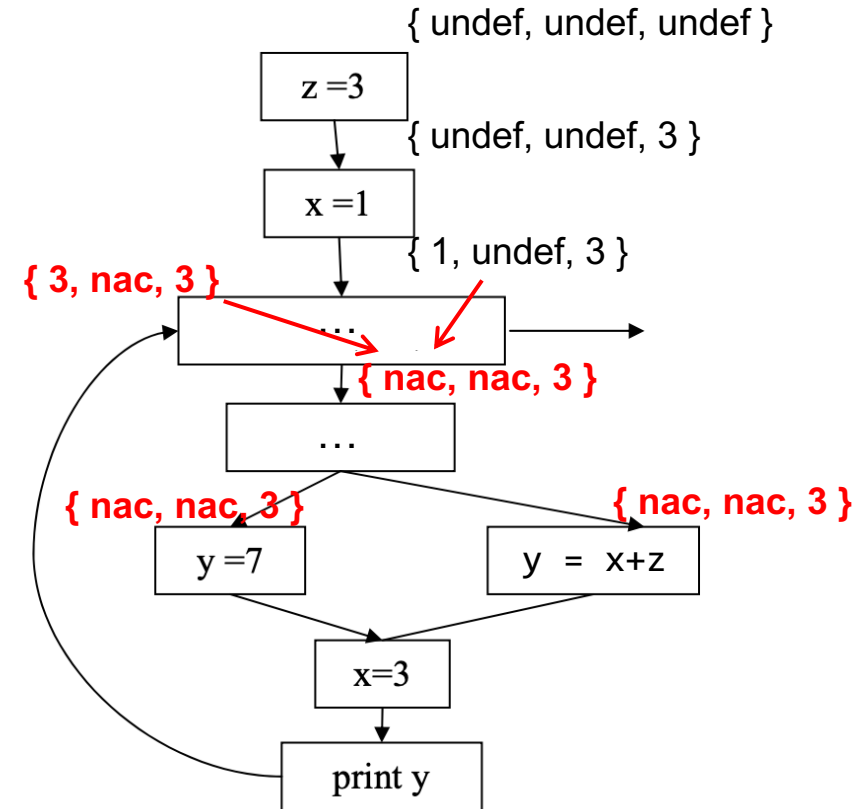


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

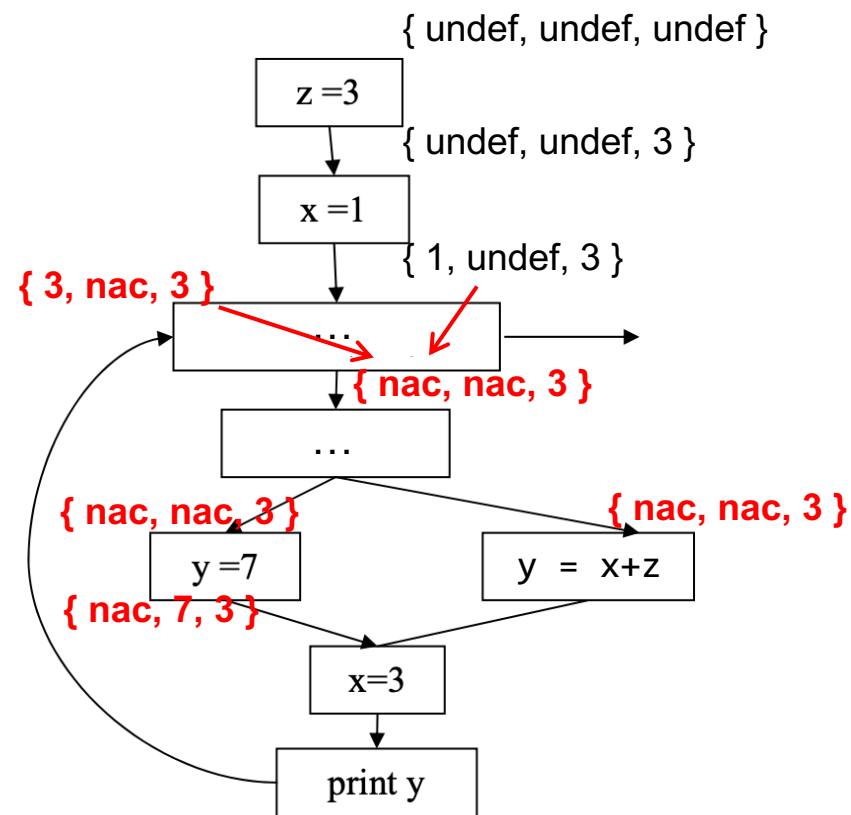


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

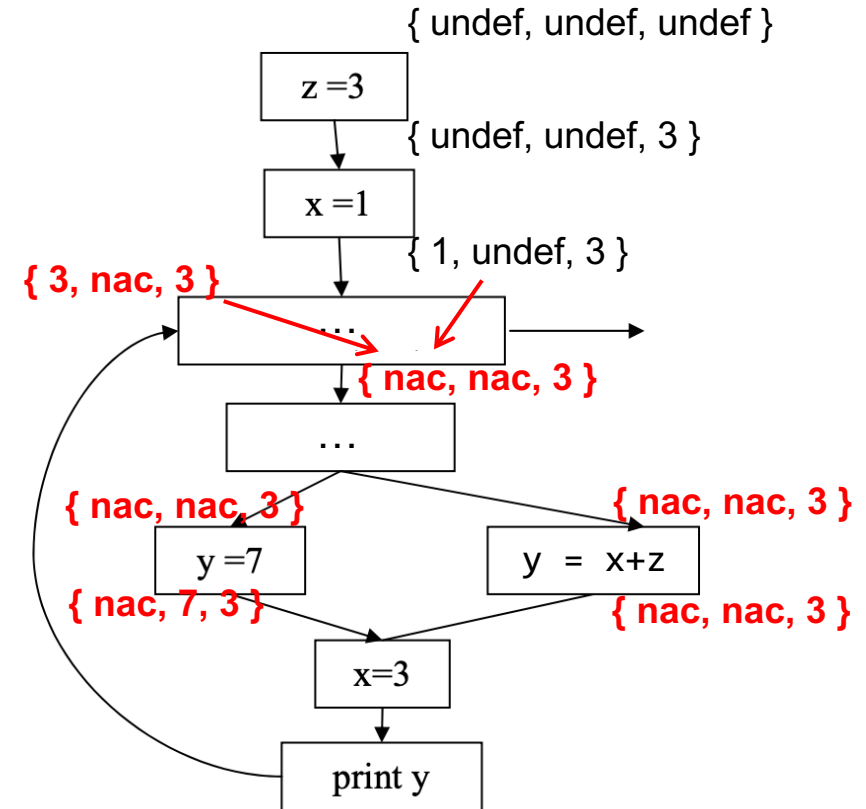


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

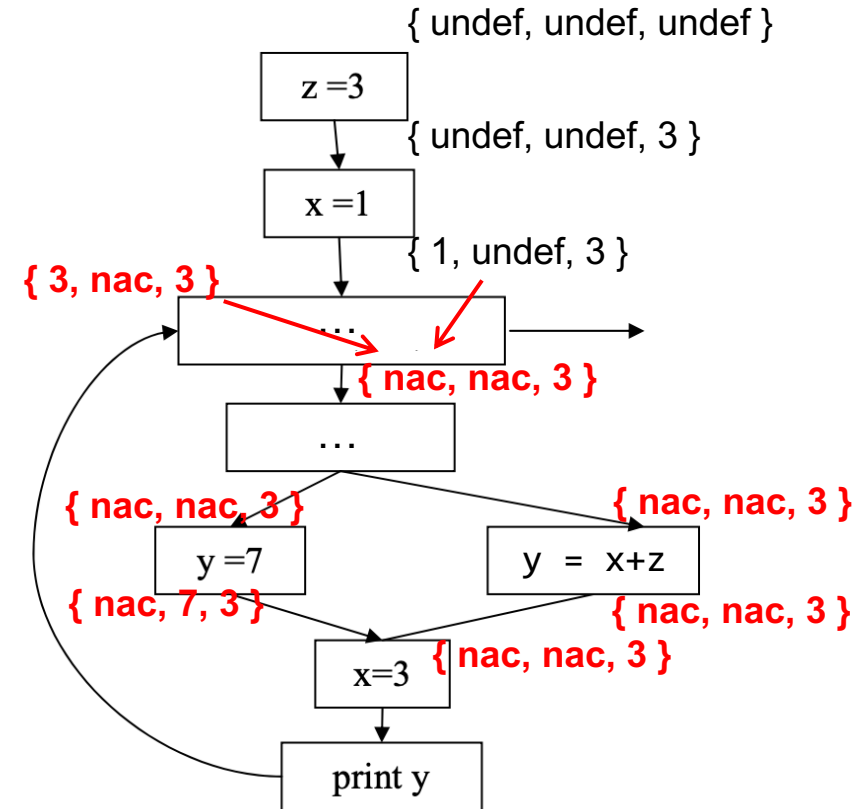


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

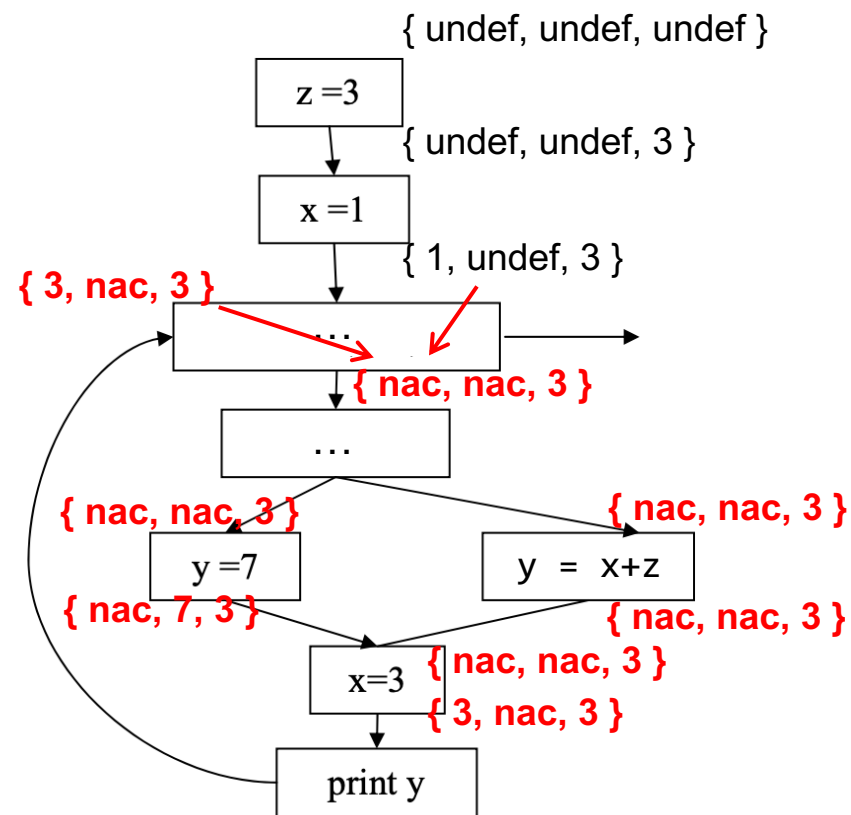


Constant Propagation

- Each variable $v \rightarrow \text{UNDEF, Constant, NAC}$, denoted as $m(v)$

Merge Function: $m(x) \wedge m(x')$

- $\text{UNDEF} \wedge \text{Constant} = \text{Constant}$
- $\text{NAC} \wedge \text{Constant} = \text{NAC}$
- $\text{Constant} \wedge \text{Constant} = \text{Constant}$
- $\text{Constant} \wedge \text{Constant}' = \text{NAC}$

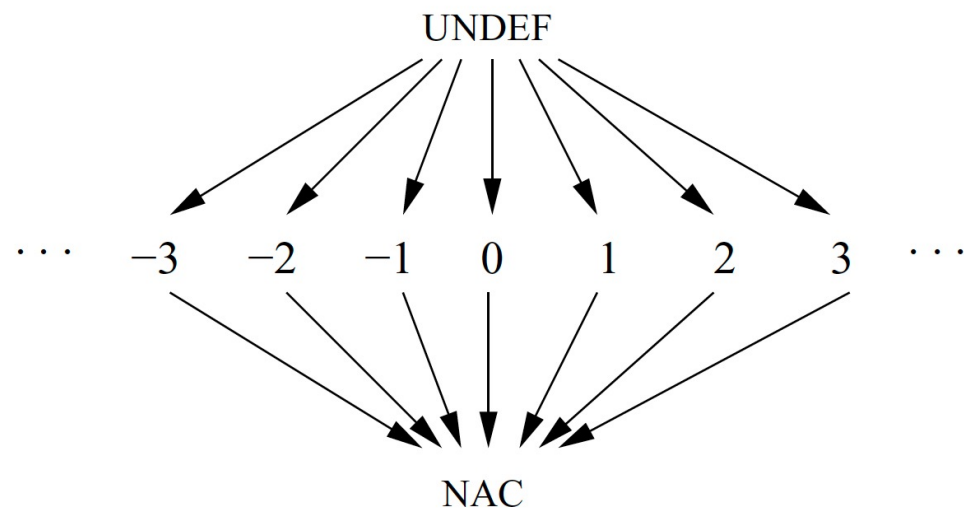


Constant Propagation

- Does a variable have a constant value at a program point?
- **Feature 1:** It has an unbounded set of possible data-flow values
- **Feature 2:** It is not distributive

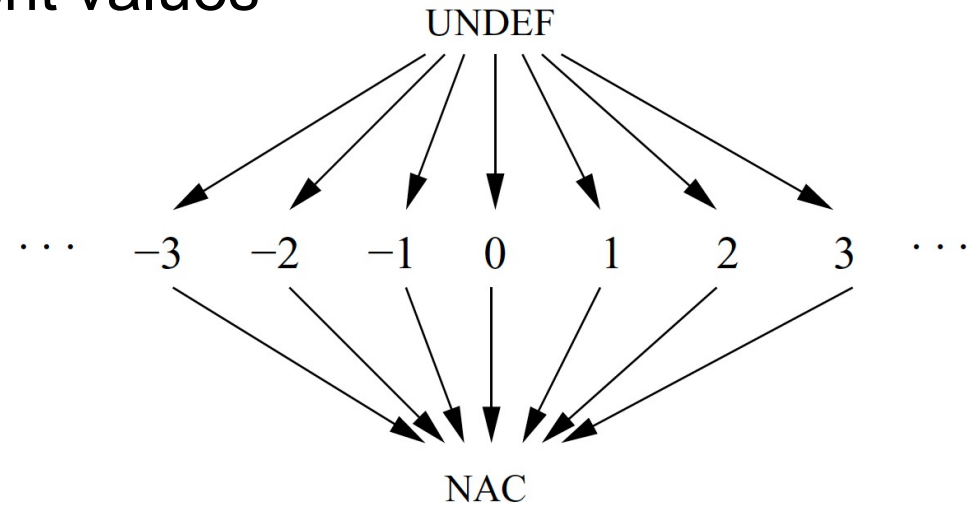
Unboundedness

- Each variable may correspond to infinite number of constants



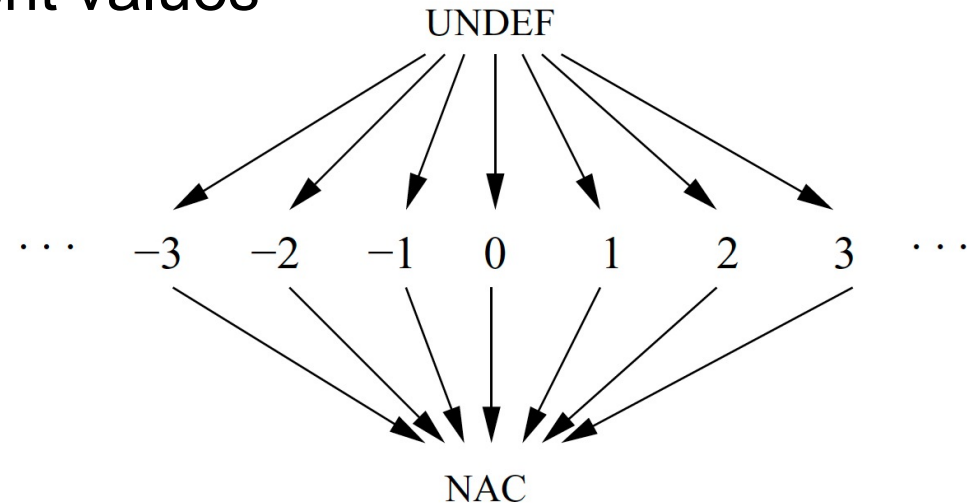
Monotonicity

- At each program point, $m(x)$ always starts with UNDEF
 - $\Rightarrow$ become constant after assignment
 - $\Rightarrow$ become NAC if assigned different values



Monotonicity

- At each program point, $m(x)$ always starts with UNDEF
 - $\Rightarrow$ become constant after assignment
 - $\Rightarrow$ become NAC if assigned different values
- At each program point
 - # NAC always increases



Non-distributivity

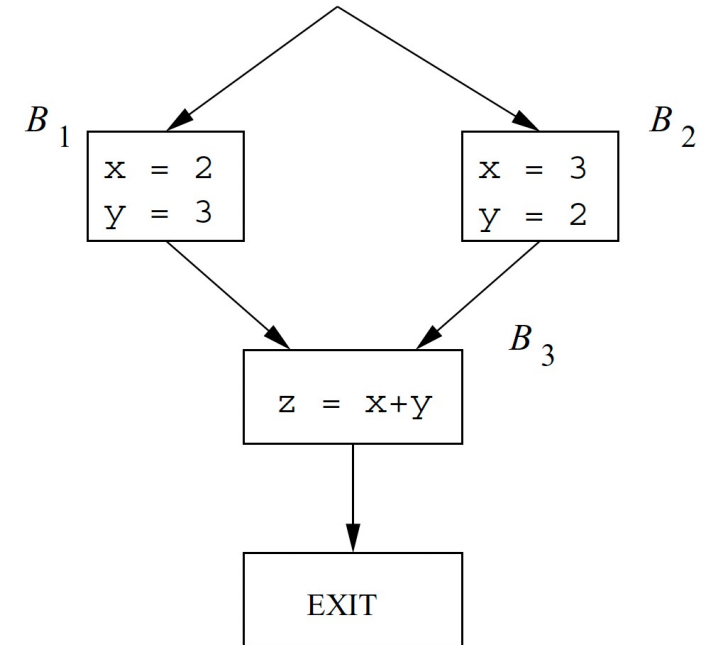
- $f(a \wedge b) \neq f(a) \wedge f(b)$

Non-distributivity

- $f(a \wedge b) \neq f(a) \wedge f(b) \Rightarrow$ Loss of precision in DFA!
- Meet-Then-Transfer $\neq$ Transfer-Then-Meet

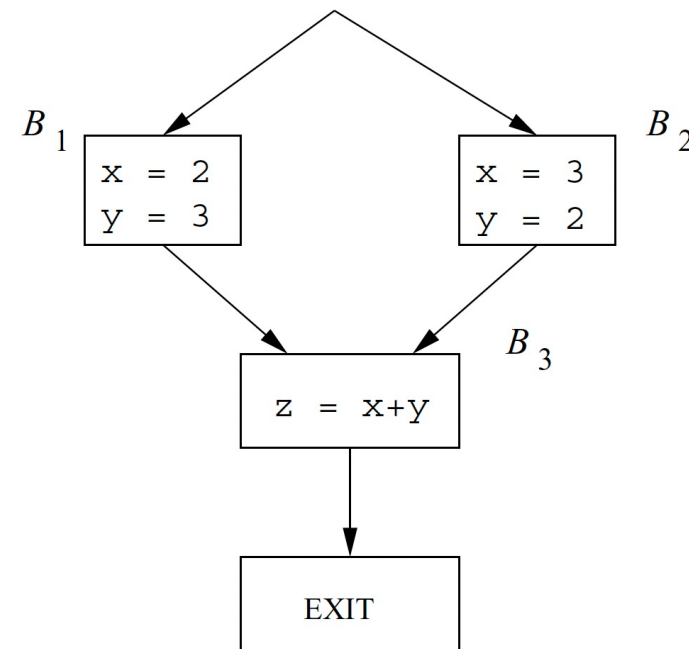
Non-distributivity

- $f(a \wedge b) \neq f(a) \wedge f(b) \Rightarrow$ Loss of precision in DFA!
- Meet-Then-Transfer $\neq$ Transfer-Then-Meet
- Meet-Then-Transfer
 - $m(x) = \text{NAC}, m(y) = \text{NAC} \Rightarrow m(z) = \text{NAC}$



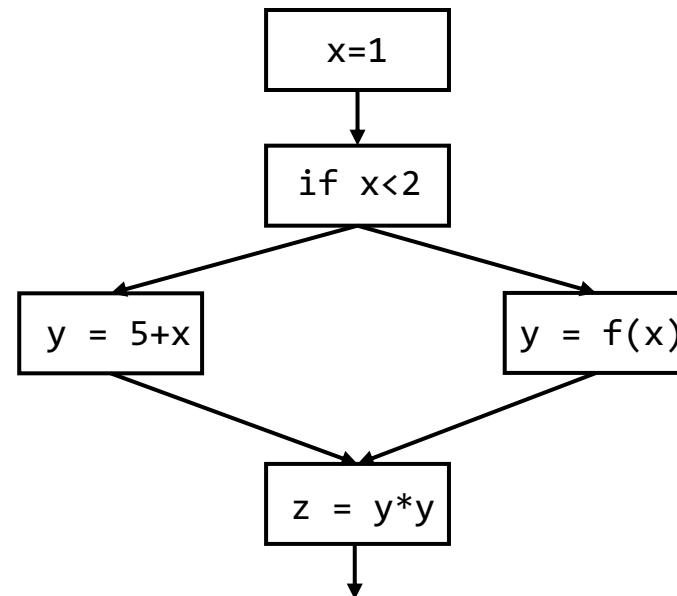
Non-distributivity

- $f(a \wedge b) \neq f(a) \wedge f(b) \Rightarrow$ Loss of precision in DFA!
- Meet-Then-Transfer $\neq$ Transfer-Then-Meet
- Meet-Then-Transfer
 - $m(x) = \text{NAC}, m(y) = \text{NAC} \Rightarrow m(z) = \text{NAC}$
- Transfer-Then-Meet
 - $m(z) = 5, m(z) = 5 \Rightarrow m(z) = 5$



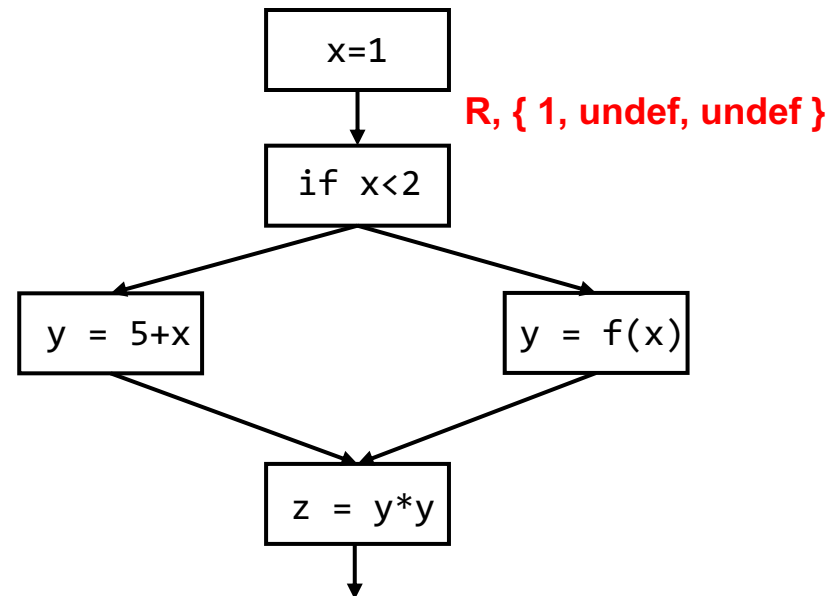
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition $\Rightarrow$ Precision improvement!



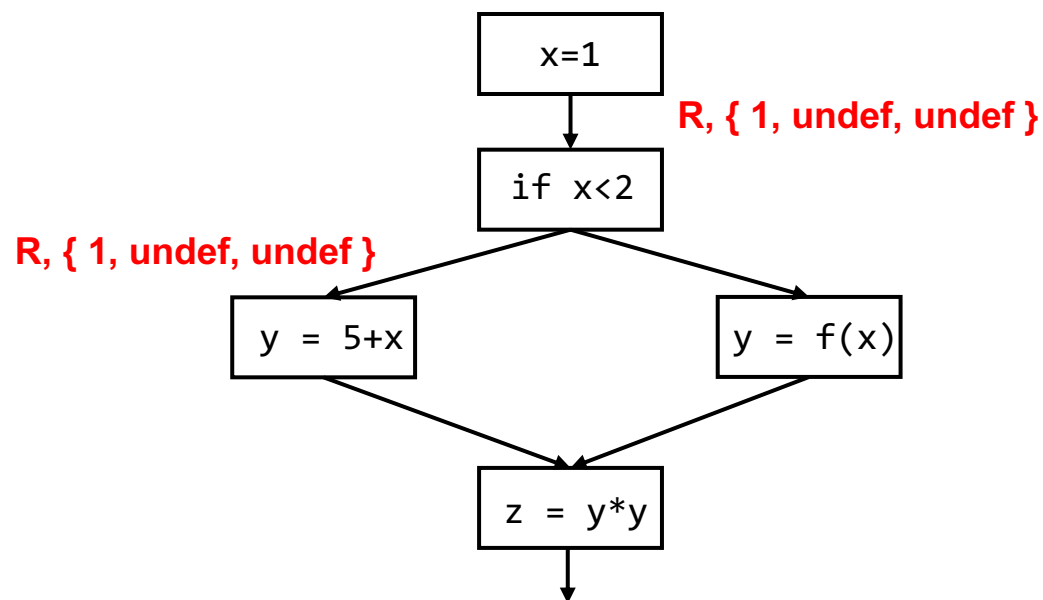
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition $\Rightarrow$ Precision improvement!



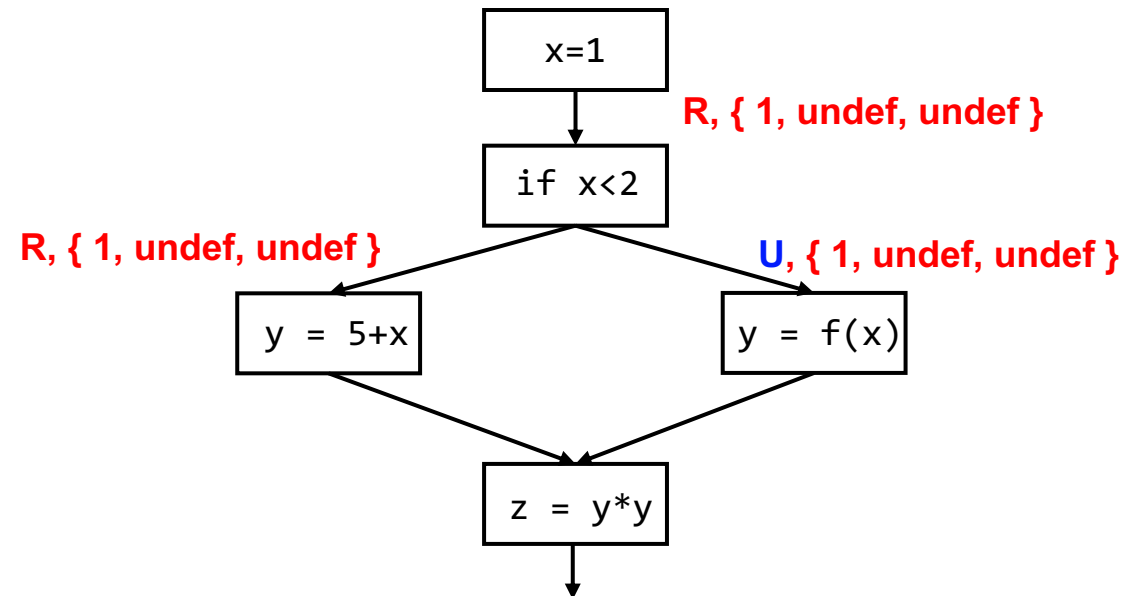
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition $\Rightarrow$ Precision improvement!



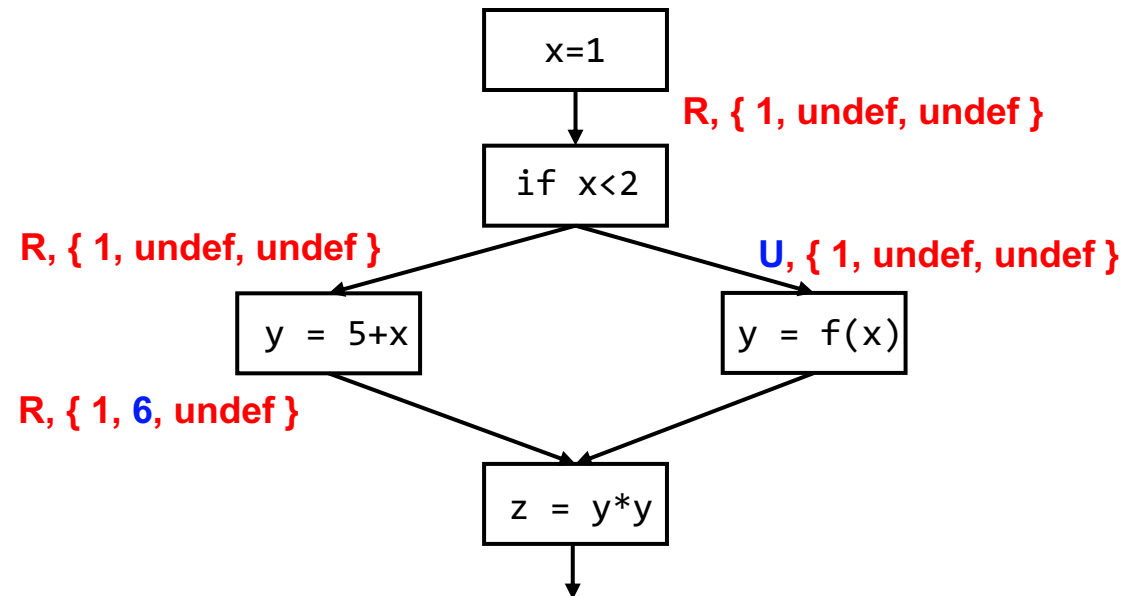
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => Precision improvement!



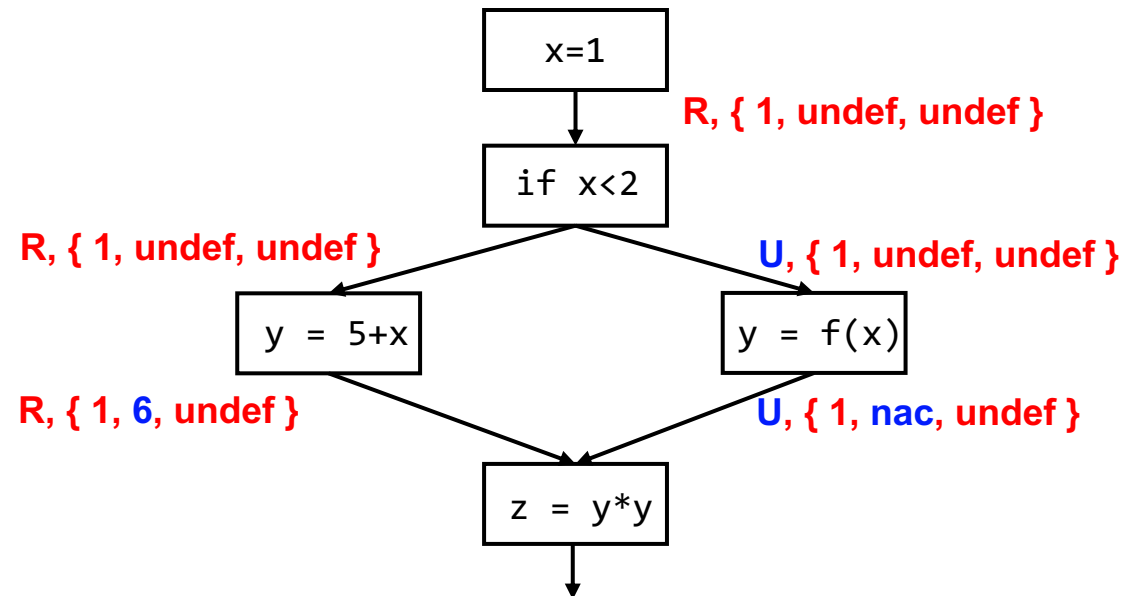
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => Precision improvement!



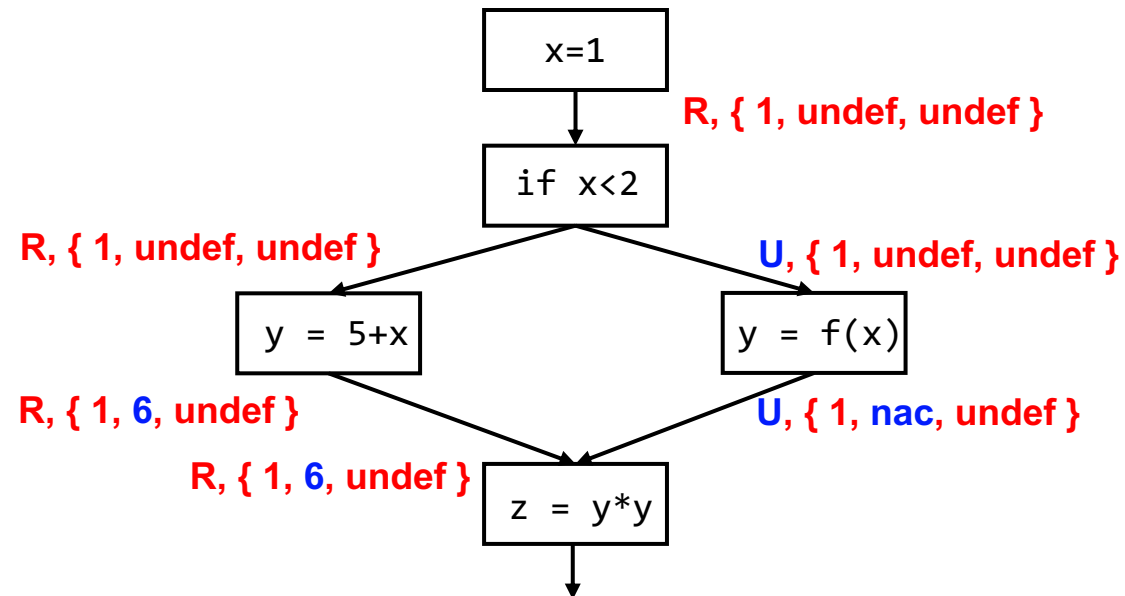
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => Precision improvement!



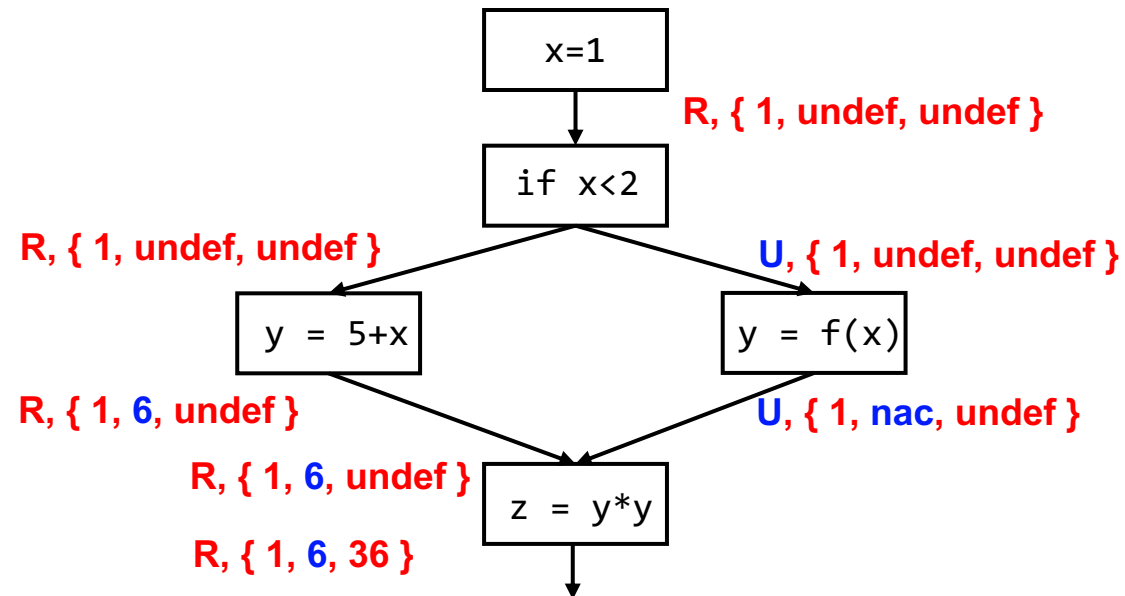
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => Precision improvement!



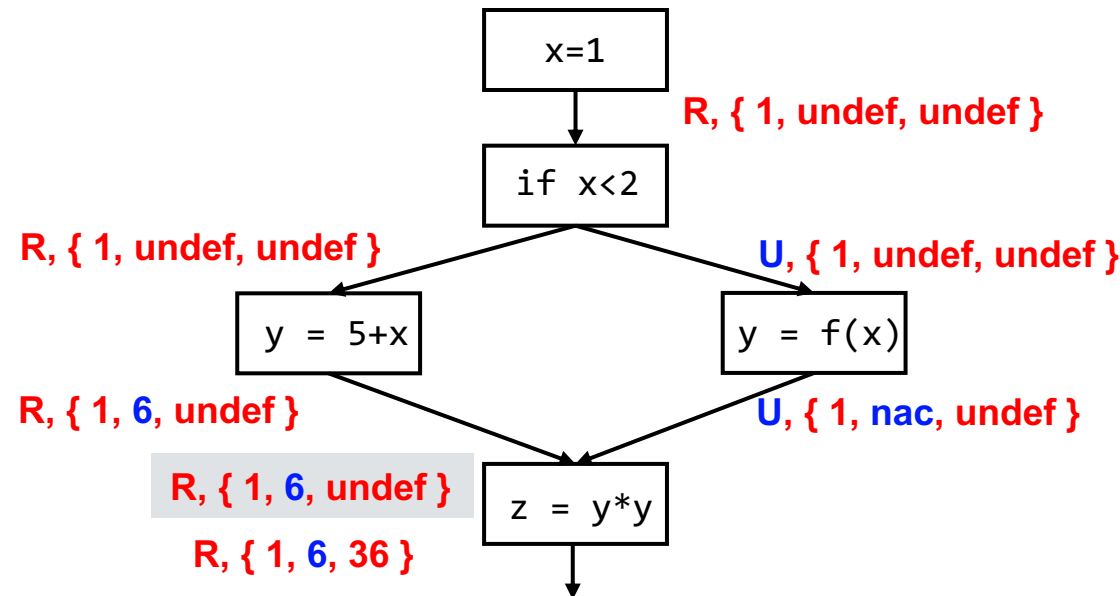
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => Precision improvement!



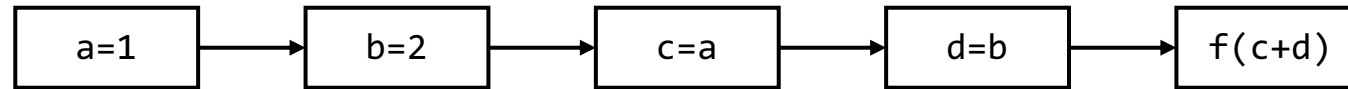
Conditional Const. Propagation

- Check the reachability of the code
- Check the branch condition => **Precision improvement!**

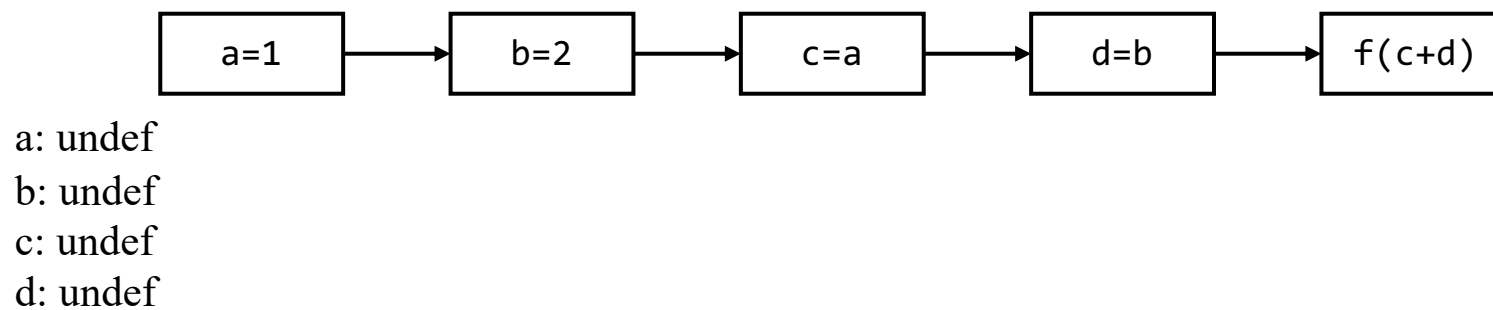


PART IV: Sparse DFA

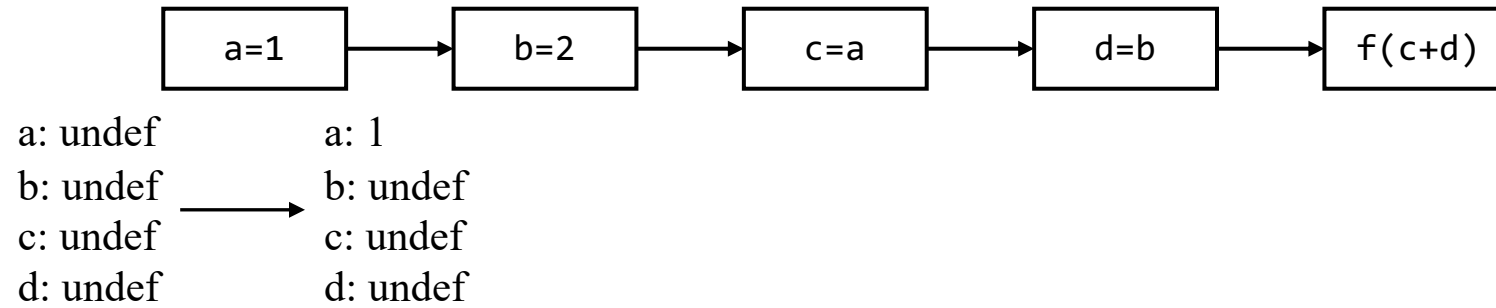
Constant Propagation



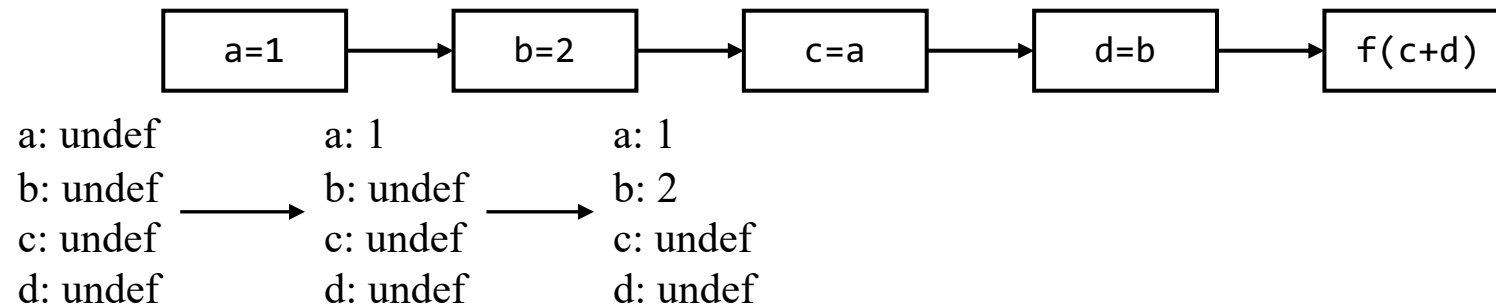
Constant Propagation



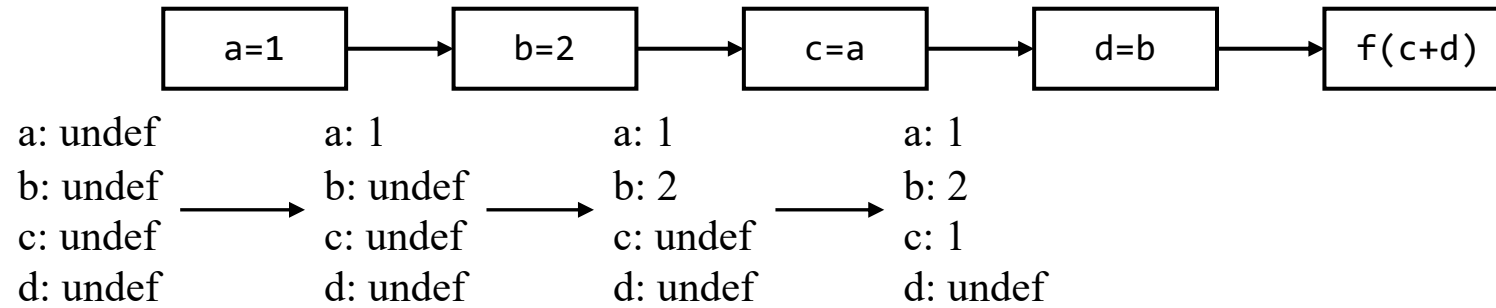
Constant Propagation



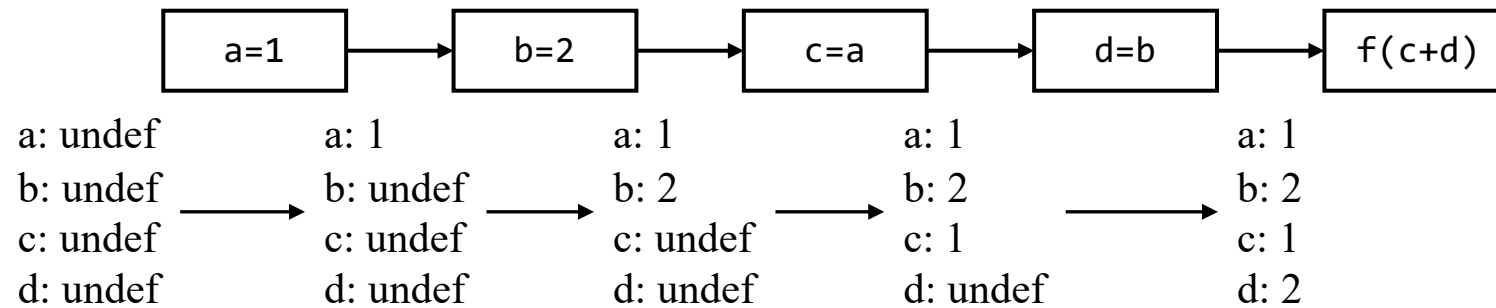
Constant Propagation



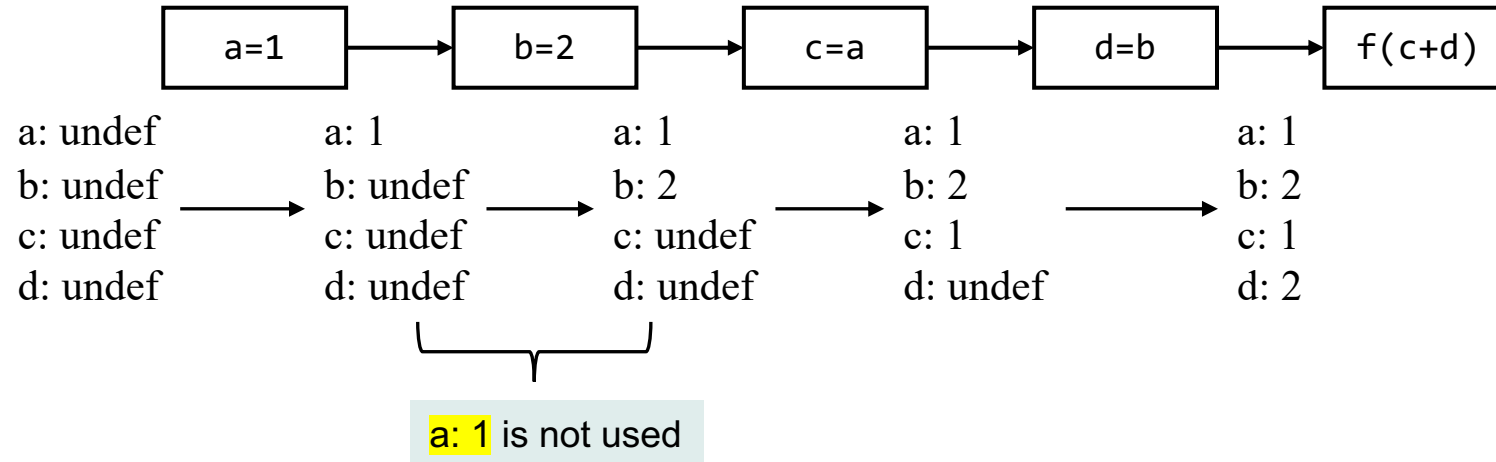
Constant Propagation



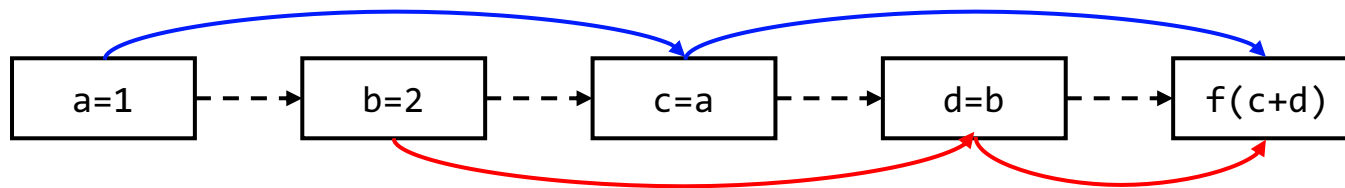
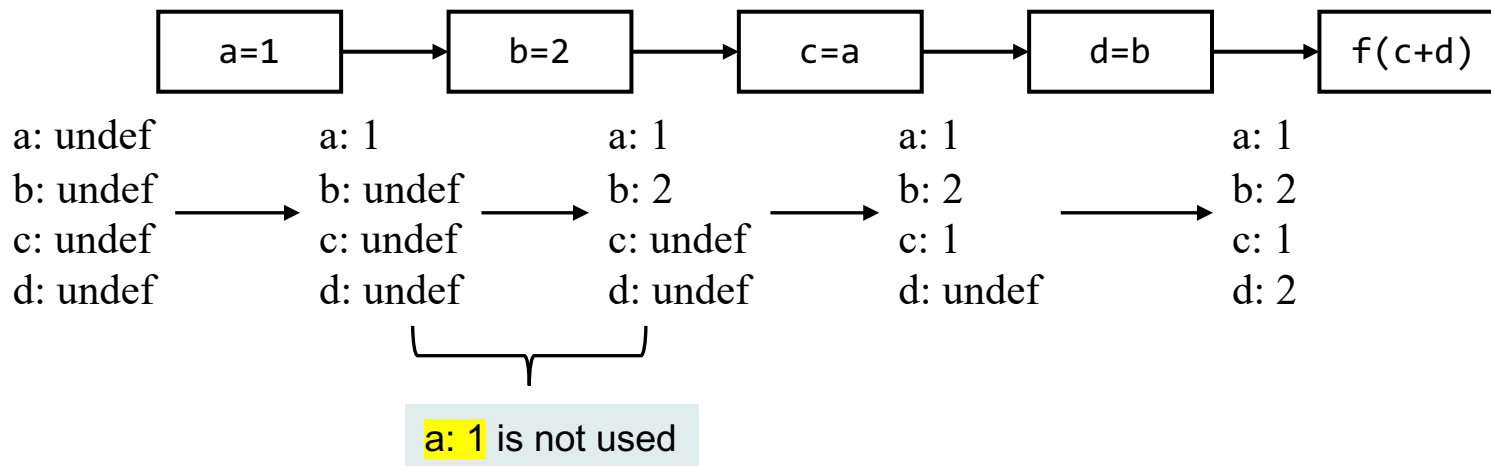
Constant Propagation



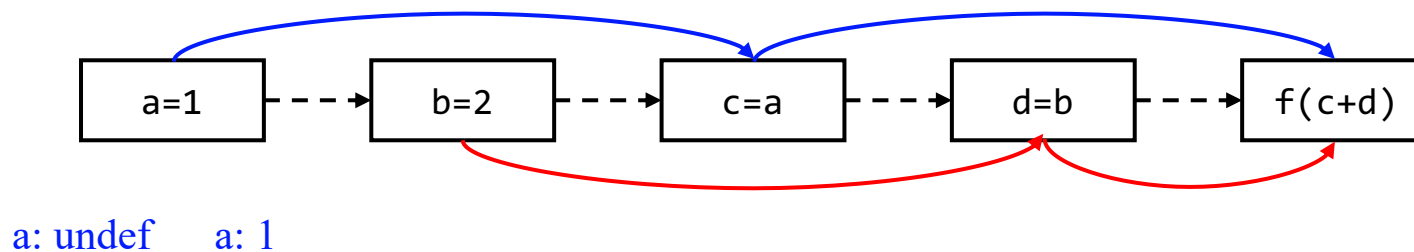
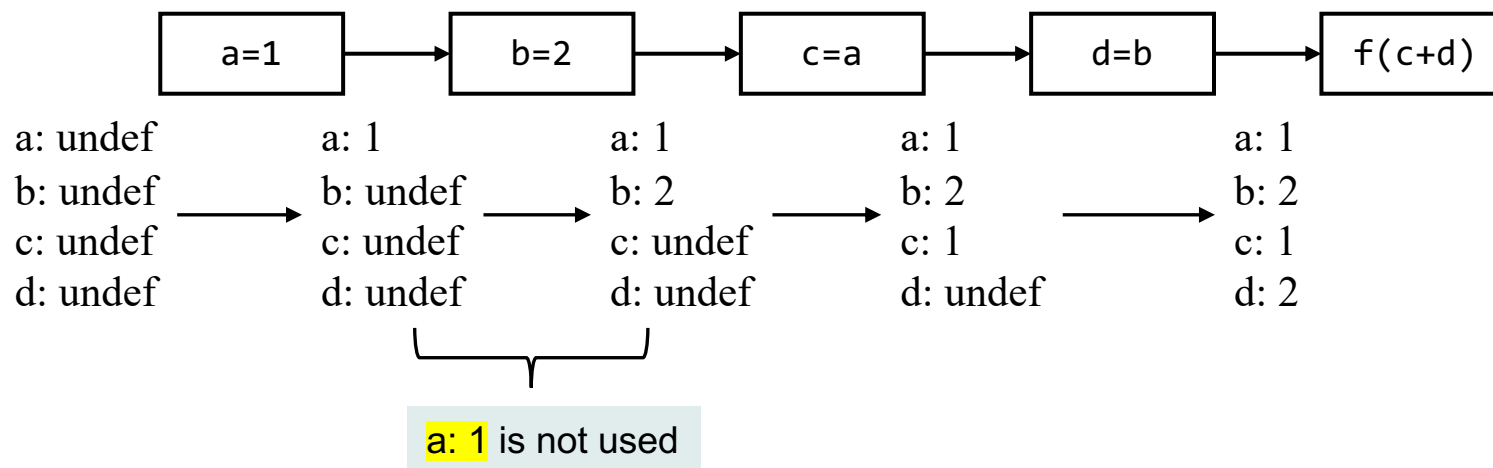
Constant Propagation



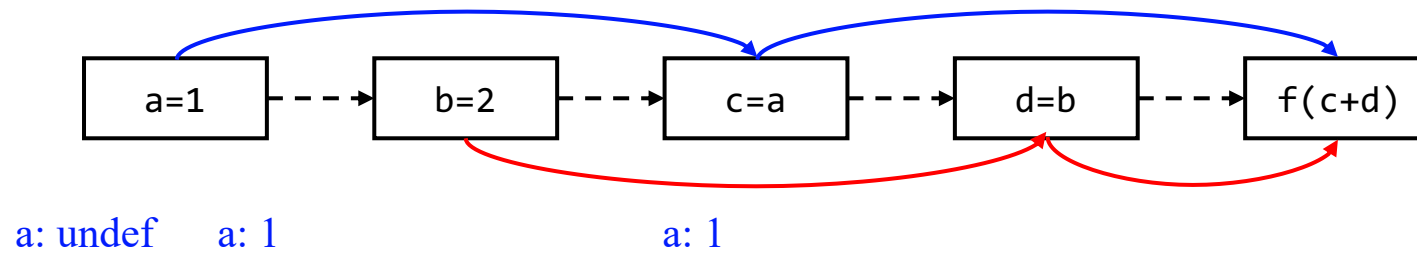
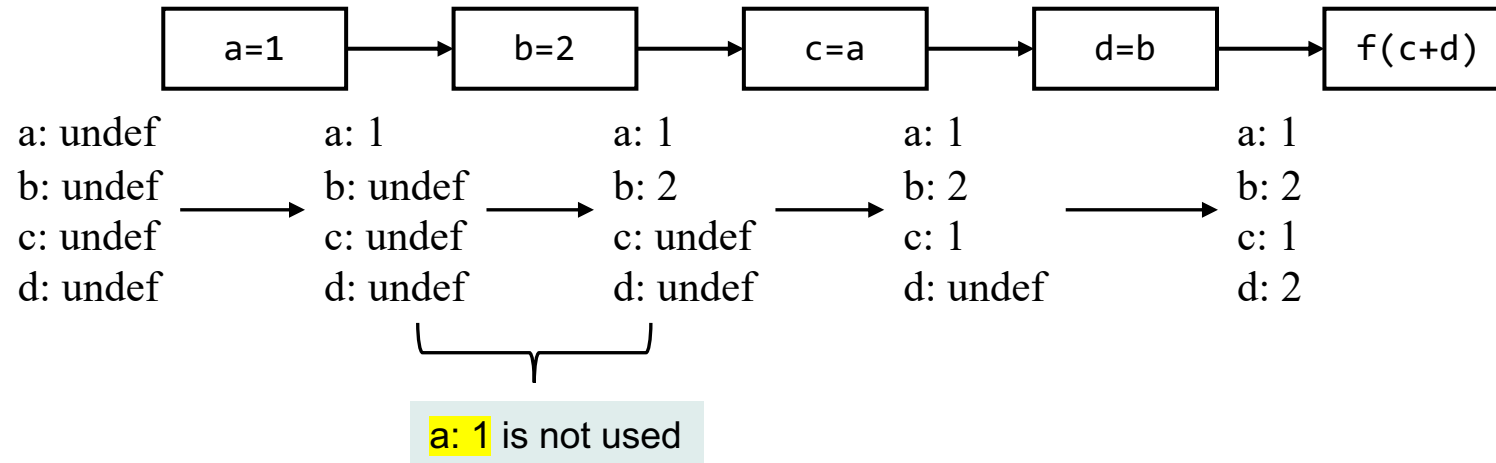
Constant Propagation



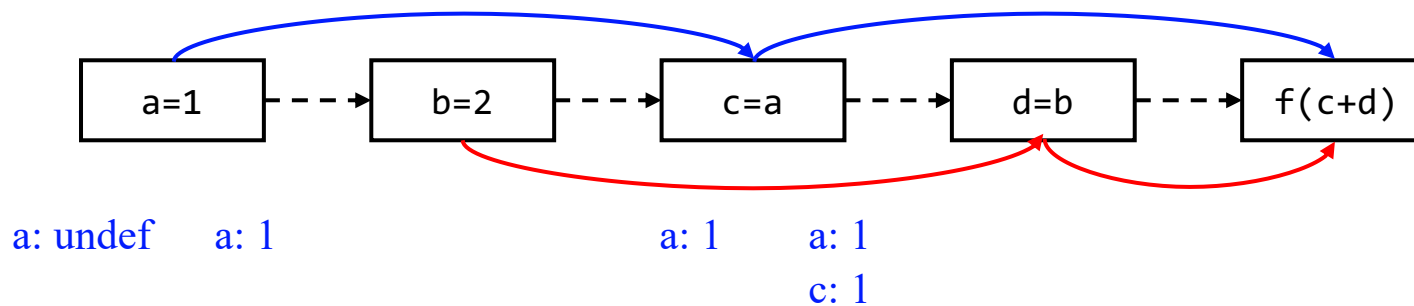
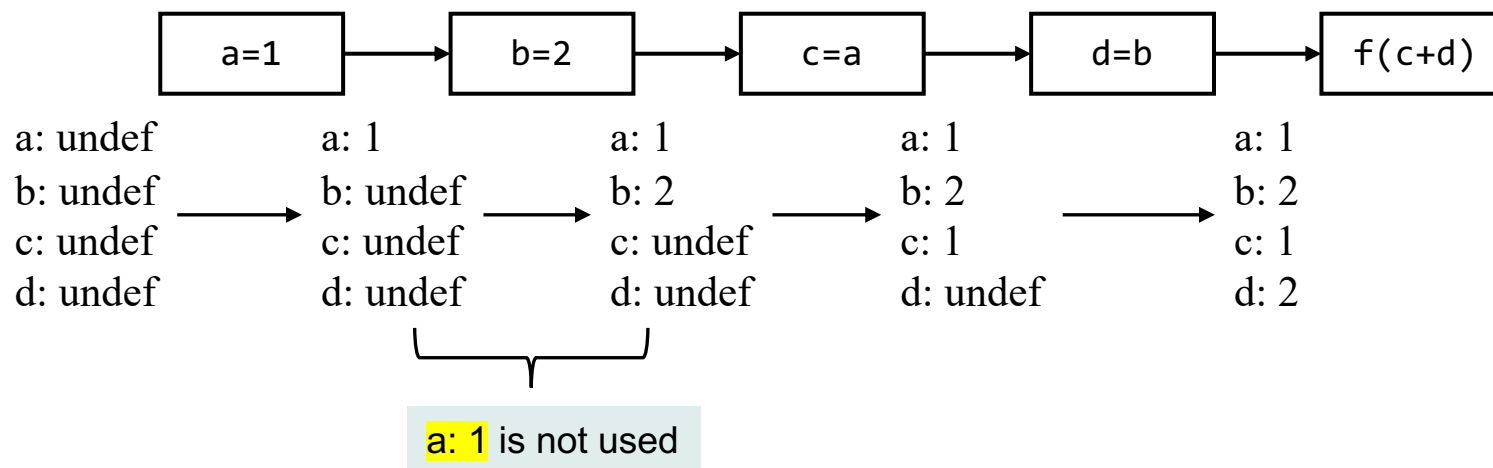
Constant Propagation



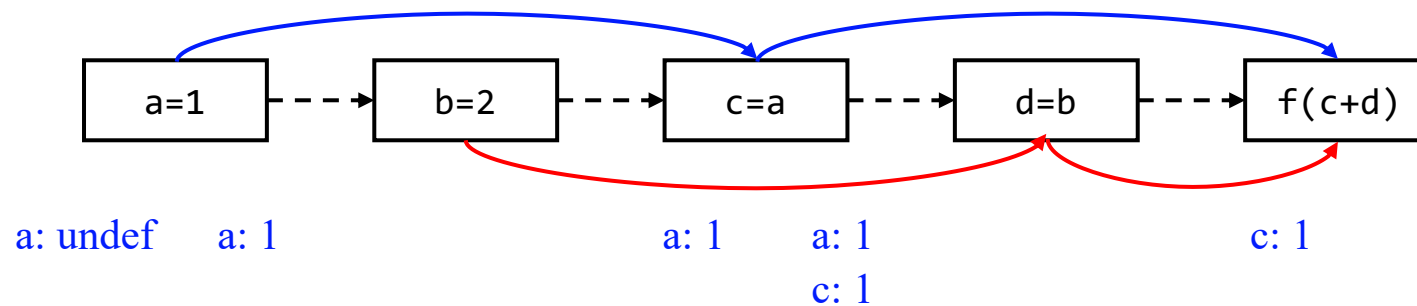
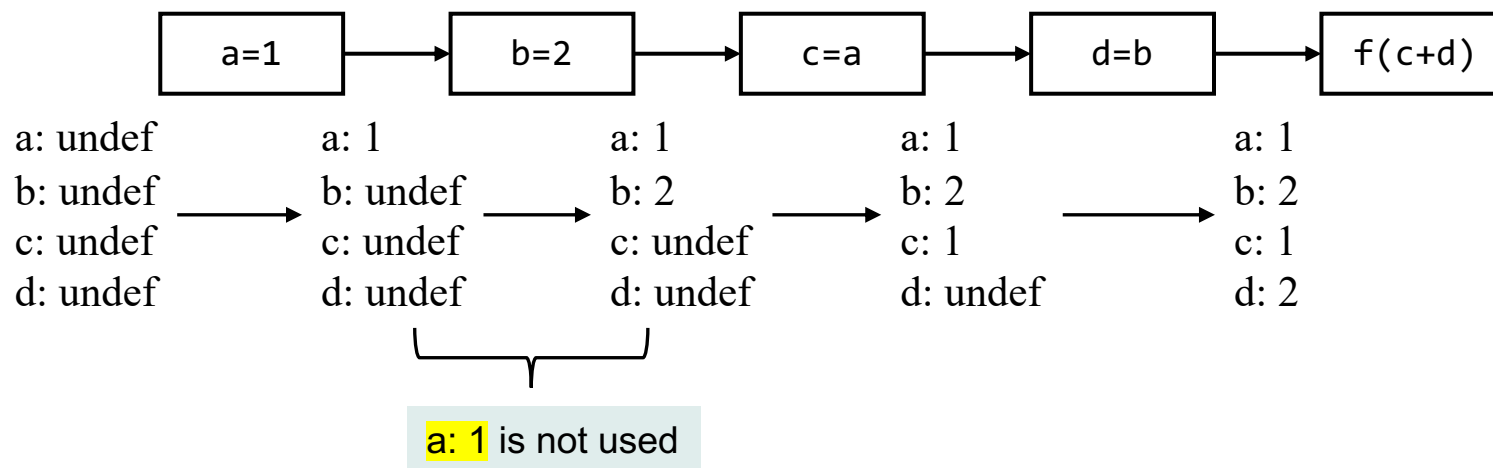
Constant Propagation



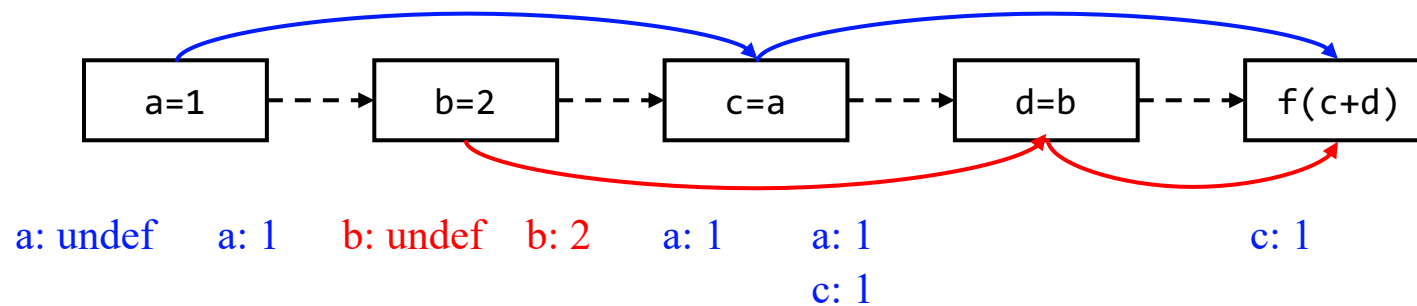
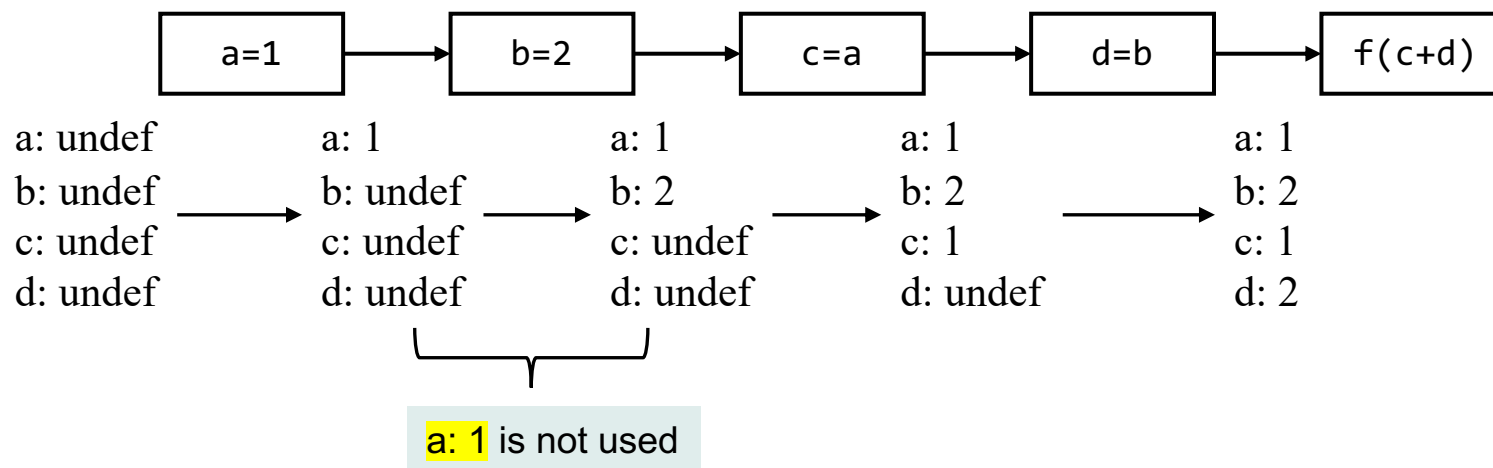
Constant Propagation



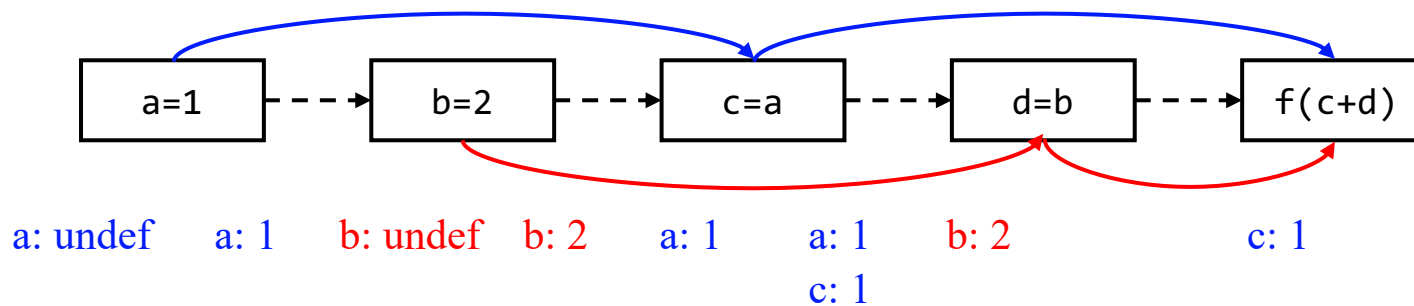
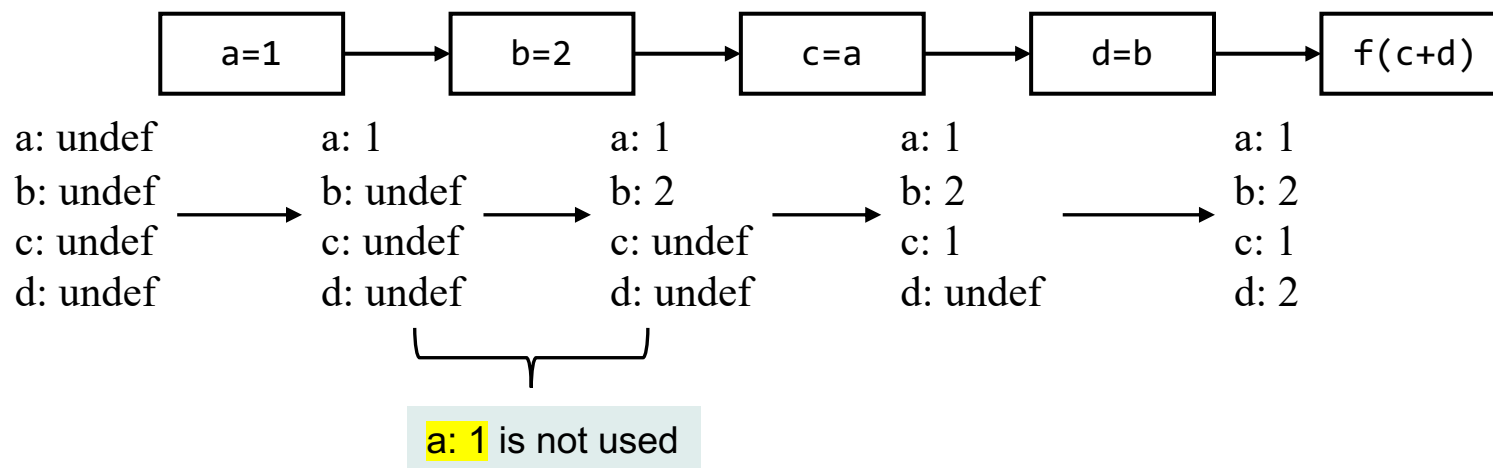
Constant Propagation



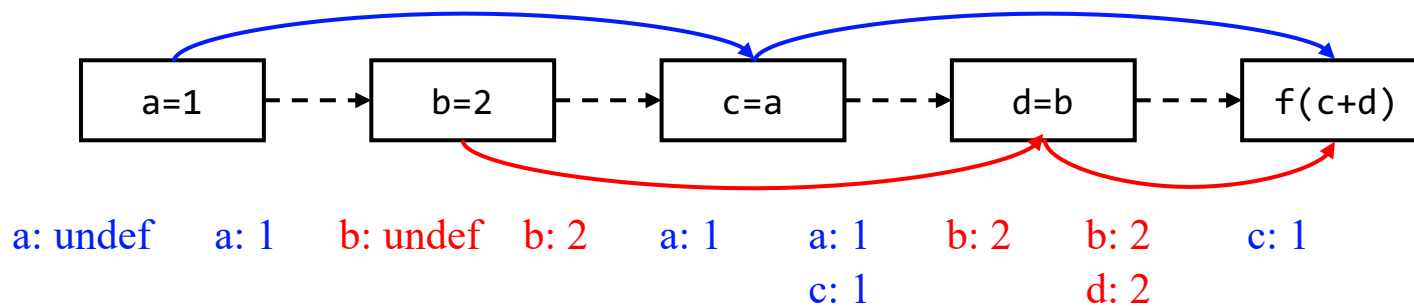
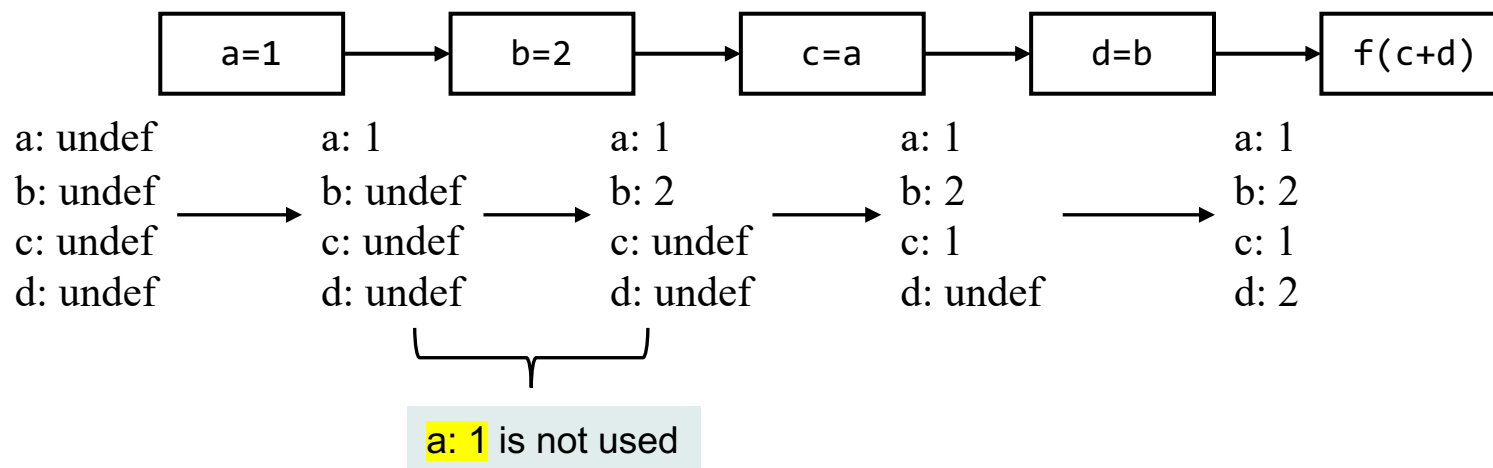
Constant Propagation



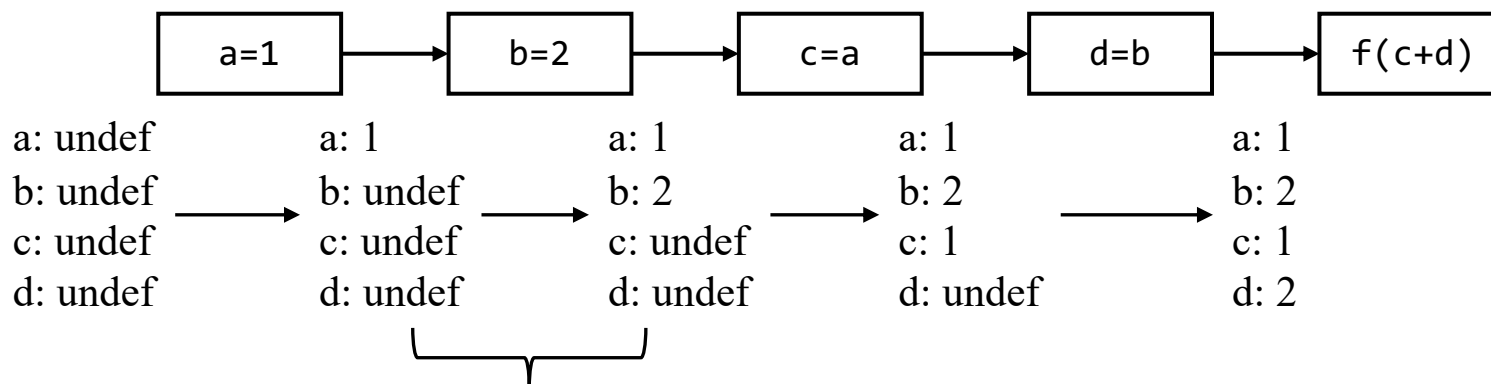
Constant Propagation



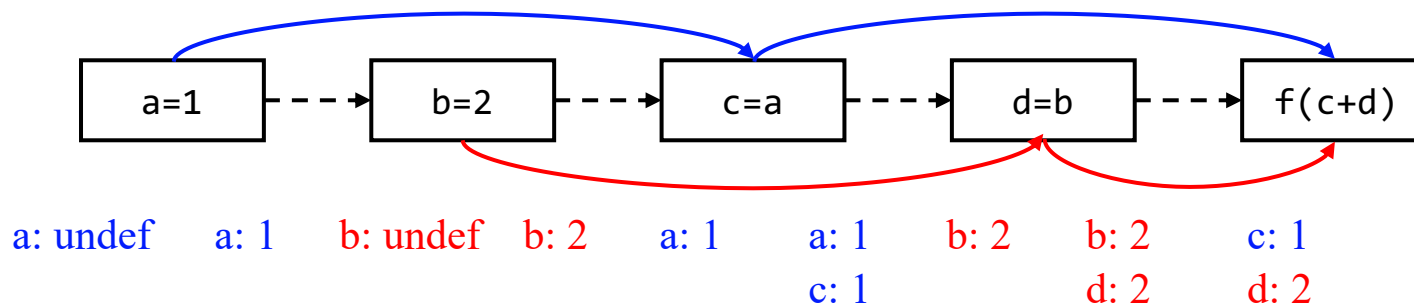
Constant Propagation



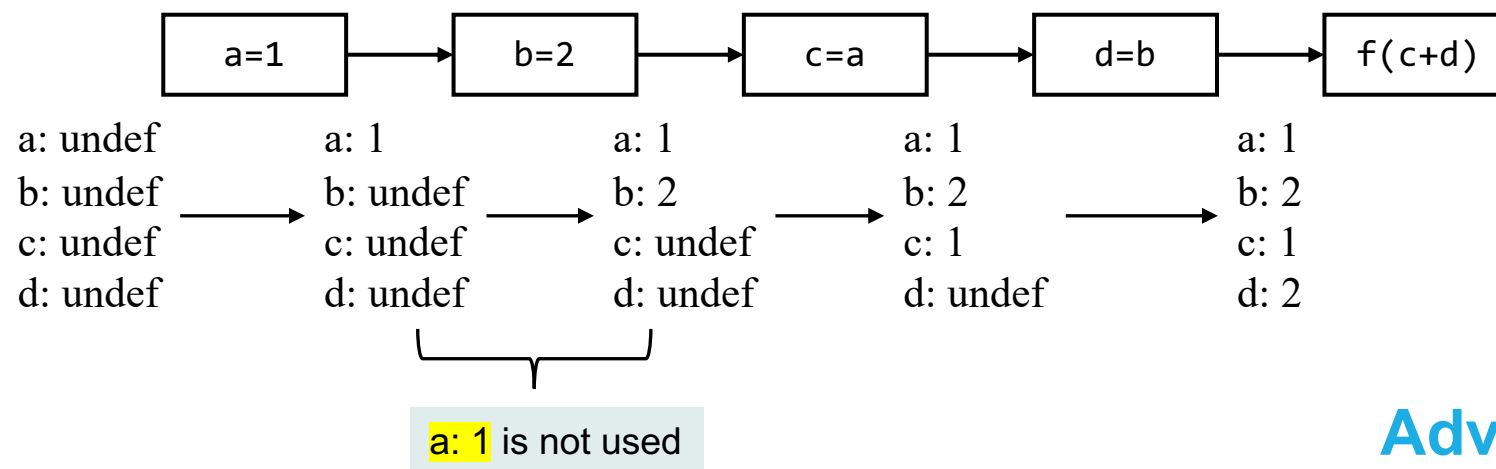
Constant Propagation



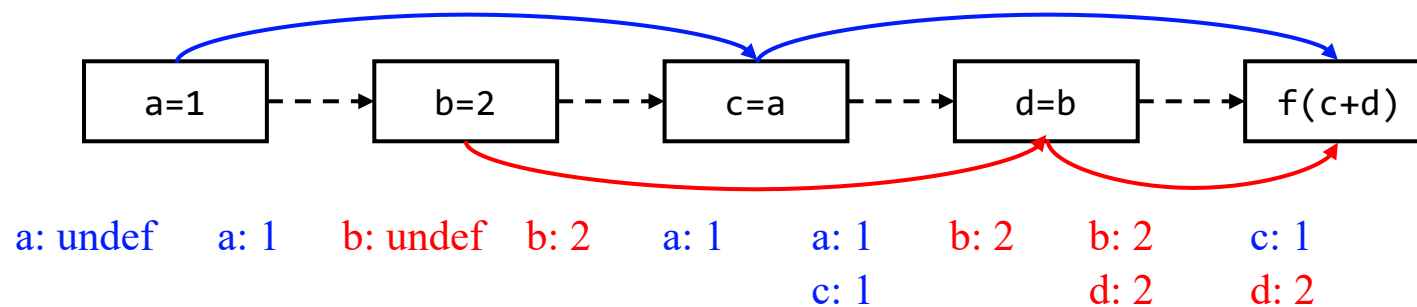
a: 1 is not used



Constant Propagation

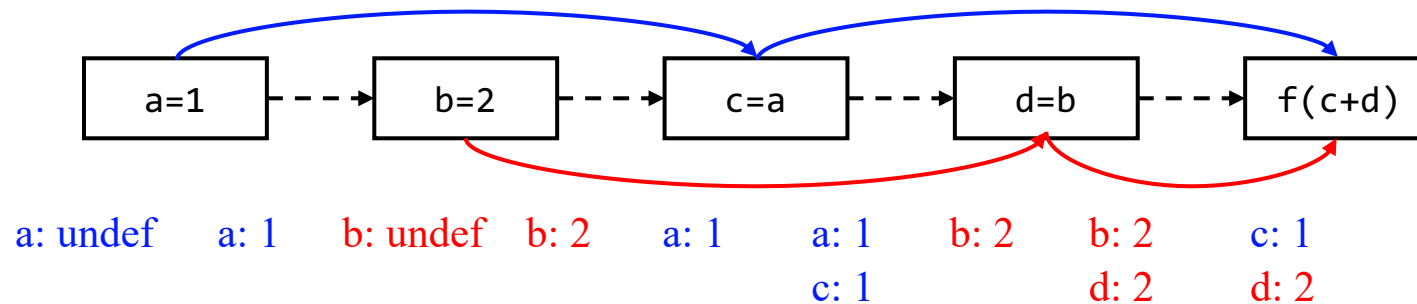


Advantages?



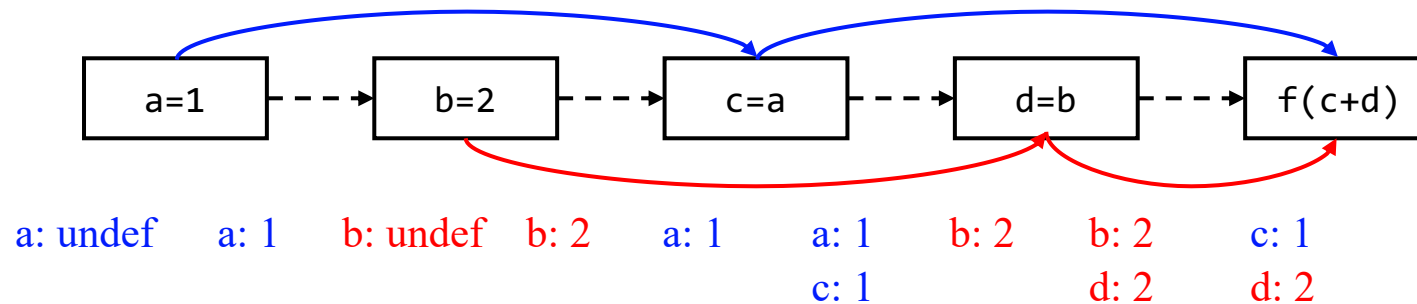
Sparse DFA

- Sparse Dataflow Analysis
 - Temporal sparsity
 - Space sparsity



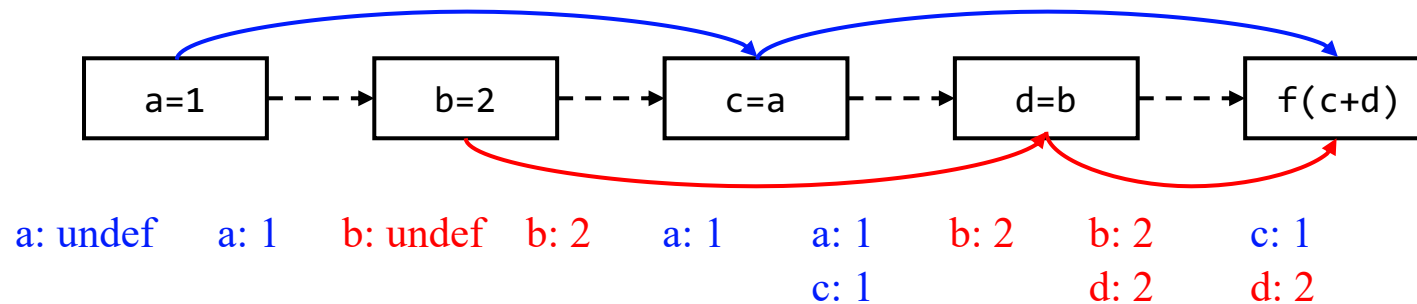
Sparse DFA

- **Sparse Dataflow Analysis**
 - **Temporal sparsity:** Skip unnecessary control flows
 - **Space sparsity**



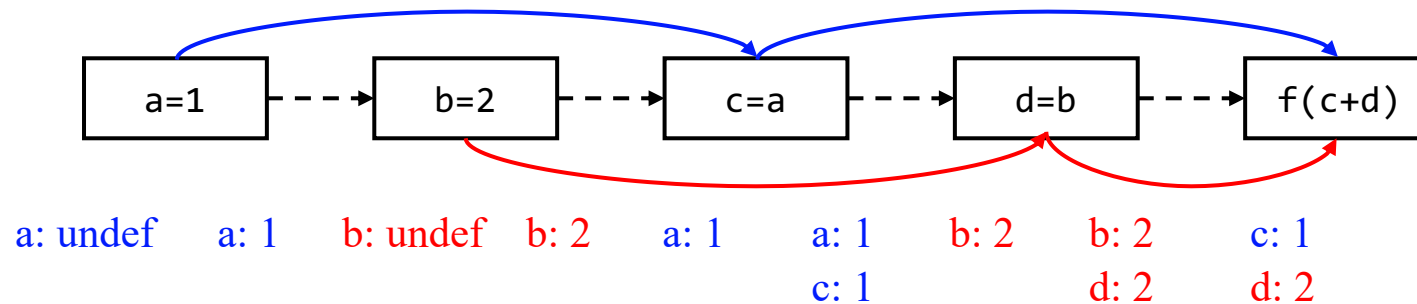
Sparse DFA

- **Sparse Dataflow Analysis**
 - **Temporal sparsity:** Skip unnecessary control flows
 - **Space sparsity:** Preserve necessary dataflow facts at a program point



Sparse DFA

- **Sparse Dataflow Analysis** **Relations to SSA?**
 - **Temporal sparsity:** Skip unnecessary control flows
 - **Space sparsity:** Preserve necessary dataflow facts at a program point



Sparse DFA for Security

- [PLDI'18](#): Pinpoint: Fast and Precise Sparse Value Flow Analysis for Million Lines of Code
- [ICSE'19](#): Smoke: Scalable Path-Sensitive Memory Leak Detection for Millions of Lines of Code
- [ICSE'20](#): Conquering the Extensional Scalability Problem for Value-Flow Analysis Frameworks
- [ICSE'20](#): Pipelining Bottom-up Data Flow Analysis
- [PLDI'21](#): Path-Sensitive Sparse Analysis without Path Conditions
- [OOPSLA'22](#): Indexing the Extended Dyck-CFL Reachability for Context-Sensitive Program Analysis
- [TOSEM'23](#): Anchor: Fast and Precise Value-Flow Analysis for Containers via Memory Orientation
- [ICSE'24](#): A Two-Layer Persistent Summary Design for Taming Third-Party Libraries
- ...
- ...



THANKS!